The Hydra-16 book

The Hydra-16 and HydraOS, from the board to the programs

The machine, the hardware and building it, the kernel and its calls,
the drivers, the shell, the languages, the tools, the emulator,
and the programming guide

2026-10-20
From the repository's book (docs/): node docs/build/docpdf.js

Contents

The Hydra-16 book

The Hydra-16 book

1. The Hydra-16 at a glance

1. The Hydra-16 at a glance

2. The hardware

2. The hardware
The architecture
The devices
The parts
Building it
The board layout
The quirks
The cards

3. The base: the kernel and the system calls

3. The base: the kernel and the system calls
The boot
The kernel
Files, servers and namespaces
The system calls
The monitor

4. The drivers

4. The drivers
The serial console
The kernel's devices
Storage
Sound
GPIO, I2C and SPI
Video: the Vera X

5. The shell: rc

5. The shell: rc

6. The languages

6. The languages
HyForth
hylang
HyBASIC

7. The tools

7. The tools

8. The emulator and the PC tools

8. The emulator and the PC tools

9. The programming guide

9. The programming guide
Assembly
C
Servers and drivers
Libraries
An operating system of your own
The system call reference

The Hydra-16 book

The Hydra-16 is a multitasking computer built around the 65C02, and HydraOS is its operating system. This book is the whole of it, in the order you'd meet it: what the machine is, the board and how to build it, the kernel the board runs, the drivers, the shell, the languages, the tools, the emulator that runs it all on a PC, and finally how to write programs for it. Each chapter builds on the ones before it, and each starts with the big picture before the detail, so you can read as far as you need and stop.

If you're new, read chapter 1 first: it explains the ideas the rest of the book takes for granted, and ends with an hour's tour of the running system, in the emulator, with no hardware needed.

Contents

1. **The Hydra-16 at a glance**: what it is, the ideas behind it (sixteen tasks in hardware, banked memory, paged ROM, prioritised interrupts), the software in layers, and a first hour with the system.
2. **The hardware**: the board, from the outside in.
    1. The architecture: the buses, the memory map, the pseudo-registers, banking, the ROMs, I/O, interrupts, clocks, reset.
    2. The devices: the VIA (GPIO, I2C, SPI, the tick), the ACIA (the serial port), the YM2151 and the audio path, the clock chip, the expansion slots.
    3. The parts: every part on every board, generated from the schematics, with what each does and what to buy.
    4. Building it: assembly, power, jumpers, programming the ROMs, the first power-up, POST, the hardware test, and what to do when something's wrong.
    5. The board layout: where everything is, the connectors and headers and their pins.
    6. The quirks: what the board does that you wouldn't expect, and what changed from 1.8C to 1.8D.
    7. The cards: the memory daughter card, the bus breakout card, and the Vera X video card.
3. **The base: the kernel and the system calls**: what every system on the Hydra shares.
    1. The boot: reset, POST, the drivers, init.
    2. The kernel: tasks and the scheduler, interrupts, memory and banks, shared segments, semaphores, notes, the clock.
    3. Files, servers and namespaces: the file model every device is part of.
    4. The system calls: how a program calls the kernel, and what it can ask for.
    5. The monitor: Woz's monitor, the base's own init.
4. **The drivers**: the devices as files.
    1. The serial console: `ser` and `cons`, the windows, the PC tool's `/pc`.
    2. The kernel's devices: `kdev`: the modules, `/proc`, pipes, the environment, the clock.
    3. Storage: SD cards, the RAM disks, the ROM disk, HydraFS.
    4. Sound: the YM2151, the Vera X's PSG and PCM, `/dev/snd`.
    5. GPIO, I2C and SPI: the VIA's port A, the I2C bus, the SPI devices.
    6. Video: the Vera X: `vid`, its screen, graphics and sprites, and its keyboard and mouse.
5. **The shell: rc**: commands, pipes, variables, scripts, namespaces and windows.
6. **The languages**: HyForth, hylang and HyBASIC, and the numbers they share.
7. **The tools**: the core tools, `edit` and `ed`, `as` and `dis`, `db`, `play`, `xmodem`, `calc`.
8. **The emulator and the PC tools**: running the Hydra on a PC, in a terminal or a browser; the tests; the PC tool, card images and writing a real card.
9. **The programming guide**: programs of your own.
    1. Assembly: the assembly SDK, modules and RAM programs.
    2. C: the C SDK, cc65, the library, how C reaches the system calls.
    3. Servers and drivers: serving files, owning interrupts.
    4. Libraries: the number and asm libraries, `XCALL`.
    5. An operating system of your own: a system of your own on the base.
    6. The system call reference: every call, generated from the specification.

The design notes, the plans each part was built from, and the project's status are in design/: they're history, and not needed to use the system.

How to read it

- **Names in** `code` are what you type, a file's path, or a name in the source: `ls /rom`, `base/kernel/sched.s`, `SPAWN`. Paths without a folder at the front are in the repository: `base/` is the base, `reborn/` HydraOS, `board/` the KiCad files, `docs/` this book.
- **Numbers with** `$` are hexadecimal, as the 6502's assemblers write them: `$FF00` is 65280.
- **References like U7 or J18** are the schematic's: a chip (U, IC), a connector or jumper (J), a resistor (R) or capacitor (C). The board layout shows where each one is.
- **Prompts** show where a line is typed: `/>` HyForth (the login shell), `%` rc, `hylang>` hylang, `>` HyBASIC, `T1 00>` the monitor, and `$` a shell on the PC.

Printing it

hydra-16.pdf is the whole book as one PDF, with its contents as bookmarks: `node docs/build/docpdf.js` makes it again (it needs Edge or Chrome). The parts list is made from the schematics by `node docs/build/bom.js`, and the call reference from the specification by `node docs/build/callref.js`: both run whenever the board or the calls change, and the repository's checks fail if they haven't.

1. The Hydra-16 at a glance

The Hydra-16 is a computer you can build: a single board around a WDC W65C02S, the modern CMOS version of the 6502 that ran the Apple II, the Commodore 64 and the BBC Micro. What makes it different from those machines is that its memory system was designed for **multitasking**. The board itself keeps sixteen separate programs' memory apart, so the operating system can switch from one program to another with a single register write, and a program can't trample another's memory by accident.

On top of that hardware runs **HydraOS**, an operating system in the style of Plan 9 (Bell Labs' successor to Unix): every device is a file, every program has its own view of the file tree, and the system is built from small pieces that each run in a task of their own. It has a shell, three programming languages, an assembler, a debugger, a screen editor and a music player, all running on the 65C02.

This chapter explains the ideas the rest of the book is built on, then takes you through an hour with the running system. You don't need a board for any of it: the emulator runs the same ROM images the board does.

Contents

1. The machine in one page
2. The ideas behind it
3. The software, in layers
4. The repository
5. A first hour
6. Where to go next

The machine in one page

CPU	WDC W65C02S at 3.58 MHz (jumpers for 1.79 and 0.89 MHz too; with the stated parts, 3.58 MHz is the fastest valid speed)
Tasks	16, each with its own 32K of RAM (zero page and stack included) and its own bank selections, kept apart by the hardware
RAM	512K of task RAM (32K for each task); 2 MB of shared RAM on the board; up to 15 memory cards of 2 MB each, all seen through an 8K window
ROM	4 MB of paged ROM in 16K banks (the operating system's modules and its ROM disk); a 128K BIOS ROM in 8K pages (the kernel)
Interrupts	16 interrupt lines in priority order, each with its own handler address
On the board	A 65C22 VIA (timers, GPIO, an I2C bus, an SPI bus with eight device headers), a 65C51 ACIA (an RS-232 serial port), a YM2151 FM sound chip with a stereo mixer, a socket for a DS1747 clock
Expansion	Six card slots on an 8-bit bus, eight SPI headers (SD cards), a GPIO and I2C header, two memory card connectors
Video	The Vera X card in slot 0: the Commander X16's VERA chip (VGA at 640x480, two layers, 128 sprites, 256 colours, a 16-voice synthesizer, PCM audio, an SD card slot)
Power	A standard ATX PC power supply
Software	HydraOS 1.0: a preemptive multitasking kernel, Plan 9's files and namespaces, a windowing console, HydraFS on SD cards, the rc shell, HyForth, hylang and HyBASIC, an assembler, a debugger, a screen editor, a music player

---

The ideas behind it

A 65C02 in brief

If you've programmed a 6502, skip ahead. If not, here's what you need to follow this book.

The 65C02 is an 8-bit processor with a 16-bit address bus: it can address 64K of memory, from `$0000` to `$FFFF` (the `$` means hexadecimal). It has three 8-bit registers, **A** (the accumulator) and **X** and **Y** (index registers), an 8-bit stack pointer **S**, a program counter, and a status register **P** whose bits are its flags (carry, zero, negative, overflow, interrupt disable, decimal).

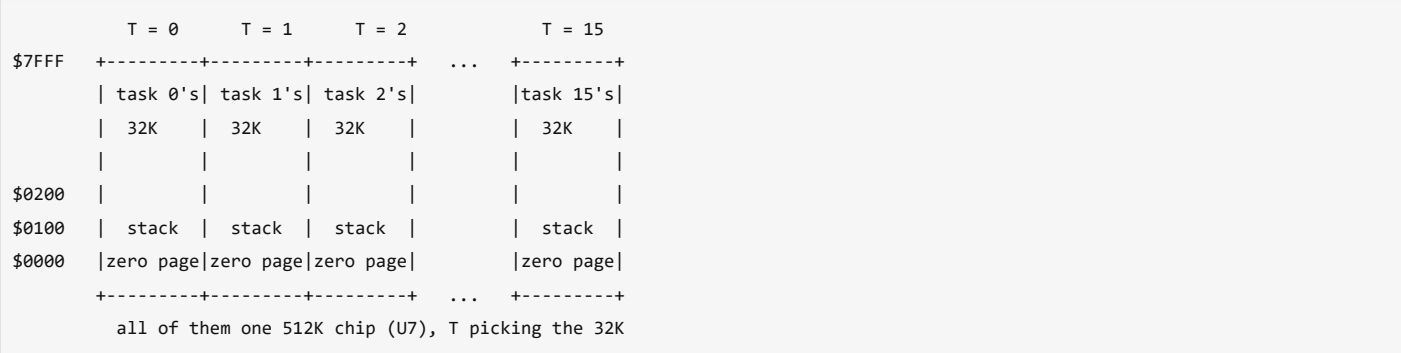
Two parts of memory are special. **Page zero**, `$0000-$00FF`, can be reached with shorter, faster instructions, and pairs of its bytes can be used as pointers; programs keep their most used variables there. **Page one**, `$0100-$01FF`, is the stack. At the top of memory, `$FFFA-$FFFF` hold three addresses the processor reads when it starts and when it's interrupted: the NMI vector, the RESET vector and the IRQ vector.

Everything else, ROM, RAM and the I/O chips' registers, is somewhere in the same 64K: the 6502 has no separate I/O instructions, so a device is read and written like memory.

Sixteen tasks in hardware

On a normal 6502 machine, every program shares one 64K. If two programs run "at once", they have to agree on who uses which memory, and they have to share page zero and the stack, which makes switching between them slow and fragile.

The Hydra-16 gives each of sixteen **tasks** its own lower 32K. The board has one 512K RAM chip for this, and a 4-bit register called `T` (at `$FFF0`) drives that chip's top four address lines. So when `T` is 3, addresses `$0000-$7FFF` reach task 3's 32K of the chip; when `T` is 4, the same addresses reach task 4's. Each task has its own page zero and its own stack.



Switching tasks is one write: the kernel saves the running task's registers on its own stack, writes the next task's number to `T`, and restores that task's registers from its stack, which is now in place. No memory is copied.

More memory than 64K: the bank window

32K of RAM per task isn't much. The upper half of the address space gives more, through windows that show part of a much larger memory at a time. The first is the **bank window** at `$8000-$9FFF`: 8K that shows one **bank** of memory, chosen by the task's **RAM bank register**, a byte written at address `$0000`.

- Bank numbers `$00-$EF` are **task banks**, on memory cards. Each task has its own: task 3's bank `$05` and task 4's bank `$05` are different memory, so tasks stay apart here too. Each memory card gives each task 16 banks (128K); the board takes up to 15 cards.
- Bank numbers `$F0-$FF` are **shared banks**, on the board itself: the same memory for every task, so tasks can share data. There are 256 of them (2 MB), shown 16 at a time; which 16 is chosen by another register, `U` (`$FFF1`).

Each task's RAM bank register is its own: when `T` changes, so does which bank is in the window, with no further writes. The operating system hands out banks so programs don't have to agree among themselves.

The paged ROM

The second window, `$A000-$DFFF`, shows 16K of the **paged ROM**: 4 MB of flash in eight chips, 256 banks of 16K. Each task chooses its own ROM bank with its **ROM bank register**, a byte written at address `$0001`.

This is where the operating system lives. The drivers, the shell and the languages are **modules** in the paged ROM, each in banks of its own or, if it's small, sharing a bank with others, and each **runs in place**: a task that runs the shell sets its ROM bank register to the shell's bank and jumps to it. Nothing is copied into RAM, so a task keeps nearly all of its 32K for its data, and sixteen tasks can run the same module at once. The rest of the paged ROM is a small read-only disk, `/rom`, with the tools (programs read into RAM to run) and files the system needs.

The BIOS ROM and its pages

The top 8K, `$E000-$FFFF`, is the **BIOS ROM**, where the kernel lives. It's paged too: a 128K flash chip, 16 pages of 8K, the page chosen by a fourth register, `W` (`$FFF3`). Changing `W` changes the code being run under the processor's feet, so the kernel keeps the same small block of code at the same address on every page, and moves between pages only through it. The kernel's 192 system calls are a **jump table** on page 0, at `$F800`: a program calls the system with `JSR` to a call's slot.

Interrupts with priorities

A 6502 has a single interrupt input, so a normal machine's interrupt handler has to ask each device in turn which one interrupted. The Hydra-16 has **sixteen interrupt lines** in priority order (the VIA first, then the serial port, then the card slots and the sound chip), a pair of priority encoders that find the highest one waiting, and a small RAM of sixteen handler addresses that the processor reads instead of the ROM's IRQ

vector. So each line goes straight to its own handler. The kernel gives each line to the driver that owns it.

I/O and the card slots

The I/O chips' registers are at `$FF00-$FFEF`: fifteen **ports** of 16 bytes each. Port 0 is the VIA, port 1 the serial port, port 4 the sound chip, and the rest belong to the six card slots, two ports and two interrupt lines each. `$FFF0-$FFF3` are the four registers `T`, `U`, `V` and `W` (`V` chooses which handler address a software interrupt uses).

The whole map

```
$FFFF +------------------------------+
      | $FFFE-$FFFF: the IRQ vector   |  from the vector RAM: the active line's handler
      | $FFFA-$FFFD: NMI, RESET       |  from the BIOS ROM
      | $FFF0-$FFF3: T U V W          |  the pseudo-registers
      | $FF00-$FFEF: I/O ports 0-14   |  the VIA, the ACIA, the YM2151, the slots
$E000 | BIOS ROM, 8K page (W)         |  the kernel
      +------------------------------+
$DFFF |                              |
      | paged ROM, 16K bank ($01)     |  the system's modules, the ROM disk
$A000 |                              |
      +------------------------------+
$9FFF | RAM bank, 8K ($00)            |  a task bank, or a shared bank (with U)
$8000 |                              |
      +------------------------------+
$7FFF |                              |
      | the task's own 32K (T)        |  its program, data, stack and page zero
      |                              |
$0000 +------------------------------+
```

Chapter 2 describes how the board does all this, chip by chip.

The software, in layers

The base

The **base** is what every system on the Hydra-16 shares, and what the kernel needs to be useful. It's in the repository's `base/` folder:

- **The kernel**, the whole BIOS ROM: the boot and the power-on self test (POST), the scheduler, the interrupt path, tasks and memory, files and namespaces, and the 192 system calls. It runs as task 0.
- `ser`, the console driver: the serial port as a file. It runs as task F (15).
- `kdev`, the kernel's devices: the list of modules a program is started from, the tasks (`/proc`), pipes, the environment. It runs as task E (14).
- `wozmon`, a monitor: Steve Wozniak's Apple 1 monitor, to look at memory, change it and run code. It runs as task 1, the first program, called **init**.

The base boots on its own and gives you the monitor's prompt. It exists so that someone can write a different operating system on the same kernel: chapter 3 is the base.

HydraOS

HydraOS is the operating system built on the base, in the repository's `reborn/` folder (it was rebuilt from scratch in 2026, hence the name). It uses the same BIOS ROM, byte for byte, and its own paged ROM:

Task	Module	What it does
0	the kernel	The scheduler, interrupts, system calls, memory, files and namespaces
1	`init`, then the shell	`init` sets up the RAM disks and the namespace, then becomes window 0's shell (`EXEC`): no task is kept just to wait. The kernel starts `init` again when that shell ends, and once for each window made
2 and up	programs	The other windows' shells, the tools, the languages, your programs
B (11)	`vid`	The Vera X video card (it ends at once if there's no card)

Task	Module	What it does
C (12)	`snd`	Sound: the YM2151, and the Vera X's synthesizer and PCM
D (13)	`kdev`	The kernel's devices (the base's), and with them the GPIO header, the I2C bus and the Vera X's keyboard and mouse
E (14)	`storage`	SD cards, the RAM disks, the ROM disk, and the HydraFS file system
F (15)	`cons`	The console: the serial port, windows, the PC tool's `/pc` folder

`cons` is the base's `ser` with a windowing system on top: they share the same serial-port code.

The Plan 9 ideas

HydraOS borrows three ideas from Plan 9, and they explain most of how it works.

Every device is a file. The serial console is `/dev/cons`; you read it to get a line typed, and write it to print. The sound chip is `/dev/snd` and `/dev/sndctl` (`echo note 0 60 >/dev/sndctl` plays middle C). The tasks are directories in `/proc`. A device is a **server**, a driver running in its own task that answers file requests, so the same few system calls (`OPEN`, `READ`, `WRITE`, `CLOSE`) reach every device.

Every program has its own namespace. The file tree a program sees is built by **binds** and **mounts** ("put this directory here"), and a program can change its own without affecting anyone else's. `/bin`, where programs are found, is a **union** of several directories (the RAM disk's, an SD card's, the ROM's), searched in order, so there are no search paths.

Notes, not signals. When you press Ctrl-C, the console sends a **note** called `interrupt` to the programs in that window's **note group**. A program can catch a note with a handler, or let the default end it.

Programs and languages

A **program** is either a module in the paged ROM (it runs in place) or a file, usually ending in `.hyx`, that the system loads into a task's RAM at `$0800` and runs. You can write programs in assembly (with ca65 on a PC, or with `as` on the Hydra itself) or in C (with cc65 on a PC).

HydraOS has three languages of its own: **HyForth** (Forth 2012, and the login shell), **hylang** (a Lisp) and **HyBASIC** (in the QuickBASIC style). All three, and C, share one number system: exact integers of any size, exact fractions and decimals, in any base.

The repository

Folder	What's there
`docs/`	This book; `docs/design/` the design notes and plans
`board/`	The KiCad 9 schematics and PCBs: the main board, the memory daughter card, the bus breakout card, the Vera X glue board
`base/`	The base: the kernel (`kernel/`), the system calls' specification (`spec/`), `ser`, `kdev` and `wozmon` (`modules/`), the emulator (`sim/`), its tests, its ROM images (`bin/`)
`reborn/`	HydraOS: its modules, its ROM disk (`romfs/`) and SD card (`sdcard/`), the SDKs (`sdk/`), its tests, its ROM images (`bin/`)
`old/`	The Hydra's first operating system (the 1.8C software), kept as it was

The ROM images are kept in Git, so you can program a board without building anything: `base/bin/` has the base's, `reborn/bin/` HydraOS's. To build them yourself you need Node.js 18 or later and cc65.

A first hour

This is a tour of the running system. Everything here works the same on a board; we'll use the emulator.

Starting it

You need Node.js (18 or later) and the repository. From the repository's `reborn/` folder:

```
$ node sim/run.js -i
```

That boots HydraOS's images (`reborn/bin/`) in the emulator, in real time, with your terminal as the Hydra's serial console. (To build the images yourself first, `node build.js`, which needs cc65 too.) The boot prints the kernel's banner, POST's results, each driver starting, and then the shell's prompt:

```
Hydra-16: kernel 0.1, ABI 1
POST ZP:0 ST:0 OS:0 HI:0 SH:S W:0
RAM U:0 F0:0/00/0000 F4:0/00/0000 F8:0/00/0000 FC:0/00/0000 00:0/00/0000 10:0/00/0000
POST ok
RAM modules: 02
task F: cons
...
HydraOS 1.0 for the Hydra-16
init: up in task 01
...
HyForth (Forth 2012), bye to end
/>
```

The emulator takes keys after **Ctrl-A**: Ctrl-A x quits, Ctrl-A r presses the reset button, Ctrl-A h lists the rest.

If you'd rather not use a terminal, open `reborn/bin/hydra-16.html` (in Git, made by every build; `node sim/web.js` makes it alone): the whole emulator as one web page, which needs nothing installed. Open it in a browser, and the serial console is a terminal in the page, with the Vera X's screen beside it and a Sound button.

The shell

The prompt, `/>`, is the current directory and `>`. The login shell is **HyForth working as a shell**: a line that starts with a Forth word or a number is Forth, and any other line is a command for **rc**, the Plan 9 shell. So both work at the same prompt:

```
/> 2 3 + .
5
/> : sq dup * ;
/> 7 sq .
49
/> echo hello | wc
      1       1       6
```

Forth works on a stack: `2 3 + .` pushes 2 and 3, adds them, and prints the result. `: sq dup * ;` defines a new word, `sq`, that squares a number. `echo hello | wc` is an rc command line: `echo` prints its arguments, `|` sends them to `wc`, which counts the lines, words and characters.

The console edits a line as you type: Backspace, the arrow keys, Home and End, Ctrl-U to clear the line, and Up and Down for the lines you typed before. **Ctrl-C** stops whatever's running.

Files

Everything is in one tree of names:

```
/> ls /
bin/
dev/
env/
lib/
mnt/
pc/
proc/
ram/
rom/
sd/
sram/
tmp/
/> ls /rom
README
bin/
doc/
lib/
songs/
```

Directory	What's there
`/rom`	The ROM disk: read only, always there
`/ram`	Your own area of the RAM disk (each shell has its own)
`/sram`	The shared RAM disk, the same for every program
`/sd/0`	The SD card in SPI device 0 (`/sd/1` the card in device 1, and so on)
`/bin`	The programs: a union of `/ram/bin`, the cards' `bin`, `/rom/bin` and the ROM's modules
`/dev`	The devices: `cons`, `sndctl`, `vid`, `gpio`, `i2c` ...
`/proc`	The tasks, a directory each
`/env`	The environment variables, a file each
`/pc`	A folder on your PC, with the PC tool (or the emulator's `--pc-dir`)

`cd /rom` goes into a directory and `cd` alone goes back to `/`; `cat /rom/README` prints a file. Redirection and pipes are rc's: `echo hi >/ram/f` writes a file, `cat /ram/f | wc -c` counts its characters.

To make a file, use **edit**, the screen editor: `edit /ram/todo`, type some lines, then **Ctrl-O** and Enter to save and **Ctrl-X** to leave. The bottom two lines of the screen list the other keys, and Ctrl-G shows them all.

The SD card

The emulator puts HydraOS's **SD card** in SPI device 0, so it's at `/sd/0`. It holds what isn't part of the system: the sample programs, the sample songs and the benchmarks.

```
/> ls /sd/0
bench/
sample/
songs/
/> ls /sd/0/sample/c
chorus
code
ctest
hello
jukebox
keys
ntest
philo
prodcons
race
round
shapes
sketch
tones
upper
/> /sd/0/sample/c/hello world
hello from C, world
```

On a board, write the same card to a real SD card with `node sim/tools/sdwrite.js` (see chapter 8).

Tasks and windows

Every program runs in a task of its own. `ps` lists them:

```
/> ps
task  state   parent     cpu group  name
   0  ready   -          1.0     0  kernel
   1  wait    -          0.4     1  forth
   2  ready    1         0.0     1  ps
  12  idle    -          0.0    12  snd
  13  idle    -          0.1    13  kdev
  14  idle    -          1.5    14  storage
  15  event   -          0.5    15  cons
```

`ps` itself is task 2, started by the shell (task 1). Only six of the sixteen tasks are in use with a shell at its prompt (seven with a Vera X): the rest are for your programs and more windows. `sleep 100 &` runs a program in the background (`ps` shows its task, say 3); `kill 3` ends it; `/proc/3/status` is its state as a file.

The console has **windows**, as Plan 9's rio has: each one a whole console with its own shell, one shown at a time on your terminal. The windows' keys start with **Ctrl-]**: Ctrl-] c starts a new window with a shell, Ctrl-] and a digit shows that window, Ctrl-] w lists them, Ctrl-] s splits the screen and starts a shell below, Ctrl-] ? lists all the keys. A window you aren't looking at keeps running, and its output is there when you come back.

The multitasking demos show tasks working together, sharing memory and taking turns with semaphores:

```
/> bind -a /sd/0/sample/c /bin
/> race
/> philo
/> round
```

`bind -a` adds the card's samples to the end of `/bin`, so they run by name. `race` shows several tasks losing each other's updates to a shared counter, then not losing any once they take turns with a mutex; `philo` is the dining philosophers; `round` is a four-voice round, each voice a task keeping its own time (turn on the emulator's sound with `node sim/run.js -i --sound` to hear it).

The languages

HyForth is the shell you're in already. **hylang** is a Lisp:

```
/> hylang
hylang> (+ 1/3 1/6)
=> 1/2
hylang> (map (fn {n} {* n n}) {1 2 3})
=> {1 4 9}
hylang> exit
```

HyBASIC is BASIC in the QuickBASIC style, with numbered lines or none:

```
/> basic
> PRINT 1 / 3 + 1 / 6; 2 ^ 70
 1/2  1180591620717411303424
> 10 FOR i = 1 TO 3: PRINT i; i / 4: NEXT
> RUN
 1  1/4
 2  1/2
 3  3/4
> SYSTEM
```

The numbers are exact: `1/3 + 1/6` is `1/2`, not 0.4999. Chapter 6 covers all three.

A program of your own

The SDKs are in `reborn/sdk`. To build a sample in a folder of your own (this needs cc65):

```
$ mkdir hi
$ cp sdk/asm/samples/hi/hi.s hi/
$ node build.js prog hi
$ node sim/run.js -i --pc-dir hi
```

`--pc-dir hi` gives the Hydra your folder as `/pc`, as the PC tool does for a real board over the serial line:

```
/> /pc/hi.hyx one two
Hello, one!
Hello, two!
I'm task 5, in /, in window 0.
```

The Hydra can assemble it itself, too: `as /sd/0/sample/as/hi.s /ram/hi` makes the same program.

Where to go next

- **Building a board?** Chapter 2, then building it.
- **Writing an operating system of your own?** Chapter 3, the base.
- **Using the system?** The shell, the languages and the tools.
- **Writing programs?** The programming guide, after the system calls.
- **Working on a PC?** The emulator.

2. The hardware

This chapter is the Hydra-16's board, from the outside in. It starts with how the board works as a whole: what the processor sees at each address, and which chips make each part of that happen. Then come the devices on the board, the parts you need to build one, how to build it and bring it up, where everything is on the board, what it does that you wouldn't expect, and the cards that plug into it.

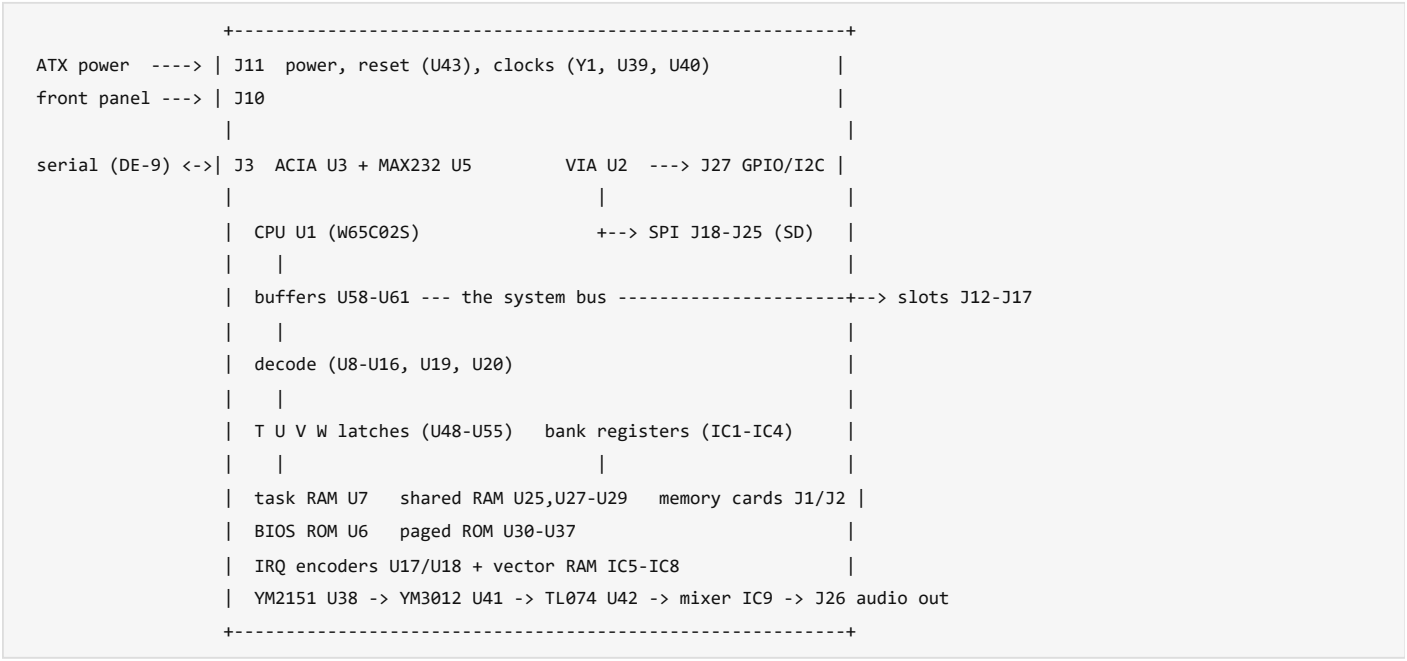
It describes **board revision 1.8D**, the schematic in `board/` (`board/hydra-16.kicad_sch` and its sheets; KiCad 9). The 1.8D board runs the same software as 1.8C: it changes some parts for reliability, and nothing a program can see. The quirks lists what changed.

Reference designators (U25, J18, R28) are the schematic's. The registers' names (`T_REGISTER`, `VIA_T1CL`, ...) are those of `base/include/hw.inc`, which every program on the Hydra includes.

Contents

1. The architecture: how the board works, part by part.
   - The memory map and the pseudo-registers `T`, `U`, `V`, `W`
   - Task RAM and the bank registers, the RAM window, the paged ROM, the BIOS ROM
   - I/O space, interrupts, clocks, reset, power and the bus
2. The devices: the VIA and its GPIO, I2C and SPI buses, the ACIA and the serial port, the YM2151 and the audio path, the clock chip, the expansion slots.
3. The parts: every part on every board, generated from the schematics, with what to buy.
4. Building it: assembly, the first power-up, programming the ROMs, POST, the hardware test, and finding faults.
5. The board layout: a drawing of each board, where each part is, and the connectors' pins.
6. The quirks: what the board does that you wouldn't expect, and what changed from 1.8C to 1.8D.
7. The cards: the memory daughter card, the bus breakout card, and the Vera X video card.

The board in one picture



The board layout has the real drawing, made from the PCB.

The architecture

This page is how the board works: what the processor sees at each address, and which chips make it so. The overview gave the ideas; here they're taken down to the gates. The sheet named under each heading is the schematic sheet to look at (`board/NAME.kicad_sch`).

Contents

1. The board at a glance
2. The memory map
3. The pseudo-registers T, U, V, W
4. Task RAM and the bank registers
5. The RAM window
6. The paged ROM
7. The BIOS ROM
8. I/O space
9. Interrupts
10. Clocks
11. Reset, power and the bus
12. The chips by function

The board at a glance

CPU	WDC W65C02S (U1), 3.58 MHz as built (J5-J7 choose 0.89, 1.79 or 3.58 MHz; J8's 7.16 MHz is beyond the stated parts)
Task RAM	512K (U7, HM628512, or a DS1747 for a clock that keeps time): 32K at `$0000-$7FFF` for each of 16 tasks
The RAM window	8K at `$8000-$9FFF`. **Task banks:** up to 15 memory cards of 2 MB, each giving every task 16 banks of its own. **Shared banks:** 2 MB on the board (U25, U27-U29), the same for every task: 256 banks, 16 at a time
Paged ROM	4 MB (U30-U37, eight SST39SF040), seen in 16K banks at `$A000-$DFFF`
BIOS ROM	U6, an SST39SF010A (128K; a '020A or '040 fits too), seen in 8K pages at `$E000-$FFFF`
I/O	15 ports of 16 bytes at `$FF00-$FFEF`, and the system port at `$FFF0-$FFFF`
Interrupts	16 IRQ lines in priority order, each with its own vector (a 16-entry vector RAM read at `$FFFE`)
Devices	A 65C22 VIA (timers, GPIO, I2C, SPI), a 65C51 ACIA (RS-232 on a DE-9), a YM2151 and YM3012 (stereo FM sound) with a mixer
Expansion	Six slots (62-pin card edge, 8-bit bus), eight SPI device headers, a GPIO and I2C header, the memory card connectors
Video	The Vera X in slot 0 (the cards)
Power	An ATX PC supply (+5 V, +3.3 V, +12 V, -12 V)

There are no programmable logic chips on the board: everything is 74-series logic, so every signal can be followed on the schematic and probed on the board.

The memory map

What the CPU sees at each address, and what chooses it:

Addresses	What	Chosen by
`$0000`	RAM bank register (writes); task RAM (reads)	per task
`$0001`	ROM bank register (writes); task RAM (reads)	per task
`$0002-$7FFF`	Task RAM: page zero, the stack, the program's RAM	`T`
`$8000-$9FFF`	The RAM window: a task bank (`$00-$EF`) or a shared bank (`$F0-$FF`)	`$00`; and `T` for task banks, `U` for shared banks
`$A000-$DFFF`	Paged ROM, a 16K bank	`$01`
`$E000-$FEFF`	BIOS ROM, an 8K page	`W`
`$FF00-$FFEF`	I/O ports 0-14, 16 bytes each	
`$FFF0-$FFF3`	The pseudo-registers `T`, `U`, `V`, `W`	

Addresses	What	Chosen by
`$FFF4-$FFF9`	Not decoded: reads give `$FF` (1.8D)	
`$FFFA-$FFFD`	BIOS ROM: the NMI and RESET vectors	`W`
`$FFFE-$FFFF`	The IRQ/BRK vector, from the vector RAM (16 entries)	the highest-priority active IRQ line, or `V`

Every task has its own `$0000-$7FFF` and its own `$00` and `$01`. All tasks share the I/O space, the shared RAM banks, both ROMs, and the pseudo-registers themselves.

The pseudo-registers T, U, V, W

(Sheet `FFF_Registers`)

Four 8-bit registers in the system port. Each is a 74F573 latch, written by the CPU's write strobe, and read back through a 74F541 buffer, so a read gives back all 8 bits last written.

Address	Register	Latch / read-back	The bits the hardware uses
`$FFF0`	`T`, the task	U48 / U49	T0-T3: which task's RAM (U7's A15-A18; the memory cards' task lines)
`$FFF1`	`U`, the shared macro-page	U50 / U51	U0-U3: the shared RAM's A13-A16
`$FFF2`	`V`, the vector select	U52 / U53	V0-V3: the vector RAM's entry when no IRQ line is active. V4-V7 go nowhere: the kernel keeps a software interrupt's number there
`$FFF3`	`W`, the BIOS ROM page	U54 / U55	W0-W5: the BIOS ROM's A13-A18 (16 pages on the '010A, 64 on a '040)

- **They power up random.** The latches have no reset: the boot code sets them all. Pressing the reset button doesn't change them either.
- **The unused bits are kept.** Bits 4-7 of `T` and `U`, and 6-7 of `W`, are stored and read back, but drive nothing.
- `T` **is the one that matters most.** Writing it switches every per-task thing at once: the task RAM, the bank registers' outputs (so the RAM window's bank and the ROM bank), and the memory cards' task lines.

Task RAM and the bank registers

(Sheets `ZPMirrorRAM`, `AddressDecode`)

Task RAM is one 512K static RAM (U7, an HM628512 or a DS1747) on the main board:

- CPU A0-A14 go straight to it; CPU A15 is its chip enable, low for `$0000-$7FFF`.
- `T0-T3` drive its A15-A18, so each value of `T` selects a different 32K.
- Page zero and the stack page are part of it, so every task has its own zero page and stack, and a task switch is a single write to `$FFF0`.

The DS1747 option. U7's socket also takes a Dallas DS1747 (the 5 V part, not the 3.3 V DS1747W): a 512K battery-backed RAM with a real-time clock, pin compatible, in a 600-mil module. Its clock's registers are the chip's top 8 bytes, which are **task F's** `$7FF8-$7FFF`. The kernel looks for the clock at boot and sets the time from it; the rest of the chip is task RAM as before, kept while the power's off. The devices say more.

The bank registers `$00` and `$01` are four 74LS219 chips, small 16 x 4-bit RAMs, addressed by `T0-T3`: so each task has its own pair of registers.

Register	Chips	Drives
`$00`: the RAM bank	IC1 (bits 0-3), IC2 (bits 4-7)	`RAMB0-7`: which bank is at `$8000-$9FFF`
`$01`: the ROM bank	IC3 (bits 0-3), IC4 (bits 4-7)	`ROMB0-7`: which paged ROM bank is at `$A000-$DFFF`

- A write to `$0000` or `$0001` stores the byte in the current task's entry. The decoding (U8-U10) spots a write to `$000x`, and U16 picks the register by A0.
- The same write also goes to task RAM as usual. The registers can't be read; a **read** of `$00` or `$01` comes from task RAM, which holds the same value. So a program can read its own bank selections back.
- The chips' outputs are always on, so the bank lines follow `T` at once: a task switch brings the new task's RAM bank and ROM bank in with no further writes.

- On this board, bank bits 2 and 3 are crossed on their way to the memory. The software never notices, but it matters when you trace a fault to a chip: see the quirks.

The RAM window

(`$8000-$9FFF`; sheets `ZPMirrorRAM`, `SharedMem`, `AddressDecode`)

`$8000-$9FFF` shows one 8K bank. The **bank ID** is the current task's `$00`:

Bank ID	What
`$m0-$mF` (m = `$0-$E`)	Memory card m: 16 banks **for each task** (the card's chips also get `T0-T3`)
`$F0-$FF`	Shared RAM: 16 banks, the same for every task, from shared macro-page `U`

How it's decoded:

- **The module select.** The bank ID's upper nibble (`RAMB4-7`) goes to a 74LS154 (U23), enabled by the window's select (`nBRA_S`: `$8000-$9FFF` while PHI2 is high). Its outputs `nBRC0-nBRC14` go to the memory card connector (J1), one for each module; `nBRC15` selects the shared RAM on the board.
- **The memory card connector.** The lower nibble and `T` are buffered to it by U21 (74F541), as `RAMB_M0-3` and `T_M0-3`, with the read and write strobes (`RWB_M`, `!RWB_M`). J2 carries D0-D7, A0-A12 and +5 V.
- **A memory card** (the cards) is four HM628512, 2 MB, set to its module number by a jumper. `T_M0-3` go to its chips' A13-A16, so each task has its own 16 banks; `RAMB_M0-1` go to A17-A18; and `RAMB_M2-3` pick the chip. 16 tasks x 16 banks x 8K is the card's 2 MB.
- **Shared RAM** is four HM628512 on the main board, U25, U27, U28 and U29 (surface mount, in SOP-32). `RAMB_M2-3` pick the chip, through a 74F139 (U26) enabled by `nBRC15`. `RAMB_M0-1` go to the chips' A17-A18, and `U0-U3` to A13-A16, so the 16 bank IDs `$F0-$FF` are seen in 16 macro-pages (`U` = 0-F): 256 shared banks, 2 MB. The software numbers them **shared bank IDs**, `U << 4 | (bank & $0F)`.

Bank IDs (`$00`)	Shared RAM chip	Shared bank IDs
`$F0-$F3`	U25	`$x0-$x3` for each `U` = x
`$F4-$F7`	U28 (the crossed bits 2 and 3)	`$x4-$x7`
`$F8-$FB`	U27 (the crossed bits)	`$x8-$xB`
`$FC-$FF`	U29	`$xC-$xF`

A bank with no chip behind it, a missing memory card, reads `$FF` on 1.8D (the data bus's pull-ups, RN4). POST finds the cards that are there (`RAM modules: 03` at boot), and the kernel hands out banks only on those.

The HM628512's pins, for tracing a bad line (POST and the hardware test report bad lines by number):

Signal	Pin	Signal	Pin
A0-A7	12, 11, 10, 9, 8, 7, 6, 5	D0-D2	13, 14, 15
A8, A9, A10, A11, A12	27, 26, 23, 25, 4	D3-D7	17, 18, 19, 20, 21
A13-A16 (`U0-U3` on shared RAM; `T_M0-3` on a card)	28, 3, 31, 2	~CE, ~OE, ~WE	22, 24, 29
A17, A18 (`RAMB_M0`, `RAMB_M1`)	30, 1	VCC, GND	32, 16

(The SOP-32 package has the same pin numbers as the DIP.)

The paged ROM

(`$A000-$DFFF`; sheets `BankedROM`, `ZPMirrorRAM`)

Eight SST39SF040 flash chips (512K each, U30-U37) make 4 MB, seen as 256 banks of 16K. The bank is the current task's `$01`:

- `ROMB0-4` go to the chips' A14-A18: 32 banks in each chip.
- `ROMB5-7` pick the chip through a 74F138 (U24), enabled by the window's select (`nBRO_S`: `$A000-$DFFF` while PHI2 is high).

Chip select (the 74F138's output)	Chip	Its image (HydraOS's build)
0	U31	`bin/prom0.bin`
1	U32	`bin/prom1.bin`
2	U34	`bin/prom2.bin`

Chip select (the 74F138's output)	Chip	Its image (HydraOS's build)
3	U36	
4	U30	
5	U33	
6	U35	
7	U37	

- **The images are the chips' view, not the CPU's.** Two things on the board shuffle the banks: each bank's two 8K halves are swapped (CPU `$A000` reads the chip's offset `$2000` in the bank, and `$C000` its offset `$0000`), and the bank number's bits 2 and 3 are crossed. The build allows for both (`base/tools/romimg.js`), so you burn each `promN.bin` whole, at offset 0, in the socket the table gives, and never think about it. The quirks explain.
- **Only the sockets that are used need chips.** HydraOS fills three (U31, U32, U34); the base alone fills one (U31). The others can stay empty.
- **A card can take the window over.** `nBROMD`, a slot pin pulled up on the board, disables the whole paged ROM when a card pulls it low, so the card can answer `$A000-$DFFF` itself.
- **The ROMs can't be written in circuit.** The chips' ~OE is the inverted R/W, and their ~WE is tied high: program them in a programmer (building it).

The BIOS ROM

(U6, the root sheet)

An SST39SF010A in a PLCC-32 socket: 128K, 16 pages of 8K. (The '020A, 32 pages, and the '040, 64 pages, fit the same socket.)

- CPU A0-A12 go to the chip's A0-A12, and `W0-W5` to its A13-A18: each value of `W` selects an 8K page.
- It's selected for `$E000-$FEFF` and for `$FFFA-$FFFD`, the NMI and RESET vectors. The I/O space and the vector RAM take the rest of `$FF00-$FFFF`.
- The build's `bin/bios_rockwell.bin` and `bin/bios_wdc.bin` (one for each serial chip U3 can have) are 128K, the whole '010A.

Changing `W` changes the code being run. The next instruction is fetched from the new page, at the same address. So the kernel keeps identical code at the same address on every page: the COMMON block at `$FD00` (the IRQ entry and exit, and the far call between pages) and the reset stub at `$E000`. And since `W` isn't reset, **every page starts with the reset code**: the RESET vector on every page is `$E000`, which sets `W` to 0. The kernel explains how the pages are used.

I/O space

(Sheet `AddressDecode`)

`$FF00-$FFFF` is decoded when A8-A15 are all 1 (U14, a 74F30), while PHI2 is high (`nIO_S`). A 74LS154 (U19) splits it by A4-A7 into 16 ports of 16 bytes, `nIOP0_S-nIOP15_S`. Ports 0-14 are for devices; port 15 is the system port.

Port	Addresses	Use
0	`$FF00-$FF0F`	VIA (65C22, U2)
1	`$FF10-$FF1F`	ACIA (65C51, U3): 4 registers, repeated every 4 bytes
2	`$FF20-$FF2F`	Slot 0, select A (the Vera X: its registers 0-15)
3	`$FF30-$FF3F`	Slot 0, select B (the Vera X: its registers 16-31)
4	`$FF40-$FF4F`	YM2151 (U38): `$FF40` address and status, `$FF41` data (repeated)
5-9	`$FF50-$FF9F`	Slots 1-5, select A
10-14	`$FFA0-$FFEF`	Slots 1-5, select B
15	`$FFF0-$FFFF`	The system port (below)

The system port is decoded by a 74F138 (U20) on A0-A2, enabled by `nIOP15_S` and by A3 differing from A2 (U12):

Address	Select	Use
`$FFF0-$FFF3`	`nIOB0-3`	`T`, `U`, `V`, `W`
`$FFF4-$FFF9`	none	Not decoded (reserved)

Address	Select	Use
`$FFFA-$FFFD`	(the BIOS ROM)	The NMI and RESET vectors, from the current BIOS page
`$FFFE`, `$FFFF`	`nIOB14`, `nIOB15`	The vector RAM, low and high bytes

Each slot's port selects (`nIOA_S`, `nIOB_S`) and its two IRQ lines are listed with the slots. A card can also decode addresses of its own from the whole I/O page, with `nIO_S` and A0-A7.

Interrupts

(Sheet `IRQ_P_E`, the IRQ priority encoder)

Sixteen active-low IRQ lines, `nIRQ0-nIRQ15`, each pulled up (RN2, RN3). Line 0 has the highest priority.

Line	Source	Line	Source
0	VIA	8	Slot 4, A
1	ACIA	9	Slot 5, A
2	Slot 0, A (the Vera X)	10	Slot 1, B
3	Slot 0, B	11	Slot 2, B
4	YM2151	12	Slot 3, B
5	Slot 1, A	13	Slot 4, B
6	Slot 2, A	14	Slot 5, B
7	Slot 3, A	15	Not connected: the software interrupt

How it works:

- Two cascaded 74LS148 priority encoders (U17 for lines 0-7, U18 for 8-15) find the highest-priority active line.
- Any active line asserts the CPU's ~IRQ (through U44, then U22). The IRQ input is level-triggered: a device holds its line low until its handler clears the cause.
- A 74F157 (U45) forms a 4-bit index, `q0-q3`: the encoders' output while any IRQ line is active, `V0-V3` otherwise.
- The index addresses the **vector RAM**, four 74LS219 (IC5-IC8): a 16-entry table of 16-bit vectors. The CPU reads its IRQ/BRK vector from `$FFFE/$FFFF`, so it gets the entry for the active line: each line has its own handler.

```
nIRQ0-7 --> U17 '148 --+
                       +--> index q0-q3 --> U45 '157 --> IC5-IC8 vector RAM --> D0-D7 at $FFFE/$FFFF
nIRQ8-15 -> U18 '148 --+                     ^
                                  V0-V3 ----+ (when no line is active)
```

The index is the line number XOR 7. The '148s encode active-low inputs with the highest-priority input as 7, so line n gives index `n ^ 7`: line 0 is entry 7, line 7 entry 0, line 8 entry 15, line 15 entry 8. `hw.inc`'s `IRQ_INDEX(line)` is `line ^ 7` for this reason.

Writing a vector. A write to `$FFFE` or `$FFFF` stores the low or high byte of entry `q`, which is `V0-V3` when no IRQ line is active. So: interrupts off, set `V` to the entry, write the two bytes. The kernel's boot does this once (`base/kernel/reset.s`): every line's entry is the kernel's one IRQ path, which then calls the line's driver.

BRK. `BRK` also reads `$FFFE/$FFFF`, and with no IRQ line active that's entry `V0-V3`. The kernel leaves `V` at line 15's entry (line 15 has no hardware), so a `BRK` comes to the kernel by that vector: a program's stray `BRK` is a note to it, and the debugger's breakpoints are `BRK`s.

The lines can't be masked. There's no mask register: a line held low keeps interrupting until the device lets it go. So a driver must quiet its device in its handler, and a device that can only be quieted slowly (an I2C device, say) mustn't be given a line.

NMI (`NMIB`, pulled up, on every slot) uses the BIOS ROM's `$FFFA` vector. Nothing on the board drives it.

Clocks

(Sheet `Clocks`)

A 14.31818 MHz crystal (Y1) with a 74ACT14 (U39) oscillator, divided by a 74F191 counter (U40):

Clock	Frequency	Goes to
HS_CLK	14.318 MHz	The slots
Q0	7.159 MHz	CPU clock jumper J8
SND_CLK (Q1)	3.580 MHz	The YM2151's clock; the slots; CPU clock jumper J7
SER_CLK (Q2)	1.790 MHz	The ACIA's clock (XTAL1); the slots; CPU clock jumper J6
Q3	0.895 MHz	CPU clock jumper J5

- **The CPU clock** (PHI2) is whichever of J5-J8 is fitted: fit exactly one. The software is built for **J7, 3.58 MHz**. 0.89 and 1.79 MHz work electrically, but the software's timing (the tick, the serial port) isn't built for them.
- **With the stated parts, 3.58 MHz is the fastest valid speed.** At J8's 7.16 MHz the bank registers, the decoders and the RAMs don't make the shorter cycle (there are no wait states), and the power-on self test fails nearly every RAM test (measured on 1.8D, 2026-10-13); the YM2151 is too slow for that bus too. J8, and the software's `node build.js --clock 2`, are for a board with faster parts.
- **DMA stops the clock.** The CPU's PHI0 input is PHI2 gated by DMAB (U11), so a card pulling DMAB low stops the CPU (the W65C02S is fully static).
- PHI1, the inverse of PHI2 (U22), qualifies the address decoding: every select is only active while PHI2 is high.
- **1.8D's terminations.** Each clock leaves its driver through a 22 ohm resistor (R23-R27): a clock's driver is the net X_SRC, and the board and the slots see X. R22 (330 ohms) limits the crystal's drive.

Reset, power and the bus

Power comes from an ATX-24 connector (J11):

- +5 V for the logic, +3.3 V, and +12 V and -12 V for the audio op-amps and the slots.
- +5 V standby is brought out as +5VA.
- The slots have a -5 V pin, fed from ATX pin 20 (1.8D): older ATX 1.x supplies have -5 V there; newer ones leave it unconnected. Don't count on it.

The front panel (J10, 2 x 6):

Pin	Signal
7	RESB: a push button from here to ground resets the machine
8	+5 V
11	ATX PS_ON#: a switch from here to ground turns the supply on
12	The power LED's anode, through R2 (470 ohms) from +5 V (1.8D); its cathode to ground
5, 6, 9, 10	Ground
1-4	Not connected

Reset (RESB):

- A DM1813 supervisor (U43) holds reset low at power-up and during brown-outs.
- The front panel's button and the slots (open-collector) can pull it low too.
- It resets the CPU, the VIA, the ACIA, the YM2151 and the YM3012. It **doesn't** reset the pseudo-registers, the bank registers or any RAM: the boot code sets those.

The bus:

- RDY is pulled up by R28 (10K). The W65C02S drives RDY low itself while it waits in WAI, so RDY must never be driven high directly. (1.8C had a jumper, J4, for the pull-up; 1.8D has the resistor instead.)
- BE, the bus enable, is pulled up, and on the slots.
- **DMA:** a card pulls DMAB low to take the bus. That stops the CPU's clock and turns off the address buffers (U58, U60, 74F541), the data transceiver (U59, 74ACT245) and the R/W buffer (U61, 74F125); the card then drives the address and data buses itself. Nothing uses DMA yet.
- **The data bus's pull-ups** (RN4, 10K, 1.8D) make a read that nothing answers, an empty slot or an unused port, give $FF.
- SYNC, the CPU's opcode-fetch signal, is on the slots, for single-stepping hardware or a logic analyser.
- MLB and VPB aren't used. SOB is tied high.

The chips by function

Function	Parts
CPU	U1 W65C02S
Bus buffers	U58, U60 74F541 (address); U59 74ACT245 (data); U61 74F125 (R/W)
Address decoding	U8, U9 74LS260; U10 74F20; U11 74ACT08; U12 74F86; U13, U16 74F32; U14 74F30; U15 74F00; U19 74LS154 (the I/O ports); U20 74F138 (the system port); U21 74F541 (the memory card buffer); U22 74ACT240
Pseudo-registers	U48, U50, U52, U54 74F573; U49, U51, U53, U55 74F541; U56 74F32; U57 74F02
Bank registers	IC1-IC4 74LS219; U23 74LS154 (the module select); U24 74F138 (the paged ROM's chip select)
Interrupts	U17, U18 74LS148; U44 74F00; U45 74F157; IC5-IC8 74LS219 (the vector RAM); RN2, RN3 pull-ups
Task RAM	U7 HM628512 (or DS1747)
Shared RAM	U25, U27, U28, U29 HM628512 (SOP-32); U26 74F139
Paged ROM	U30-U37 SST39SF040
BIOS ROM	U6 SST39SF010A (PLCC-32)
Clocks	Y1 14.31818 MHz; U39 74ACT14; U40 74F191
Reset	U43 DM1813
VIA, SPI	U2 R65C22; U4 74F138 (the SPI device select)
Serial	U3 R65C51; U5 MAX232
Sound	U38 YM2151; U41 YM3012; U42 TL074; IC9 NJM4556A (the mixer)

The parts has every part, with its value and package.

The devices

The chips on the board that do I/O, and the connectors they reach. Each section gives the chip's port and IRQ line, how it's wired, and which driver serves it: chapter 4 is the software side.

Contents

1. The VIA (65C22): timers, GPIO, I2C, SPI
2. The SPI bus and its device headers
3. The I2C bus
4. The ACIA (65C51) and the serial port
5. The YM2151 and the audio path
6. The clock chip
7. The expansion slots

The VIA (65C22)

U2, a Rockwell R65C22 (or WDC W65C22N). Port 0, `$FF00-$FF0F`; IRQ line 0, the highest priority.

The VIA is the board's general-purpose I/O chip: two 8-bit ports, two timers, and two handshake lines on each port. On the Hydra:

Part of the VIA	Used for
Port A, PA0-PA1	The I2C bus: PA0 is SCL, PA1 is SDA (bit-banged)
Port A, PA2-PA7, CA1, CA2	General-purpose I/O, on header J27
Port B	The SPI bus (bit-banged): its clock, data, select and device number
Timer 1	The kernel's tick: free-running, 200 interrupts a second, which drives the scheduler and the clock
Timer 2	The serial driver's send pacing (below)
CB1, CB2, the shift register	Not brought out: the shift register can't drive the SPI lines

J27, the VIA port A header (2 x 6, 2.54 mm):

Pin	Signal	Pin	Signal
1	GND	2	+5 V
3	PA0: `I2C_SCL`	4	PA1: `I2C_SDA`
5	PA2	6	PA3
7	PA4	8	PA5
9	PA6	10	PA7
11	CA1	12	CA2

Every pin of J27, and the SPI bus's clock and data lines, pass through **J28**, an SP720 ESD protection array (a chip in a DIP-16 socket, though the schematic calls it a connector). It clamps static from fingers and cables to the supply rails. The board works without it, but fit it if anything's ever plugged into J27.

HydraOS serves the pins as files (in `kdev`): `/dev/gpio/0` to `/dev/gpio/7`, the whole port as `port`, `ctl` to set each pin's direction, and `ca1`, which can interrupt (GPIO, I2C, SPI and the clock). PA0 and PA1 belong to the I2C bus, so don't use them as GPIO.

The SPI bus

VIA port B, bit-banged; U4 (74F138) decodes the device select.

Port B bit	Signal
PB0	`SPI_CLK`: the clock (SCLK)
PB1	`nSPI_CS`: low selects the device given by PB3-PB6
PB2	`SPI_MOSI`: data out

Port B bit	Signal
PB3-PB5	`SPI_A0-A2`: the device, 0-7
PB6	`SPI_A3`: 0 for the board's devices 0-7 (U4); 1 for devices 8-15, which a card in a slot decodes itself
PB7	`SPI_MISO`: data in

So a transfer is: set PB3-PB6 to the device, take PB1 low, clock bits out on PB2 and in from PB7 by toggling PB0, then take PB1 high. U4 turns `nSPI_CS` and `SPI_A0-A3` into one select line for each of the eight headers. Every slot carries the whole bus, `nSPI_CS` and `SPI_A0-A3` included, so a card can be devices 8-15.

The eight SPI device headers, J18-J25 for devices 0-7, are each 1 x 6, 2.54 mm:

Pin	Signal
1	~CS (the device's select)
2	SCLK
3	MOSI
4	MISO
5	+5 V
6	GND

SD cards. An SD card goes on a header through a common SD card module, one with its own 3.3 V regulator and level shifter: the headers give +5 V and 5 V logic, which would destroy a bare card. Device 0 (J18) is the usual place. The storage driver serves a card on any device as `/dev/sd/N` (N the device, in hex) and its file system at `/sd/N` (storage). Any SPI device can also be used raw, as `/dev/spi/N`.

The I2C bus

VIA PA0 (SCL) and PA1 (SDA), bit-banged; pull-ups in RN1.

The I2C bus goes to J27 (pins 3 and 4) and to every slot (pins 28 and 30), with its pull-ups on the board, so a device on a card needs none of its own. HydraOS serves it as `/dev/i2c`: its `ctl`, and a file for each device that answers. The Vera X's input controller (the keyboard and mouse) is on it at `$42`.

Being bit-banged, it's slow (some tens of kHz), and an I2C device can't be given an interrupt line: the board's lines can't be masked, and a device can only be quieted by an I2C transaction, too slow for an interrupt handler. So I2C devices are polled.

The ACIA (65C51) and the serial port

U3, a Rockwell R65C51 (or WDC W65C51N). Port 1, `$FF10-$FF13` (repeated through `$FF1F`); IRQ line 1.

The serial port is the Hydra's console: everything in this book is typed at a terminal on it.

The chip and its clock:

- The ACIA's clock is `SER_CLK`**, 1.790 MHz**, not the 1.8432 MHz its baud rate table assumes. So every rate is 2.9% slow: 9600 baud is really about 9,320. Terminals and USB serial adapters accept that.
- The rates the board has are the ACIA's table's: 300, 600, 1200, 2400, 4800, 9600, 19200, and 115200 (the ACIA's clock divided by 16). Both the base and HydraOS start at **9600 baud, 8 data bits, no parity, 1 stop bit**, with RTS/CTS. HydraOS can change the rate (`echo b115200 >/dev/serctl`).
- DCD and DSR are tied active (to ground).
- A **WDC W65C51N** works too, with the BIOS ROM for it (`bios_wdc.bin`): its transmitter's status bit and interrupt don't work, so the drivers time each byte instead.

Sending is paced. At 115200 the 2.9% error and the MAX232's slow edges leave the receiver little margin, so long output sent back to back loses characters. So the serial driver sends each byte from VIA timer 2, not from the ACIA's "transmitter empty" interrupt: a character's time and two idle bits at 115200, one at the other rates. Every build does this, on either chip.

The DE-9 connector (J3, a female socket) is driven by a MAX232 (U5), and wired like a modem (DCE):

DE-9 pin	Signal	Direction
2	The Hydra transmits (ACIA TxD)	out

DE-9 pin	Signal	Direction
3	The Hydra receives (ACIA RxD)	in
5	Ground	
7	ACIA ~CTS, from the terminal's RTS (through J30)	in
8	ACIA ~RTS, to the terminal's CTS	out
1, 4, 6, 9	Not connected	

The cable. A USB serial adapter's male plug goes straight into J3. An extension between them is a **straight-through** male-to-female cable (pin 2 to pin 2, 3 to 3, and so on), not a null-modem cable.

J30, the CTS jumper (1.8D). The ACIA only transmits while its ~CTS is low:

- **1-2** (fit this): ~CTS comes from the line's pin 7, so the terminal's RTS can stop the Hydra. The cable must carry pin 7, and the terminal must assert RTS (they do by default).
- **2-3**: ~CTS is held low: the Hydra always sends. For a three-wire cable (pins 2, 3 and 5 only).
- With no jumper, ~CTS floats and the Hydra may send nothing at all.

The YM2151 and the audio path

U38, a Yamaha YM2151 (OPM). Port 4: `$FF40` address and status, `$FF41` data (repeated through `$FF4F`); IRQ line 4. Sheets `Sound` and `Mixer`.

The chip. The YM2151 is an 8-channel, 4-operator FM synthesizer, the chip in many 1980s arcade machines and the Commander X16. It's clocked by `SND_CLK`, 3.58 MHz, its design frequency.

- Writing `$FF40` selects a register; writing `$FF41` writes it. Reading `$FF40` gives the status: bit 7 is busy, bits 0 and 1 are timer A's and timer B's flags.
- After a data write the chip is busy for 64 of its clocks, about 18 µs, and a write while it's busy is lost: so the driver checks bit 7 before each write.
- Its two timers can interrupt on line 4. The sound driver doesn't use them for timing songs (the system's tick is steadier: the quirks).
- **CT1 and CT2**, its two general-purpose outputs, are on header J9.

The audio path:

```
YM2151 U38 --serial--> YM3012 U41 (DAC) --> TL074 U42 (buffers, filters) --> C90/C91 --+
                                                                                       |
slot 0's SND_CL0 / SND_CR0 (the Vera X) ------------------------------------------- +--> mixer IC9 --> R30/R31 --> C92/C93 --> J26
                                                                                       |   (NJM4556A,
line input J29 ----------------------------------------------------------------------+    ±12 V)
```

- The **YM3012** (U41) turns the YM2151's serial output into two analog channels, centred on its COM pin (about 2.5 V). A **TL074** (U42) buffers and filters them. In 1.8D, C90 and C91 (10 µF) keep that 2.5 V out of the mixer.
- The **mixer** (IC9, an NJM4556A on ±12 V) is two inverting summers, one for each channel, each at unity gain (10K in, 10K feedback). Each adds three inputs: the YM3012's channel, **slot 0's** audio (`SND_CL0`, `SND_CR0`: the Vera X's PSG and PCM), and the line input on **J29** (pin 1 left, pin 2 right). On 1.8D the other slots' audio pins aren't connected; 1.8C mixed all six slots (the quirks).
- **The output** (1.8D): through R30/R31 (33 ohms, which keep the op-amp stable into a cable and safe from a shorted plug), and C92/C93 (220 µF, non-polar), to the 3.5 mm stereo jack **J26**: left on the tip, right on the ring. R32 and R33 hold the jack's side at 0 V so a plug going in doesn't click. The NJM4556A drives headphones (16-32 ohms) directly.
- **The grounds.** The audio's ground, GNDA, meets the logic's ground once, at R29 (0 ohms) beside the YM3012, and at the ATX connector.

The sound driver serves the YM2151 and the Vera X's PSG as one synthesizer, `/dev/snd` (sound).

The clock chip

The board has no clock chip of its own: the time comes from a **Dallas DS1747** in the task RAM's socket, U7, in place of the HM628512. The DS1747 is a 512K static RAM with a battery and a real-time clock inside, in the same pinout; its clock's eight registers are its top 8 bytes. Since `T0-T3` drive U7's A15-A18, those are task F's `$7FF8-$7FFF`.

- Task F's program (the serial driver, `ser` or `cons`) keeps its RAM below `$7FF8`, so nothing else writes there.

- The kernel's `RTC` call reads and sets the chip's registers; `kdev` sets the system's clock from the chip as it starts, and `/dev/time` (`date`) sets both the system's clock and the chip's.
- The rest of the chip is task RAM as before, kept while the power's off.
- A new DS1747 comes with its battery disconnected and its oscillator stopped: setting the time once starts it.
- Without a DS1747 the clock starts at 2000-01-01 each boot, until something sets it.

Use the 5 V DS1747, not the 3.3 V DS1747W.

The expansion slots

J12-J17, slots 0-5: 62-pin card edge connectors, the "Hydra bus 8-bit". Sheet `Connectors`*.*

All six slots carry the same bus, except for each slot's two port selects and its two IRQ lines (and its audio pair, which on 1.8D only slot 0's reaches the mixer):

Slot	Connector	`nIOA_S` **(pin 51)**	`nIOB_S` **(pin 49)**	`IRQA` **(pin 55)**	`IRQB` **(pin 53)**
0	J12	port 2, `$FF20`	port 3, `$FF30`	line 2	line 3
1	J13	port 5, `$FF50`	port 10, `$FFA0`	line 5	line 10
2	J14	port 6, `$FF60`	port 11, `$FFB0`	line 6	line 11
3	J15	port 7, `$FF70`	port 12, `$FFC0`	line 7	line 12
4	J16	port 8, `$FF80`	port 13, `$FFD0`	line 8	line 13
5	J17	port 9, `$FF90`	port 14, `$FFE0`	line 9	line 14

Slot 0's lines have the highest priority of the slots, which is why the video card goes there; the B lines of slots 1-5 have the lowest.

The pinout, the same on every slot (~ active low):

Pin	Signal	Pin	Signal
1	GND	2	~`nBROMD`: disable the paged ROM
3	~`RESB`	4	D7
5	+5 V	6	D6
7	~`NMIB`	8	D5
9	-5 V (ATX pin 20, if the supply has it)	10	D4
11	`BE`	12	D3
13	-12 V	14	D2
15	`SER_CLK` (1.79 MHz)	16	D1
17	+12 V	18	D0
19	GND	20	`RDY`
21	R/~W	22	`SND_CLK` (3.58 MHz)
23	~`DMAB`	24	`SND_CR`: audio in, right (slot 0 only)
25	~`nIO_S`: any I/O port	26	`SND_CL`: audio in, left (slot 0 only)
27	`SYNC`	28	`I2C_SDA`
29	`SPI_SCLK`	30	`I2C_SCL`
31	`SPI_MOSI`	32	A15
33	`SPI_MISO`	34	A14
35	~`nSPI_CS`	36	A13
37	`PHI1`	38	A12
39	`PHI2`	40	A11
41	`SPI_A3`	42	A10
43	`SPI_A2`	44	A9
45	`SPI_A1`	46	A8
47	`SPI_A0`	48	A7
49	~`nIOB_S`: the slot's port B	50	A6

Pin	Signal	Pin	Signal
51	~`nIOA_S`: the slot's port A	52	A5
53	~`IRQB`	54	A4
55	~`IRQA`	56	A3
57	+5 V	58	A2
59	`HS_CLK` (14.318 MHz)	60	A1
61	GND	62	A0

Designing a card:

- **Select your registers** with `nIOA_S` or `nIOB_S`: each is low for the slot's 16 bytes, only while PHI2 is high, with the address already steady. Decode A0-A3 for the register. For more than 32 bytes, decode `nIO_S` and A0-A7 yourself, at addresses no other card uses.
- **Interrupts:** drive `IRQA`/`IRQB` open-collector (pull low, or let go): the board pulls each line up. Hold the line until the handler clears the cause, and give the card a way to be quieted from a register.
- **Timing:** the bus has no wait states. A read's data must be on the bus 10 ns before PHI2 falls (about 140 ns of PHI2 high at 3.58 MHz, 70 ns at 7.16), and a write's data is held only 10 ns after PHI2 falls, so latch it with a strobe that ends with PHI2.
- **An empty slot reads** `$FF` (1.8D's pull-ups), so software can look for a card by reading its registers.
- **Power:** +5 V, +12 V and -12 V are always there; -5 V only with an old ATX supply. +3.3 V isn't on the slots.

The cards describes the cards made so far.

The parts

Every part on every board, with what to buy. The tables between the markers are made from the schematics by `node docs/build/bom.js` (from `board/`): don't edit them by hand, edit the schematic and run it again. The prose around them is written by hand.

Contents

1. Reading the tables
2. Buying the parts: the chips that need care, substitutes, sockets
3. The main board
4. The memory daughter card (through-hole and surface mount)
5. The bus breakout card
6. The Vera X glue board
7. What each chip does

Reading the tables

- **Qty** is how many of that part, value and package the board takes.
- **Part** is the schematic's value. Capacitors and resistors are marked the way their bodies are, with the three-digit EIA code, and the plain value beside it: **104** is 10 x 10^4 pF, 100 nF; **103** is 10 x 10^3 ohms, 10K; **220** is 22 ohms (22 x 10^0). **NP** is a non-polar (bipolar) electrolytic.
- **Package** is the footprint's: DIP-n is a through-hole chip with n pins, 0.3" wide unless it says 0.6"; SOP-32 a surface-mount chip; "ceramic, through-hole" a small disc or MLCC capacitor at 2.54 or 5.08 mm pitch; "radial" an electrolytic, its diameter given.
- **Socket** is the socket the footprint is drawn for. Every DIP chip on the main board is socketed, which makes a fault easy to fix and a chip easy to try in another place. **Sockets to buy** totals them.
- **References** are the schematic's, in ranges: C1-C4 is C1, C2, C3 and C4. The board layout shows where each is.

Buying the parts

Most of the board is ordinary 74-series logic, resistors and capacitors from any distributor. A few parts need care.

The CPU, W65C02S (U1). Buy the current WDC part, W65C02S6TPG-14 (DIP-40), from a distributor: it's still made. An old NMOS 6502 won't do: the software uses the 65C02's instructions throughout, the kernel's idle loop uses the W65C02S's `WAI`, and DMA relies on its static core (it stops the CPU's clock).

The VIA and the ACIA (U2, U3). The schematic has Rockwell's R65C22 and R65C51, which are long out of production. WDC's W65C22N and W65C51N are current:

- **The W65C22N** is a drop-in replacement (the N has an open-drain IRQ output, as the R65C22 has).
- **The W65C51N** has a known fault: its transmitter's status bit is stuck "empty" and it gives no transmitter interrupt. The software allows for it: program U6 with `bios_wdc.bin` (programming the ROMs).

Both WDC chips have run on a 1.8D (2026-10-13: a W65C22 in U2 with no change, a W65C51N in U3 with the WDC build). U2's IRQ pin is the priority encoder's own input (`nIRQ0`), not a shared line, so the push-pull W65C22S works there as well as the open-drain N.

The sound chips (U38 YM2151, U41 YM3012). These are only available used, pulled from old equipment, and fakes are common: buy from a seller with a good record, and test them (the hardware test checks the YM2151's busy flag and timers). The board works without them; leave U38 and U41 out and the system just has no FM sound. A modern alternative is the **OPM2151** (an FPGA re-implementation of the YM2151, pin-compatible and cycle-accurate, sold on Tindie by Joe's Retro Sto'), with an optional I2S DAC board that fits U41's socket in place of a YM3012: close the OPM2151's solder jumper JP2 ("I2S Mode") for that. Untested on the Hydra at the time of writing.

The 74LS219s (IC1-IC8): 16 x 4-bit RAMs, the bank registers and the vector RAM. They're out of production, but easy to find as surplus. The **74LS189** is the same chip with inverted outputs, and **won't** do.

The RAMs, HM628512 (512K x 8, 55 or 70 ns). Hitachi's are surplus now; Alliance's **AS6C4008** (55 ns) is current and pin compatible: the DIP-32 for U7 and the memory cards, the SOP-32 for the shared RAM (check that its body fits the footprint, 11.3 mm wide). U7 can be a **DS1747** instead, for a clock that keeps time (the devices).

The flash ROMs. Microchip's SST39SF040 (512K, DIP-32, 70 ns) for the paged ROM, and the SST39SF010A (128K) in PLCC-32 for the BIOS ROM. Both are current. Buy only as many SST39SF040 as the system needs: HydraOS uses three (U31, U32, U34), the base alone one (U31).

The reset supervisor (U43, DM1813 in TO-92): the DS1813-10 (Dallas, now Analog Devices).

The mixer's op-amp (IC9, NJM4556A): a dual op-amp that drives headphones. An LF353 fits the socket but distorts into headphones (it's what 1.8C had).

The ESD array (J28, SP720): Littelfuse's SP720APP, in a DIP-16 socket. Optional, but fit it if J27 is used.

The crystal (Y1, 14.31818 MHz, HC-18 or HC-49). The NTSC colour-burst frequency times four, so it's cheap and common.

Families. The schematic's families matter in a few places:

- **74F and 74ACT** where it says so: the decoding must be fast, and the 1.8D board uses ACT where a part drives a CMOS input (the CPU's clock, IRQ and data inputs). An F part in an ACT place works on most boards, but out of specification.
- **74LS** parts can be LS or ALS; **74F** parts mustn't be LS (they're too slow at 7.16 MHz).
- Don't substitute HC or HCT parts: they're slower than F and ACT.

Jumper shunts (2.54 mm): one for the CPU clock (J7), one for the CTS jumper (J30), and one for each memory card.

Power. Any ATX power supply with a 24-pin connector: the board needs far less than any of them gives.

The main board

`board/hydra-16.kicad_sch`, revision 1.8D. The memory cards, the Vera X and its glue are separate (below).

<!-- parts:main:begin -->

ICs

Qty	Part	Package	Socket	References
1	74ACT08	DIP-14	DIP-14 (0.3")	U11
1	74ACT14	DIP-14	DIP-14 (0.3")	U39
1	74ACT240	DIP-20	DIP-20 (0.3")	U22
1	74ACT245	DIP-20	DIP-20 (0.3")	U59
2	74F00	DIP-14	DIP-14 (0.3")	U15, U44
1	74F02	DIP-14	DIP-14 (0.3")	U57
1	74F125	DIP-14	DIP-14 (0.3")	U61
3	74F138	DIP-16	DIP-16 (0.3")	U4, U20, U24
1	74F139	DIP-16	DIP-16 (0.3")	U26
1	74F157	DIP-16	DIP-16 (0.3")	U45
1	74F191	DIP-16	DIP-16 (0.3")	U40
1	74F20	DIP-14	DIP-14 (0.3")	U10
1	74F30	DIP-14	DIP-14 (0.3")	U14
3	74F32	DIP-14	DIP-14 (0.3")	U13, U16, U56
7	74F541	DIP-20	DIP-20 (0.3")	U21, U49, U51, U53, U55, U58, U60
4	74F573	DIP-20	DIP-20 (0.3")	U48, U50, U52, U54
1	74F86	DIP-14	DIP-14 (0.3")	U12
2	74LS148	DIP-16	DIP-16 (0.3")	U17, U18
2	74LS154	DIP-24	DIP-24 (0.6")	U19, U23
8	74LS219	DIP-16	DIP-16 (0.3")	IC1-IC8
2	74LS260	DIP-14	DIP-14 (0.3")	U8, U9
1	DM1813-10	TO-92 Inline Wide		U43
1	HM628512LP-5	DIP-32	DIP-32 (0.6")	U7
4	HM628512LP-5	SOP-32		U25, U27-U29
1	MAX232	DIP-16	DIP-16 (0.3")	U5

Qty	Part	Package	Socket	References
1	NJM4556AD	DIP-8	DIP-8 (0.3")	IC9
1	R65C22-P2	DIP-40	DIP-40 (0.6")	U2
1	R65C51-P2	DIP-28	DIP-28 (0.6")	U3
8	SST39SF040	DIP-32	DIP-32 (0.6")	U30-U37
1	SST39SF0x0A	PLCC-32	PLCC-32	U6
1	TL074	DIP-14	DIP-14 (0.3")	U42
1	WDC65C02S	DIP-40	DIP-40 (0.6")	U1
1	YM2151	DIP-24	DIP-24 (0.6")	U38
1	YM3012	DIP-16	DIP-16 (0.3")	U41

Crystals and oscillators

Qty	Part	Package	Socket	References
1	14.31818MHz	crystal HC18		Y1

Capacitors

Qty	Part	Package	Socket	References
68	104 (100 nF)	ceramic, through-hole		C1-C3, C5, C6, C12-C46, C57-C64, C66-C85
5	105 (1 µF)	ceramic, through-hole		C4, C7-C10
2	106 (10 µF)	radial 5.0 mm		C90, C91
5	107 (100 µF)	radial 6.3 mm		C51, C53-C56
2	227 NP (220 µF)	radial 8.0 mm		C92, C93
2	330 (33 pF)	ceramic, through-hole		C47, C48
2	332 (3.3 nF)	ceramic, through-hole		C50, C52
1	475 (4.7 µF)	ceramic, through-hole		C49
4	476 (47 µF)	radial 6.3 mm		C86-C89

Resistors and networks

Qty	Part	Package	Socket	References
1	0 (a link)	axial resistor		R29
1	101 (100 Ω)	axial resistor		R3
11	103 (10K)	axial resistor		R4-R7, R13, R19-R21, R28, R32, R33
1	103 (10K)	SIP-9 network		RN4
1	105 (1M)	axial resistor		R1
5	220 (22 Ω)	axial resistor		R23-R27
2	330 (33 Ω)	axial resistor		R30, R31
1	331 (330 Ω)	axial resistor		R22
3	332 (3.3K)	SIP-9 network		RN1-RN3
1	471 (470 Ω)	axial resistor		R2

Connectors, headers and jumpers

Qty	Part	Package	Socket	References
1	0.89489MHz	header 2x01, 2.54 mm		J5
1	1.78977MHz	header 2x01, 2.54 mm		J6
1	3.57955MHz	header 2x01, 2.54 mm		J7
1	7.15909MHz	header 2x01, 2.54 mm		J8
1	ATX-24	Molex Mini-Fit Jr 5566-24A2 2x12 P4.20mm Vertical		J11
1	Audio In	header 2x01, 2.54 mm		J29
1	Audio Jack	Jack 3.5mm CUI SJ1-3525N Horizontal		J26
1	CTS	header 1x03, 2.54 mm		J30

Qty	Part	Package	Socket	References
1	Conn_01x24	socket 1x24, 2.54 mm		J2
1	Conn_01x32	socket 1x32, 2.54 mm		J1
1	DSUB_9	L77SDE09SA4CH3R		J3
1	Front Panel	header 2x06, 2.54 mm		J10
6	SLOT 0-5	TE 7-5530843-0		J12-J17
1	SND CT1/2	header 2x01, 2.54 mm		J9
1	SP720APP	DIP-16	DIP-16 (0.3")	J28
8	SPI Device 0-7	socket 1x06, 2.54 mm		J18-J25
1	VIA Port A	header 2x06, 2.54 mm		J27

Sockets to buy

Qty	Socket
15	DIP-14 (0.3")
19	DIP-16 (0.3")
13	DIP-20 (0.3")
3	DIP-24 (0.6")
1	DIP-28 (0.6")
9	DIP-32 (0.6")
2	DIP-40 (0.6")
1	DIP-8 (0.3")
1	PLCC-32

216 parts on the main board, from `board/hydra-16.kicad_sch`.

<!-- parts:main:end -->

The memory daughter card

A task RAM module is 2 MB, which gives each task 16 more banks; the main board takes up to 15 modules. There are two versions of the card:

Through-hole

`board/MemoryDaughterCard/MemDaughterCard.kicad_sch`: four DIP-32 RAMs, one module, 2 MB.

<!-- parts:memory:begin -->

ICs

Qty	Part	Package	Socket	References
1	74LS139	DIP-16		U1
4	HM628512LP-7	DIP-32	DIP-32 (0.6")	IC1-IC4

Capacitors

Qty	Part	Package	Socket	References
5	.1uF	ceramic, through-hole		C1-C5

Connectors, headers and jumpers

Qty	Part	Package	Socket	References
1	Conn_01x24	header 1x24, 2.54 mm		J2
1	Conn_01x32	header 1x32, 2.54 mm		J1
1	Conn_02x15_Odd_Even	header 2x15, 2.54 mm		J3

Sockets to buy

Qty	Socket
4	DIP-32 (0.6")

13 parts on the memory daughter card (through-hole), from `board/MemoryDaughterCard/MemDaughterCard.kicad_sch`.

<!-- parts:memory:end -->

Surface mount

`board/MemoryDaughterCard/MemDaughterCard_SurfaceMount.kicad_sch`: eight SOP-32 RAMs, which make it **two** modules, 4 MB. J3 chooses the first module's number (IC1-IC4) and J4 the second's (IC5-IC8): the cards.

<!-- parts:memory-smd:begin -->

ICs

Qty	Part	Package	Socket	References
1	74LS139	DIP-16		U1
8	HM628512LFP-7	SOIC127P1414X300-32N		IC1-IC8

Capacitors

Qty	Part	Package	Socket	References
5	.1uF	ceramic, through-hole		C1-C5

Connectors, headers and jumpers

Qty	Part	Package	Socket	References
1	Conn_01x15_Pin	header 1x15, 2.54 mm		J4
1	Conn_01x24	header 1x24, 2.54 mm		J2
1	Conn_01x32	header 1x32, 2.54 mm		J1
1	Conn_02x15_Odd_Even	header 2x15, 2.54 mm		J3

18 parts on the memory daughter card (surface mount), from `board/MemoryDaughterCard/MemDaughterCard_SurfaceMount.kicad_sch`.

<!-- parts:memory-smd:end -->

The bus breakout card

`board/HydraBusBreakoutCard/HydraBusBreakoutCard.kicad_sch`: a slot card with every bus signal on headers, for a logic analyser or for wiring a prototype. It has no chips.

<!-- parts:breakout:begin -->

Connectors, headers and jumpers

Qty	Part	Package	Socket	References
1	Address Bus	header 1x16, 2.54 mm		J3
1	Audio	header 1x03, 2.54 mm		J9
1	Bus Control	header 1x06, 2.54 mm		J7
1	Bus_Hydra_8bit	BUS Hydra 8Bit		J1
1	Clocks	header 1x05, 2.54 mm		J6
1	Data Bus	header 1x08, 2.54 mm		J2
1	I2C	header 1x02, 2.54 mm		J10
1	Interrupts & I/O	header 1x06, 2.54 mm		J5
1	Power	header 1x05, 2.54 mm		J11
1	Reset	header 1x01, 2.54 mm		J8
1	SPI	header 1x08, 2.54 mm		J4

11 parts on the bus breakout card, from `board/HydraBusBreakoutCard/HydraBusBreakoutCard.kicad_sch`.

<!-- parts:breakout:end -->

The Vera X glue board

`board/vera-x/vera-x.kicad_sch`: the protoboard that connects the Vera X to a breakout card in slot 0, with the keyboard and mouse controller (the cards). The Vera X card itself is bought (Joe Burks's VERA X 6.1), and isn't listed.

<!-- parts:vera-x:begin -->

ICs

Qty	Part	Package	Socket	References
1	74ACT00	DIP-14		U2
1	74AHCT138	DIP-16		U1
1	ATtiny861 (x16-smc)	DIP-20		U4

Capacitors

Qty	Part	Package	Socket	References
5	100nF	ceramic, through-hole		C2-C6
1	10uF	radial 5.0 mm		C1

Resistors and networks

Qty	Part	Package	Socket	References
1	10K	axial resistor		R1

Connectors, headers and jumpers

Qty	Part	Package	Socket	References
1	Breakout J10: I2C	header 1x02, 2.54 mm		J10
1	Breakout J11: power (pin 3 is -5 V!)	header 1x05, 2.54 mm		J11
1	Breakout J2: data bus	header 1x08, 2.54 mm		J2
1	Breakout J3: address bus (A15 at 1)	header 1x16, 2.54 mm		J3
1	Breakout J5: interrupts & I/O	header 1x06, 2.54 mm		J5
1	Breakout J6: clocks	header 1x05, 2.54 mm		J6
1	Breakout J7: bus control (pin 5 is /DMAB)	header 1x06, 2.54 mm		J7
1	Breakout J8: reset	header 1x01, 2.54 mm		J8
1	Breakout J9: audio	header 1x03, 2.54 mm		J9
1	Data: 8 shunts (without U3)	header 2x08, 2.54 mm		JP1
1	PS/2 keyboard			J12
1	PS/2 mouse			J13
1	VERA X 6.1 (2x13 socket)	socket 2x13, 2.54 mm		J1

Switches

Qty	Part	Package	Socket	References
1	Power (press once at power-up)	SW PUSH 6mm		SW1

24 parts on the Vera X glue board, from `board/vera-x/vera-x.kicad_sch`.

<!-- parts:vera-x:end -->

What each chip does

The main board's chips, by what they're for. The architecture explains each part of the board.

Chip	Part	What it does
U1	W65C02S	The CPU
U2	65C22	The VIA: the tick, GPIO, I2C, SPI
U3	65C51	The ACIA: the serial port
U4	74F138	The SPI device select (headers J18-J25)
U5	MAX232	The serial port's RS-232 levels
U6	SST39SF010A	The BIOS ROM: the kernel
U7	HM628512 or DS1747	Task RAM, 32K for each task
U8, U9	74LS260	Address decoding: `$000x` (the bank registers' writes)
U10	74F20	Address decoding
U11	74ACT08	Address decoding; the CPU's clock, gated by DMA
U12	74F86	The system port's decoding
U13, U16	74F32	Address decoding: the windows' selects, qualified by PHI1
U14	74F30	The I/O page, `$FFxx`
U15	74F00	Address decoding
U17, U18	74LS148	The interrupt priority encoders
U19	74LS154	The 16 I/O ports
U20	74F138	The system port: `T`, `U`, `V`, `W` and the vector RAM
U21	74F541	The memory card connector's buffer
U22	74ACT240	Drives IRQB, PHI1, DMA and !RWB
U23	74LS154	The RAM window's module select
U24	74F138	The paged ROM's chip select
U25, U27-U29	HM628512 (SOP-32)	Shared RAM, 2 MB
U26	74F139	The shared RAM's chip select
U30-U37	SST39SF040	The paged ROM, 4 MB
U38	YM2151	FM sound
U39	74ACT14	The crystal oscillator, and the clocks' drivers
U40	74F191	The clock divider
U41	YM3012	The YM2151's DAC
U42	TL074	The DAC's buffers and filters
U43	DM1813	The reset supervisor
U44	74F00	The IRQ to the CPU
U45	74F157	The vector RAM's index: the active line, or `V`
U48, U50, U52, U54	74F573	The latches of `T`, `U`, `V`, `W`
U49, U51, U53, U55	74F541	Their read-back buffers
U56	74F32	The latches' strobes
U57	74F02	The latches' strobes
U58, U60	74F541	The address bus buffers
U59	74ACT245	The data bus transceiver
U61	74F125	The R/W buffers
IC1-IC4	74LS219	The bank registers, `$00` and `$01`
IC5-IC8	74LS219	The vector RAM
IC9	NJM4556A	The audio mixer and headphone driver
J28	SP720	ESD protection for J27 and the SPI lines

Building it

How to go from a bare board to a Hydra-16 at a shell prompt: the tools, the order to solder in, the checks before power, programming the ROMs, the first boot, what POST tells you, the hardware test, and what to do when something's wrong.

Contents

1. What you need
2. Assembly
3. Before the chips go in
4. Programming the ROMs
5. Fitting the chips and jumpers
6. The first boot
7. POST: the power-on self test
8. The hardware test
9. When something's wrong

What you need

- **The parts**: the parts list, and the board (`board/hydra-16.kicad_pcb`, sent to any PCB maker; it's a six-layer board, 304.8 x 243.8 mm, with ground, +5 V and ±12 V planes inside).
- **A soldering iron** with a fine tip, solder, flux, and desoldering braid. Everything is through-hole except the four shared RAM chips, which are SOP-32 (1.27 mm pitch): easy with flux and a fine tip.
- **A multimeter.** A logic probe or an oscilloscope helps a lot if something doesn't work.
- **A flash programmer** that handles the SST39SF040 (DIP-32) and the SST39SF010A (PLCC-32: with a PLCC-32 adapter), such as the XGecu T48 or TL866II+.
- **An ATX power supply**, and a switch, a push button and an LED for the front panel (J10), or a PC case's.
- **A terminal**: a PC with a serial port or a USB serial adapter, a **straight-through** DE-9 cable (female to female), and a terminal program (PuTTY, Tera Term, `screen`, `minicom`).
- **At least one memory card** (the cards): the system runs without one, but HydraOS needs task banks for most programs.
- **An SD card module** for J18 (one with its own 3.3 V regulator and level shifter), for HydraOS's file system.
- **The Vera X**, for a screen, keyboard and mouse: optional; the serial console is all you need.

Assembly

Solder from the flattest parts up, so the board lies flat on the bench while you work. **Don't put any chips in their sockets yet.**

1. **The shared RAM** (U25, U27, U28, U29, SOP-32), first, while nothing else is in the way. Mind pin 1 (the dot or bevel on the chip, the marked pad on the board). Tack two opposite corners, check the alignment, then solder the rest; clean any bridges with braid and flux. Check each pin under a magnifier.
2. **The resistors**: R1-R33 as the list gives them, and R29, a 0 ohm link (a piece of wire does).
3. **The IC sockets**: all the DIP sockets, and the PLCC-32 socket for U6. Line each socket's notch up with the notch on the silkscreen: the socket is your guide when the chips go in. Solder two opposite corners first, check it sits flat, then the rest.
4. **The small capacitors**: the 100 nF decoupling capacitors (104) at every chip, C47 and C48 (33 pF, at the crystal), C50 and C52 (3.3 nF), C49 (4.7 µF), C4 and C7-C10 (1 µF, at the MAX232).
5. **The resistor networks** RN1-RN4: pin 1, the common pin, is marked with a dot on the part and a square pad on the board.
6. **The crystal** Y1, and **U43**, the reset supervisor (TO-92: match the flat side to the silkscreen).
7. **The electrolytics**: C51 and C53-C56 (100 µF), C86-C89 (47 µF), C90 and C91 (10 µF): the long lead, +, to the square pad. C92 and C93 (220 µF) are non-polar and go either way.
8. **The headers**: the clock jumpers J5-J8, J9, J10, J27, J29, J30, and the SPI headers J18-J25 (sockets).
9. **The connectors**: the memory card sockets J1 and J2, the DE-9 (J3), the audio jack (J26), the ATX connector (J11), and last the six slots (J12-J17), which stand tallest. Solder one pin of each slot, check it's square to the board, then the rest.

Then look over the whole board for unsoldered pins, bridges and splashes, and clean off the flux.

Before the chips go in

With **no chips** in their sockets and **no jumpers**:

1. **No shorts.** Measure the resistance from +5 V to GND (a socket's VCC pin to its GND pin, or J10 pin 8 to pin 5): it should be well above a few ohms (the capacitors charge, so it rises). Do the same for +12 V and -12 V (slot pins 17 and 13) to GND. A dead short means a bridge: find it before power.
2. **The power.** Plug in the ATX supply and turn it on by connecting J10 pin 11 to ground (the front panel's switch). Measure:
   - +5 V at every socket's VCC pin (pin 14 of a DIP-14, 16 of a DIP-16, 20 of a DIP-20, 32 of the RAMs and ROMs, 40 of the CPU and VIA, and so on). 4.75 to 5.25 V.
   - +12 V and -12 V at IC9 (pins 8 and 4) and U42 (pins 4 and 11).
   - The power LED on J10 pin 12 lights.
3. Turn it off.

Programming the ROMs

The ROM images are already built, in the repository:

System	BIOS ROM (U6, SST39SF010A)	Paged ROM (SST39SF040)
HydraOS (`reborn/bin/`)	`bios_rockwell.bin` or `bios_wdc.bin`	`prom0.bin` in **U31**, `prom1.bin` in **U32**, `prom2.bin` in **U34**
The base alone (`base/bin/`)	`bios_rockwell.bin` or `bios_wdc.bin`	`prom0.bin` in **U31**

The BIOS ROM is the one image that depends on the serial chip in U3: `bios_rockwell.bin` for a **Rockwell R65C51**, `bios_wdc.bin` for a **WDC W65C51N** (its fault). The paged ROM is the same for both: its serial drivers ask the BIOS which chip it's built for (`BIOS_ACIA`, `$FDFE`). Changing the chip in U3 means reprogramming U6 alone.

- Burn each file whole, from offset 0, and verify it. Each is exactly one chip (128K or 512K).
- The other paged ROM sockets stay empty.
- Mark each chip with its file and socket before it leaves the programmer: the four paged ROMs look alike.
- The images are built for **3.58 MHz** (J7). For 7.16 MHz (J8), build them again with `node build.js --clock 2` (it needs cc65; the emulator and the PC tools say how), for a board with faster parts than the stated ones: with the stated parts, 3.58 MHz is the fastest valid speed (the RAM path doesn't make the cycle, and the YM2151 never could).
- The paged ROM's images already allow for the board's swapped bank halves and crossed bank bits: burn them as they are.
- The BIOS ROM and the paged ROM must come from the **same build**: the modules are built against that kernel's calls, and the hardware test checks every chip against the checksums the build stored. (Each BIOS ROM page has two bytes, at `$FEFE`, that make its checksum 0: so the stored checksums are the same for either BIOS image.)

The PLCC socket. U6's chip has one bevelled corner and a dot over pin 1; the socket has the same bevelled corner. Press the chip in evenly. To take it out, use a PLCC extractor (or two thin hooks in the socket's corner slots), never a screwdriver.

Fitting the chips and jumpers

With the power off:

1. **Every chip** in its socket, pin 1 to the notch: the CPU (U1), the VIA and ACIA (U2, U3), all the logic, U7 (an HM628512 or a DS1747), the ROMs, the sound chips. The board works without the sound chips (U38, U41, U42, IC9) and without J28 (the SP720), if you don't have them yet.
2. **The jumpers**: **J7** (3.58 MHz) and **J30 on 1-2** (the serial port's CTS). Only one of J5-J8, ever.
3. **A memory card** in J1/J2, set to **module 0** (the cards); more cards as modules 1, 2, and so on.
4. **An SD card module** on J18 (device 0), with a card, if you have one.
5. **The serial cable** from J3 to the terminal.

The terminal's settings: 9600 baud, 8 data bits, no parity, 1 stop bit (9600 8N1), hardware flow control (RTS/CTS), and VT100 or xterm emulation.

The first boot

Turn the power on. Within a second the terminal shows the kernel's banner and POST's results. With HydraOS's ROMs, the drivers then start, one task each, and the login shell's prompt comes up:

```
Hydra-16: kernel 0.1, ABI 1
POST ZP:0 ST:0 OS:0 HI:0 SH:S W:0
RAM U:0 F0:0/00/0000 F4:0/00/0000 F8:0/00/0000 FC:0/00/0000 00:0/00/0000 10:0/00/0000 20:0/00/0000
POST ok
RAM modules: 03
task F: cons
task E: storage
task D: kdev
task C: snd
task B: vid
task 1: init
HydraOS 1.0 for the Hydra-16
init: up in task 01
...
HyForth (Forth 2012), bye to end
/>
```

(`task A: vid` starts and ends at once without a Vera X.) With the base's ROMs, Woz's monitor comes up instead:

```
Hydra-16: kernel 0.1, ABI 1
POST ZP:0 ST:0 OS:0 HI:0 SH:S W:0
RAM U:0 F0:0/00/0000 F4:0/00/0000 F8:0/00/0000 FC:0/00/0000 00:0/00/0000 10:0/00/0000
POST ok
RAM modules: 02
task F: ser
task E: kdev
task 1: wozmon

Hydra-16 monitor (Woz's): XXXX examine, XXXX.YYYY a block, XXXX: BB store, XXXXR run;
L instructions, K bytes; Escape the line, Ctrl-C back here.
T1 00>
```

Try it: `ls /rom` lists HydraOS's ROM disk, `ps` the tasks; `FF00.FF0F` in the monitor dumps the VIA's registers. Chapter 1's first hour goes on from here.

If nothing appears, or something's wrong, read on.

POST: the power-on self test

POST runs first at every reset, before anything else: in task 0, with interrupts off, and with its own polled output, so it works when little else does. It checks what the task system depends on, then the paged RAM's lines. All zeros is good.

The first line: the task system.

Field	Good	Checks
`ZP:`	`0`	Page zero (`$00FF`) is each task's own. Tasks 8, 4, 2, 1 and 0 each write their number there, then read it back. The value is a mask of the `T` lines (bit n = Tn) that don't keep them apart
`ST:`	`0`	The same for the stack page (`$0100`)
`OS:`	`0`	The same at `$03FF`
`HI:`	`0`	The same at `$7FF0`, the top of task RAM
`SH:`	`S`	Shared bank `$F0` is **S**hared between tasks 0 and 1 (`X`: it isn't: the bank registers or the shared RAM are at fault)

Field	Good	Checks
`W:`	`0`	A mask of the `W` lines that don't select their BIOS ROM page (each page has its number at `$FDFF`)

A bad `T` line (the same bits in `ZP`, `ST`, `OS` and `HI`) is U48 (the `T` latch), U21 (its buffer to the memory cards) or U7's A15-A18 (pins 31, 2, 30, 1).

The second line: the paged RAM. Each field is a hex mask of **bad** lines (bit n set = line n bad):

- `U:x`: the `U` register's lines U0-U3, tested on shared bank `$F0`.
- `bb:x/dd/aaaa`: bank `bb`, tested at `$8000-$9FFF`:
  - `x`: the bank register's lines 0-3 (writes to bank `bb` XOR 1, 2, 4 and 8 mustn't land in `bb`);
  - `dd`: the data lines D0-D7 (a walking one);
  - `aaaa`: the address lines A0-A12 (`$8000 + 2^n` for each line n).

The banks tested are the first bank of each shared RAM chip (`F0`, `F4`, `F8`, `FC`, with U = 0), then the first bank of each memory card (`00`, `10`, `20`, ...). A missing chip shows as bad lines. The test overwrites paged RAM, which doesn't matter at reset.

The third line is `POST ok`, or `POST found faults:` and what they are: `T lines`, `shared RAM`, `W lines`, `RAM (left unused)`. The system starts anyway, as far as it can:

- A bad memory card is treated as not there, and a bad shared RAM chip's banks are never handed out.
- A bad `T` or `W` line is beyond help: tasks or BIOS pages land on each other, and the system may not start.

Finding the chip and pin. The shared RAM chips by bank: `F0` is U25, `F4` is U28, `F8` is U27, `FC` is U29 (the crossed bank bits swap the middle two: the quirks). The HM628512's pins are in the architecture. For example:

- `F0:0/00/0001` is A0 (pin 12) on U25;
- `F8:0/04/0000` is D2 (pin 15) on U27;
- `10:0/00/0400` is A10 (pin 23) on memory card 1.

The line after POST, `RAM modules: 03`, is how many memory cards were found (in hex) and are good.

For a closer look at any fault, run the hardware test.

The hardware test

A test of as much of the board as software can reach, for bringing a board up or chasing a fault. It's a program of its own in paged ROM bank 1, the same in both systems' ROMs. It takes the machine over: interrupts off, its own polled serial I/O at 9600 8N1 (it finds out for itself whether the serial chip is a Rockwell R65C51 or a WDC W65C51N, so the one image serves both, whatever the system was built for), no calls into the system. Its memory tests overwrite everything, so it ends with a reset.

Starting it:

- **From POST**: hold down `T` as you press reset (or type it while POST runs). This works even when the system can't start: a bad shared RAM chip, an IRQ line held active.
- **From HydraOS**: `hwtest` at the shell.

The menu. Type a test's key to run that test, or:

Key	Does
`A`	All the tests, quick: every bank's marks, and every byte of a sample of the memory (about 15 seconds)
`F`	All the tests, full: every byte of every bank, in every task (about 3 minutes at 3.58 MHz with three memory cards)
`L`	All the quick tests, again and again until a key: a count of passes and failures after each (for an intermittent fault)
`R`	Reset: the machine starts again, through POST

Each test prints its name, what it found, then `ok` or `FAIL` and the fault. A run ends with `hwtest: all passed` or `hwtest: failed: N`:

```
CPU ................ ok
RAM modules ........ 0 1 2 ok
shared RAM ......... FAIL bank 04 8000 bits 08
CPU clock .......... 3.58 MHz ok
SPI devices ........ SD cards 0 ok
hwtest: failed: 1
```

The tests, in the order `A` runs them. The memory tests write a pattern that depends on the address, so a stuck or crossed address line shows up as well as a bad data line. "Bits" are hex masks of the bad bits.

Key	Test	Checks	A fault shows
`1`	CPU	A sample of the W65C02S's instructions and flags: binary and decimal arithmetic, and the 65C02's own (`STZ`, `TSB`/`TRB`, `BRA`, `PHX`, `RMB`/`SMB`, `BBR`/`BBS`, `(zp)`)	`check N`
`2`	T U V W registers	Each stores and reads back all 8 bits	the register and its bad bits
`3`	shared RAM	Each of the 256 banks (`U` 0-F, IDs `$F0-$FF`) has its own mark; every byte of the first bank of each chip (full: of every bank)	Address faults: each chip and the lines it doesn't see, with pins, e.g. `U29: A18 (pin 1)`. The same A13-A16 line on all four chips is the `U` register's line itself (test 2). `no mark`: a data fault or no chip. Otherwise a byte fault: `bank UB`, where, the bad bits
`4`	RAM bank registers	Each task's `$00` (IC1, IC2): set to each bank and read at once; then all 16 tasks set to their own banks first and read afterwards; then lines 4-7	`at once: task F: D>F` (set to `$FD`, it read bank `$FF`'s mark: a cell that doesn't hold its value); `all set, then read: ...` (right at once, wrong later: writing one register disturbs another). `skipping banks ...` is information: task 0 alone reads those wrong, so the shared RAM is at fault (test 3)
`5`	task RAM	Each task's RAM is its own (a mark in each), then every task's page zero, stack page and `$0200-$7FFF`	the task, where, the bad bits; or `task F has task B's mark: U7 A17 (pin 30)`
`6`	RAM modules	Which memory cards there are; each bank of each has its mark in each task; every byte of each card's first bank (full: every bank, in every task)	the bank, the task, where, the bad bits
`7`	BIOS ROM	A CRC of each 8K page against the build's checksums	`page N`, and `(W line n)` if it reads as another page
`8`	paged ROM	A CRC of each 16K bank in use, and the bank lines	`bank NN`, or `bank line n`
`I`	interrupts	No IRQ line active with the devices quiet; the vector RAM (all 16 entries, two patterns); `BRK` through entry `V`; the VIA's, the ACIA's (a Rockwell's: a WDC chip's transmitter can't interrupt) and the YM2151's interrupts on their lines, one at a time, then all at once in priority order	`an IRQ line is held active`; `vector entry N wrote .. read ..`; the device and `no IRQ` or the line it came on; `unasked: IRQ on line N`
`V`	VIA	Timer 1's latches, the shift register and IER read back; the timers counting and their flags	the part
`Y`	sound chip	The YM2151's busy flag (set by a write, then clear), and its timers' flags	`always busy (no chip?)`, `never busy`, `timer A`, ...
`K`	CPU clock	The CPU's clock against the YM2151's (`SND_CLK`): shows 0.89, 1.79, 3.58 or 7.16 MHz, which must be what the ROM is built for	the clock and what the ROM expects, or `not a clock the board has`
`S`	serial port	DCD and DSR active; the control and command registers; a programmed reset; a character's time at 9600 baud (on a Rockwell chip; a WDC chip, whose TDRE never clears, is named instead)	the register, or the character's time in cycles
`P`	SPI devices	Information: each of devices 0-7 is sent an SD card's reset (CMD0); the SD cards that answer are shown	
`C`	I2C bus	SCL and SDA high when released and each low alone when pulled low; then information: the devices that answer, at `$08-$77`	the lines' levels
`X`	slot cards	Information: the slot ports (`1A` = slot 1, select A) that don't read as empty	
`H`	hold a task (probe)	Not a test, and not in `A`, `F` or `L`: every task's RAM bank register set to `$F0` + the task, then the task you type (0-F) held in `T`, its bank read over and over until a key. Meanwhile, measure T0-T3 (IC1-IC4 pins 1, 15, 14, 13) and `RAMB0-7` (IC1 and IC2 pins 5, 7, 9, 11) with a probe	

The CPU clock and serial tests time things with VIA timer 1, so a bad VIA shows up there too: run the tests in order when chasing a fault.

When something's wrong

Work from the bottom up: power, then reset and the clock, then the CPU fetching from the BIOS ROM, then the serial port. Every step can be checked with a multimeter or a logic probe.

Nothing on the terminal at all:

1. **Power**: +5 V at U1 pin 8 (VDD) and pin 21 (VSS, 0 V).
2. **Reset**: U1 pin 40 (RESB) low for a moment at power-up, then high. If it stays low, U43 or the front panel's button (J10 pin 7) is holding it.
3. **The clock**: a square wave at U1 pin 37 (PHI2 in), 3.58 MHz. None: check J7 is fitted (one jumper only), Y1 and U39 (pin 2 oscillates), U40, R22-R27.
4. **RDY** (U1 pin 2) high, and **BE** (pin 36) high.
5. **The CPU running**: SYNC (U1 pin 7) pulses. If it doesn't, or the address lines don't change, the CPU isn't fetching: check the BIOS ROM (U6's orientation, that it's programmed, its CE at pin 22 pulsing), the address and data buffers (U58-U60), and the decoding.
6. **The serial port**: the terminal's settings (9600 8N1), the cable (straight-through, not null-modem), **J30 on 1-2** with a cable that carries pin 7 (or 2-3 for a three-wire cable), the MAX232's capacitors (C4, C7-C10: check their polarity, if they're electrolytics) and its ±8 V or so at pins 2 and 6. Probe U3 pin 10 (TxD) for activity.

Garbled characters: the terminal's rate isn't 9600, or the CPU clock isn't the ROM's (J7 for the standard images). At other rates than 9600, set both ends.

POST reports faults: POST says which chip and pin; reseat the chip, check its socket's solder joints, swap it with a known-good one of the same kind. An SOP shared RAM chip: reflow its pins.

It stops after POST, or a driver doesn't start: run the hardware test (`T` at reset), `A` for all the tests. An IRQ line held active (test `I`) stops the system as soon as interrupts go on: it's usually a device that isn't reset or a slot card. Take the cards out and try again.

No sound: the YM2151's test (`Y`) passes but nothing comes out: check U41, U42, IC9 (±12 V at their supply pins), and the jack's wiring. `Y` says `always busy`: U38 isn't there or isn't seen (its CS from port 4).

No SD card (`ls /sd/0` fails): the hardware test's `P` shows which devices answer. Check the module's power (+5 V on its header) and that it has a 3.3 V regulator and level shifter, and that the card is formatted (storage says how to make a card).

Intermittent faults: the hardware test's `L` runs the tests until a key, with a count of passes and failures. Warm the board (a hair dryer) or tap chips while it runs, to find a bad joint.

The board layout

Where everything is on each board. The drawings and the tables are made from the PCB files by `node docs/build/layout.js`: don't edit them by hand.

Reading the drawings

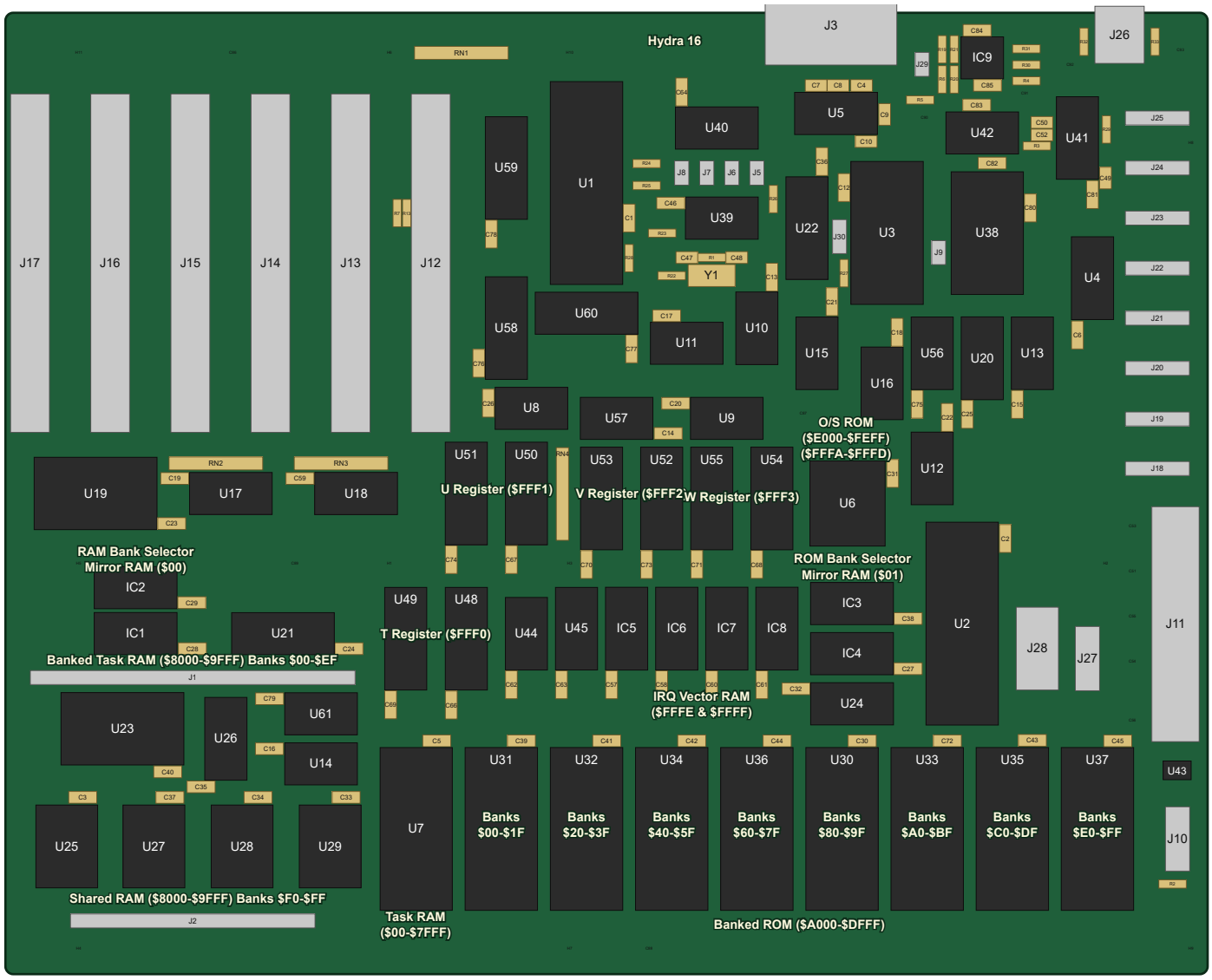
- **The board** is drawn from its outline (the PCB's Edge.Cuts layer), seen from the front (the component side), with its top left corner at 0, 0.
- **Each part** is the box its footprint's courtyard takes up, labelled with its reference: **ICs** dark grey, **connectors** light grey, **passive parts** (resistors, capacitors, the crystal) tan, anything else blue. A part on the back of the board is drawn faint, with a dashed edge.
- **The labels** in pale yellow are the board's own silkscreen: the names of its areas ("Shared RAM", "T Register", the paged ROM's banks).
- **In the tables**, a part's place is its centre in millimetres from the top left corner (x across, y down), its side, and how far it's turned. To find a part on the board, look it up in the table, then measure, or find it in the drawing: the references are there too, and a browser's search finds them in the SVG.

The drawings are SVG files beside this page; open one on its own to zoom in.

Contents

1. The main board
2. The connectors and jumpers
3. The memory daughter card, surface mount
4. The bus breakout card

The main board



The board's areas:

- **The top edge** is the back panel: the serial port (J3) and the audio jack (J26) stand over it. Below them, the serial chips (U3, U5), the sound chips and the mixer (U38, U41, U42, IC9).
- **The left side** is the six slots, slot 5 (J17) at the edge and slot 0 (J12), the Vera X's, innermost.
- **The top middle** is the CPU (U1), its buffers (U58-U60), the clocks (Y1, U39, U40, with the clock jumpers J5-J8 in a row) and the decoding.
- **The right edge** is the SPI headers (J18 at the bottom, up to J25), the VIA (U2) and its header J27, the ATX connector (J11), the reset supervisor (U43) and the front panel (J10).
- **The middle** is the pseudo-registers (each labelled: `T`, `U`, `V`, `W`), the bank registers (IC1-IC4) and the interrupt logic with the vector RAM (U17, U18, U45, IC5-IC8), with the BIOS ROM (U6, "O/S ROM").
- **The bottom** is the memory: the shared RAM (U25, U27-U29) and the memory card connectors (J1, J2) at the left, the task RAM (U7), and the eight paged ROMs, each labelled with its banks.

<!-- layout:main:begin -->

The board is 304.8 mm wide and 243.8 mm high; a part's place is its centre, in millimetres from the board's top left corner (x across, y down), seen from the front.

Part	Value	x	y	Side	Turned
C1	104 (100 nF)	158.1	49.6	front	270°

Part	Value	x	y	Side	Turned
C2	104 (100 nF)	253.5	130.8	front	270°
C3	104 (100 nF)	22.1	199.0	front	180°
C4	105 (1 µF)	218.3	18.4	front	180°
C5	104 (100 nF)	111.8	184.7	front	180°
C6	104 (100 nF)	271.8	79.2	front	270°
C7	105 (1 µF)	204.3	18.4	front	
C8	105 (1 µF)	212.4	18.4	front	180°
C9	105 (1 µF)	222.9	24.5	front	270°
C10	105 (1 µF)	219.5	32.6	front	180°
C12	104 (100 nF)	212.6	41.8	front	270°
C13	104 (100 nF)	194.3	64.6	front	270°
C14	104 (100 nF)	170.5	106.7	front	180°
C15	104 (100 nF)	256.5	101.8	front	90°
C16	104 (100 nF)	64.4	186.7	front	
C17	104 (100 nF)	170.1	76.8	front	180°
C18	104 (100 nF)	226.1	78.5	front	270°
C19	104 (100 nF)	45.4	118.1	front	180°
C20	104 (100 nF)	167.4	99.1	front	
C21	104 (100 nF)	209.6	70.8	front	270°
C22	104 (100 nF)	238.8	100.1	front	270°
C23	104 (100 nF)	44.7	129.5	front	180°
C24	104 (100 nF)	84.6	161.3	front	
C25	104 (100 nF)	243.8	99.4	front	270°
C26	104 (100 nF)	122.4	101.5	front	90°
C27	104 (100 nF)	226.4	166.4	front	
C28	104 (100 nF)	44.9	161.3	front	
C29	104 (100 nF)	44.7	149.6	front	
C30	104 (100 nF)	219.7	184.7	front	180°
C31	104 (100 nF)	224.9	119.3	front	90°
C32	104 (100 nF)	202.9	171.5	front	180°
C33	104 (100 nF)	88.9	199.0	front	180°
C34	104 (100 nF)	66.5	199.0	front	180°
C35	104 (100 nF)	52.1	196.3	front	180°
C36	104 (100 nF)	207.0	40.2	front	90°
C37	104 (100 nF)	44.2	199.0	front	180°
C38	104 (100 nF)	226.3	153.7	front	
C39	104 (100 nF)	128.3	184.7	front	
C40	104 (100 nF)	38.6	192.5	front	
C41	104 (100 nF)	149.9	184.7	front	
C42	104 (100 nF)	171.5	184.7	front	
C43	104 (100 nF)	257.9	184.5	front	
C44	104 (100 nF)	193.1	184.7	front	
C45	104 (100 nF)	279.5	184.7	front	
C46	104 (100 nF)	171.2	48.3	front	180°
C47	330 (33 pF)	174.0	62.1	front	180°
C48	330 (33 pF)	184.2	62.1	front	
C49	475 (4.7 µF)	278.9	40.7	front	270°
C50	332 (3.3 nF)	264.0	27.8	front	180°

Part	Value	x	y	Side	Turned
C51	107 (100 µF)	285.8	144.7	front	90°
C52	332 (3.3 nF)	264.0	31.0	front	180°
C53	107 (100 µF)	285.8	133.3	front	90°
C54	107 (100 µF)	285.7	167.6	front	90°
C55	107 (100 µF)	285.7	156.2	front	90°
C56	107 (100 µF)	285.7	176.5	front	270°
C57	104 (100 nF)	153.7	167.9	front	270°
C58	104 (100 nF)	166.4	167.9	front	270°
C59	104 (100 nF)	77.2	118.1	front	180°
C60	104 (100 nF)	179.1	167.9	front	270°
C61	104 (100 nF)	191.8	167.9	front	270°
C62	104 (100 nF)	128.3	167.9	front	270°
C63	104 (100 nF)	141.0	167.9	front	270°
C64	104 (100 nF)	171.5	22.6	front	90°
C66	104 (100 nF)	113.0	173.1	front	270°
C67	104 (100 nF)	128.3	136.2	front	270°
C68	104 (100 nF)	190.5	137.4	front	270°
C69	104 (100 nF)	97.7	178.0	front	90°
C70	104 (100 nF)	147.3	142.5	front	90°
C71	104 (100 nF)	175.3	142.4	front	90°
C72	104 (100 nF)	241.3	184.7	front	180°
C73	104 (100 nF)	162.6	142.5	front	90°
C74	104 (100 nF)	113.0	141.2	front	90°
C75	104 (100 nF)	231.1	101.8	front	90°
C76	104 (100 nF)	120.0	91.4	front	90°
C77	104 (100 nF)	158.8	82.7	front	270°
C78	104 (100 nF)	123.2	58.6	front	90°
C79	104 (100 nF)	69.5	174.0	front	180°
C80	104 (100 nF)	259.8	47.0	front	270°
C81	104 (100 nF)	275.6	43.6	front	270°
C82	104 (100 nF)	247.7	38.1	front	
C83	104 (100 nF)	243.8	23.4	front	
C84	104 (100 nF)	243.8	4.4	front	
C85	104 (100 nF)	246.5	18.4	front	
C86	476 (47 µF)	54.7	10.2	front	
C87	476 (47 µF)	199.4	101.7	front	
C88	476 (47 µF)	160.2	237.4	front	
C89	476 (47 µF)	70.4	139.7	front	
C90	106 (10 µF)	235.6	26.7	front	180°
C91	106 (10 µF)	258.4	23.2	front	90°
C92	227 NP (220 µF)	266.1	13.2	front	
C93	227 NP (220 µF)	297.9	13.4	front	90°
H1	MountingHole_Pad	95.2	139.7	front	
H2	MountingHole	276.8	139.7	front	
H3	MountingHole_Pad	141.0	139.7	front	
H4	MountingHole_Pad	16.5	237.5	front	
H5	MountingHole_Pad	16.5	139.7	front	
H6	MountingHole_Pad	95.2	10.2	front	

Part	Value	x	y	Side	Turned
H7	MountingHole_Pad	141.0	237.5	front	
H8	MountingHole_Pad	298.4	33.0	front	
H9	MountingHole_Pad	298.4	237.5	front	
H10	MountingHole_Pad	141.0	10.2	front	
H11	MountingHole_Pad	16.5	10.2	front	
IC1	74LS219	41.9	153.7	front	270°
IC2	74LS219	41.9	142.0	front	270°
IC3	74LS219	223.5	146.0	front	270°
IC4	74LS219	223.5	158.8	front	270°
IC5	74LS219	161.3	165.1	front	180°
IC6	74LS219	174.0	165.1	front	180°
IC7	74LS219	186.7	165.1	front	180°
IC8	74LS219	199.4	165.1	front	180°
IC9	NJM4556AD	243.8	15.2	front	90°
J1	Conn_01x32	86.9	168.6	front	270°
J2	Conn_01x24	76.7	230.4	front	270°
J3	DSUB_9	204.0	11.1	front	180°
J5	0.89489MHz	190.5	41.9	front	90°
J6	1.78977MHz	184.2	41.9	front	90°
J7	3.57955MHz	177.8	41.9	front	90°
J8	7.15909MHz	171.4	41.9	front	90°
J9	SND CT1/2	236.6	59.6	front	270°
J10	Front Panel	295.9	203.2	front	
J11	ATX-24	293.4	178.2	front	90°
J12	SLOT 0	107.8	63.5	front	270°
J13	SLOT 1	87.5	63.5	front	270°
J14	SLOT 2	67.2	63.5	front	270°
J15	SLOT 3	46.9	63.5	front	270°
J16	SLOT 4	26.6	63.5	front	270°
J17	SLOT 5	6.2	63.5	front	270°
J18	SPI Device 0	285.7	115.6	front	90°
J19	SPI Device 1	285.7	103.0	front	90°
J20	SPI Device 2	285.7	90.2	front	90°
J21	SPI Device 3	285.7	77.4	front	90°
J22	SPI Device 4	285.7	64.7	front	90°
J23	SPI Device 5	285.7	52.0	front	90°
J24	SPI Device 6	285.7	39.3	front	90°
J25	SPI Device 7	285.7	26.6	front	90°
J26	Audio Jack	282.4	4.9	front	180°
J27	VIA Port A	275.6	170.2	front	180°
J28	SP720APP	265.4	170.2	front	180°
J29	Audio In	232.4	14.4	front	90°
J30	CTS	211.5	54.2	front	
R1	105 (1M)	176.5	62.1	front	
R2	471 (470 Ω)	298.4	221.0	front	180°
R3	101 (100 Ω)	264.0	33.8	front	180°
R4	103 (10K)	261.4	17.3	front	180°
R5	103 (10K)	229.4	22.1	front	

Part	Value	x	y	Side	Turned
R6	103 (10K)	237.5	14.4	front	270°
R7	103 (10K)	99.3	53.3	front	90°
R13	103 (10K)	101.7	53.3	front	90°
R19	103 (10K)	237.5	11.8	front	90°
R20	103 (10K)	240.5	14.4	front	270°
R21	103 (10K)	240.5	6.7	front	270°
R22	331 (330 Ω)	166.4	66.7	front	
R23	220 (22 Ω)	169.0	55.9	front	180°
R24	220 (22 Ω)	165.1	38.2	front	180°
R25	220 (22 Ω)	165.1	43.8	front	180°
R26	220 (22 Ω)	194.7	44.7	front	270°
R27	220 (22 Ω)	212.6	63.5	front	270°
R28	103 (10K)	158.1	59.7	front	270°
R29	0 (a link)	279.1	27.1	front	270°
R30	330 (33 Ω)	256.3	13.2	front	
R31	330 (33 Ω)	256.3	9.1	front	
R32	103 (10K)	273.4	9.9	front	90°
R33	103 (10K)	291.5	9.9	front	90°
RN1	332 (3.3K)	105.4	10.2	front	
RN2	332 (3.3K)	43.2	114.3	front	
RN3	332 (3.3K)	74.9	114.3	front	
RN4	103 (10K)	141.2	132.2	front	90°
U1	WDC65C02S	154.9	67.3	front	180°
U2	R65C22-P2	250.2	179.1	front	180°
U3	R65C51-P2	215.9	39.3	front	
U4	74F138	279.4	76.2	front	180°
U5	MAX232	219.5	21.7	front	270°
U6	SST39SF0x0A	214.8	119.4	front	
U7	HM628512LP-5	96.5	188.0	front	
U8	74LS260	125.7	104.1	front	90°
U9	74LS260	175.3	106.7	front	90°
U10	74F20	186.7	72.4	front	
U11	74ACT08	165.1	87.6	front	90°
U12	74F86	231.1	108.0	front	
U13	74F32	264.2	94.0	front	180°
U14	74F30	72.4	194.3	front	90°
U15	74F00	201.9	78.7	front	
U16	74F32	218.4	86.4	front	
U17	74LS148	48.3	125.7	front	90°
U18	74LS148	80.0	125.7	front	90°
U19	74LS154	36.8	114.3	front	270°
U20	74F138	251.4	96.5	front	180°
U21	74F541	81.8	153.7	front	270°
U22	74ACT240	199.4	43.2	front	
U23	74LS154	43.6	174.0	front	270°
U24	74F138	205.7	179.1	front	90°
U25	HM628512LP-5	15.5	211.5	front	
U26	74F139	59.7	193.0	front	180°

Part	Value	x	y	Side	Turned
U27	HM628512LP-5	37.6	211.5	front	
U28	HM628512LP-5	60.0	211.5	front	
U29	HM628512LP-5	82.3	211.5	front	
U30	SST39SF040	204.5	188.0	front	
U31	SST39SF040	118.1	188.0	front	
U32	SST39SF040	139.7	188.0	front	
U33	SST39SF040	226.1	188.0	front	
U34	SST39SF040	161.3	188.0	front	
U35	SST39SF040	247.7	188.0	front	
U36	SST39SF040	182.9	188.0	front	
U37	SST39SF040	269.2	188.0	front	
U38	YM2151	241.3	41.9	front	
U39	74ACT14	174.0	55.9	front	90°
U40	74F191	171.4	33.0	front	90°
U41	YM3012	275.6	40.6	front	180°
U42	TL074	240.0	34.3	front	90°
U43	DM1813-10	294.5	192.5	front	
U44	74F00	135.9	165.1	front	180°
U45	74F157	148.6	165.1	front	180°
U48	74F573	120.7	170.2	front	180°
U49	74F541	105.3	170.2	front	180°
U50	74F573	135.9	133.3	front	180°
U51	74F541	120.7	133.3	front	180°
U52	74F573	170.2	134.6	front	180°
U53	74F541	154.9	134.6	front	180°
U54	74F573	198.1	134.6	front	180°
U55	74F541	182.9	134.6	front	180°
U56	74F32	238.8	94.0	front	180°
U57	74F02	162.6	99.1	front	270°
U58	74F541	130.8	91.4	front	180°
U59	74ACT245	130.8	50.8	front	180°
U60	74F541	158.8	72.4	front	270°
U61	74F125	72.4	181.6	front	90°
Y1	14.31818MHz	176.6	66.6	front	

<!-- layout:main:end -->

The connectors and jumpers

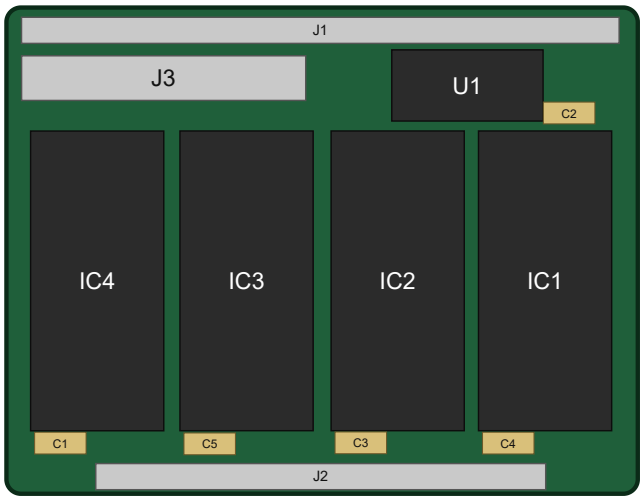
What each connector on the main board is for, and where its pins are described. Pin 1 of each header is marked on the silkscreen with a square pad.

Ref	What	Its pins
J1, J2	The memory card connectors (1 x 32, 1 x 24 sockets)	The cards
J3	The serial port, a DE-9 male	The devices
J5-J8	The CPU clock jumpers: 0.89, 1.79, **3.58** (fit) MHz; J8's 7.16 MHz is beyond the stated parts	The architecture
J9	The YM2151's CT1 and CT2 outputs	The devices
J10	The front panel: reset, power switch, power LED	The architecture
J11	ATX-24 power	The ATX standard's

Ref	What	Its pins
J12-J17	Slots 0-5	The devices
J18-J25	SPI devices 0-7 (an SD card module on J18)	The devices
J26	Audio out, a 3.5 mm stereo jack	The devices
J27	The VIA's port A: GPIO, I2C, CA1, CA2	The devices
J28	The SP720 ESD array (a chip, in a socket)	The devices
J29	The mixer's line input: pin 1 left, pin 2 right	The devices
J30	The ACIA's CTS: **1-2** (fit) or 2-3	The devices

The jumpers to fit on a new board: **J7** (3.58 MHz) and **J30 1-2**. Nothing else.

The memory daughter card



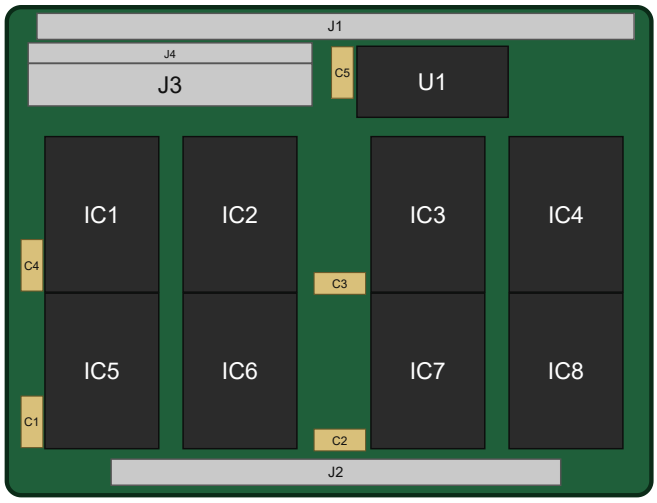
<!-- layout:memory:begin -->

The board is 87.0 mm wide and 67.0 mm high; a part's place is its centre, in millimetres from the board's top left corner (x across, y down), seen from the front.

Part	Value	x	y	Side	Turned
C1	.1uF	4.9	60.0	front	
C2	.1uF	75.0	14.5	front	
C3	.1uF	46.3	60.0	front	
C4	.1uF	66.7	60.0	front	
C5	.1uF	25.5	60.1	front	
IC1	HM628512LP-7	81.9	56.7	front	180°
IC2	HM628512LP-7	61.5	56.7	front	180°
IC3	HM628512LP-7	40.7	56.7	front	180°
IC4	HM628512LP-7	20.1	56.7	front	180°
J1	Conn_01x32	82.6	3.1	front	270°
J2	Conn_01x24	72.5	64.6	front	270°
J3	E D C B A 9 8 7 6 5 4 3 2 1 0	39.4	8.5	front	270°
U1	74HC139	72.4	6.9	front	270°

<!-- layout:memory:end -->

The memory daughter card, surface mount



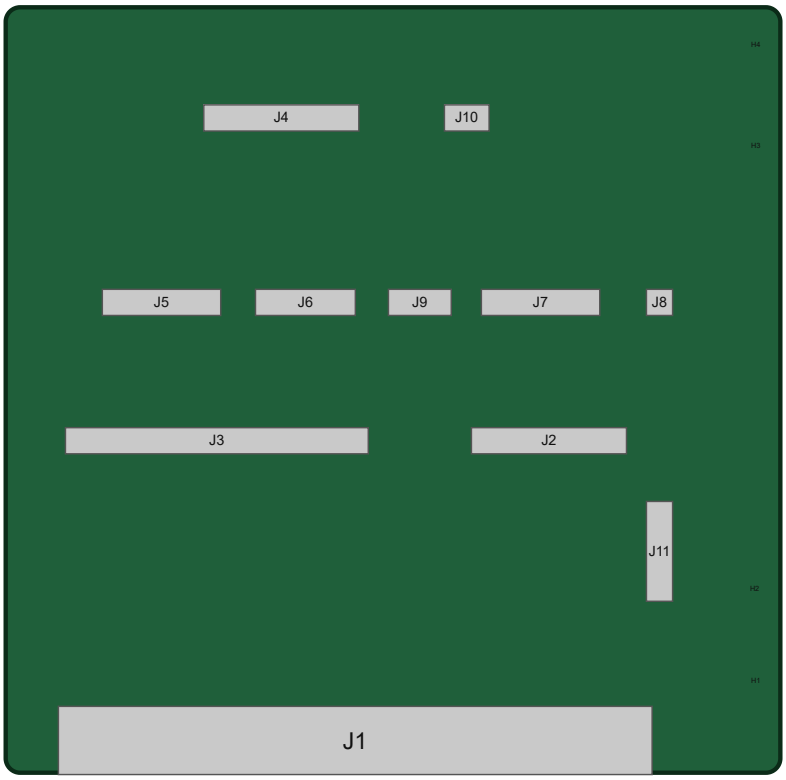
<!-- layout:memory-smd:begin -->

The board is 88.9 mm wide and 67.3 mm high; a part's place is its centre, in millimetres from the board's top left corner (x across, y down), seen from the front.

Part	Value	x	y	Side	Turned
C1	.1uF	3.5	59.7	front	90°
C2	.1uF	48.4	59.7	front	180°
C3	.1uF	48.4	38.1	front	180°
C4	.1uF	3.5	38.1	front	90°
C5	.1uF	46.3	6.5	front	270°
IC1	HM628512LFP-7	13.1	28.6	front	180°
IC2	HM628512LFP-7	32.2	28.6	front	180°
IC3	HM628512LFP-7	58.1	28.6	front	180°
IC4	HM628512LFP-7	77.1	28.6	front	180°
IC5	HM628512LFP-7	13.1	50.2	front	180°
IC6	HM628512LFP-7	32.2	50.2	front	180°
IC7	HM628512LFP-7	58.1	50.2	front	180°
IC8	HM628512LFP-7	77.1	50.2	front	180°
J1	Conn_01x32	84.7	2.7	front	270°
J2	Conn_01x24	74.6	64.1	front	270°
J3	Conn_02x15_Odd_Even	40.3	9.3	front	270°
J4	Conn_01x15_Pin	40.3	6.7	front	270°
U1	74LS139	49.8	14.1	front	90°

<!-- layout:memory-smd:end -->

The bus breakout card



<!-- layout:breakout:begin -->

The board is 106.7 mm wide and 105.4 mm high; a part's place is its centre, in millimetres from the board's top left corner (x across, y down), seen from the front.

Part	Value	x	y	Side	Turned
H1	MountingHole_3.2mm_M3	101.6	92.7	front	
H2	MountingHole_3.2mm_M3_ISO7380_Pad_TopBottom	101.6	80.0	front	
H3	MountingHole_3.2mm_M3	101.6	19.1	front	
H4	MountingHole_3.2mm_M3	101.6	5.1	front	
J1	Bus_Hydra_8bit	46.8	102.1	front	
J2	Data Bus	83.7	59.7	front	270°
J3	Address Bus	48.1	59.7	front	270°
J4	SPI	29.1	15.2	front	90°
J5	Interrupts & I/O	27.8	40.6	front	270°
J6	Clocks	46.4	40.6	front	270°
J7	Bus Control	80.0	40.6	front	270°
J8	Reset	90.0	40.6	front	180°
J9	Audio	59.6	40.6	front	270°
J10	I2C	64.8	15.2	front	270°
J11	Power	90.0	69.8	front	

<!-- layout:breakout:end -->

The Vera X glue board has no PCB: it's built on a protoboard (the cards).

The quirks

What the board does that you wouldn't expect. None of it matters to a program that goes through the system's calls: the kernel and the build allow for all of it. It matters when you write your own kernel, trace a fault to a chip, or design a card.

Contents

1. The crossed bank bits
2. The paged ROM's swapped halves
3. Registers that power up random
4. Write-only bank registers
5. Changing W changes the code
6. Interrupts: XOR 7, and no masks
7. No wait states
8. The serial port's clock
9. The YM2151's timer B
10. The clock chip in task F's RAM
11. Only slot 0's audio is mixed
12. The power rails
13. The schematic's own quirks
14. What changed from 1.8C to 1.8D

The crossed bank bits

On the board as built, one pair of bank bits trades places between the bank registers and the memory, in **both** bank registers (`$00` for RAM, `$01` for ROM): **bits 2 and 3**. Data bit 2 drives `RAMB3/ROMB3`, and bit 3 drives `RAMB2/ROMB2` (IC1 and IC3, the registers' low nibbles). Bits 4 to 7 reach the memory straight: on the drawing the high nibbles' chips (IC2 and IC4) have data bits 7 and 6 crossed into their inputs *and* their outputs labelled `RAMB6/RAMB7` the other way round, two crossings that cancel. So a bank number's bit 5 is chip-select bit 0, bit 6 is bit 1 and bit 7 is bit 2, and chip k in the decoder's order holds banks 32k to 32k + 31. (Bits 6 and 7 were thought crossed too, until 2026-10-13, when the first ROM to put files in a third chip read an empty socket on the board: bank `$40` reaches U34, bank `$80` nothing.)

So the bank a program selects as `b` is the hardware's bank `swap(b)`, with bits 2 and 3 exchanged. It's always the same memory for the same `b`, so **the software never needs to care**, with two exceptions:

- **Tracing a fault to a chip.** Shared bank IDs `$F4-$F7` are on **U28**, and `$F8-$FB` on **U27** (not the other way round). IDs whose bits 2 and 3 are equal (`$F0-$F3`, `$FC-$FF`) aren't affected, and a memory card answers the module number it's jumpered as (bits 4 to 7 are straight).
- **Building a ROM image.** Paged ROM bank `b` sits at bank `swap(b)` in the chips: within a chip, banks 4-7 and 8-11 change places. The build (`base/tools/romimg.js`) writes the chips' view, and the emulator (`base/sim/lib/machine.js`) reads it the same way, so a `promN.bin` is burned as it is, and chip k (U31, U32, U34 ... in the architecture's table) holds banks 32k to 32k + 31.

The schematic shows the board as built; don't "fix" it, or every ROM image would have to change.

The paged ROM's swapped halves

The paged ROM's window, `$A000-$DFFF`, is 16K: CPU A13 picks the half. But in the window, `$A000-$BFFF` has A13 = 1 and `$C000-$DFFF` has A13 = 0, and A13 goes to the chips unchanged. So **CPU `$A000` reads the chip's offset `$2000` in the bank, and CPU `$C000` its offset `$0000`**: each bank's two halves are swapped in the chip.

The build writes each bank's halves swapped, so the CPU sees them in order. Anything else that writes a paged ROM image must do the same.

Registers that power up random

`T`, `U`, `V` and `W` are 74F573 latches, and the bank registers and the vector RAM are 74LS219 RAMs: none has a reset. They power up random, and **pressing reset doesn't change them**. So:

- The CPU may start with any BIOS page in place. That's why **every BIOS page starts with the reset code**, and every page's RESET vector points to it: it sets `W` to 0 first.
- The boot code sets `T`, `U` and `V`, every task's bank registers, and all 16 vectors, before it turns interrupts on.
- After a reset, RAM keeps what it had (it isn't cleared), which the hardware test and the debugger can use.

Write-only bank registers

The bank registers `$00` and `$01` can't be read: a read of `$0000` or `$0001` comes from task RAM. But a write goes to both, so task RAM holds the same value, and a program reads its own bank selections back from there. Only a DMA card writing `$0000` or `$0001` could make the two differ.

Changing W changes the code

`W` chooses the BIOS ROM's page at `$E000-$FFFF`, where the code that changes `W` is running. After `STA W_REGISTER` the next instruction comes from the new page, at the same address. So code that changes `W` must be at the same address, with the same bytes, on every page: the kernel's COMMON block at `$FD00` does all its page changes (the kernel).

Interrupts: XOR 7, and no masks

- **The vector RAM's index is the line number XOR 7**: line 0's handler is in entry 7, line 7's in entry 0, line 8's in entry 15. It's how the 74LS148s encode, and `hw.inc`'s `IRQ_INDEX(line)` does the arithmetic.
- **There's no mask register.** Each line is wired straight to the encoders, so a device that holds its line low keeps interrupting until its handler quiets it. A driver's handler must always clear its device's interrupt, and a device that can't be quieted quickly (an I2C device) must be polled instead of given a line.
- **Line 15 has no hardware**, so its entry is free for `BRK`: the kernel sets `V` to it.

No wait states

`RDY` has only a pull-up: nothing on the board stretches a bus cycle. Every device must answer within the CPU's cycle, and at 7.16 MHz neither the YM2151 nor the RAM path (the bank registers, the decoders, the RAMs) can: **with the stated parts, 3.58 MHz is the fastest valid speed** (at J8's 7.16 MHz the power-on self test fails nearly every RAM test: measured on 1.8D, 2026-10-13). A card in a slot can pull RDY low to add wait states of its own, but RDY must never be driven high (the W65C02S pulls it low during `WAI`): use an open-collector output.

The serial port's clock

The ACIA runs from 1.790 MHz, not the 1.8432 MHz its baud rates are made for, so every rate is **2.9% slow**: 9600 is about 9,320, 115200 about 111,860. Terminals accept it. But at 115200, characters sent back to back have too little margin, so the serial driver paces its sending by VIA timer 2, with idle bits between characters (the devices).

The WDC W65C51N has a hardware fault of its own: its "transmitter empty" status bit is always set and it never interrupts for sending. The BIOS ROM built for it (`bios_wdc.bin`) times each character instead, and tells the paged ROM's serial drivers (`BIOS_ACIA`), which do the same.

The YM2151's timer B

On the board, the YM2151's timer B didn't keep its period when it was re-armed every tick (its flag reset through register `$14`, which seems to restart it): songs timed by it ran up to twice as fast, by a different amount each run. So the sound driver times songs by the system's tick (VIA timer 1). The hardware test's timer checks (one period from a clean start) pass.

The clock chip in task F's RAM

A DS1747 in U7's socket puts its clock's registers at the chip's top 8 bytes, which are task F's `$7FF8-$7FFF`. So task F's program must keep out of its own last 8 bytes, and only the kernel's `RTC` call should touch them. The base and HydraOS run their serial driver in task F, and keep its RAM below `$7F00`.

Only slot 0's audio is mixed

Every slot has an audio pair (`SND_CL`, `SND_CR`, pins 26 and 24), but on 1.8D only **slot 0's** reaches the mixer, with the YM2151's and the line input (J29). 1.8C mixed all six slots' audio, each through a 10K resistor; 1.8D keeps only slot 0's, R7 and R13. A sound card goes in slot 0, or brings its audio out on a connector of its own.

The mixer's inputs are DC-coupled: a card whose outputs sit at a DC level (a DAC biased at half its supply) should couple them through a capacitor.

The power rails

- **-5 V** reaches the slots (pin 9) only from an old ATX 1.x supply's pin 20; newer supplies leave that pin unconnected. Don't count on it.
- **+3.3 V** comes in on the ATX connector but only reaches a bulk capacitor (C55): it isn't used on the board or carried to the slots.
- **+5 V standby** (`+5VA`) likewise only reaches C51.

The schematic's own quirks

Things in the KiCad files that don't match what's on the board, or look wrong but aren't:

- **J28 is a chip**, the SP720 ESD array, drawn with a DIP-16 socket's symbol.
- **The bus breakout card's labels** disagree with the main board's slot on two pins: J11 pin 3, "+5VA", is **-5 V**, and J7 pin 5, "!RWB", is `DMAB` (the cards).
- **J1 pin 4**, on the memory card connector, isn't connected.
- **The pseudo-registers' unused bits** (`T` and `U` 4-7, `W` 6-7, `V` 4-7) are latched and read back, but drive nothing: the software uses `V`'s top bits to remember a software interrupt's number.

What changed from 1.8C to 1.8D

Reliability changes only. The address map, the decoding, every register and every timing a program can see are 1.8C's, so the same ROMs and software run on both boards. (`board/CHANGES-1.8D.md` has the designer's notes.)

Parts swapped (the same footprints and pinouts):

Part	1.8C	1.8D	Why
U11	74F08	74ACT08	It drives the CPU's clock input; the W65C02S wants a CMOS high (0.7 VDD), and an F part guarantees only about 2.5 V
U22	74F240	74ACT240	It drives IRQB, PHI1, DMA and !RWB
U59	74F245	74ACT245	It drives the CPU's data bus on every read
IC9	LF353	NJM4556A	The mixer drives the headphone jack: the LF353 current-limits into 16-32 ohm headphones and distorts; the 4556A drives about 70 mA

Parts added:

Part	Value	What
R22	330 ohms	Limits the crystal's drive (between U39's output, the crystal and C48)
R23-R27	22 ohms	Series terminations at the source of HS_CLK, SND_CLK, SER_CLK, PHI2 and PHI1
R28	10K	A fixed pull-up on RDY, in place of 1.8C's jumper J4
C86-C89	47 µF	Bulk capacitance on +5 V, spread over the board
C80-C85	100 nF	Decoupling at the YM2151, the YM3012, and the op-amps' supplies

Part	Value	What
R29	0 ohms	Joins the audio ground to the logic ground once, beside the YM3012
C90, C91	10 µF	Keep the YM3012's 2.5 V out of the mixer
R30, R31	33 ohms	In series with each output: the op-amp stable into a cable, safe from a shorted plug
C92, C93	220 µF non-polar	AC-couple each output to the jack
R32, R33	10K	Hold the jack's side of C92/C93 at 0 V, so a plug going in doesn't click
RN4	10K x 8	Pull-ups on D0-D7: a read nothing answers gives `$FF`
J30	1 x 3 header	The ACIA's CTS: 1-2 from the RS-232 line (as 1.8C), 2-3 always clear

Parts removed: J4 (the RDY jumper: R28 instead), and the mixer inputs for slots 1-5 (above).

Wiring:

- The headphone jack (J26) has left on its tip and right on its ring, the usual way. 1.8C had them swapped.
- The power LED (J10 pin 12) is lit from +5 V through R2, not from the ATX supply's PWR_OK (a logic signal, not meant to drive an LED).
- ATX pin 20 feeds the slots' -5 V pin.
- U26's unused second half has its inputs tied, and U22's spare inputs are grounded.
- The unused pins are marked no-connect.

What software sees: only that a read of an empty slot or an unused I/O port gives `$FF` every time, instead of whatever the bus last held. Nothing relied on that.

At 7.16 MHz, ACT parts are a few ns slower than F parts; the faster clock fails on 1.8D with the stated parts (POST's RAM tests): 3.58 MHz is the fastest valid speed.

The cards

The boards that plug into the main board: the memory daughter card (task banks), the bus breakout card (every bus signal on a header), and the Vera X (video, a second sound chip, a keyboard and a mouse). Their parts are in the parts list, and the drawings of the first two in the board layout.

Contents

1. The memory daughter card
2. The bus breakout card
3. The Vera X
    1. What it is, on the Hydra
    2. The glue board: the plan, the breakout card's headers, the glue logic, the Vera X's header, the data path, power, audio, the interrupt
    3. The keyboard and mouse
    4. Timing, bringing it up, the carrier card to come

The memory daughter card

`board/MemoryDaughterCard`*: a through-hole version and a surface-mount one.*

A **module** of task RAM is 2 MB: four HM628512, which give each of the 16 tasks 16 banks of 8K of its own (the architecture). The main board takes up to 15 modules, numbered 0-14, as bank IDs `$00-$EF`: module m is banks `$m0-$mF`.

The cards:

- **Through-hole** (`MemDaughterCard`): one module, four DIP-32 chips.
- **Surface mount** (`MemDaughterCard_SurfaceMount`): **two** modules, eight SOP-32 chips (IC1-IC4 the first, IC5-IC8 the second).

How a card works:

- It plugs into the main board's memory connectors, J1 (1 x 32) and J2 (1 x 24): every module sees the same signals, and its number decides which bank IDs it answers.
- A 74LS139 (U1) picks one of the four chips by `RAMB_M2-3` (each half of it for one module on the surface-mount card).
- `RAMB_M0-1` go to the chips' A17-A18, `T_M0-3` to A13-A16 (so each task has its own banks), and A0-A12 and the data lines come from J2.

Setting the module number. J3 is a 2 x 15 header. Its odd pins (1, 3, ... 29) are the module selects `nBRC0-nBRC14`; its even pins are all the card's select. **A shunt across pins 2m+1 and 2m+2 makes the card module m**: pins 1-2 for module 0, 3-4 for module 1, and so on. On the surface-mount card, J4 (1 x 15) is the second module's select, beside J3's odd row: a shunt from J4's pin m+1 to J3's pin 2m+1 makes the second module m.

- Give every module a different number. Start at 0 and count up.
- A module answers the number it's jumpered as: the crossed bank bits are 2 and 3, inside a module's sixteen banks, not the module number's. (Until 2026-10-13 this page said a module jumpered as 4 answered as 8; it doesn't.)
- POST finds the modules that are there and tests each one: `RAM modules: 03` is three found. A module with a bad line is left unused.

The main board's J1 (the card's J1 is the same):

Pins	Signals
1-3	A13, A14, A15 (not used by the card)
4	Not connected
5-8	`T_M0-3`: the task
9-12	`RAMB_M0-3`: the bank's low nibble
13, 14	`RWB_M`, `!RWB_M`: write and read strobes
15-17	GND
18-32	`nBRC0_S-nBRC14_S`: the module selects

J2: D0-D7 (pins 1-8), +5 V (9-11), A0-A12 (12-24).

The bus breakout card

`board/HydraBusBreakoutCard`.

A slot card with no chips: every bus signal brought out to a header, for a logic analyser, a scope, or wiring a prototype (the Vera X's glue board is wired to one).

Header	Signals
J1	The slot's edge connector
J2	The data bus: D7 at pin 1, down to D0 at pin 8
J3	The address bus: A15 at pin 1, down to A0 at pin 16
J4	SPI: A0-A3, ~CS, MISO, MOSI, SCLK
J5	NMIB, `nIO_S`, the slot's port selects and IRQs
J6	The clocks
J7	Bus control: `nBROMD`, BE, RDY, R/W, SYNC, and `DMAB`
J8	Reset
J9	Audio: the slot's pair
J10	I2C: SDA (pin 1), SCL (pin 2)
J11	Power

Two of its labels are wrong. The card's connector symbol and the main board's slot disagree, and the slot carries what the main board's says:

Header	Its label	What the slot really carries	So
J11 pin 3	`+5VA`	**-5 V** (slot pin 9, and only with an old ATX supply)	**Never use it as a 5 V supply**
J7 pin 5	`!RWB`	`DMAB` (slot pin 23)	It's not an inverted R/W: pulling it low stops the CPU

The Vera X

What it is

The Hydra's video card is Joe Burks's **VERA X 6.1**, a module carrying the Commander X16's **VERA** (Video Embedded Retro Adapter): an iCE40UP5K FPGA with 128K of video RAM inside it, running the X16 community's gateware (version 47 or later). It gives:

- **VGA video**, 640 x 480: two layers, each text (with a 256-character font), tiles, or a bitmap at 1, 2, 4 or 8 bits a pixel; 128 hardware sprites; a palette of 256 colours from 4,096; scaling, and the gateware's FX (line drawing, polygon filling, cache writes).
- **A 16-voice PSG** (each voice square, sawtooth, triangle or noise), and **a PCM FIFO** (8 or 16 bits, mono or stereo, up to 48 kHz), mixed to a stereo line output.
- **An SD card's SPI controller**, with its own card slot on the module.

The VERA's own documentation is the X16's: *The Commander X16 Programmer's Reference*, its VERA chapters.

On the Hydra

Slot	Slot 0, always: its port selects, interrupt and audio pair are the ones the software expects
Registers	The VERA's 32, at slot 0's ports 2 and 3, `$FF20-$FF3F`: `VERA_*` in `base/include/hw.inc`
Interrupt	Its `IRQ#` on slot 0's IRQ A, **line 2**: the highest priority after the VIA and the ACIA
Audio	Its line output into slot 0's audio pair, mixed with the YM2151's
Reset	`RESB` to its `RES#`: a reset makes the FPGA load itself again from its flash, and it doesn't answer until that's done (the driver looks for it for 0.3 s)
Its SD card	The VERA's own SPI controller, separate from the board's SPI bus: a card there is the storage driver's disk `v`, `/sd/v`
Keyboard and mouse	The X16's input controller (an ATtiny861 running `x16-smc`) on the I2C bus at `$42`, polled

The software: `vid` finds the card as the system starts (by its version register), shows the console on the screen, and serves the card as `/dev/vid`; the sound driver's channels 8-23 are the PSG's voices and `/dev/vid/pcm` the PCM; and `kdev` reads the keyboard and mouse (video). The emulator has the whole card (`node sim/run.js --vera --smc`).

As built (October 2026): the VERA X module alone, wired to a bus breakout card in slot 0 through the **glue board** below. A proper carrier card (below) is still to be made.

The glue board

`board/vera-x/vera-x.kicad_sch`*: a schematic for a protoboard, with no PCB.*

How to connect the VERA X 6.1 (its 2 x 13 header) to the Hydra with a **bus breakout card in slot 0**, two glue chips on a small protoboard, and wires, with the X16's input controller beside them for a keyboard and mouse. Its parts are in the parts list.

The plan

- **Slot 0** (the main board's J12). Its two port selects are ports 2 and 3, `$FF20-$FF3F`, the VERA's 32 registers; its IRQ A is line 2; its audio pair goes into the mixer. In any other slot the addresses, the interrupt and the audio would all be wrong.
- **No buffers on the bus side.** The breakout card is only headers: each slot pin to a header pin, on the system's buses themselves.
- **Glue logic.** The VERA wants `CS#`, `RD#` and `WR#` (active low). The bus has `R/W` (`RWB`), `PHI2` and an I/O page select; the strobes are made from them as the X16 makes them.

The breakout card's headers, as slot 0 drives them

Only the pins this wiring uses:

Header	Pin	Signal		Header	Pin	Signal
J2 (data bus)	1	D7		J3 (address bus)	9	A7
	2	D6			10	A6
	3	D5			11	A5
	4	D4			12	A4
	5	D3			13	A3
	6	D2			14	A2
	7	D1			15	A1
	8	D0			16	A0
J5 (interrupts & I/O)	2	`nIO_S`: the `$FFxx` page, low only while PHI2 is high		J6 (clocks)	4	PHI2
	3	`nIOB_S`: port 3, `$FF30-$FF3F`		J7 (bus control)	4	RWB (high: a read)
	4	`nIOA_S`: port 2, `$FF20-$FF2F`		J8 (reset)	1	RESB (low: reset)
	6	IRQA: IRQ line 2 (a 3.3K pull-up on the board)		J9 (audio)	2	SND_CR: the mixer's right
J11 (power)	4	+5 V			3	SND_CL: the mixer's left
	5	GND		J10 (I2C)	1, 2	SDA, SCL

The glue logic

U1, 74AHCT138: `CS#` for `$FF20-$FF3F`. Enabled by `nIO_S`, it decodes A5-A7: output Y1 is low for A7 A6 A5 = 0 0 1.

U1 pin	Function	Connect to
1	A0	J3 pin 11 (A5)
2	A1	J3 pin 10 (A6)
3	A2	J3 pin 9 (A7)
4	/E1	J5 pin 2 (`nIO_S`)
5	/E2	GND
6	E3	+5 V
8	GND	GND
14	/Y1	VERA pin 13 (`CS#`)
16	VCC	+5 V, 100 nF to GND

Leave the other outputs (7, 9-13, 15) unconnected.

U2, 74ACT00: `RD#` **and** `WR#`. `RD#` = NOT (PHI2 AND RWB): low while PHI2 is high in a read. `WR#` = NOT (PHI2 AND NOT RWB): low while PHI2 is high in a write. Gate 2 is the inverter.

U2 pin	Function	Connect to
1	1A	J6 pin 4 (PHI2)
2	1B	J7 pin 4 (RWB)
3	1Y	VERA pin 18 (`RD#`)
4	2A	J7 pin 4 (RWB)
5	2B	J7 pin 4 (RWB)
6	2Y	U2 pin 10 (NOT RWB)
7	GND	GND
8	3Y	VERA pin 15 (`WR#`)
9	3A	J6 pin 4 (PHI2)
10	3B	U2 pin 6
11	4Y	unconnected
12, 13	4A, 4B	GND (an unused CMOS input must never float)
14	VCC	+5 V, 100 nF to GND

Use the ACT and AHCT families as listed: their inputs take the TTL-level signals of the board's LS and F parts, and they're fast, which the write strobe needs (timing).

The Vera X's header, pin by pin

Its 2 x 13 header goes in a 2 x 13 socket (J1 on the glue board). Pin 1 is SCL; odd pins on one row, even on the other.

VERA pin	Signal	Connect to
1	SCL	unconnected (the VERA doesn't need it)
2	SDA	unconnected
3	+5 V	J11 pin 4 (+5 V)
4	GND	J11 pin 5 (GND)
5	D7	the data path (below), to J2 pin 1
6	D6	J2 pin 2
7	D5	J2 pin 3
8	D4	J2 pin 4
9	D3	J2 pin 5
10	D2	J2 pin 6
11	D1	J2 pin 7
12	D0	J2 pin 8
13	`CS#`	U1 pin 14
14	`RES#`	J8 pin 1 (RESB)
15	`WR#`	U2 pin 8
16	`IRQ#`	J5 pin 6 (IRQA)
17	A4	J3 pin 12
18	`RD#`	U2 pin 3
19	A2	J3 pin 14
20	A3	J3 pin 13
21	A0	J3 pin 16
22	A1	J3 pin 15
23, 24	GND	GND
25	Audio left	J9 pin 3 (SND_CL)
26	Audio right	J9 pin 2 (SND_CR)

So the VERA's registers 0-31 are `$FF20-$FF3F`, `VERA_BASE` in `hw.inc`.

The data path

The VERA X takes 5 V inputs but drives 3.3 V highs. The W65C02S's data inputs are specified high at 0.7 VDD, **3.5 V**, so a 3.3 V high is 0.2 V short of the data sheet. The X16 runs this way, since a CMOS input's real threshold is near 2.5 V, so there are two ways to wire the data lines, and the glue board has both:

- **JP1 with 8 shunts** (try this first): the VERA's data pins straight to the bus. JP1's odd pins are the VERA's D0-D7 (`VD0`-`VD7`), its even pins the bus's.
- **U3, a 74AHCT245**, fitted with JP1 empty: its TTL inputs take the VERA's 3.3 V, and its outputs give 5 V. If the card is found unreliably or reads come back wrong, fit it.

U3 pin	Function	Connect to
1	DIR	J7 pin 4 (RWB: high, A to B, the VERA onto the bus)
2-9	A1-A8	VERA pins 12, 11, 10, 9, 8, 7, 6, 5 (D0-D7)
10	GND	GND
11-18	B8-B1	J2 pins 1-8 (D7 at pin 11, down to D0 at pin 18)
19	/OE	U1 pin 14 (`CS#`)
20	VCC	+5 V, 100 nF to GND

Power, grounds, wires

- **Power**: J11 pin 4 (+5 V) to VERA pin 3 and the chips; J11 pin 5 (GND) to VERA pins 4, 23 and 24 (all three) and the chips. Use a heavier wire for the ground: the whole data bus returns through it. 10 µF (C1) and 100 nF (C2) across the VERA's pins 3 and 4, at the socket; 100 nF at each chip (C3-C6).
- **Never J11 pin 3** (-5 V, whatever the breakout card says).
- Keep the bus wires short (15 cm or less) and together; run a ground wire beside PHI2. Plug and unplug with the power off.
- The VERA X's VGA output goes to the monitor. Its SD card slot is its own (at 3.3 V, as the VERA X's notes say).

Audio

The VERA's line outputs go into the mixer's slot 0 inputs: 10K into an inverting summer on ±12 V, beside the YM2151's. A ground-centred line output connects straight in. Before wiring them, measure pins 25 and 26 against GND with the card running: if they sit at a DC level rather than about 0 V, put a 10 µF capacitor in series with each (its + toward the VERA if the level is positive), or the mixer passes the offset to the jack.

The mixer is unity gain, so the VERA comes out as loud as its line output is, which may be louder than the YM2151. If so, a divider in each line evens them (10K in series and 4.7K to GND at the mixer's side: about a third).

The interrupt

Slot 0's IRQ A is the VERA's alone, with a 3.3K pull-up on the board, so the VERA's `IRQ#` drives it directly: low for an interrupt, high (3.3 V) otherwise. If the line misbehaves, put a 74LS07 gate (open collector) between them: VERA pin 16 to its input, its output to J5 pin 6, so the card only ever pulls the line low.

The keyboard and mouse: the X16's SMC

The input controller is the X16's own: its **SMC**, an **ATtiny861** (U4) with the X16 community's firmware, `x16-smc` (github.com/X16Community/x16-smc), unchanged. A PS/2 keyboard and a PS/2 mouse plug into it, and it talks to the Hydra over the I2C bus at address `$42`, which every slot carries and the breakout card brings out on J10.

What you need: an ATtiny861 (DIP-20) programmed with an `x16-smc` release (its HEX file and fuses, as its README gives them; a TL866-class programmer does it), two 6-pin mini-DIN sockets (J12, J13), a push button (SW1), a 10K resistor (R1) and a 100 nF capacitor.

Its pins are the firmware's default build's (not its `COMMUNITYX16_PINS` one):

ATtiny861 pin	Port	The firmware's name	To
1	PB0	`I2C_SDA_PIN`	J10 pin 1 (SDA)
3	PB2	`I2C_SCL_PIN`	J10 pin 2 (SCL)
18	PA2	`PS2_KBD_CLK`	The keyboard's socket (J12), pin 5 (clock)
4	PB3	`PS2_KBD_DAT`	The keyboard's socket, pin 1 (data)
9	PB6	`PS2_MSE_CLK`	The mouse's socket (J13), pin 5

ATtiny861 pin	Port	The firmware's name	To
8	PB5	`PS2_MSE_DAT`	The mouse's socket, pin 1
14	PA4	`POWER_BUTTON_PIN`	SW1, to GND
17	PA3	`PWR_OK`	+5 V through R1 (10K): "the power is good"
5, 15	VCC, AVCC		+5 V, 100 nF to GND at the chip
6, 16	GND, AGND		GND
10	PB7	(its RESET)	Nothing (the programmer's)
20, 19, 2, 13, 12, 7, 11	PA0, PA1, PB1, PA5, PA6, PB4, PA7	`RESB_PIN`, `NMIB_PIN`, `IRQB_PIN`, `PWR_ON`, `ACT_LED`, ...	Nothing: they're the X16's

The PS/2 sockets' pin 3 is GND and pin 4 +5 V; pins 2 and 6 aren't used.

- **The I2C bus** has its pull-ups on the main board (RN1), so the SMC needs none. Nothing else on the board answers at `$42`. `IRQB_PIN` stays unconnected: the Hydra polls the SMC, as its interrupt couldn't be quieted from a handler (the quirks).
- **Press SW1 once after switching on.** The firmware is a power supply's controller first: its power-on, which the button starts, is what starts the keyboard and mouse. Until then it answers on I2C but has no keys.
- **To check it:** `ls /dev/i2c` lists `42` once the SMC is on the bus, and `ps` shows `kdev` in the state `sleep` (between its looks at the controller) and not `idle`. The controller is looked for as the system starts (three times, in its first three seconds): fit it, then reset.

Timing

- **The VERA latches a write's data as its write strobe ends**, and the 65C02S holds write data only **10 ns** after PHI2 falls. So `WR#` must end the write promptly: it's PHI2 through one 74ACT00 gate, rising 3-9 ns after PHI2 falls. (`CS#` comes through the board's decoding and ends later: as the write's end, it would miss the data.)
- **Reads**: the 65C02S needs data 10 ns before PHI2 falls. `CS#` falls some 15-20 ns after PHI2 rises, and the VERA's data follows within tens of ns. PHI2 is high about 140 ns at 3.58 MHz and 70 ns at 7.16 MHz: room at both.
- **No glitches**: `nIO_S` is low only while PHI2 is high, when the address is steady, so `CS#` can't pulse while it changes.
- **The slot's own port selects** (`CS#` = J5 pin 4 AND J5 pin 3, one gate of a 74AHCT08, in place of U1) also work at 3.58 MHz, but they come through the board's 74LS154, 25-35 ns more, which leaves too little of a 7.16 MHz cycle. U1 works at both speeds.

Bringing it up

1. **The glue board alone** (no VERA in its socket), the breakout card in slot 0, the Hydra on: check +5 V and GND at the socket's pins 3 and 4. With a logic probe or a scope, watch `CS#` (U1 pin 14), `RD#` (U2 pin 3) and `WR#` (U2 pin 8) while something touches `$FF20-$FF3F`: in HyBASIC, `10 POKE 65313,85: X=PEEK(65313): GOTO 10`, `RUN` (Ctrl-C stops it). `CS#` should pulse on every access, `WR#` on the POKE, `RD#` on the PEEK, each only while PHI2 is high.
2. **The VERA X**, power off, then on: the console comes up on the monitor too (`vid` found the card). `cat /dev/vid/ctl` starts with `vera 47.0.2` (the gateware's version).
3. **Try it**: `echo border 4 >/dev/vid/ctl` (the border's colour), `echo note 8 69 >/dev/sndctl` (an A from the PSG, through the mixer; `echo off 8 >/dev/sndctl`), `play` a WAV file (the PCM).
4. **No** `/dev/vid` (no card found): check +5 V, the data lines' order (VERA pin 5 is D7), A0-A4, `CS#`, `RD#` and `WR#` (each active low), and `RES#` (high once the reset is over). Then try U3 for the 3.3 V highs.
5. **The keyboard and mouse**: `ls /dev/i2c` lists `42`; press SW1; then keys typed on the PS/2 keyboard reach the shell, and `cat /dev/vid/mouse` prints a line each time the mouse moves, the pointer following it on the screen.
6. (7.16 MHz, J8 and a build with `--clock 2`, is for a board with faster parts than the stated ones: with these, 3.58 MHz is the fastest valid speed.)

The carrier card to come

The plan for a proper card (`docs/design/plans/VIDEO.md`, option A) is a small slot card with a socket for the VERA X module and the same glue: `CS#` from either port select, `RD#` and `WR#` from PHI2 and R/W, `IRQ#` through an open-collector buffer, the audio through a divider to the mixer's level, and the SMC with its PS/2 sockets on the slot's I2C. Until it exists, the glue board above does the same job.

3. The base: the kernel and the system calls

The **base** is the part of the Hydra-16's software that every system on the board shares: the kernel in the BIOS ROM, its system calls, the interrupt path, a serial console driver, the kernel's own devices, and a monitor. HydraOS is built on it, and it's enough to build an operating system of your own on. It lives in `base/`.

This chapter is the kernel as a whole: how the machine starts, how tasks, memory and interrupts work, the file model everything else is built on, how a program calls the system, and the monitor the base starts when it runs alone. Chapter 4 is the drivers that serve the devices as files, and chapter 9 how to write programs and drivers of your own.

Contents

1. The boot: from reset to init: the reset stub, the kernel task, POST, the drivers, init.
2. The kernel: the BIOS ROM's pages, tasks and the scheduler, interrupts, memory and banks, shared segments, semaphores, notes, the clock, how the kernel reaches other tasks.
3. Files, servers and namespaces: fds and channels, servers and requests, device letters, namespaces and unions, the environment, the kernel's own devices.
4. The system calls: the jump table, the registers, the groups of calls, errors, and calling from each language.
5. The monitor: Woz's monitor, the base's own init.

The base in one page

Booted on its own, the base is four tasks:

Task	Module	What it is
0	the kernel	The BIOS ROM (`base/bin/bios_rockwell.bin` or `bios_wdc.bin`, from `base/kernel/`): the boot and POST, the scheduler and the tick, tasks, memory and banks, semaphores, shared segments, notes, the clock, files and the server protocol, namespaces, the loader; 89 system calls in a jump table of 192 slots at `$F800`
F	`ser`	The console driver: the serial port as files (`#c`: `cons`, `consctl`, `ser`, `serctl`)
E	`kdev`	The kernel's devices: the modules (`#m`), `null` and the kernel's messages (`#n`), the tasks (`#p`), pipes (`#|`), the environment (`#e`), shared segments by name (`#s`), the ticks (`#t`), raw RAM (`#r`), the root (`#/`)
1	`wozmon`	The monitor, as init: Woz's Apple 1 monitor, with a disassembler

Tasks 2 to D are free for whatever runs on it. The paged ROM image (`base/bin/prom0.bin`) has the module directory, the hardware test, and the three modules; it fills one chip.

HydraOS uses the same kernel, byte for byte: its BIOS ROM is the base's. It replaces `ser` with `cons` (the same files and more: windows), adds its own drivers, and starts `init` instead of the monitor (chapter 4). A program written for the base runs on HydraOS as it is.

The base's tree

Folder	What's there
`base/kernel/`	The kernel: `bios.cfg` (the BIOS ROM's 16 pages), `reset.s` (the reset stub, the vectors, the boot), `common.s` (the COMMON block: the IRQ entry and exit, the far call), `sched.s`, `irq.s`, `call.s` (SCALL, KCALL, kcopy), `task.s`, `load.s` (SPAWN and the loader), `mem.s`, `sem.s`, `notes.s`, `time.s`, `file.s`, `ns.s`, `env.s`, `info.s`, `debug.s`, `cons.s` (the bring-up console), `sys.s`, `post.s`, `far.s`
`base/include/`	`hw.inc` (the board as the software sees it: every register's name) and `layout.inc` (where every piece of the kernel's state is)
`base/spec/`	`api.def` (the system calls: the jump table, `hydra.inc` and the reference are made from it) and `errors.def` (the error codes and their texts)
`base/modules/`	The base's modules, a folder each (`ser`, `kdev`, `wozmon`); `rom.txt`, the paged ROM's list and its init; `module.cfg` ... `module8.cfg`, a module's link (one bank to eight)
`base/lib/`	Sources shared with HydraOS's modules: `serial.inc` (the serial port's layer: `ser`'s and `cons`'s), `dis.inc` and `w65c02.inc` (the disassembler)
`base/sdk/asm/`	The assembly SDK's core: `hydra.inc` (the calls, the errors, the constants: made from `spec/`, and in Git), `hyx2.inc` and `hyx2.cfg` (a program's header and link), `macros.inc`, `srvlib.inc` and `srvlib.s` (the server library)
`base/tools/`	`apigen.js` (the spec into the jump table, the error texts, `hydra.inc`), `romimg.js` (a paged ROM image), `check.js`, `budget.js`
`base/sim/`	The emulator, cycle by cycle (chapter 8)
`base/tests/`	The base's tests and their modules

Folder	What's there
`base/bin/`	The images, in Git: `bios_rockwell.bin` and `bios_wdc.bin` (one for each serial chip), `prom0.bin`

Building it needs Node.js 18 or later and cc65 (`ca65` and `ld65`), found in `$CC65_BIN`, `$CC65_HOME/bin`, or on the PATH:

```
$ cd base
$ node build.js          # the kernel (bin/bios_*.bin), the modules, the paged ROM (bin/prom0.bin), hydra.inc
$ node sim/run.js -i     # the base in the emulator, its serial console in your terminal (Ctrl-A x quits)
$ node sim/test.js       # the base's tests
```

The images are in Git, so `node sim/run.js -i` works without cc65.

The boot

What happens from the moment the reset line goes high to the first prompt. The code is `base/kernel/reset.s`, with POST in `post.s` and the drivers' start in `task.s`.

Contents

1. The reset stub
2. Step 1: every task's state
3. Step 2: the console, the banner and POST
4. Step 3: the interrupts
5. Step 4: the boot drivers
6. Step 5: the tick, init, and the idle loop
7. The module directory
8. After the boot: the kernel's messages
9. Rebooting, and the hardware test

The reset stub

When the reset line goes high, the W65C02S reads the RESET vector at `$FFFC` and jumps there. But `$FFFC` is in the BIOS ROM, whose page is chosen by `W`, and `W` isn't reset: it powers up random and keeps its value through a reset (the quirks). So the CPU may start on any of the 16 pages.

The kernel deals with it by putting the same thing on every page:

- Every page's RESET vector points to `$E000`.
- Every page's first instruction, at `$E000`, is `stz W_REGISTER`: it selects page 0. The CPU fetches the next instruction from page 0, at `$E003`.
- Page 0 goes on from `$E003`: interrupts off (`sei`), decimal mode off, `T` = 0 (the kernel task's RAM and stack), the stack pointer at `$FF`, then `BOOT`.

From here the boot runs in **task 0, the kernel task**, with interrupts off, in five steps.

Step 1: every task's state

Nothing in RAM can be trusted after power-up, and the bank registers are random too. For each task, 15 down to 0, the boot sets `T` to the task and then, in that task's RAM:

- its bank registers `$00` and `$01` to 0;
- its OS zero page (`$80-$FF`): the task free (`ST_FREE`), no flags, no preemption hold, no notes, its saved stack pointer;
- its OS area: no irq entry and no serve entry, every interrupt line unowned in its copy of the owners' table, all 16 fds closed.

It can't use the stack while it does this: changing `T` changes the stack page under it. Then, back in task 0:

- the kernel task is made ready, and never preemptible (its `TK_PREEMPT` is 1 for ever);
- `U` = 0, and `V` = line 15's vector entry, the one `BRK` uses;
- the kernel task's tables are cleared: the parents, the tasks' banks, semaphores, exit records, CPU times, shared segments, the note groups (each task its own), the channel table (no open files), the device table (no devices), the namespaces;
- the kernel task's serve entry is set to the KCALL dispatcher, and its irq entry to the tick's handler.

Step 2: the console, the banner and POST

- **The bring-up console**: the serial port, polled, at 9600 8N1 (`cons.s`). Everything the kernel prints until a console driver takes the port over goes out this way, and it's also kept in the kernel's message buffer (below).
- **The banner**: `Hydra-16: kernel 0.1, ABI 1`. The ABI is the system calls' version: a program built for ABI 1 runs on any kernel that says 1.

- **POST** (BIOS page 4, `post.s`): it checks that the tasks' RAM is kept apart (the `T` lines), that the shared RAM is shared, the `W` lines, and every RAM bank chip's lines, and finds the memory cards. Building it explains its report. Bad RAM is recorded and left unused. A `T` typed during POST starts the hardware test instead of the system.
- **The environments' banks**: each task's environment is a RAM bank of the kernel task's, from the first good memory card; they're emptied. With no memory card, there are no environments.
- `RAM modules: NN`: how many memory cards POST found good.

Step 3: the interrupts

`IRQ_INIT` (`irq.s`) writes all 16 entries of the vector RAM: each line's entry points to its stub in the COMMON block, on every page the same. It sets every line's owner to nobody, then gives the **VIA's line, 0, to the kernel**: the VIA's timer 1 is the system's tick. The VIA's line also carries timer 2 and CA1, which the kernel passes on as lines of their own (16 and 17) to the drivers that own them (the kernel).

Step 4: the boot drivers

`K_TASK_BOOT` (`task.s`, BIOS page 1) reads the paged ROM's **module directory** and sets up a task for each module that's a **driver** with the flag `HF_BOOT`, in the directory's order: the first in **task F**, the next in **task E**, and down. Each is set up as `SPAWN` would set it up: its bank at `$A000`, its data copied into its RAM, its BSS cleared, and its first instruction its `init` entry. None runs yet: interrupts are still off, and the kernel task holds the CPU.

The base's directory has two boot drivers: `ser` (task F) and `kdev` (task E). HydraOS's has five: `cons` (F), `storage` (E), `kdev` (D), `snd` (C), `vid` (B). Each prints a line as it's set up: `task F: ser`.

Step 5: the tick, init, and the idle loop

- **The tick**: VIA timer 1 is set free-running at **200 interrupts a second** (`TICK_HZ`), and its interrupt enabled.
- **Interrupts on.** From now on the scheduler runs, and the boot drivers start: each runs its `init` (registering its device letters, `SRV_REGISTER`, and owning its interrupt lines, `IRQ_OWN`), then waits for requests. While a driver's init runs, it's busy: a request to it waits.
- **The boot waits** for every boot driver's init to end, two seconds at most. A driver whose init fails (it returns C = 1) ends; the system goes on without it. A driver whose init doesn't end, because it has work of its own to do between its calls, counts as started at its first `SRV_NAP` (the kernel).
- **Init**: the module the directory names as init starts in **task 1**, as a program, with an empty namespace of its own and no fds. It builds both: it opens the console (fds 0, 1 and 2) and binds what the system's namespace needs. **Init is the system**: in the base it's `wozmon`, the monitor; in HydraOS it's `init`, which reads the namespace file, sets the windows up and becomes the login shell (`EXEC`).
- **Init again**: the kernel starts init again, in a task of its own with a number as its argument, when it's asked to (`INIT_RUN`: HydraOS's console asks for window n's when the user makes one), and by itself with `0` when the task the boot's init was in has `EXEC`ed and then ends: a login shell that ends is started again, with no task kept waiting for that. An init that ends without an `EXEC` (the base's monitor, a test) is left ended.
- **The idle loop**: task 0 becomes the idle task. It runs only when no other task can, and then waits for an interrupt with `WAI`, which stops the CPU's clock until one comes. It also takes a turn when an interrupt handler has queued notes to a note group for it to send.

The base reaches the monitor's prompt in under a second, most of it spent printing at 9600 baud.

The module directory

The paged ROM's bank 0 holds the **module directory**, at `$A200` (written by `base/tools/romimg.js` from a `rom.txt`):

- a signature, `HYMD`, and its version;
- how many modules there are (127 at most), and which is init;
- whether bank 1 holds the hardware test;
- 16 bytes for each module: its first bank, how many banks it spans, its type (program, driver, library), its flags, its name (10 characters at most), and its **page**: where its header is in its first bank, `$A000 + 256 * page`.

`MODINFO` reads an entry, and kdev serves the directory as files: `#m/NAME` is a module's image, which `SPAWN` reads; `#m/bin` lists the programs, and is bound at `/bin`.

A `rom.txt` names the modules in order, and init:

```
init        wozmon
module      ser         ; (task F: the console's driver)
module      kdev        ; (task E: the kernel's devices)
module      wozmon      ; (task 1: the monitor)
```

Bank 0's first 512 bytes hold a signature line and the ROM disk's partition table (HydraOS's `/rom`); bank 1 is the hardware test with the checksums of every ROM; the modules start at bank 2. A module of two banks or more is at `$A000` of its first, its banks in a row. A module of one bank is **packed**: the build links it at the page where `romimg.js` finds room for it (in bank 0 after the directory, in the free top of another module's last bank, or in a bank of its own), so several small ones share a bank. The kernel reads a module's header at its page, and sets its task's ROM bank register to its bank as for any other. In HydraOS, `vid` shares bank 0 with the directory, and `kdev` and `init` share the console driver's last bank.

After the boot: the kernel's messages

Everything the kernel prints on the bring-up console, from the banner on, is also kept: its last 4K, in the kernel task's RAM. The `KMESG` call reads them, and kdev serves them as `#n/kmesg`, bound at `/dev/kmesg` (Plan 9's name): `cat /dev/kmesg` shows the boot's lines long after they've scrolled away. A driver's own messages from its init go there too.

Rebooting, and the hardware test

`REBOOT` starts the system again, as the reset button does: it quiets the VIA's and the ACIA's interrupts (only the button resets the chips), then jumps to the reset stub. With `.A = REBOOT_HWTEST`, POST starts the hardware test instead of the system, as a `T` typed during POST does: HydraOS's `hwtest` command is this. Every task ends where it is; files written are on their disks already, since the disks write through.

The kernel

How the kernel works: where it lives, how it runs tasks, handles interrupts, gives out memory, and lets tasks work together. Each section names its source in `base/kernel/`; the design behind all of it is in `docs/design/reimplementation-from-scratch.md`.

Contents

1. The kernel's principles
2. The BIOS ROM's pages
3. A task's memory
4. Tasks: states, the scheduler, starting and ending, programs and drivers
5. Interrupts
6. Reaching other tasks: SCALL, KCALL, kcopy, quick looks
7. Memory: the break, pages, banks, shared segments
8. Semaphores
9. Notes
10. Time
11. Debugging support

The kernel's principles

- **The kernel is the BIOS ROM, and nothing else.** Drivers, even the kernel's own devices, are modules in the paged ROM running in tasks of their own.
- **Every task is kept apart by the hardware.** A task's `$0000-$7FFF` is its own; it reaches another task's memory only through the kernel.
- **Nothing outside the kernel changes `T`, `V` or `W`.** Programs run on BIOS page 0 (`W = 0`) always. (`base/tools/check.js` checks every module for writes to them.)
- **The kernel task is never preempted.** Its work (a KCALL) runs to its end, so the kernel's tables need no locks.
- **A server never waits.** When it can't answer yet, it says so, and the kernel makes the client wait (files).
- **Interrupts are never off for long**: no stretch of code anywhere holds them off for more than 200 cycles, since a character arrives every 320 cycles at 115200 baud.
- **Every call answers the same way**: C = 0 success, C = 1 failure with the error in `.A`.
- **One specification** (`base/spec/api.def`) makes the jump table, the SDK's includes, the languages' bindings and the reference.

The BIOS ROM's pages

`bios.cfg`, `common.s`, `far.s`

The kernel is 128K: 16 pages of 8K, all at `$E000-$FFFF`, chosen by `W`.

Page	What's there
0	What runs often: the interrupt path, the scheduler, SCALL and kcopy, the waiting side of the calls, the bring-up console, and the **jump table** at `$F800` (192 slots of 3 bytes)
1	Tasks and memory: the kernel task's side of the task calls (setting a task up, the boot's drivers), memory, notes, `TASKINFO`
2	Files, environments, pipes
3	Namespaces, `SPAWN` and the loader
4	POST
5-15	Empty: room to grow

On **every** page:

- `$E000`: the reset stub (the boot).
- `$FD00-$FDFE`: the **COMMON block**, the same bytes on every page: the interrupt entry and exit, and `K_FAR`, the kernel's far call between pages. Since changing `W` changes the code under the CPU, every page change happens in here.

- `$FDFE`: the serial chip the BIOS is built for (`BIOS_ACIA`: 0 a Rockwell R65C51, 1 a WDC W65C51N), which the paged ROM's serial drivers read.
- `$FDFF`: the page's number (POST reads it).
- `$FEFE`: two bytes that make the page's checksum (the hardware test's CRC of `$E000-$FEFF`) 0, set by the build.
- `$FFFA`: the vectors.

Page 0 is the scarce one (about 1.5K free); the others have plenty. A system call whose routine is on another page is marked `far` in `api.def`: its jump table slot goes to a 6-byte stub on page 0 that makes the far call, which costs about 100 cycles. `node tools/budget.js` shows each page's use.

A task's memory

`base/include/layout.inc`

Every task has its own `$0000-$7FFF`, laid out the same way:

Where	What
`$00`, `$01`	Its bank registers: the RAM bank at `$8000`, the paged ROM bank at `$A000`
`$02-$21`	`r0-r15`, the **call registers**: two bytes each, low first
`$22-$7F`	**The program's own zero page**: no call ever touches it
`$80-$FF`	The OS zero page: the task's state for the kernel (`TK_*`), the far call's and kcopy's pointers, the call stubs' scratch. One byte is the program's to write: its event count, `TASK_EVENT` (`$BD`), for servers
`$0100-$01FF`	Its stack. A task that isn't running has its saved frame on top
`$0200-$03FF`	The OS area (`TA_*`): a server's request being served (`TASK_INBOX`), its entries (serve, irq, note handler), its name, its copy of the interrupt lines' owners, its page and bank maps, the request it's making, its fds, its current directory, its arguments (`TASK_ARGS`, `$0350`)
`$0400-$7FFF`	**The program's**: its data and BSS, its break, and pages given out from the top down. A RAM program loads at `$0800`
`$8000-$9FFF`	The RAM bank `$00` selects: one of the task's own, or a shared one
`$A000-$DFFF`	The paged ROM bank `$01` selects: a module running in place has its bank here (its header at `$A000`, or at its page of a bank it shares)
`$E000-$FFFF`	The BIOS ROM, page 0: the jump table at `$F800`; the I/O at `$FF00`

In task F, the top of RAM is `$7EFF`: the DS1747's clock registers may be at `$7FF8-$7FFF`. The **kernel task** (task 0) keeps the kernel's state: its zero page (`K0_*`) and its RAM from `$0400` (the `K_*` tables: tasks, channels, devices, namespaces, semaphores, segments), and its own RAM banks (the tasks' environments). Every fixed address is in `layout.inc`, and nowhere else.

Tasks

`sched.s`, `task.s`, `load.s`

States

A task is in one state at a time (`TK_STATE`):

State	Meaning
`FREE`	Not in use
`NEW`	Set up, not yet run
`READY`	Runnable: the only state the scheduler runs
`WAIT`	Waiting for a `WAKE`
`SLEEP`	Waiting for a tick count
`CALL`	Calling another task (a server), which runs the call in its own task
`BLOCKED`	Waiting to call a task that's busy with another call
`EVENT`	Waiting for a server's event count to change (a read with nothing to read yet)
`IDLE`	A driver between calls

A task that's **stopped** (by the debugger, or `/proc/N/ctl`'s `stop`) keeps its state but isn't run.

Waiting is always done the same way: set the state, `YIELD`, and look again when back. A wake for nothing costs a look. A wake that comes before the wait makes the wait return at once, so nothing is lost between deciding to wait and waiting.

The scheduler

- **A task that isn't running** keeps one frame on its own stack, whatever stopped it (an interrupt, `YIELD`, a wait): `U Y W X A P PCL PCH`. A switch saves the stack pointer, picks the next task, writes `T`, loads that task's stack pointer, and unwinds its frame. No memory is copied.
- **Round robin** over tasks 1-15: each `READY` task in turn. A task runs until it waits, yields, or the tick takes the CPU away: **preemption**, 200 times a second.
- **The scheduler ends three kinds of wait itself**, as it looks for the next task: `SLEEP` once the time has come (so the tick's interrupt only counts), `BLOCKED` once the task it wants to call is free, and `EVENT` once the server's event count has changed (so an interrupt handler wakes its clients by adding 1 to a byte).
- **Task 0** runs only when nothing else can (its idle loop: `WAI`), or when interrupt handlers have queued notes for it to deliver.
- `PREEMPT_OFF` holds the CPU for timing-critical work (interrupts still run; a switch that comes due waits for `PREEMPT_ON`). Keep it short: a few ticks at most.

Starting and ending

- `SPAWN` starts a program in a task of its own: the **lowest** free task (init is task 1). It opens the program's file through the caller's namespace and reads its header:
  - **a module of the paged ROM runs in place**: its task's `$01` selects the module's bank, so its code is at `$A000` in that task alone; only its data is copied into the task's RAM, and its BSS cleared, by the task itself as it starts. Sixteen tasks can run the same module at once. A `SPAWN` of a module takes about 14 ms.
  - **any other program is a RAM program**: the new task gets the file as an fd and loads itself, at `$0800`, as it starts, so `SPAWN` doesn't wait for the load.
  - **a bank program** (`HF_BANK`) loads itself the same way, but into a RAM bank of its task's own, at `$8000` (below).
- **Drivers** take the **highest** free task, task F first. A driver runs its `init`, then is `IDLE`, running only for calls (requests) and interrupts. A driver with work of its own between its calls (a device to look at now and then, things to do when a key comes) doesn't return from its `init`: it waits in `SRV_NAP`, for a time or till its own event count changes. While it waits there it serves its calls as any driver does; when it runs again it's busy (a call waits) till its next `SRV_NAP`. So such work takes no task of its own: HydraOS's `kdev` looks at the Vera X's keyboard this way, and its `cons` hands keys to the windows with no program asking.
- **The child gets** the caller's fds 0, 1 and 2 (or a map of them), its current directory, its namespace (shared, or empty), its note group (or a new one), and a copy of its environment. Its parent is the caller.
- `EXEC` runs another program **in the calling task**, in place of the one that calls it (Plan 9's `exec`). The program is found as `SPAWN` finds it; the task keeps its number, its parent, its note group, every fd, its directory, its namespace and its environment; what was the old program's goes as at its end (its interrupt lines, segments, semaphores, served devices, note handler, pages and banks). The parent's `WAIT` gets the new program's status. rc uses it for the last command of `rc -c`, so a command typed at HyForth's prompt takes one task, not two.
- `RUN` runs a **bank program in the calling task**, as a subroutine, and waits for it: no task is made, and the program stays loaded for the next time. Bank programs, below. `SPAWN` with `SPAWN_RUN` does the same for a caller that would wait for the program at once, and gives back its own task, whose `WAIT` gets the program's status; a program that isn't a bank program starts in a task of its own as ever.
- `EXITS` ends a task with a status: a code (0 for success) and a message (31 characters). Returning from a program's entry point is `EXITS` with 0. Everything the task had is given back: its fds, pages, banks, segments, semaphores, interrupt lines.
- `WAIT` takes a child's status. The status of a child that ended first waits for its parent's `WAIT`, and its task isn't reused until then. A parent that ends first leaves its children **nobody's**: each is let go as it ends, its task free at once. (They used to go to init, which had to stay to wait for them.)
- **Init again.** `INIT_RUN` asks the kernel task to start init again in a task of its own, with a number as its argument (HydraOS's console: the window the user made); and the kernel does it itself, with `0`, when the task init was in has `EXEC`ed and then ends (the boot).

Bank programs

`run.s` *(page 5, its calls' entries on page 0)*

A **bank program** is a program built to run from a RAM bank: an o65 module (as ld65 links it, its text at `$8000`, its zero page `$22` up, its relocations kept) after a HYX2 header with `HF_BANK` set. Most of the tools in `/rom/bin` are bank programs. The kernel can run one in a task of its own, as `SPAWN` runs any program, or in the **caller's own task** (`RUN`):

- **The code cache.** The kernel keeps a task's bank programs in RAM banks of the task's own (eight at most, from `BANKS_ALLOC`), several to a bank, one above the other. Each bank's last 256 bytes are its directory: where each program is, its file's identity (`STAT`'s qid, version, length and time, so a changed file is loaded again), when it was last used, and whether it's running. A program is put in the lowest gap it fits; with none, in a new bank; with no bank to be had, the program used least lately is given up, and the next, till it fits. The cache goes with the task (at its end, or an `EXEC`).
- **Loading.** The first time, the program is read into its place and relocated there: its code and data by where it went, its zero page by where that goes. A program placed where it was linked, at a bank's start, isn't relocated at all; in a task of its own it's read in one piece, its text landing where it was linked (its file's `HX_LENGTH`, the build's, says how much). A copy of its data is kept, so each run starts with its data as it was loaded and its BSS cleared, as a new load's would.
- **Running in place.** The program runs on the caller's stack, with its bank at `$8000` and its zero page where the caller's program's is (the caller's bytes kept and put back). It gets what `SPAWN` would give it: its arguments at `r0`, the caller's fds 0, 1 and 2 or a map of them, no note handler. With `RUN_LEND` it may move the break past the caller's (its memory for `BREAK`; put back after), so a caller says whether the RAM past its break is free. It ends as any program does, returning or `EXITS` or a note's default action (Ctrl-C), and then everything is the caller's again: its fds (the program's closed), note handler, zero page, banks and break. A note that ended the program (but a kill) is posted again, so the caller's own handler sees it. A program running in place can `RUN` another.
- **What can't run in place**: a program marked `HF_TASK` (one that changes what's the caller's, or needs more RAM than a caller can lend) runs only in a task of its own; and a program that isn't a bank program at all (a RAM program, a module) is `SPAWN`ed.

Children are bank programs of a program's own: modules whose functions the program calls itself, on its own C stack (`sdk/c/include/bankprog.h`; Dingus Quest Zero's pieces). `BANK_LOAD` puts one in the same code cache, its imports found by name in the caller's exports file (`NAME.exp`, the build's), its zero page where the caller says, and tells the caller its bank and directory entry; the caller maps the bank (`$00` alone), calls the entry and maps its own bank back, checking first that the entry is still the same load. As it loads a child the kernel borrows a RAM bank of the task's, the **loader bank**: the child's imports' names and addresses (each export looked for through hash chains), and the child's file and the exports' file read 2K at a time; it's given back at the load's end. Only the child's exports' table stays past its BSS, so a child's code, data and BSS may fill a bank's 7.5K. `BANK_FLUSH` empties the cache (and sets how many banks it may have).

Interrupts

`irq.s`, `common.s`

There's **one path for every interrupt line**:

1. The line's entry in the vector RAM points to its stub in the COMMON block, the same on every page.
2. The stub saves `W` and goes to the dispatcher on page 0, with `.A` = the line, and the interrupted task's frame begun on its own stack.
3. The dispatcher finds the line's **owner** and switches `T` to it: the owner's interrupt entry runs **in its own task**, with its zero page, its stack and its banks (its module at `$A000`), with `.A` = the line and `.X` = the task that was interrupted.
4. The entry returns `.A = 0`, or `IRQ_RESCHED` to ask for a task switch (the tick always asks). The interrupted task is switched out, unless it holds preemption.

The rules for an interrupt entry:

- It runs with interrupts off, and **never waits**.
- It has **about 85 cycles**: the whole path takes about 115 of the 200 that interrupts may be held off.
- It wakes its clients by **adding 1 to its event count** (`inc TASK_EVENT`), never with `WAKE`, and sends a note to a note group with `NOTE_QUEUE` (the console's Ctrl-C).
- A line must be owned (`IRQ_OWN`) **before** its device interrupts: a line nobody owns is counted and ignored, and, being level-triggered, comes straight back.

The lines:

Line	Owner	Notes
0	The kernel	The VIA: timer 1 is the tick. The stub checks the **ACIA first**, since at 115200 a byte can't wait behind the tick
1	The serial driver (`ser` or `cons`)	The ACIA
2	`vid` (HydraOS)	Slot 0's A: the Vera X's VSYNC
3	Free	Slot 0's B
4	Free	The YM2151: the sound driver doesn't use its interrupt (songs are timed by the tick)
5-14	Free	The other slots

Line	Owner	Notes
15	(none)	No hardware: the vector entry `BRK` uses
16	The serial driver	**VIA timer 2**, a line of its own: owning it is owning the timer (the serial port's paced sending)
17	`kdev` (HydraOS), while `/dev/gpio/ca1` is open	**VIA CA1**, a line of its own

`BRK` comes through the same path as line 15's: a `BRK` in a program is the note `NOTE_BRK` to that program (or, under the debugger, a breakpoint).

Reaching other tasks

`call.s`, `kdefs.inc`

Tasks are kept apart, but the kernel has to reach across. It does so in four ways:

- **SCALL** runs a task's **serve entry in that task**: its zero page, stack and banks. It's how a request reaches a server: the client is `CALL` meanwhile, and the server `READY`, so the call can be preempted like any code. A task serves one call at a time; a second caller is `BLOCKED` until it's free.
- **KCALL** is an SCALL into the kernel task, to a kernel routine. Since the kernel task is never preempted, KCALLs run one at a time, to their ends, and the kernel's tables need no locks.
- **kcopy** copies bytes between this task's memory and another's, each side as its own task sees it (its RAM, its bank at `$8000`, its ROM bank at `$A000`): `T` switched for each byte's read and write, interrupts off for 4 bytes at a time. `CLIENT_READ` and `CLIENT_WRITE` are kcopy.
- **Quick looks**: a moment in another task's memory with `T` switched and interrupts off, no stack used (the stack page changes with `T`), then back. For a byte or two of another task's state.

`T` is written only by quick looks, the interrupt path, the scheduler, SCALL and kcopy.

Memory

`mem.s`

The kernel manages memory in **pages** (256 bytes) and **banks** (8K). Finer allocation (`malloc`, Forth's `ALLOT`, hylang's heap) is each language's own.

A task's RAM:

- `BREAK` moves the end of the program's data (its break), as Unix's `brk`: up from where its BSS ends.
- `PAGES_ALLOC` gives a run of pages from the **top** of RAM down; `PAGES_FREE` gives them back. The break can't rise over a page given out.

Banks: each task has 16 banks of 8K **for each memory card** (module) installed: module m's are `$m0-$mF`, and they're the task's alone.

- `BANKS` says how many, and which modules are good.
- `BANKS_ALLOC` gives a run from the lowest free (`BANKS_ALLOC_IN` from a range), `BANKS_FREE` gives them back. The bitmap is only so that libraries in one program don't collide: a program **selects a bank by writing** `$00`, and reads and writes it at `$8000-$9FFF`.

```
        lda     #4
        jsr     BANKS_ALLOC                     ; Four banks: .A = the first
        bcs     @nomem
        sta     $00                             ; The first at $8000
        lda     #$5A
        sta     $8000
```

Shared segments: a run of shared banks that several tasks attach to.

- `SEG_CREATE` makes one of `.A` banks, attached to the caller; `SEG_ATTACH` and `SEG_DETACH` count tasks in and out; the last out (or the last to end) frees it.
- `SEG_MAP` says how to reach bank `.X` of it: `.A` for `U` (`$FFF1`) and `.X` for `$00`.
- Segments use the shared bank IDs `$80-$FF`; `$00-$7F` are HydraOS's shared RAM disk's.
- To find a segment by name, `kdev` keeps names for them: `name NAME N` written to `/dev/seg/ctl`, and any task reads `/dev/seg/NAME` for its number.

```
        lda     seg
        ldx     #0                              ; Its first bank
        jsr     SEG_MAP
        sta     $FFF1                           ; U: the shared macro-page
        stx     $00                             ; Then the bank: at $8000
```

Memory POST found bad is never given out.

Semaphores

`sem.s`

The kernel keeps **16 semaphores**, every task's by number. Each is a count and the tasks waiting on it.

Call	What it does
`SEM_NEW`	Make one: `.A` its count; `.X` 0, or `SEM_MUTEX` for a mutex (a count of 1, given back only by the task that took it)
`SEM_ACQUIRE`	Take one of its count, waiting (using no CPU) until there's one. A note ends the wait (`E_INTR`)
`SEM_TRY`	The same without the wait: `E_AGAIN` if there's none
`SEM_RELEASE`	Give one back; the waiters wake, and the first to run takes it
`SEM_WAIT`	`SEM_ACQUIRE` with a time limit (`.X` ticks): `E_AGAIN` if the time comes first; a release ends the wait at once
`SEM_FREE`	Free it; its waiters wake with `E_INVAL`

A task's end frees the semaphores it made and gives back the mutexes it holds. A mutex's holder that asks again gets `E_BUSY`, not a wait for ever.

Notes

`notes.s`

Notes are Plan 9's signals, numbered:

Note	Name	Default	Sent by
1	`interrupt`	ends the task, status 130	Ctrl-C at the console, to the window's note group
2	`kill`	ends the task, status 137; **never** the handler's	Ctrl-\, `kill`, `/proc/N/ctl`'s `kill`
3	`hangup`	ends the task	
4	`alarm`	ends the task	
5	`brk`	ends the task	a `BRK` instruction in the program
16-31	a program's own	ends the task	

- `NOTE` sends one to a task, or to a **note group** (`NOTE_GROUP | g`): every task in it. A child is in its parent's group unless `SPAWN_NEWGROUP` gives it one of its own.
- **A task takes its notes in its own code, never inside the kernel**: when the scheduler switches to it and its frame is in its own code, or at the end of a system call that waits, which then returns `E_INTR`.
- `NOTIFY` sets a handler: it's called with `.A` = the note, and returns C = 0 to go on where the task was, or C = 1 for the default. With no handler, the default ends the task with 128 + the note's Unix number.
- A note that comes while the handler runs waits for it (except a kill).
- A program that runs code in more than one bank keeps its handler in its RAM: the kernel calls it with whatever bank is at `$A000` when the note comes.

Time

`sched.s`, `time.s`

- **The tick**: 200 a second (`TICK_HZ`), from VIA timer 1. `TICKS` is its count, 16 bits (it wraps after about 5.5 minutes).
- `SLEEP` waits a number of ticks; `SLEEP_UNTIL` until a tick count, for steady timing; `YIELD` lets the others run.

- **The clock**: `TIME` is seconds since 2000-01-01 (in `r0/r1`), kept as the boot's time plus the ticks since. `TIME_SET` sets it. With a DS1747 in U7, `RTC` reads and sets the chip, and kdev sets the clock from it at boot (the devices).

Debugging support

`debug.s`, `info.s`

- `TASKINFO` says what the kernel knows of a task: its state, flags, parent, CPU time, name, note group. `TASKREAD` reads its arguments, current directory, environment, saved registers, or an open fd. `ps` and `/proc` are built on them.
- `TASKSTOP` stops a task where it is, or starts it again.
- `TASKSTEP` runs one instruction of a stopped task **out of line**: a copy of the instruction in the task's own zero page with a `BRK` after it, run as the task, which stops it again. `JMP`, `JSR`, `RTS` and `RTI` it does itself, on the task's frame; a `JSR` into the kernel always runs whole. So a step needs no trace flag and patches no ROM.
- **Breakpoints** are the debugger's: a `BRK` written into the program's RAM. With breakpoints on, any `BRK` stops the task instead of sending it the `brk` note.
- `SPAWN_STOPPED` starts a program stopped at its entry point.

These are a driver's calls (kdev's): programs use them through `/proc/N/ctl` (files), and `db`, the debugger, is built on that (the tools).

Files, servers and namespaces

On the Hydra, as in Plan 9, **everything is a file**: a disk's files, the console, the serial port, the sound chip, another task's memory, a pipe, the environment. A program uses them all with the same few calls (`OPEN`, `READ`, `WRITE`, `CLOSE`), and controls a device by writing text commands to its `ctl` file. This page is the model: what a file is to the kernel, who serves it, how names find it, and the kernel's own devices. The code is `base/kernel/file.s`, `ns.s` and `env.s`, and `base/modules/kdev`.

Contents

1. Fds and channels
2. Servers and requests
3. Waiting
4. Names
5. Namespaces: binds, mounts and unions
6. The environment
7. The kernel's own devices: kdev
8. Every device letter

Fds and channels

A task has **16 fds**: 0 is its standard input, 1 its standard output, 2 its standard error. Each fd names a **channel**, an open file in the kernel task's table (48 of them): its server, the server's handle for the file (its **fid**), the mode it was opened with, and its **offset**.

- `DUP`, `DUP2`, and a child's inheritance give another fd for the same channel, so they share the offset (as Unix's do).
- A child gets its parent's fds 0, 1 and 2, or with `SPAWN_FDMAP` whichever the parent chooses.
- `CLOSE` closes an fd; the channel closes when no fd names it.

Call	What it does
`OPEN`	`r0` a path, `.A` a mode (`O_READ`, `O_WRITE`, `O_RDWR`, with `O_TRUNC`, `O_NONBLOCK`): `.A` an fd
`CREATE`	A file made (or emptied) and opened; `.X` = `DM_DIR` makes a directory
`READ`, `WRITE`	`.A` an fd, `r0` a buffer, `r1` a count: `.A`/`.X` the count done. `READ` gives 0 at the end of the file
`SEEK`	An fd's offset (`r0`/`r1`: 32 bits), from the start, the offset, or the end
`CLOSE`, `DUP`, `DUP2`	As Unix's
`STAT`, `FSTAT`	A file's **stat record**: 64 bytes (`SR_*`): its name, qid, mode, length, time, device
`WSTAT`, `FWSTAT`	Change a file from a stat record: rename it, change its mode or length (fields left all `$FF` stay)
`REMOVE`	A file, or an empty directory
`PIPE`	Two fds, a pipe's ends: `.A` to read, `.X` to write (512 bytes held between them)
`FD2PATH`	The name an fd was opened by, whole and clean
`CHDIR`, `GETCWD`	The current directory

A directory reads as stat records, one after another (`SR_SIZE` each): there's no text to take apart. `ls` reads a directory and prints each record.

`PUTC`, `PUTS` and `GETC` are a byte or a string written to fd 1, and a byte read from fd 0. A task with no fds (the kernel, a driver) gets the bring-up console instead: the serial port, polled.

Servers and requests

Every file is served by a **server**: a task that has registered a **device letter** (`SRV_REGISTER`) and answers requests for it. A **driver** is a server that's a module of the paged ROM, started in a task of its own.

When a program calls `OPEN`, `READ` or any file call, the kernel:

1. fills a **request block** (`RQ_*`, 32 bytes) in the program's own OS area: the request (`R_OPEN`, `R_READ`, `R_WRITE`, `R_STAT`, `R_CLUNK` ...), the fid, the mode, the offset, the count, the buffer's address, the client's task and note group;

2. **SCALLs** the server: the server's serve entry runs in the server's own task, while the client waits in its call;
3. the server takes the request into its `TASK_INBOX` (`SRV_TAKE`), moves the data between its memory and the client's buffer with `CLIENT_READ` and `CLIENT_WRITE` (kcopy: each side as its own task sees it), and answers (`SRV_REPLY`): C and `.A`, the count done, a new fid.

`READ` and `WRITE` ask for 512 bytes (`IO_UNIT`) at most a request. A `READ` goes on until its count is done, the file ends, or a request comes back short; a `WRITE` until its count is done.

srvlib (`base/sdk/asm/srvlib.inc` and `srvlib.s`) does nearly all of a server's work: it keeps the fids, makes stat records, walks a tree of files described as a table, makes text files' contents on each read, and parses commands written to `ctl` files. A server is mostly that table and a routine for each file (servers and drivers).

Control is text. A device is set by writing a command to its `ctl` file, its words after it (`echo b115200 >/dev/serctl`), and read back as text (`cat /dev/serctl`). A bad command is `E_INVAL` and changes nothing.

Waiting

A server never waits. If it can't answer yet (a pipe is empty, a console has no line), it answers `E_AGAIN`, and the kernel has the client wait:

- each server has an **event count**, a byte in its zero page (`TASK_EVENT`);
- the client waits (`EVENT` state, using no CPU) until the server's event count changes from what it was when the server took the request, or a `WAKE`; then the kernel sends the request again;
- the server adds 1 to its event count whenever something its clients may be waiting for happens: a byte came in, there's room to send. Its interrupt entry can do this in a single instruction (`inc TASK_EVENT`).

A note ends a client's wait: the server gets `R_FLUSH`, and the client's call returns `E_INTR`. An fd opened with `O_NONBLOCK` gets `E_AGAIN` instead of waiting.

So a server stays simple (one request at a time, never blocked), a slow device never holds up the system, and Ctrl-C always works.

Names

A name is made **whole and clean** before it's looked up: a relative name is put after the current directory, then `.`, `..` and empty elements are taken out. `cd /sd/0/src; cat ../readme` reads `/sd/0/readme`.

A name that starts with `#` is a device's own: `#c/cons` is the console server's `cons`, found in no namespace. What follows the letter, up to the `/`, is its **spec**, which picks one of the device's trees: `#c2/cons` is the console's window 2, `#f` with spec `x` the ROM disk's file system.

Any other name is found in the task's **namespace**.

Namespaces

A namespace is a table of **mount entries** (`ns.s`): each says that a device's tree (a letter, a spec and a path in it) appears at a name, a **mount point**. Several entries at one mount point make a **union**, in order.

Looking a name up: its mount point is the longest one it starts with, in whole elements. The union's members are its **candidates**, each with the rest of the name: `OPEN`, `STAT` and `REMOVE` try them in turn until one has the name. `CREATE` goes to the member marked for creating (`MCREATE`), or the first.

A **directory at a union** reads as every member's records, one member after another: `ls /bin` lists the programs from every place bound there.

Call	What it does
`BIND`	`r0` new at `r1` old: `MREPL` in place of what's there, `MBEFORE` or `MAFTER` in its union; `| MCREATE` to take creates
`MOUNT`	A device's tree (`.X` its letter, `r0` its spec) at old, with the same flags
`UNMOUNT`	All of old, or its member new
`NSINFO`	One entry of a task's namespace (`/proc/N/ns` reads them as `bind` and `mount` lines)

A bind is resolved when it's made, as in Plan 9: `bind /rom/bin /bin` binds whatever `/rom/bin` is at that moment.

Who shares a namespace: a child shares its parent's until either changes it, which gives the changer a copy. `SPAWN_NEWNS` gives a child an empty one. So a shell can change its own view of the files (bind a directory over `/bin`, mount a card) without changing anyone else's.

In the base, the monitor needs no namespace: it opens `#c/cons` by its device name. HydraOS's init builds the system's namespace from a file, `/rom/lib/namespace`, and so does each new shell (rc):

```
bind '#/' /
bind -a '#c' /dev           # cons consctl ser serctl
bind -a '#n' /dev           # null zero kmesg
bind '#d' /dev/sd
mount '#e' /env             # the environment
mount '#p' /proc
mount '#f' /sd              # the cards: /sd/0 ... /sd/f
mount '#f' /rom x           # the ROM disk
mount '#f' /sram s          # the shared RAM disk
mount -c '#f' /ram r/$task  # this shell's own area of the RAM disk
bind -c /ram/bin /bin
bind -a /sram/bin /bin
bind -a /sd/0/bin /bin      # (added only if the card has one)
bind -a /rom/bin /bin       # programs on the ROM disk
bind -a '#m/bin' /bin       # the ROM's program modules
...
```

The environment

`env.s`

Each task has its own **environment**: 8K of variables, each a name and a value of any bytes, kept in a RAM bank of the kernel task's. `SPAWN` gives a child a copy of its parent's (`SPAWN_NOENV`: an empty one).

Call	What it does
`ENV_GET`	A variable's value, or part of it, into a buffer
`ENV_PUT`	Set a variable, or part of its value
`ENV_DEL`	Remove one
`ENV_NAME`	The variables' names, one at a time

Each takes a task's number (`$FF`: this task's), so a program can read another's. kdev serves the caller's environment as files, `#e`, mounted at `/env` (Plan 9's): `cat /env/home`, `echo hi >/env/greeting`. rc keeps its variables there (a list's words each ending with a zero byte) and its functions as `fn#NAME`; C's `getenv`, HyForth's `getenv` and hylang's `$name` read them. With no memory card, there are no environments.

The kernel's own devices

The kernel's devices are served by **kdev**, a boot driver like any other (task E in the base, D in HydraOS), not by the kernel task: the kernel keeps only tables.

Letter	Bound at	Files
`#/`	`/`	The root: an empty directory for each mount point the default namespace uses (`bin`, `dev`, `env`, `lib`, `mnt`, `pc`, `proc`, `ram`, `rom`, `sd`, `sram`, `tmp`; and in `dev`: `gpio`, `i2c`, `mod`, `sd`, `seg`, `spi`, `vid`), so `ls /` shows them
`#n`	`/dev`	`null` (reads as nothing, takes every write), `zero` (reads as zeros), `kmesg` (the kernel's messages: the boot)
`#t`	`/dev`	`ticks` (the tick count, in decimal), `time` (the date and time, as text: write one to set the clock, and the DS1747 if there's one), `rtc` (the DS1747's registers)
`#m`	`/dev/mod`, and `#m/bin` at `/bin`	A file for each module of the paged ROM, its image (`SPAWN` reads its header); `bin`, the programs alone
`#p`	`/proc`	A directory for each task (below)
`#|`		Pipes: opening `pipe` makes one; `PIPE` makes both ends. 512 bytes each, 8 at once
`#e`	`/env`	The asking task's environment, a file a variable

Letter	Bound at	Files
`#s`	`/dev/seg`	Shared segments by name: `ctl` (`name NAME N`, `free NAME`), and a file each name, read as its segment's number and banks
`#r`	(init's alone)	Raw RAM, read only: `task` (every task's RAM, task t's at `t * $8000`) and `shared` (the shared banks)

/proc

`/proc/N` is task N, as files:

File	What it is
`status`	Its name, state, parent, CPU time in ticks, and note group
`args`	Its arguments
`cwd`	Its current directory
`env`	Its environment
`fd`	Its open files
`ns`	Its namespace, as `bind` and `mount` lines
`regs`	Its saved registers
`mem`	Its 64K as it sees it, read and written as a file (any task's but the kernel's and a driver's)
`ram`	Its banks: bank b's byte o at `b * $2000 + o`
`note`	Write a note's name or number to send it
`ctl`	Commands: `kill`, `interrupt`, `note N`, `stop`, `start`, `step`, `next`, `break`, `nobreak`

`ps` and `top` read `status`; `kill` and `slay` write `ctl`; `db`, the debugger, uses `mem`, `regs` and `ctl`. A task's memory is reachable **only** this way: the kernel's `TASKMEM` serves only kdev.

Every device letter

The base's devices are kdev's and the console's. HydraOS adds the rest (chapter 4):

Letter	Device	Server	Bound at
`#/`	The root	kdev	`/`
`#c`	The console (`#cN`: window N)	`ser` (base), `cons` (HydraOS)	`/dev`
`#P`	`/pc`: a folder on the PC	`cons`	`/pc`
`#n`, `#t`	null, zero, kmesg; ticks, time, rtc	kdev	`/dev`
`#m`	The paged ROM's modules	kdev	`/dev/mod`, `/bin`
`#p`	The tasks	kdev	`/proc`
`#|`	Pipes	kdev	
`#e`	The environment	kdev	`/env`
`#s`	Shared segments' names	kdev	`/dev/seg`
`#r`	Raw RAM	kdev	(init's)
`#d`	The disks	`storage`	`/dev/sd`
`#f`	HydraFS, a disk's file system	`storage`	`/sd`, `/rom`, `/ram`, `/sram`
`#S`	The SPI devices	`storage`	`/dev/spi`
`#g`, `#i`	GPIO, I2C	`kdev` (HydraOS's)	`/dev/gpio`, `/dev/i2c`
`#a`	Sound	`snd`	`/dev`
`#v`	The Vera X	`vid`	`/dev/vid`

The system calls

How a program asks the kernel for something: the jump table, the registers, how errors come back, what each group of calls is for, and how to make a call from each language. The call reference lists every call in full.

Contents

1. The jump table
2. Registers
3. Errors
4. The groups
5. A program, walked through
6. From each language
7. Stability: the ABI

The jump table

A program calls the system with `jsr` **to the call's slot** in the jump table, on BIOS ROM page 0 from `$F800`:

```
            jsr         OPEN                            ; ($F8C0)
```

Each slot is 3 bytes, a `jmp` to the kernel's routine (or, for a call whose routine is on another page, to a stub that makes the far call). The names and addresses are in `hydra.inc` for assembly and `hydracalls.h` for C, both made from `base/spec/api.def` by the build.

Programs always run with BIOS page 0 in place (`W = 0`), so the table is always there.

Registers

Where	What
`.A`, `.X`, `.Y`	Small arguments and results. A 16-bit value is `.A` (low) and `.X` (high)
`r0-r15` (`$02-$21`)	The **call registers**: pointers and 16-bit values, two bytes each, low first. A 32-bit value is two of them, `r0/r1` (`r0` the low half)
C	**0: success. 1: failure, with the error code in `.A`.** Always, for every call
`$22-$7F`	The program's own zero page: no call touches it

A call may change `.A`, `.X`, `.Y`, `r0-r15` and the flags: keep what you need in `$22-$7F` or in your RAM. Strings are zero-terminated, passed by address in a call register.

Errors

A call that fails returns **C = 1 and an error code in `.A`**: `E_NOENT` (`$20`, "not found"), `E_INTR` (`$05`, "interrupted"), `E_AGAIN` (`$04`, "not yet: try again") and so on, from `base/spec/errors.def`. `ERRSTR` turns a code into its text. The usual pattern:

```
            jsr         OPEN
            bcs         @error                          ; .A = the error code
            sta         fd
            ...
@error:
            LDR         r0, buffer                      ; Its text, for a message
            jsr         ERRSTR
```

Three errors come from the kernel's own machinery, not the call's purpose:

- `E_INTR`: a note came while the call waited. The call stopped, and the note is taken (its handler runs, or the default ends the task) as the call returns.

- `E_AGAIN`: the file was opened `O_NONBLOCK` and there's nothing yet.
- `E_NOSYS`: no such call (a slot not yet used, or withdrawn).

The reference has every code.

The groups

The table is in groups, each with room to grow:

Group	Slots	What's there
system	`$F800`, 16	`SYSINFO` (the ABI, the memory cards, the free tasks), `ERRSTR`, `KMESG` (the kernel's messages), `REBOOT`, `XCALL` (call a library in another bank)
task	`$F830`, 32	`SPAWN`, `EXEC`, `RUN`, `EXITS`, `WAIT`, `GETPID`, `GETPPID`, `YIELD`, `SLEEP`, `SLEEP_UNTIL`, `TICKS`, `PREEMPT_OFF/ON`, `TASKINFO`, `TASKREAD`, `NOTIFY`, `NOTE`, `MODINFO`, the environment (`ENV_GET`, `ENV_PUT`, `ENV_DEL`, `ENV_NAME`), semaphores (`SEM_NEW`, `SEM_ACQUIRE`, `SEM_TRY`, `SEM_RELEASE`, `SEM_FREE`, `SEM_WAIT`)
memory	`$F890`, 16	`BREAK`, `PAGES_ALLOC/FREE`, `BANKS`, `BANKS_ALLOC/FREE/ALLOC_IN`, shared segments (`SEG_CREATE`, `SEG_ATTACH`, `SEG_DETACH`, `SEG_MAP`, `SEGINFO`, `SEG_CREATE_IN`), the code cache (`BANK_LOAD`, `BANK_FLUSH`)
file	`$F8C0`, 32	`OPEN`, `CREATE`, `CLOSE`, `READ`, `WRITE`, `SEEK`, `STAT`, `FSTAT`, `WSTAT`, `FWSTAT`, `REMOVE`, `DUP`, `DUP2`, `CHDIR`, `GETCWD`, `PIPE`, `FD2PATH`
name	`$F920`, 16	`BIND`, `MOUNT`, `UNMOUNT` (of one thing, or of everything: an empty namespace), `NSINFO`
cons	`$F950`, 16	`PUTC`, `PUTS`, `GETC`, `PUTHEX`: a byte or a string to fd 1, a byte from fd 0
time	`$F980`, 16	`TIME`, `TIME_SET`, `RTC`
server	`$F9B0`, 32	For drivers: `SRV_REGISTER`, `SRV_TAKE`, `SRV_REPLY`, `CLIENT_READ`, `CLIENT_WRITE`, `IRQ_OWN`, `IRQ_RELEASE`, `WAKE`, `PAUSE`, `NOTE_POST`, `NOTE_QUEUE`, `ROMREAD`, `TASKMEM`, `TASKSTOP`, `TASKSTEP`, `SRV_NAP` (a driver's wait between turns of its own work), `INIT_RUN` (init started again)
dbg	`$FA10`, 16	Debugging and tests: unstable, may change in any version

The kernel and files pages explain what the calls do; the reference has each one's registers.

A program, walked through

`sdk/asm/samples/hi/hi.s`, a program that greets each of its arguments, then says its task and directory:

```
.include "hydra.inc"                                ; The calls and constants (made from spec/api.def)
.include "hyx2.inc"                                 ; The header: HYX2_PROGRAM
.include "macros.inc"                               ; LDR, MOVR, PRINT, CALL, CHECK

            HYX2_PROGRAM "hi", main                 ; Its name, and where it starts

.zeropage
arg:        .res        2                           ; An argument (its zero page: $22-$7F)

.bss
cwd:        .res        PATH_MAX + 1                ; Its current directory
```

`main` gets its arguments at `r0`: strings one after another, each with a zero byte, and an empty one after the last (the program's name isn't one of them). For each it prints `Hello, NAME!`:

```
@name:
            PRINT       "Hello, "                   ; "Hello, NAME!"
            MOVR        r0, arg
            CALL        PUTS
            PRINT       s_bang
```

then its task and its directory:

```
        CALL    GETPID                  ; .A = this task (0-15)
        ...
        LDR     r0, cwd
        CALL    GETCWD                  ; Its current directory, into cwd
        PRINT   cwd
```

and a variable of its environment, which may not be there:

```
        LDR     r0, s_window            ; Its window, if its environment has one
        LDR     r1, window
        LDR     r2, 7
        stz     r3
        stz     r3 + 1
        lda     #$FF                    ; (This task's)
        CALL    ENV_GET
        CHECK   @end                    ; (Not there: C = 1, .A = E_NOENT)
```

`rts` from `main` ends it with code 0; `EXITS` ends it anywhere, with a code and a message: its **status**, which its parent gets from `WAIT` (and rc shows as `$status`).

The SDK's macros (`macros.inc`):

Macro	What it does
`CALL name`	A system call (the same as `jsr name`, but it reads as one)
`CHECK label`	On to `label` if the call before it failed (C = 1)
`LDR reg, value`	A call register = a 16-bit value (an address, a number)
`MOVR to, from`	One call register (or zero-page word) = another
`PRINT label`, `PRINT "text"`	A string to fd 1

None defines a label (but unnamed ones), so the code around them keeps its cheap locals (`@name`).

From each language

Every call a program makes is reachable from every language on the Hydra:

Language	How	Example
Assembly	`jsr NAME`, with `hydra.inc`	`jsr GETPID`
C	`hy_call (HY_NAME, &regs)`, or `hydra.h`'s wrappers, or the standard library over them (`open`, `read`, `printf`)	`hy_task ()`
HyForth	`sys-name`, its registers on the stack (`hydra.fl`)	`sys-getpid .`
hylang	`(sys-name ...)`, its arguments by kind	`(sys-getpid)`
HyBASIC	`SYS "NAME", a, x, y`: `r0`-`r15` POKEd first (bytes 2-33); `RREG a, x, y, p` reads the registers after	`SYS "GETPID": RREG A, X, Y, P`

The reference shows each call's HyForth and hylang forms; the C guide how C reaches them.

Stability: the ABI

The calls are the kernel's promise to programs. **A slot never moves once it's published**: a new call is only ever added at the end of its group, and a withdrawn call keeps its slot and answers `E_NOSYS`. So a program built against `hydra.inc` today runs on later kernels. The banner and `SYSINFO` give the **ABI version** (1); it changes only if that promise has to be broken.

The `dbg` group is excepted: its calls are for the tests and may change in any version.

The monitor

When the base runs on its own, its init (task 1) is **wozmon**: Steve Wozniak's monitor from the Apple 1, as the Hydra-16's first software had it, with a disassembler added. It's a way to look at memory, change it, and run what's there, from the serial console: enough to bring up a new board, try an instruction, or poke a device's registers.

It's a program on the kernel's calls (`base/modules/wozmon/wozmon.s`), not code in the BIOS ROM, so it sees the machine as any task does, and a program it runs can call the kernel.

Contents

1. The prompt and the commands
2. A session
3. What it sees
4. Writing a program with it
5. Without a console driver

The prompt and the commands

```
T1 00>
```

The prompt is the task (`T1`), the RAM bank at `$8000` (`00`; a shared bank, `$F0` up, shows its `U` too: `T1 F2(3)>`), and `>`. A line holds any number of commands, as Woz's did, separated by spaces; hex digits in upper or lower case:

Command	What it does
`XXXX`	**Examine**: the byte at `XXXX`, on a line of its own (`XXXX: BB`)
`XXXX.YYYY`	**A block**: `XXXX` to `YYYY`, 8 bytes a line. `.YYYY` alone goes on from where the last stopped
`XXXX: BB BB ...`	**Store**: the bytes from `XXXX` on (it shows `XXXX`'s old byte first). `: BB` alone goes on after the last stored
`XXXXR`	**Run**: a `JSR` to `XXXX` (`R` alone: the last address shown). The program's `RTS` comes back to the prompt
`L`	From now on, an examine shows **instructions**: one a line, its bytes and its text as the assembler `as` writes it (`lda #$41`)
`K`	Back to bytes

Keys: Backspace (or Delete) takes back a character; Escape gives up the line (Woz's `\`). **Ctrl-C** stops whatever is running, a program started with `R` too, and brings the prompt back; so does any other note (a `BRK` in a program is the note `brk`).

A session

Store a six-byte program at `$1000`, list it as instructions, run it, then look at the VIA's registers and the start of the jump table (captured from the emulator):

```
T1 00> 1000: A9 48 20 50 F9 60
1000: 2E
T1 00> L 1000.1005
1000: A9 48     lda #$48
1002: 20 50 F9  jsr $F950
1005: 60        rts
T1 00> 1000R
1000: A9 48     lda #$48H
T1 00> K FF00.FF0F
FF00: FF FF 00 00 04 21 E8 45
FF08: 0F 03 00 40 00 00 E0 FF
T1 00> F800.F80F
F800: 4C 77 EB 4C B0 EB 4C FD
F808: ED 4C 2B EB 4C 4C EB 4C
```

- The store showed `$1000`'s old byte (`2E`: RAM powers up random).
- `$F950` is `PUTC`, so the program prints `H` (`$48`) and returns. `R` showed the address it ran first, as an instruction since `L` was on.
- `FF00.FF0F` is the VIA, port 0: its timers counting.
- `F800` is the jump table: a `jmp` (`4C`) for each call.

What it sees

Task 1's view of the machine, as any task's:

- **RAM**, `$0000-$7FFF`: the task's own.
  - `$02-$21` are the call registers `r0-r15`, and `$22-$7F` the monitor's own zero page.
  - `$80-$FF` and `$0200-$03FF` are the kernel's areas for the task: don't write them.
  - `$0100-$01FF` is the stack, and the monitor's RAM starts at `$0400` (a few hundred bytes).
  - `$1000` **up is free** for programs.
- **The RAM bank** at `$8000-$9FFF`: the one `$00` selects. `0: 05` puts bank 5 there; `$F0-$FF` are shared banks (with `U` at `$FFF1`).
- **The paged ROM's bank** at `$A000-$DFFF`: the one `$01` selects, which is the monitor's own module, since it runs from there. Changing `$01` pulls the monitor out from under itself; a program run with `R` may change it if it puts it back before it returns.
- **The BIOS ROM's page 0** at `$E000-$FFFF`, and the I/O at `$FF00-$FFEF`. Examining a device's register reads it, with whatever that does (reading the ACIA's data register takes a byte).

`T`, `V` and `W` belong to the kernel: changing them from the monitor will stop the system.

Writing a program with it

A program put in RAM with `:` and run with `R` is an ordinary subroutine in task 1: it can call any system call with `jsr` (the addresses are in `base/sdk/asm/hydra.inc`, and in the call reference), and ends with `RTS`. For anything longer than a few bytes, write it with the assembler and load it as a program (assembly); on HydraOS, `as` assembles on the Hydra itself.

`L` shows any code as instructions, the kernel's and the ROM modules' too: `E000.E00F` with `L` on is the reset stub.

Without a console driver

With no task F (a paged ROM without `ser`), the monitor uses the kernel's bring-up console: the serial port, polled. Everything works but Ctrl-C (nothing catches it), which is useful when bringing up a new board or a new console driver.

4. The drivers

A **driver** is a module of the paged ROM that serves a device as files, in a task of its own. This chapter is each of them, from the serial port, which everything else depends on, to the Vera X: what files each serves, what you can do with them from the shell, and how each works inside. Chapter 3 explained the file model they all follow; servers and drivers is how to write one.

Contents

1. The serial console: `ser` (the base's) and `cons` (HydraOS's): the console, its line editor and raw mode, the windows, the serial port, `/pc`.
2. The kernel's devices: `kdev`: the modules, `/proc`, pipes, the environment, shared segments' names, the clock; and in HydraOS, in the same task, GPIO, I2C and the Vera X's keyboard and mouse.
3. Storage: `storage`: the SPI bus, SD cards, the RAM disks, the ROM disk, and HydraFS.
4. Sound: `snd`: the YM2151 and the Vera X's PSG as one synthesizer, and the PCM.
5. GPIO, I2C and SPI: the VIA's port A, its pins, the I2C bus (`kdev`'s); the SPI devices as files.
6. Video: `vid`: the Vera X's screen, its files, drawing, claiming the chip; and the keyboard and the mouse.

The drivers at boot

The boot starts each boot driver in a task of its own, from task F down, in the order of the paged ROM's module directory (the boot):

Task	Base	HydraOS	Devices	Bound at
F	`ser`	`cons`	`#c` the console (and `#P`, `/pc`, in HydraOS)	`/dev`, `/pc`
E	`kdev`	`storage`	`#S` SPI, `#d` disks, `#f` HydraFS	`/dev/spi`, `/dev/sd`, `/sd`, `/rom`, `/ram`, `/sram`
D		`kdev`	`#/`, `#n`, `#t`, `#m`, `#p`, `#|`, `#e`, `#s`, `#r`; `#g` GPIO, `#i` I2C	`/`, `/dev`, `/proc`, `/env`, `/dev/seg`, `/dev/gpio`, `/dev/i2c`
C		`snd`	`#a` sound	`/dev`
B		`vid`	`#v` the Vera X (none without a card: it ends)	`/dev/vid`

Few tasks are kept standing. There are only sixteen, so what can share a task does: GPIO and I2C are in `kdev` (HydraOS's `kdev` is the base's with them added: `reborn/modules/kdev`), and so is the Vera X's keyboard and mouse, which `kdev` looks at between its calls (`SRV_NAP`), where once a program, `input`, did. `init` doesn't stay either: it becomes the shell.

`ps` shows them (the tasks in decimal; this is the emulator without a Vera X, so `vid` has ended):

```
/> ps
task  state   parent     cpu group  name
   0  ready   -          1.0     0  kernel
   1  wait    -          0.4     1  forth
   2  ready    1         0.0     1  ps
  12  idle    -          0.0    12  snd
  13  idle    -          0.1    13  kdev
  14  idle    -          1.5    14  storage
  15  event   -          0.5    15  cons
```

How a driver works

Every driver has the same shape (servers and drivers):

- `init` runs once as it starts: it registers its device letters (`SRV_REGISTER`), owns its interrupt lines (`IRQ_OWN`), and sets its hardware up. A driver with work of its own between its calls doesn't return from it, but waits in `SRV_NAP` between its turns (the kernel): `cons` is shown as `event` for that, and `kdev` as `sleep` when there's a Vera X keyboard to look at.
- `serve` runs for each request a client makes, in the driver's own task. Nearly every driver uses srvlib's, with a table of its files and a routine for each. A driver **never waits**: when it can't answer yet it says so (`E_AGAIN`), and adds to its event count when things change (waiting).
- `irq` runs for its interrupt lines, in its own task, with interrupts off and about 85 cycles to spend.
- Its state is in its own RAM and banks; nothing else touches its hardware. **One driver owns each piece of hardware**: the serial port is the console driver's, the SPI bus and every disk the storage driver's, the YM2151 the sound driver's, port A `kdev`'s (its GPIO and I2C), the

VERA the video driver's. That's why there are no locks: two requests never meet on a chip.

Drivers rarely call each other, and only in one direction, so none can wait on another: `kdev` writes the Vera X's keys to the console (`#c/kbin`) and its mouse to the video driver (`#v/mousein`); the console rings the sound driver's bell (`#a/bell`) and writes the screen (`#v/term`); and the sound driver writes the PSG's registers through the video driver (`#v/psg`). `vid` calls nobody, and nobody calls `kdev` while serving.

The serial console

The serial port is the Hydra's console: the terminal everything in this book is typed at. Its driver runs in **task F** and owns the ACIA and VIA timer 2. There are two:

- `ser`, the base's (`base/modules/ser/ser.s`): the serial port as a console, as bare as one can be.
- `cons`, HydraOS's (`reborn/modules/cons/`): the same files and more: **windows**, each a whole console, a VT100 screen kept for each, the Vera X's screen as a second terminal, and `/pc`, a folder on the PC over the same line.

Both are built on the same serial layer (`base/lib/serial.inc`: the rates, the send ring, timer 2's pacing), and `ser`'s files are a part of `cons`'s, so a program written for the base runs on HydraOS unchanged.

Contents

1. The serial line
2. The console's files
3. Reading a line: cooked mode
4. Raw mode and keys
5. Notes from the keyboard
6. Windows: groups, the keys, tiles, the scrollback, the cut buffer
7. The chrome: the bar, headers and footers
8. A window's screen, and its size
9. Two terminals: the serial port and the screen
10. /pc: a folder on the PC
11. How it works

The serial line

- **9600 baud, 8 data bits, no parity, 1 stop bit**, with RTS/CTS, as the system starts. The rates are 300, 600, 1200, 2400, 4800, 9600, 19200 and 115200 (the ACIA): `echo b115200 >/dev/serctl`, then set the terminal to match. `cat /dev/serctl` says the rate and the flow control (`x1`, `x0`), then two lines of counts for chasing an unreliable line: `/pc`'s requests, and how many of their replies were cut short, unanswered or damaged, how many requests the PC had damaged (its NAK), and how often the PC was given up; and the ACIA's receive **overruns** (the system was late taking a byte: another came first) and **framing errors** (noise, or a rate the terminal doesn't match), and the bytes **dropped** because the receive ring was full (nothing was reading, and the terminal didn't stop). The PC tool's Ctrl-A s has the PC's side: the frames from the Hydra that came damaged or stopped part-way, and the replies it was asked for again. An `unanswered` here with a reply `asked for again` there is a reply that was lost or late on its way to the Hydra; with a frame `stopped part-way` there, a request cut short on its way to the PC.
- **Every byte sent is paced** by VIA timer 2: a character's time and a margin (two idle bits at 115200, one at the other rates), so the board's slightly slow clock and the MAX232's edges never lose a character.
- **Bytes received** go into a 256-byte ring as they come (the ACIA's interrupt). A byte that arrives when the ring is full is dropped.
- **Any terminal** works: a VT100 or xterm emulation, PuTTY, Tera Term, `screen`, `minicom`. The PC tool (`reborn/sim/tools/hydrapc.js`) is a terminal that also serves `/pc` (below).

The console's files

`#c`, bound at `/dev`:

File	`ser`	`cons`	What it is
`cons`	yes	yes	The console: a read gets a line (or keys, raw), a write goes to the terminal
`consctl`	yes	yes	Its mode: `rawon`, `rawoff`, `group`; `cons` adds the key modes, scrolling, the terminals, the size
`ser`	yes	yes	The serial port, raw: bytes in and out as they are
`serctl`	yes	yes	The rate: `b300` ... `b115200`; the flow control: `x1`, `x0`. Reads as both, and the counts (above)
`wctl`		yes	The windows: new ones, which is shown, their chrome, layouts, the keys

File	ser	cons	What it is
label		yes	The window's title
wnew		yes	A read waits for Ctrl-] c, makes a window, and gives its number (init's)
kbdin		yes	A write is typed into the window as keys
kbin		yes	The console's keyboard: the Vera X's keys, written by kdev
text		yes	The window's scrollback and screen, as text
snarf		yes	The cut buffer, shared by all the windows

In cons, each window has its own: #c2/cons is window 2's console, #c/cons window 0's. A shell in window 2 has #c2 bound at /dev, so its /dev/cons is its own window's.

Reading a line: cooked mode

A read of cons gets **a line**, edited at the console before the program sees it:

Key	What it does
Backspace, Delete	Rub out a character
Left, Right, Home, End (Ctrl-A, Ctrl-E)	Move in the line (cons)
Up, Down	The lines typed before (cons)
Ctrl-U	Rub out the line
Enter	Send the line (it ends with an LF; CR, LF or CR LF from the terminal are one end)
Ctrl-D	On an empty line: the end of the input (a read of 0)

Each key is echoed as it's typed. A line longer than the window's row wraps onto the next rows. A write to cons goes to the terminal, each LF as CR LF.

Raw mode and keys

Writing rawon to consctl turns line editing off: a read gets each key as it comes, with no echo. Raw mode lasts until the last fd open on consctl closes (Plan 9's way), so a program opens consctl, writes rawon, and keeps it open while it wants raw keys. rawoff turns it off.

cons can give a raw reader's keys two ways (a consctl command):

- keys hydra (as it starts): the cursor and function keys as **one code each** (KEY_UP, KEY_F1 ...: the constants are in hydra.inc), and KEY_RESIZE when the window's size changes. An Escape alone is a key of its own once a moment passes with nothing after it.
- keys vt: the keys as a VT100 sends them, escape sequences and all, following the window's modes (cursor keys, keypad, VT52). For programs that parse them themselves.
- keys mods: as hydra, plus a key the terminal sent with modifiers as KEY_MOD, the modifiers (1 Shift, 2 Alt, 4 Ctrl), then the key.

The window's screen answers a program's queries into its keys, as a VT100 does (where the cursor is, what the terminal is, its size: CSI 18 t). With ?1000 set (and ?1006), the mouse's presses and releases come as xterm's reports too, at the window's cells.

Notes from the keyboard

In either mode, at any time:

- **Ctrl-C** sends the interrupt note to the shown window's note group: the program running stops, unless it catches it.
- **Ctrl-** sends kill, which nothing catches.

consctl's group makes the window's notes go to the writer's note group: a shell does this for each command it runs in the foreground.

Windows

`cons` gives the one terminal **several consoles**: windows, Plan 9's way (rio's, on a text terminal), not Unix's job control. Each window is a whole console with its own `cons`, line editor, raw mode, note group and screen. One window is **shown**, and gets the keys; the others run on, their output kept on their screens, their reads waiting for keys. Showing a window paints it on the terminal exactly as it was.

- **Up to 16 windows**, as the console's RAM banks allow. **Window 0** is init's, and the login shell's. A window goes away when its program ends (its last `cons` closes), except window 0.
- **Groups**: each window belongs to a group, a shell session. Ctrl-] c makes a new group with a shell in it; a window made by a program (`new-window`) joins that program's group. Each group remembers the window it last showed.
- `new-window [-g] [command]` makes a window and shows it, running the command (or a shell); `-g` puts it in a group of its own.

The keys

Every window key is **Ctrl-]** and then a key:

Keys	What they do
Ctrl-] 0 ... 9	Show that window (windows 10 and up: from the list, Ctrl-] w)
Ctrl-] c	A new group, with a shell
Ctrl-] n, Ctrl-] p	The next or previous group
Ctrl-] Tab, Ctrl-Tab	The group's next window; Ctrl-] Shift-Tab, Ctrl-Shift-Tab, the previous
Ctrl-] and an arrow	The group's previous (up, left) or next (down, right) window
Ctrl-] w	The list of windows: a window's number, or the arrows and Enter, shows it
Ctrl-] s, Ctrl-] v	Split: a new window with a shell, tiled below or beside
Ctrl-] z	Zoom: a tiled group's focus alone, or tiled again
Ctrl-] [, Shift-PgUp	The scrollback (below)
Ctrl-] y	Paste the cut buffer
Ctrl-] h	Hold the window's output (the VT100's No Scroll), until Ctrl-] h again
Ctrl-] x	A hangup note to the window's group
Ctrl-] ?	The keys, in a popup
Ctrl-] Ctrl-]	A Ctrl-] typed into the window

They're **bindings**: `wctl`'s `key` lines change them (`key prefix ctrl-a` makes Ctrl-A the prefix; `key t new` makes Ctrl-] t a new group). Init writes `/lib/windows`'s lines to `wctl` as it starts, and a card's `/lib/windows` comes first, so a card can carry your own.

Tiles and floating windows

A group's windows are shown one at a time (`layout tabs`), or **tiled** together: `echo layout rows >/dev/wctl` (or `columns`, `grid`). A window can **float** over its group's in a box: `echo float 40 10 >/dev/wctl` (40 x 10, centred), `float X Y C R` at a place, `float off`.

The scrollback and the cut buffer

Each window keeps rows past its screen: its **scrollback**, 64 by default (`wctl`'s `history N`, up to 128). **Ctrl-] [** (or Shift-PgUp) shows it, while the program goes on: the arrows, PgUp, PgDn, Home and End move; Space marks a line, and the lines from it to the cursor are the selection; Enter copies them to `/dev/snarf`, the cut buffer (rio's), and leaves; `q`, or Escape twice, leaves. **Ctrl-] y** pastes the cut buffer into the window shown as its keys, between bracketed-paste marks for a program that asked (`?2004`). `/dev/snarf` is a file: `cat /dev/snarf`, or `echo text >/dev/snarf` to replace it (8K at most).

`/dev/text` reads the window's scrollback and screen as text, a line a row.

The chrome: the bar, headers and footers

A terminal can show **chrome** around the window: the **bar** (a row for the whole console, at the top or bottom: the groups, the time), and the window's **header** and **footer** (a row each, above and below it). By default they're on the screen (with a Vera X), and off on the serial port.

Each is drawn from a **format**: `%n` the window's number, `%l` its label, `%p` its program, `%s` its status line, `%w` the group's windows, `%G` the groups, `%c` `%r` its size, `%m` its modes, `%t` `%d` the time and date, `%L` the keyboard's LEDs, `%y` the scrollback's place, `%=` the rest at the right, `%[1;33]` SGR's rendition, `%%` a `%`. The defaults, `/lib/windows`:

```
bar top
bar %[7] %G%=%t
default header %[1]%n %l%=%w
default footer %s
default chrome screen on
default chrome serial off
```

`wctl` takes the same lines, for the window that writes them: `echo chrome serial on >/dev/wctl` shows it on the serial port too; `echo header %n %p >/dev/wctl`; `echo status compiling >/dev/wctl` sets the footer's `%s`. The window's **label** is `/dev/label` (`echo build >/dev/label`), which OSC 0 and 2 set too; empty, it's the program's name.

A window's screen, and its size

Each window's screen is kept in the console driver's RAM banks, written by a **VT100**: the VT100's and VT102's sequences and reports, VT52 mode, the alternate screen, double width and height, the character sets. So a full-screen program (`edit`, `top`, `more`) works in any window, and a window shown again is painted exactly as it was.

A window's size is the smaller of the terminals it's shown on, less their chrome: 80 x 24 on a serial terminal until it says otherwise; 80 x 57 on the Vera X's screen alone. `consctl` reads with a line `size C R`; a raw reader with `keys hydra` gets `KEY_RESIZE` when it changes. The serial terminal's size is asked as the console starts (`CSI 18 t`) and can be set: `echo terminal size 132 43 >/dev/consctl`. The PC tool tells the Hydra its window's size as it changes.

Two terminals: the serial port and the screen

With a Vera X, the console has a second terminal, the screen (`vid`'s `#v/term`), and keys from its keyboard (`#c/kbin`, written by `kdev`). `consctl` chooses how they're used:

Command	What it does
`both`	The window shown on both (as it starts)
`screen`, `serial`	The windows on one alone. With `screen` alone, output isn't held back by the serial line
`seats`	Both on, each a **seat** of its own: its own window shown, its own keys (the keyboard's are the screen's), its own Ctrl-C. A group's windows are sized to the terminals showing it

`cat /dev/consctl` shows which (`terminal both`), and once the screen's keyboard has typed, a line `keyboard screen`: someone is at the screen, so a program may take it alone (Dingus Quest Zero does, with `terminal both`) without leaving the serial terminal, then the only keyboard, blind.

/pc: a folder on the PC

`/pc` is a folder on the PC, served over the **same serial line** by the PC tool, so files go between the Hydra and the PC with `cp`:

```
$ node sim/tools/hydrapc.js COM3 work              # the PC: a terminal, and the folder work as /pc
/> ls /pc
/> cp /pc/hello.hyx /ram/bin/hello
```

- The PC tool needs Node's `serialport` package (`npm install` in `reborn/sim`); `--list` shows the PC's serial ports, `--baud N` sets the rate, `--read-only` keeps the folder from being changed. Its own keys are Ctrl-A and a key: Ctrl-A x quits.
- The PC tool is the Hydra's terminal too; its requests go in **frames** between the console's bytes, with a CRC, so a byte lost on the line means a frame sent again, never a wrong file.
- One request at a time; with no PC tool on the line, a request fails (`E_IO`) after a moment.
- The emulator plays the PC tool's part with `--pc-dir DIR` (the emulator).
- While a program has `/dev/ser` open for reading (`xmodem`), the line is its: `/pc` and the windows' output wait.

`xmodem -r file` and `xmodem -s file` move files with any terminal program instead (the tools).

How it works

- **Receiving**: the ACIA's interrupt puts each byte into the receive ring and adds 1 to the driver's event count, so the clients waiting look again. Before each request, the driver hands the keys to the windows' queues, acting on Ctrl-] and the key after it there. Ctrl-C and Ctrl-\ are sent as notes from the interrupt entry itself (`NOTE_QUEUE`), so they work even when the system's busy.
- **Sending**: the shown window's output goes into a send ring as there's room; VIA timer 2 runs a character's time, and its interrupt sends the next byte. The interrupts' work is a few dozen cycles each.
- **The screens**: `vt.s` (the driver's second bank) is the VT100: it writes each window's cells into the driver's RAM banks (a pool of 64 rows of 128 cells, three planes: characters, attributes, colours), and the terminal follows the window shown as its output is parsed. Showing another window paints it from its cells.
- **The irq entry is in RAM**: `cons` is two banks, so its interrupt entry (and what that calls) lives in its RAM, as the rule for a module of several banks says (assembly).
- `ser` is the same without windows, screens or `/pc`: a receive ring, a send ring, the line editor, and the four files.

The kernel's devices

kdev (`base/modules/kdev/kdev.s`) serves the kernel's own devices: the root of the file tree, `null` and the kernel's messages, the clock, the paged ROM's modules, the tasks (`/proc`), pipes, the environment, shared segments' names, and raw RAM. It's a boot driver like any other (task E in the base, task D in HydraOS), not the kernel: the kernel keeps the tables, and kdev turns them into files. HydraOS's kdev (`reborn/modules/kdev`: the base's source, with its own added) also serves GPIO and I2C and reads the Vera X's keyboard and mouse, so that those take no tasks of their own. Files, servers and namespaces lists its devices; this page is how to use them.

Contents

1. The root
2. null, zero and the kernel's messages
3. The clock
4. The modules
5. /proc: the tasks
6. Pipes
7. The environment
8. Shared segments by name
9. Raw RAM

The root

`#/` is the root, bound at `/`: an empty directory for each mount point the default namespace uses, so `ls /` and `ls /dev` show them even before anything's mounted there. `/` holds `bin`, `dev`, `env`, `lib`, `mnt`, `pc`, `proc`, `ram`, `rom`, `sd`, `sram` and `tmp`; `/dev` holds `gpio`, `i2c`, `mod`, `sd`, `seg`, `spi` and `vid`.

---

null, zero and the kernel's messages

`#n`, bound at `/dev`:

- `/dev/null` reads as nothing and takes every write: `ls >/dev/null`.
- `/dev/zero` reads as zeros.
- `/dev/kmesg` is the kernel's messages: everything it printed on the bring-up console, the boot and POST included, its last 4K (the boot).

```
/> cat /dev/kmesg | head -4

Hydra-16: kernel 0.1, ABI 1
POST ZP:0 ST:0 OS:0 HI:0 SH:S W:0
RAM U:0 F0:0/00/0000 F4:0/00/0000 F8:0/00/0000 FC:0/00/0000 00:0/00/0000 10:0/00/0000 20:0/00/0000
```

The clock

`#t`, bound at `/dev`:

- `/dev/ticks`: the tick count (200 a second), in decimal.
- `/dev/time`: the date and time as text. Write one to set the clock, and the DS1747 with it if there's one: `echo 2026-10-11 09:30:00 >/dev/time`. `date` reads it.
- `/dev/rtc`: the DS1747's registers, as text (`none` without one).

```
/> cat /dev/ticks
1115
/> cat /dev/time
2000-01-01 00:00:10
```

Without a DS1747 the clock starts at 2000-01-01 each boot.

The modules

`#m`, bound at `/dev/mod`: a file for each module of the paged ROM, its image (a module's header first, which `SPAWN` reads). `#m/bin` lists the programs alone, and is bound into `/bin`, so `rc` runs the ROM's `rc`.

```
/> ls /dev/mod | head -6
bin/
init
cons
storage
kdev
snd
```

`mods` lists the modules with their banks and types.

/proc: the tasks

`#p`, mounted at `/proc`: a directory for each task in use, by its number.

```
/> ls /proc/1
status
ctl
args
cwd
fd
ns
env
regs
mem
ram
note
/> cat /proc/1/status
init wait - 37 1
```

`status` is the task's name, state, parent (`-`: none), CPU time in ticks, and note group. The other files:

File	What it is
`args`	Its arguments
`cwd`	Its current directory
`fd`	Its open files, a line each: `0 rw #c 291 #c/cons` (the fd, its mode, the device, its qid, its name)
`ns`	Its namespace, as the `bind` and `mount` lines that would make it
`env`	Its environment
`regs`	Its saved registers (a stopped or waiting task's)
`mem`	Its 64K as it sees them: read and write it as a file, the offset the address
`ram`	Its own banks: bank b's byte o at `b * $2000 + o`
`note`	Write a note's name or number to send it: `echo interrupt >/proc/5/note`
`ctl`	Commands (below)

```
/> cat /proc/1/ns | head -4
bind '#/' /
bind '#/dev' /dev
bind -a '#c' /dev
bind -a '#n' /dev
```

ctl

Command	What it does
`kill`	Sends the kill note: the task ends
`interrupt`	Sends the interrupt note (Ctrl-C's)
`note N`	Sends note N
`stop`, `start`	Stops the task where it is, or starts it again (`ps` shows `stopped`)
`step`	Runs one instruction of a stopped task
`next`	The same, but a `JSR`'s subroutine runs whole
`break`, `nobreak`	A `BRK` stops the task (a debugger's breakpoint), or is the program's own again

`ps`, `top`, `kill`, `slay` and `db` are built on these files. A task's memory, registers and notes are anyone's but the kernel's and a driver's: their tasks are the system's.

Pipes

`#|`: opening its `pipe` file makes a new pipe; the `PIPE` call makes both ends at once (`.A` the read end, `.X` the write end). A pipe holds 512 bytes; a reader waits for bytes, a writer for room, and the reader sees the end of the file when every write end is closed. Eight pipes can be open at once. rc's `|` uses them.

The environment

`#e`, mounted at `/env`: **the asking task's environment**, a file for each variable. Every task has its own (a copy of its parent's, made as it starts), so a variable a program sets is its own and its children's, not its parent's.

```
/> rc
% greeting=hello
% cat /env/greeting
hello% echo $greeting
hello
% ls /env
prompt
path
task
greeting
```

rc keeps its variables here (a list's words each followed by a zero byte, which is why `cat` showed no new line), and its functions as `fn#NAME`. `/proc/N/env` is any task's. Note that at HyForth's prompt each rc line runs in an `rc` of its own, so a variable set there ends with the line: start `rc` for a session that keeps them.

Shared segments by name

`#s`, bound at `/dev/seg`: names for shared segments, so tasks that didn't start each other can find one.

- `echo name scores 3 >/dev/seg/ctl` names segment 3 `scores`. The name holds a reference of its own, so the segment stays while it's named.
- `cat /dev/seg/scores` gives its number and banks (`3 2`: segment 3, of 2 banks); a task then attaches to it (`SEG_ATTACH`).
- `echo free scores >/dev/seg/ctl` lets the name go.

`free` shows the shared RAM: what the segments have, what's free.

Raw RAM

`#r`, for init alone (it isn't in the default namespace): `task` is every task's RAM (task t's at `t * $8000`, 512K), and `shared` the shared banks (shared bank ID i's at `i * $2000`, 2M), read only. It's the system's raw view, for init's own use.

Storage

storage (`reborn/modules/storage/`, task E) owns the SPI bus and every disk, and the file system on them, HydraFS. One driver for all of it, so an SD card's transfer never meets another device's on the bus. It's HydraOS's; the base has no storage.

Contents

1. The disks
2. Using them: the cards, the ROM disk, the RAM disks
3. The disks' files: `data` and `ctl`
4. Getting a card ready
5. Writes, and taking a card out
6. The tools: `df`, `mkfs`, `fsck`, `label`
7. HydraFS: what's on a disk
8. SPI devices as files
9. How it works

The disks

Disk	What it is	Its files at	Speed
`0-7`	An SD card on SPI device 0-7, the board's headers J18-J25	`/sd/0` ... `/sd/7`	About 12K a second
`8-f`	An SD card on SPI device 8-15, a card in a slot	`/sd/8` ...	
`v`	An SD card on the Vera X's own slot (the VERA's SPI controller)	`/sd/v`	Nearly twice as fast
`x`	**The ROM disk**: the paged ROM's banks after the modules, read only	`/rom`	
`r`	**The RAM disk**: banks of the storage driver's own, 256K as it starts. Each shell has its own area of it	`/ram`	At once
`s`	**The shared RAM disk**: a shared segment, 512K as it starts	`/sram`	At once

```
/> df
disk  kind   size        free        label
0     sdhc   16380 KB    15956 KB    HYDRA-16
x     rom    684 KB      0 KB        ROM
r     ram    255 KB      252 KB      RAM
s     sram   511 KB      510 KB      SRAM
```

A disk is listed once it's started: a card when something first opens it, the RAM disks as init starts them.

Using them

The disks' file systems are mounted by the default namespace (namespaces):

- `/sd/N` is card N's, `/sd/0` the usual one. A card's `bin` and `lib`, if it has them, join `/bin` and `/lib`, so a card can carry programs and libraries of your own.
- `/rom` is the ROM disk: the tools, which run from RAM (`/rom/bin`: `ls`, `cat`, `cp`, `ed`, `as`, `mkfs`, `grep`, `db`, `edit`, `calc` ...: the tools), the languages' libraries (`/rom/lib/forth`, `/rom/lib/hylang`, `/rom/lib/basic`), the assembler's include files (`/rom/lib/as`), the system's files (`/rom/lib/namespace`, `profile`, `shell`, `windows`), a song, and the call reference (`/rom/doc/api.md`).
- `/ram` is the shell's own area of the RAM disk (`r/N`, N its task): fast and private, gone when the power goes. `/ram/bin` comes first in `/bin`, so a program put there runs before the ROM's of the same name.
- `/sram` is the shared RAM disk: every shell sees the same one.

```
/> ls /rom
README
bin/
doc/
lib/
songs/
/> ls /sd/0
bench/
sample/
songs/
```

HydraOS's own card (`reborn/bin/sdcard.img`) has the SDK's samples (`/sd/0/sample`, the C ones in `/sd/0/sample/c`, the assembler's sources in `/sd/0/sample/as`), the sample songs (`/sd/0/songs`) and the benchmarks (`/sd/0/bench`).

The disks' files

`#d`, bound at `/dev/sd`: a directory for each disk started, each with two files.

- `data`: the disk as a file of bytes (its first 4 GB), at the fd's offset. `cat /dev/sd/0/data` reads the raw card.
- `ctl`: reads as the disk and its file system, and, once a card has refused a command or a block's data or stayed busy after a write, an `errors` line counting each (the cards', all of them: a bad SPI contact or a bad card shows here); takes commands.

```
/> cat /dev/sd/0/ctl
sdhc 17 MB 34816 blocks
hydrafs label=HYDRA-16
partition at block 2048
free 15956 KB of 16380 KB
/> cat /dev/sd/r/ctl
ram 256 KB 512 blocks
hydrafs label=RAM
free 252 KB of 255 KB
banks $00-$1f
```

Command	What it does
`init`	Start the card again (after it's been changed)
`sync`	Write the block kept back (below) now
`format [-f] [-p] [-s SIZE] [LABEL]`	A new, empty HydraFS (`-f` a full format; `-p` in a partition of its own after the card's others)
`label TEXT`	Set the label
`check [fix]`	Check the file system (and fix what's wrong)
`start SIZE [FROM-TO]`	A RAM disk of SIZE (8K banks, or `256K`, `1M`), from a range of memory cards; an empty HydraFS on it
`stop`	A RAM disk stopped, its banks given back (not while it's open)

`echo start 128K 1-1 >>'#d/r/ctl'` starts a RAM disk on memory card 1's banks (`>>` because a stopped disk's directory isn't listed).

Getting a card ready

Any SD card (SDSC or SDHC) works, on an SD card module with its own 3.3 V regulator and level shifter (the devices). HydraFS isn't a PC's file system, so a card is made one of two ways:

- **On the PC**: write HydraOS's card image to it, `node sim/tools/sdwrite.js bin/sdcard.img DISK` from `reborn/` (`--list` shows the PC's disks first: it refuses the system's disk, and one that isn't removable without `--force`). `sim/tools/hydrafs.js` makes images of your own from a folder (the PC tools).
- **On the Hydra**: `mkfs 0 MYCARD` makes an empty HydraFS on card 0. With `-p`, it goes in a partition of its own after the card's others, so a card can keep a PC partition too.

Writes, and taking a card out

A card's writes are **kept back a block at a time**: the block a file's last write changed stays in the storage driver's buffer until another block's wanted, the file's closed, or `sync`. So a program writing a few bytes at a time isn't slowed by the card. But a block still kept back is lost if the card is taken out or the power goes:

- **Close the file** (end the program writing it), or `echo sync >/dev/sd/0/ctl`, before taking a card out or switching off.
- After changing a card, `echo init >/dev/sd/0/ctl` (or just open it: a card's first open starts it). `init` drops a block kept back, since the card may be another.

The RAM disks' writes go at once. HydraFS writes in a safe order (a file's data, then the free map, then its directory entry), so a card pulled out loses at most what was being written, not the file system.

The tools

Tool	What it does
`df`	Each disk started: its kind, its file system's size and what's free, its label
`mkfs [-fp] disk [label ...]`	A new, empty HydraFS on the disk (`0-f`, `r`, `s`): `-f` a full format; `-p` in a partition of its own
`fsck [-f] disk`	The disk's HydraFS checked (`-f`: and its faults fixed): what was lost, unmarked, or used twice
`label disk [text ...]`	The disk's label set, or said

`mkfs`, `fsck` and `label` are RAM programs on the ROM disk; each writes the disk's `ctl`.

HydraFS

HydraFS is the Hydra's own file system, designed for the 65C02 and Plan 9's file model. Its full format is in `docs/design/plans/HYDRAFS.md`; in short:

- **Blocks** are the card's 512-byte sectors; space is given out in **clusters of 8 blocks** (4K) on a card. The superblock says a volume's own size, 1K to 4K: a RAM disk's are 1K, and so are the ROM disk's (its many small files wasted half a cluster each at 4K: 14 banks of the paged ROM).
- **Block 0** is the superblock: the magic `HYDRAFS1`, the sizes, the free map's place, the root directory's entry, the label. Then the **free map**, a bit a cluster, then the **data area**.
- **A directory** is a file of 64-byte entries, and **the entry is the file's metadata**: its name (31 bytes, any byte but `/` and 0, case-sensitive), its mode, its qid (an id unique on the disk, kept across renames, and a version that goes up at every change), its size (to 4 GB), its time, and where its data is.
- **A file's data** is a list of **extents**: runs of contiguous clusters. Two fit in the entry; more go in a chain of extent blocks. The allocator grows a file's last extent when it can, so files stay contiguous and a read is a run of blocks. A hole (a sparse file's zeros) takes no space.
- **Partitions**: a card can have a partition table (a PC's MBR), HydraFS in its partition of type `$7F`, so a card can hold a HydraFS next to a FAT partition a PC reads.
- No journal: writes go in a safe order instead, and `fsck` finds and fixes what an interrupted write leaves.

SPI devices as files

`#S`, bound at `/dev/spi`: a directory for each SPI device, `0-f`. Any SPI part on a header or a card can be used from a program or the shell, without a driver of its own:

- `N/data`: a write sends its bytes with device N selected, one transaction a request (256 bytes at most), and keeps the bytes the device sent back meanwhile; a read gives those bytes. With none kept, a read clocks in as many as it asks for (sending `$FF`).
- `N/ctl`: `mode 0` (the clock idles low) or `mode 3` (it idles high); both send and sample on the clock's rising edge, most significant bit first.

A device is one opener's at a time, and not while it's a card's (`E_BUSY`). HyForth's `lib spi` and hylang's `spi` library are words over these files (GPIO, I2C and SPI).

How it works

- **Two banks**: the first is srvlib, the SPI bus, the SD card layer, the disks, `#S` and `#d`; the second is HydraFS (`#f`).
- **SPI** is bit-banged on the VIA's port B, which only this task touches: 18 cycles a bit in, 33 out (the loops are the first Hydra software's, proven on the board). The Vera X's card is on the VERA's SPI controller (12.5 MHz), which this task touches too; the rest of the VERA is `vid`'s.
- **A cache**: a card's blocks are cached, besides the block buffer, so a file read again doesn't go to the card.
- **The ROM disk** is read through the kernel's `ROMREAD`: block n is in bank n / 32 of the paged ROM's disk area. **A RAM disk**'s block n is in bank n / 16 of its banks.

Sound

snd (`reborn/modules/snd/`, task C) makes the Hydra's sound chips one synthesizer of 24 channels: the YM2151's 8 FM channels on the board, and, with a Vera X, its PSG's 16 voices. Every language makes sound the same way, by writing text commands to `/dev/sndctl`. The Vera X's PCM (sampled sound) is the video driver's (video), and `play` plays songs, scores and WAV files on all of it.

Contents

1. The chips
2. The files
3. Making a sound from the shell
4. The commands
5. Claiming channels
6. play: songs, scores and WAV files
7. Scores
8. From each language
9. How it works

The chips

Channels	Chip	What it has
0-7	**The YM2151** (OPM), on the board	8 channels of 4-operator FM: 8 algorithms, envelopes, an LFO, a noise generator. The sound of 1980s arcade machines and the Commander X16
8-23	**The Vera X's PSG**	16 voices: pulse (with a width), sawtooth, triangle or noise, a volume and a stereo place each; no envelope of its own

Without a Vera X there are 8 channels; commands to 8-23 fail (`E_NODEV`). Both chips come out of the board's mixer and its headphone jack (the audio path).

The files

`#a`, bound at `/dev`:

File	Read	Write
`sndctl`	The state: `volume 100`, `channels 24`, then `claimed` and the channels claimed	Commands, a line each (below)
`snd`	The YM2151's registers as they were written (256 bytes: the chip's can't be read)	Register/value byte pairs, for the YM2151
`psg`	The PSG's 64 registers as written	Register/value pairs for the PSG (0-63), as ZSM songs write them
`bell`		Any write: a short beep on channel 7 (the console rings it at a BEL)

Making a sound from the shell

```
% echo note 0 60 >/dev/sndctl           # middle C on channel 0, in its patch
% echo off 0 >/dev/sndctl               # and off
% echo patch 1 40 >/dev/sndctl          # channel 1: General MIDI's violin (patch 40)
% echo note 1 69 >/dev/sndctl           # the A above middle C, 440 Hz
% echo note 8 72 >/dev/sndctl           # the PSG's first voice (with a Vera X)
% echo volume 50 >/dev/sndctl           # everything quieter
% play -m 0 t180 o4 l8 c d e f g        # a line of a tune
% play /rom/songs/scom.zsm              # a song
```

The commands

Write them to `/dev/sndctl`. A channel `CH` is 0-23; notes are MIDI's (60 is middle C, 69 the A at 440 Hz).

Command	What it does
`patch CH P`	The channel's sound: one of the driver's patches (the X16's, General MIDI's numbering). On the PSG, a patch below 4 is a waveform
`note CH N`	Note N on (a new note on the channel replaces the last)
`off CH`	The note off (it fades as its envelope says)
`level CH V`	The channel's volume, 0-127
`pan CH left|right|both`	Which speakers (or 0-3)
`bend CH B`	Bend the pitch, -128 to 127
`drum CH N`	A General MIDI drum sound, N
`freq CH HZ`	A frequency, not a note (0: off)
`glide CH N`	Slide to note N (legato)
`wave CH pulse|saw|triangle|noise [WIDTH]`	A PSG voice's waveform (and a pulse's width, 0-63, 63 a square)
`sens CH PMS AMS`	The YM2151 channel's sensitivity to the LFO (pitch, amplitude)
`lfo RATE PMD AMD WAVE`	The YM2151's LFO
`noise N|off`	The YM2151's noise generator (on channel 7)
`reg R V [R V ...]`	Registers written directly
`volume N`	The master volume, 0-200 (100 as written; more is louder, to 23 dB)
`claim N [P]`, `release N [P]`	Claim channels (below)
`reset`	The chips and every setting cleared (claims stay)

The master volume and each channel's level apply to everything, a song's raw register writes too: the driver adds them to what reaches the chip and keeps what was written.

Claiming channels

A program that wants channels to itself claims them: `claim N [P]`, N a mask of the YM2151's channels (bit n, channel n) and P of the PSG's (bit n, channel 8 + n). A claimed channel is its claimer's alone: other tasks' writes to it are dropped, until it's released or the claimer closes its last file of `#a` (its end does: the channels are keyed off then). Claiming a channel another task has gives `E_BUSY`, and claims nothing. `play` claims the channels a song uses.

---

play: songs, scores and WAV files

Command	What it does
`play [-l] song.zsm [n]`	A **ZSM song** (the X16's format; Furnace exports it) on the YM2151, and with a Vera X its PSG and PCM parts: once, its loop n more times, or `-l` until Ctrl-C
`play [-l] score.mml`	A **score** (below), compiled as it plays
`play -o score.mml song.zsm`	The score compiled into a ZSM file instead
`play file.wav`	A **WAV file** on the Vera X's PCM: 8 or 16 bits, mono or stereo, up to 48,828 Hz (from a card, 8 bits mono to about 11 kHz)
`play [-l] -m ch MML ...`	A line of MML on channel ch: `play -m 0 t180 o4 l8 c d e f g`
`play [-l] -c ch NOTES ...`	A chord: each note on the next channel from ch: `play -c 0 o4 l2 I0 c e g`
`play -x ...`	With `-m` or `-c`: the X16's own MML (FMPLAY's and PSGPLAY's): `play -x -m 0 T120 O4 L8 CDEFG`

`/rom/songs` has a few songs; HydraOS's SD card has more in `/sd/0/songs`. `scom` plays one.

Scores

A score is music as text, the language of the PC's score compiler too (`reborn/sim/tools/hysong.js`). `play` plays one as it is, compiling it as it goes into the same register writes at the same ticks as `hysong.js` would put in a ZSM file:

```
; A score: ; to the line's end is a comment
#tempo 120                       ; quarter notes a minute (120); #rate N: its ticks a second (200)
@piano { gm 0 }                  ; an instrument: one of the driver's patches, or a voice's operators
@lead { wave pulse 24 env 2 14 12 18 }   ; a PSG instrument: a waveform, an envelope (in the song's ticks)
A @piano o4 l8 c d e f g4 r4 c2  ; channel 0 (A-H: 0-7, the YM2151's); a channel's lines are joined in order
B @piano o3 l2 c [g e]2 c        ; [ ... ]N: repeated N times
I @lead o5 l8 e a b > c          ; channel 8 (I-X: 8-23, the Vera X's PSG's voices 0-15)
```

Notes and rests: `c` to `b` (`+` or `#` sharp, `-` flat), a length (1 a whole note ... 64; 3, 6, 12, 24, 48 triplets), dots, `^` ties; `r` a rest; `x N` a General MIDI drum.

Commands: `o`, `>`, `<` the octave; `l` the default length; `q` the part of a note held (eighths); `v` the volume (0-127); `p l|r|c|0` the speakers; `k` transpose; `D` detune (64ths of a semitone); `M` and `L` the LFO; `N` the noise; `y reg,val` a register; `_` a slide to the next note; `&` legato; `I N` the driver's patch N as the instrument.

The PSG's channels (I-X) take the same notes, lengths, rests, `o` `l` `q` `v` `p` `k` `D`, slides and legato, and instruments of their own: `wave W [WIDTH]` (`pulse`, `saw`, `triangle`, `noise`) and `env A D S R` (A ticks rising from silence to the note's volume, D falling by S steps to what it holds until the note's off, then R falling to silence). `x`, `M`, `L` and `N` are the YM2151's alone.

A score is read whole into memory (about 17K at most); a mistake is reported with its channel (`play: x.mml: channel 2: no such drum`). `/sd/0/songs/vera.mml` uses both chips.

From each language

Language	How
rc	`echo note 0 60 >/dev/sndctl`, `play`
C	`snd.h`: `snd_note`, `snd_off`, `snd_patch`, `snd_mml`, `snd_chord` ... (C)
HyForth	`lib sound`: `snd-note`, `snd-off`, `snd-patch`, `snd-mml`, `snd-chord` ... (HyForth)
hylang	`(use "snd")`: `(snd-note ch n)`, `(snd-mml ch "...")` ... (hylang)
HyBASIC	`SOUND` and `PLAY` (HyBASIC)

How it works

- **The YM2151's library** is the first Hydra software's, ported: a shadow of every register as written (the chip can't be read back); each carrier's total level written with its channel's volume and the master volume added, so a volume works on anything; register numbers below `$20` (which the chip doesn't have) are the driver's commands, which `sndctl`'s text turns into.
- **Pitch**: the YM2151's key code at its 3.58 MHz clock (octave and note), and a key fraction of 1/64 of a semitone for bends and detune.
- **The PSG** belongs to the video driver (the VERA is `vid`'s): `snd` keeps the PSG's registers as written and sends each request's changes to `vid` in one write (`#v/psg`). A note sets its voice's frequency and full volume; off sets the volume to 0; volumes have the same attenuation as the YM2151's.
- **No interrupts**: the driver owns no line. The YM2151's timer B didn't keep its period on the board (the quirks), so `play` times songs by the system's tick.
- **In the emulator**, `node sim/run.js -i --sound` plays it in a browser, and `--wav FILE` records it (the emulator).

GPIO, I2C and SPI

kdev serves the VIA's port A on header J27 (`reborn/modules/kdev/gpio.inc`, in HydraOS's `kdev`: it was a driver of its own, `gpio`, and took a task): its eight pins and its handshake lines as `#g`, and the I2C bus on two of them as `#i`. One driver for both, so their changes to port A never meet. The SPI devices are the storage driver's (storage), since SD cards share the bus.

Contents

1. The pins: /dev/gpio
2. CA1: waiting for an edge
3. The I2C bus: /dev/i2c
4. SPI devices: /dev/spi
5. From each language
6. An example: a button and an LED

The pins: /dev/gpio

`#g`, bound at `/dev/gpio`. The pins are on J27 (the VIA): PA0-PA7 at pins 3-10, CA1 and CA2 at 11 and 12, with +5 V and ground. PA0 and PA1 are the I2C bus's while `/dev/i2c` is used.

File	Read	Write
`0` ... `7`	The pin's level, `0` or `1`	`0` or `1`: sets it, and makes the pin an output
`port`	All 8 pins' levels, a byte	A byte: sets the outputs (the inputs keep theirs)
`ctl`	A line a pin (`2 out 1`), then `ca1 fall 3` (CA1's edge, and the edges counted) and `ca2 in`	`in N`, `out N` (a pin's direction), `ddr N` (all 8: bit n pin n, 1 an output), `ca1 rise`, `ca1 fall` (CA1's active edge), `ca2 0`, `ca2 1` (CA2 an output, low or high), `ca2 in`
`ca1`	Waits for CA1's next active edge, then the edges counted so far	

```
% echo 1 >/dev/gpio/2                  # PA2 an output, high
% echo in 3 >/dev/gpio/ctl              # PA3 an input
% cat /dev/gpio/3
1
% cat /dev/gpio/ctl
```

The pins are 5 V CMOS, through the SP720 ESD array (J28). A VIA output can drive an LED through a resistor (1K) but not much more; use a transistor for anything bigger.

CA1: waiting for an edge

A read of `/dev/gpio/ca1` **waits** for CA1's next active edge (`ca1 fall` or `ca1 rise` in `ctl`) since that fd last read it, then gives the count of edges so far. A non-blocking fd gets `E_AGAIN` instead. CA1's interrupt is on only while a `ca1` file is open (it's the kernel's line 17, owned then by `kdev`), so a floating CA1 can't flood the system. A switch's bounce counts as edges too: debounce in hardware, or by time.

```
% cat /dev/gpio/ca1                    # waits for a press
1
```

The I2C bus: /dev/i2c

`#i`, bound at `/dev/i2c`: an I2C master, bit-banged on PA0 (SCL) and PA1 (SDA), which are pulled up on the board and go to J27 and every slot.

File	What it is
`NN`	The device at address NN (7 bits, two hex digits, `08`-`77`): a write sends its bytes to it, a read reads from it, one transaction a request (64 bytes at most). A device that doesn't answer: `E_IO`
`ctl`	`speed N` (kHz, 1-100; it tops out near 40 kHz at 3.58 MHz), `subaddress N` (0, 1 or 2)

Listing `/dev/i2c` shows the devices that answer: `ls /dev/i2c` is a bus scan.

Registers: most I2C parts (EEPROMs, sensors, the Vera X's input controller) have registers. With `subaddress 1` (or 2), the file's offset is the register: a read writes the register's number (a byte, or two, high first), then a repeated start and the read; a write writes the register's number then the data. So `/dev/i2c/50`, with `subaddress 1`, reads a 24C02 EEPROM as a file of 256 bytes.

```
% ls /dev/i2c
ctl
42
% echo subaddress 1 >/dev/i2c/ctl
```

`$42` is the Vera X's input controller, which `kdev` itself reads (video). Nothing else on the board uses the bus. As the system starts `kdev` looks for the controller three times, a second apart; with none, PA0 and PA1 aren't touched again unless something uses `/dev/i2c`.

SPI devices: /dev/spi

The storage driver serves the SPI bus as `#S`, at `/dev/spi`: `N/data` (a write is a transaction; a read gives the bytes the device sent back, or clocks new ones in) and `N/ctl` (`mode 0` or `mode 3`). Devices 0-7 are headers J18-J25; 8-15 are for cards in the slots. Storage has the details.

From each language

Each language has a library over these files, with the same words:

What	HyForth (`lib gpio`, `lib i2c`, `lib spi`)	hylang (`(use "gpio")` ...)
A pin's level	`gpio ( pin -- level )`	`(gpio pin)`
Set a pin (an output)	`gpio! ( pin level -- )`	`(gpio! pin level)`
A pin's direction	`gpio-in ( pin -- )`, `gpio-out ( pin -- )`	`(gpio-in pin)`, `(gpio-out pin)`
All 8 pins	`gpio-port ( -- byte )`, `gpio-port! ( byte -- )`	`(gpio-port)`, `(gpio-port! byte)`
I2C read, write	`i2c-read ( addr reg c-addr u -- )`, `i2c-write ( addr reg c-addr u -- )`	`(i2c-read addr n)`, `(i2c-write addr bytes)` (after `(i2c-reg-size n)`, the first bytes are the register)
I2C devices	`i2c-devices`, `i2c? ( addr -- flag )`	`(i2c-devices)`
An SPI transaction	`spi ( dev c-addr u -- )`, `spi-read ( dev c-addr u -- )`, `spi-mode ( dev mode -- )`	`(spi dev bytes)`, `(spi-read dev n)`

In C, they're files: `open ("/dev/gpio/2", O_WRONLY)` and `write`. HyBASIC's `OPEN` works the same way. The libraries are sources on the ROM disk (`/rom/lib/forth/gpio.fs`, `/rom/lib/hylang/gpio.hl`), and short: they're examples of driving a device through its files.

An example: a button and an LED

An LED (through a 1K resistor) from PA2 to ground, and a push button from CA1 to ground (with `ca1 fall`): each press turns the LED over. In rc:

```
echo ca1 fall >/dev/gpio/ctl
led=0
while () {
    cat /dev/gpio/ca1 >/dev/null        # wait for a press
    if (~ $led 0) led=1
    if not led=0
    echo $led >/dev/gpio/2
}
```

In HyForth (`lib gpio` loads `/lib/forth/gpio.fs`):

```
lib gpio
: toggle  ( -- )  0  begin  s" /dev/gpio/ca1" gpio-read 2drop  1 xor  dup 2 swap gpio!  again ;
```

Video: the Vera X

The Hydra's screen, keyboard and mouse come from the **Vera X** card in slot 0 (the card): the Commander X16's VERA, with VGA at 640 x 480, two layers of text, tiles or bitmaps, 128 sprites, a 256-colour palette, a 16-voice PSG and PCM audio, and the X16's input controller for a PS/2 keyboard and mouse. Two drivers have it:

- **vid** (`reborn/modules/vid/`, task A), the driver: it owns the chip, finds the card as the system starts, sets it up for the console, and serves it as files, `#v` at `/dev/vid`. With no card it ends, and `/dev/vid` is empty.
- **kdev** (`reborn/modules/kdev/input.inc`) reads the keyboard and mouse, on the I2C bus it serves, and passes them on. (It was a program, `input`, which init started and which took a task.)

The VERA's own documentation is the X16's: *The Commander X16 Programmer's Reference*, chapter 9 (the registers, VRAM, layers, sprites, the PSG and PCM) and chapter 10 (FX).

Contents

1. Three ways to use the screen
2. The files
3. A picture from a file
4. Drawing
5. The console's terminal
6. The keyboard and the mouse
7. PCM
8. Claiming the chip
9. VRAM's layout
10. In the emulator

Three ways to use the screen

From the easiest:

1. **As the console.** The console's windows show on the screen as well as the serial terminal, so anything a program prints, colours and cursor moves included (ANSI sequences: conio, HyForth's terminal words, hylang's `screen`), shows there. Nothing to do.
2. **Through `/dev/vid`'s files**: VRAM, the palette, the sprites and the font read and written as files; the screen's modes as commands to `ctl`; drawing by commands to `draw`; a frame waited for by reading `frame`.
3. **Directly**: a program **claims** the chip and writes its registers itself, as an X16 program does, for as long as it holds the claim.

The files

File	Read	Write
`ctl`	The state, a line each: `vera 47.0.2` (the gateware's version), `mode 80x60`, `cursor blink`, `border 0`, `bitmap off`, `output vga`, `claimed` (and the claimer's task, and `all`)	`mode 80x60`, `mode 80x30`, `mode 40x30`; `cursor blink`, `cursor on`, `cursor off`; `border N`; `bitmap 320 D`, `bitmap 640 D`, `bitmap off`; `output vga`, `output ntsc [mono] [240p]`, `output rgb [240p]`; `claim`, `claim all`, `release`; `reset`
`term`	The screen's characters, a line a row	Bytes shown as an ANSI terminal shows them (the console writes here)
`vram`	VRAM: the offset is the address, `$00000-$1FFFF`	VRAM
`pal`	The palette (256 entries of 2 bytes: `$GB`, `$0R`)	The palette
`sprites`	The sprites' attributes (128 of 8 bytes)	The attributes (sprite 0 is the console's cursor, sprite 1 the mouse's pointer)
`font`	The console's font (256 characters of 8 bytes)	A font: `cat /lib/font/cp437 >/dev/vid/font` for the PC's
`frame`	Waits for the next frame (59.5 a second), then gives the frames counted	
`draw`	The pen: `pen 15`	Drawing on the bitmap (below)
`psg`	The PSG's 64 registers as written	Register/value pairs: the sound driver's (its channels 8-23)

File	Read	Write
`pcm`, `pcmctl`	`pcmctl`: the PCM's state	Samples, and their format (below)
`mouse`	The mouse, after a change, as Plan 9's `/dev/mouse`	`m X Y`: move it there
`mousein`		The mouse's moves, as `kdev` has them from the controller: `m DX DY B`
`mousectl`	`pointer on`, `swap off`	`pointer on`, `pointer off`, `swap on`, `swap off`

Modes: `mode 80x60` (the screen's 640 x 480 in 8 x 8 characters, as it starts), `mode 80x30` and `mode 40x30` (the characters taller, or bigger). **Output**: the VERA drives VGA; `output ntsc` and `output rgb` choose composite or RGB where the card brings them out (`mono`, NTSC without colour; `240p`, progressive).

A picture from a file

A 320 x 240 picture of 8 bits a pixel, its bytes in a file:

```
cat pic.bin >/dev/vid/vram
cat pic.pal >/dev/vid/pal
echo bitmap 320 8 >/dev/vid/ctl
```

shows it on layer 0, under the console's text (layer 1, whose background colour 0 lets layer 0 show through; `bitmap 320` makes the text 40 x 30, as the picture is shown doubled). `echo bitmap off >/dev/vid/ctl` takes it away. (`cp` won't write a device's file: it makes the file it copies to, and a device's files are there already. Use `cat` and `>`.) A bitmap 640 across is 1 or 2 bits a pixel: at 4 or 8 it would be more VRAM than a program has.

Drawing

`/dev/vid/draw` draws on the bitmap, the driver doing the work in its own code, so it's quick (a line of 300 pixels is some 60,000 cycles) and the console stays on the screen over it:

Command	What it draws
`pen C`	The colour from now on (one pen for every program; 15 as it starts)
`plot X Y`	A point
`line X0 Y0 X1 Y1`	A line
`box X0 Y0 X1 Y1`, `bar X0 Y0 X1 Y1`	A rectangle's outline; a filled one
`circle X Y R`, `disc X Y R`	A circle's outline; a filled one
`text X Y STRING`	Text in the console's font, 8 x 8: each character's dots in the pen's colour, the rest left
`clear [C]`	All of the bitmap, in colour 0 or C

Coordinates are the bitmap's pixels (320 x 240, or 640 x 480), each -4096 to 4095; what falls off isn't drawn. With no bitmap a command is `E_INVAL`; while the chip's claimed, `E_BUSY`. With the gateware's FX, a line at 8 or 4 bits a pixel uses the chip's line helper, and `clear` its 32-bit cache writes.

```
echo bitmap 320 8 >/dev/vid/ctl
echo pen 4 >/dev/vid/draw; echo circle 160 120 50 >/dev/vid/draw; echo text 120 116 Hydra >/dev/vid/draw
```

Each language has the same words, and more:

What	**rc** (`/dev/vid/draw`)	**HyForth** (`lib video`)	**hylang** (`(use "video")`)	**C** (`vera.h`)
The bitmap	`echo bitmap 320 8 >/dev/vid/ctl`	`bitmap ( width depth -- )`, `bitmap-off`	`(bitmap 320 8)`, `(bitmap-off)`	`vid_bitmap (320, 8)` (0: off)
The pen	`pen C`	`pen ( c -- )`	`(pen c)`	`vid_pen (c)`
A point, a line	`plot X Y`, `line X0 Y0 X1 Y1`	`plot ( x y -- )`, `line ( x0 y0 x1 y1 -- )`	`(plot x y)`, `(line x0 y0 x1 y1)`	`vid_plot`, `vid_line`
A box, a bar	`box`, `bar X0 Y0 X1 Y1`	`box`, `bar ( x0 y0 x1 y1 -- )`	`(box ...)`, `(bar ...)`	`vid_box`, `vid_bar`

What	rc (/dev/vid/draw)	HyForth (lib video)	hylang ((use "video"))	C (vera.h)
A circle, a disc	circle, disc X Y R	circle, disc (x y r --)	(circle x y r), (disc x y r)	vid_circle, vid_disc
Text	text X Y STRING	text (x y c-addr u --)	(text x y s)	vid_text (x, y, s)
Clear	clear [C]	clear	(clear)	vid_clear ()
The turtle		cs, home, fd, bk (n --), rt, lt (deg --), pu, pd, heading, seth	(cs), (home), (fd n), (rt deg) …	
VRAM	/dev/vid/vram	vpoke (addr bank c --), vpeek (addr bank -- c), vram!, vram@	(vpoke addr v), (vpeek addr), (vram! addr bytes), (vram@ addr n)	vera_write, vera_read, vera_load; vpoke, vpeek (claimed)
The palette, sprites	/dev/vid/pal, sprites	palette! (index rgb --), sprite! (n c-addr --), sprite-at (n x y --), sprite-off (n --)	(palette! i rgb), (sprite! n bytes), (sprite-at n x y), (sprite-off n)	vid_palette, vid_sprite, vid_sprite_at, vid_sprite_off
The next frame	/dev/vid/frame	vsync	(vsync)	vera_wait_frame ()
The mouse	/dev/vid/mouse	mouse (-- x y b), mouse-wait	(mouse), (mouse-wait)	vid_mouse, vid_mouse_wait

The turtle is Logo's: it starts in the middle heading up, its pen down; `fd` draws as it goes, `rt` and `lt` turn it (degrees, clockwise), `cs` clears and takes it home. `: square 4 0 do 80 fd 90 rt loop ;` draws a square. The palette's entries are `$RGB`, 4 bits each.

cc65's TGI (`tgi.h`) draws here too, so cc65's portable graphics programs run: `tgi_install (hydra_tgi)`, then `tgi_init ()`: 320 x 240 in 256 colours, lines, bars, circles, ellipses and arcs, text. The samples `shapes` (TGI) and `sketch` (`vera.h` and the mouse) are on HydraOS's SD card (`/sd/0/sample/c`).

A game paces itself by `frame`: a read waits for the next VSYNC.

The console's terminal

The screen shows the console's windows as the serial terminal does (two terminals). The console writes the window shown into `/dev/vid/term`, an ANSI terminal:

- **80 x 60** characters (`mode 80x30` and `mode 40x30` make them bigger), in the ANSI terminal's **16 colours**, 0-15 as conio numbers them. On the screen the chrome takes three rows, so a window on the screen alone is 80 x 57.
- It takes CR, LF, BS, TAB, FF; `ESC 7` and `ESC 8`, `ESC D`, `ESC E`, `ESC M`, `ESC c`; the character sets (`ESC ( 0` the DEC Special Graphics, the VT100's line drawing; `ESC ( B` ASCII; SO and SI); CSI `A B C D E F G d H f` (moves), `J` and `K`, `m` (bold, reverse, the colours 30-37, 40-47, 90-97, 100-107), `r` (a scrolling region), `s` and `u`, `?25h` and `?25l`. Others are taken and dropped.
- **Scrolling** the whole screen moves layer 1's scroll register (the map is a ring of 64 rows), so it costs a row; a region scrolls by copying its rows in VRAM.
- **The cursor** is sprite 0, an underline, blinking (`cursor on` steadies it).
- **The font**'s first 32 characters are the DEC Special Graphics; `/lib/font/cp437` is the PC's character set.

The keyboard and the mouse

With the card's **input controller**, the X16's SMC (an ATtiny861 running the X16's firmware, a PS/2 keyboard and mouse on it, on the I2C bus at `$42`), the screen is a computer of its own. `kdev` reads it, between its calls (`SRV_NAP`: the kernel): 67 times a second while keys or the mouse are coming, and 10 times a second after 2 s of nothing (a look that finds nothing is one byte on the bus). The controller is looked for as the system starts, three times a second apart; with none, `kdev` is an ordinary driver, `idle` in `ps`.

- **The keys go to the console**, as the serial terminal's do: `kdev` writes them to `#c/kbin` as a PC terminal (xterm) sends them, so a program can't tell which keyboard they came from. Ctrl-C is the window's interrupt; Ctrl-] and a digit shows a window. The layout is the US one: Shift, Ctrl, Alt, Caps Lock; the keypad's digits with Num Lock on, its cursor keys with it off; Scroll Lock the console's hold. The locks light their LEDs.
- **The mouse in the console's windows**: `kdev` sends each press, release and turn of the wheel to `#c/kbin` too, as xterm's reports. A click focuses the window under it; a program that turns on `?1000` (and `?1006`) gets the reports in its raw keys; the wheel over a window scrolls its scrollback.
- `/dev/vid/mouse` is the mouse, as Plan 9's `/dev/mouse`: a read waits for a change, then gives 49 bytes: `m` and four fields of 11 characters each with a space after it (x, y, the buttons, and the time in milliseconds). x and y are the screen's pixels (640 x 480; 320 x 240 under

`bitmap 320`); the buttons are 1 left, 2 middle, 4 right, 8 and 16 the wheel. Button changes are queued (8 of them), so a click between two reads isn't lost. Writing `m X Y` moves it.

- **The pointer** is sprite 1, an arrow, shown once the mouse has moved. `mousectl`: `pointer off`, `pointer on`, and `swap on` (the left and right buttons swapped).
- `/dev/vid/mousein` is where the moves come in: `kdev` writes `m DX DY B`, a line a change. Anything else that reads a mouse can write it too.

```
% cat /dev/vid/mousectl
pointer on
swap off
% echo m 10 -5 1 >/dev/vid/mousein
```

and a reader of `/dev/vid/mouse` gets `m 330 235 1 8215` (from the screen's middle, where the mouse starts).

PCM

The VERA plays samples from a 4K FIFO at a rate of its own: `/dev/vid/pcm` is the FIFO, `/dev/vid/pcmctl` its settings.

```
echo rate 11025 >/dev/vid/pcmctl; echo bits 8 >/dev/vid/pcmctl; echo mono >/dev/vid/pcmctl
cat drums.raw >/dev/vid/pcm; echo drain >/dev/vid/pcmctl
```

- **The samples** are the VERA's: signed, 8 or 16 bits (16 little-endian), stereo left first. (A WAV file's 8-bit samples are unsigned: `play` makes them signed.)
- **The rate** is the VERA's nearest: steps of 381 Hz, up to 48,828 Hz (`rate 22050` plays at 22,126). `rate 0` stops it. `pcmctl` reads the rate it has.
- `pcmctl`**'s commands**: `rate HZ`, `bits 8`, `bits 16`, `mono`, `stereo`, `volume N` (0-15), `reset` (the FIFO emptied at once), `drain` (waits until it's empty, so a program's end doesn't cut its last sounds off).
- **A write** fills the FIFO when it's below a quarter full, and the rest waits for the next frame, so the FIFO holds 3K to 4K while a program keeps it fed.
- `pcm` **is one task's at a time**: another's open is `E_BUSY`.
- **How fast**: from a card, 8 bits mono to about 11 kHz; from the RAM disk to about 22 kHz. 16-bit stereo at 44.1 kHz is past a 3.58 MHz 65C02.

`play` plays WAV files and a ZSM song's PCM instruments (sound).

Claiming the chip

The VERA has one set of address registers, so two tasks can't both write it. A program that wants the chip writes `claim` to `ctl` and keeps the file open: until it writes `release` or closes its last file of `#v` (its end does), the chip is its own.

- It may write any register (`hw.inc`'s names: `VERA_ADDR_L`, `VERA_DATA0`, `VERA_CTRL` ...). The driver leaves the chip alone, and other tasks' commands and reads of the chip's files get `E_BUSY`.
- The console keeps its windows' screens itself, so it paints the window shown again once the claim ends.
- **VRAM**: a claimer may use `$00000-$1AFFF` (108K) without saying so; with `claim all`, all of it, and the console's map and font are made again at the release.
- **The release** sets the chip up for the console again: the palette, the sprites (all off but the cursor and the pointer), the layers, the scales, the interrupts.
- **The PSG** stays the sound driver's: during a claim its writes are kept and made at the release. A claimer that wants the PSG claims those channels from `/dev/sndctl` too (`claim 0 65535`).
- **Interrupts**: the driver still owns IRQ line 2 during a claim: it counts frames (so `frame` works) and clears the VSYNC, LINE and SPRCOL interrupts a claimer turns on. A program that wants raster effects polls `VERA_ISR` or `SCANLINE`, or waits on `frame`. As on the X16, keep interrupt code on data port 1 and a program's on port 0, and never leave CTRL's DCSEL other than 0 for long.

```
            LDR         r0, s_ctl                       ; "/dev/vid/ctl"
            lda         #O_WRITE
            jsr         OPEN
            sta         ctl
            LDR         r0, s_claim                     ; "claim"
            LDR         r1, 5
            lda         ctl
            jsr         WRITE
            ; ... the chip is this program's: VERA_ADDR_L ... (hw.inc's names)
```

X16 code ports to the Hydra with the registers' base changed: `VERA_BASE` is `$FF20` here.

VRAM's layout

VRAM	What
`$00000-$1AFFF`	Free for programs (108K): a bitmap (320 x 240 x 8 bits is 75K), tiles, sprites
`$1B000-$1EFFF`	The console's text map (a ring of 64 rows)
`$1F000-$1F7FF`	The console's font
`$1F800`	The cursor's image
`$1F820`	The pointer's image
`$1F9C0`	The PSG's registers
`$1FA00`	The palette
`$1FC00`	The sprites' attributes

In the emulator

`node sim/run.js --vera` puts a Vera X in slot 0 (`base/sim/lib/vera.js`: the registers, VRAM, the layers, sprites and their collisions, the scan's timing and interrupts, FX, the PCM FIFO, the PSG), and `--smc` its input controller. `--screen` prints the text layer after a run, `--frame-png FILE` saves the screen, and with `-i`, `--view` shows it live in a browser (http://localhost:8016), its keys and mouse the controller's, while the terminal stays the serial console. `--sound` adds the PSG's and the PCM's sound. The emulator has the rest.

5. The shell: rc

rc is the Hydra's shell underneath everything: Plan 9's shell, by Tom Duff, small, written in 65C02 assembly and run in place from the paged ROM (`reborn/modules/rc/`). It's the shell of scripts and of C's `system()`, and HyForth's, hylang's and HyBASIC's shells hand it every line that isn't theirs. It's also a shell of its own: type `rc` for an rc session.

This chapter is a tour, with examples run on the Hydra. If you know a Unix shell, rc will look familiar, with fewer rules: every variable is a list, quoting is one kind of quote, and there's no word splitting to surprise you. The differences are summed up at the end. Plan 9's paper, *Rc: The Plan 9 Shell*, is the full story of the language.

Contents

1. The shells on the Hydra
2. Starting rc
3. Commands
4. Words and quoting
5. Variables
6. Redirection
7. Pipelines and the background
8. Control flow
9. Functions
10. Scripts
11. Built-ins
12. Namespaces from the shell
13. Windows from the shell
14. How a shell starts: the profile
15. At HyForth's prompt
16. rc for a Unix user

The shells on the Hydra

Shell	Prompt	What it is
`forth -l`	`/>`	**The login shell**: HyForth, where a line that starts with a Forth word or a number is Forth, and any other line is rc's (below)
`rc`	`%`	rc on its own
`hylang -l`	`hylang>`	hylang as a shell, by the same rule
`basic -l`	`>`	HyBASIC as a shell, by the same rule

Each window has a shell of its own. Which shell a window gets is `/lib/shell`'s line (`/bin/forth -l`); a card's `/lib/shell` comes first in `/lib`'s union, so `/bin/rc -l` in a card's `/lib/shell` gives every window rc.

Starting rc

Command	What runs
`rc`	An rc session: a prompt (`% `), each line run, until `exit` or Ctrl-D
`rc -l`	The same as a login shell: its namespace made from `/rom/lib/namespace` (and a card's), then the profile
`rc -c 'line'`	The line run, and rc ends with its status
`rc file args`	The file's commands (`$0` the file, `$*` the args)

At the prompt, the console edits the line (the line editor): Backspace, the arrows, Home and End, Ctrl-U, and Up and Down through the lines before. Ctrl-C ends what's running, and rc goes on. A command that isn't finished (an open {) gets the second prompt, a tab.

Commands

A command is a program and its arguments, run and waited for. `/bin` is searched for a bare name (and `.`: `$path` is `(. /bin)`); a name with a `/` in it is used as it is. A command's **status** is `$status`: its exit message, or its code; empty or `0` is success. Most of `/bin`'s tools are **bank programs**: a command whose program is one runs in rc's own task (the kernel's `RUN`), from a RAM bank of rc's where it stays loaded, so the next time it starts at once; any other program, and each stage of a pipeline and each command run in the background, gets a task of its own (the kernel).

Form	Meaning
`a; b`	a, then b (a new line is the same)
`a & b`	a in the background (`$apid` its task), then b
`a && b`, `a || b`	b only if a succeeds, or only if it fails
`! a`	a, its status turned over
`{ a; b }`	a group: one command
`# words`	a comment, to the line's end

```
% ls /nosuch || echo it failed
ls: /nosuch: not found
it failed
% ! ~ a b && echo not the same
not the same
```

Words and quoting

Words are separated by spaces and tabs. `'...'` **quotes**: nothing inside is special, and `''` is a quote inside it. That's the only kind of quote.

Globbing makes names from `*` (anything), `?` (one character) and `[a-z]` (one of a set; `[~a-z]` one not in it), sorted, by reading the directories; a pattern that matches nothing stays as it is.

`^` **joins words**: two single words make one; a word and a list make a list, the word joined to each; two lists of the same length join item by item. A caret is put in for you between a word and a `$` right against it (`pre$x`).

```
% x=(apple banana cherry)
% echo $x^.txt
apple.txt banana.txt cherry.txt
% echo pre^$x
preapple prebanana precherry
% echo 'single quotes: $x stays'
single quotes: $x stays
```

`#` starts a comment, so a device's name must be quoted: `ls '#c'`.

Variables

Every variable is a list of words: `x=word`, `x=(a b c)`, `x=()` (empty: as good as unset). There's no word splitting: `$x` is its words, however many spaces they hold.

Form	Value
`$x`	its words
`$#x`	how many
`$x(n)`, `$x(n-m)`	the nth word (from 1), or those from n to m
`$"x`	its words joined by spaces, one word
`` `{command} ``	the command's output, split into words at spaces, tabs and new lines

```
% x=(apple banana cherry)
% echo $x(2) $#x
banana 3
% n=`{echo hello | wc -c}
% echo $n
6
```

Variables are the environment's (the environment): a program rc starts gets a copy (`/env/x` in its task, `getenv` in C, `$x` in hylang), a list's words each with a zero byte after it. `x=1 cmd` sets `x` for that command alone.

rc's own: `$status`, `$apid` (the last background task), `$task` (rc's task), `$path`, `$prompt` (the prompt, then the second prompt), `$*` and `$0` (a script's arguments and name), `$window` (the shell's window), `$home`.

One more goes to a program's environment, though it isn't a variable of rc's: `prog`, the path rc starts the program from (`/bin/ls`, `./round`, `/ram/round`: a `$path` directory and the name, or the name as typed), put there just before it starts. A program that starts copies of itself (the C samples) finds its own file by it (`getenv ("prog")` in C, `/env/prog` as a file).

Redirection

Form	Meaning
`< file`	fd 0 from the file
`> file`, `>> file`	fd 1 to the file, made or emptied; or added to its end
`>[2] file`	fd 2 to the file (any fd: `>[3]`, `<[4]`)
`>[2=1]`	fd 2 made a copy of fd 1
`>[2=]`	fd 2 closed

```
% ls /nosuch >[2=1] | wc -l
      1
```

Devices are files, so redirection reaches them: `echo note 0 60 >/dev/sndctl` plays a note, `cat /dev/time` reads the clock.

Pipelines and the background

`a | b` runs a and b at once, a's fd 1 into b's fd 0 (a pipe, 512 bytes held); `a |[2] b` takes a's fd 2. Each stage is a program; a stage that isn't one (a group, a function) runs as `rc -c` on its text. The pipeline's status is its last stage's.

```
% ls /rom/bin | sort -r | head -3
xmodem
xd
wc
```

`a &` runs a without waiting: `$apid` is its task, `wait` waits for the background tasks (`wait $apid` for one), and `ps` shows them.

Control flow

Form	Meaning
`if(cmd) a`	a if cmd succeeds
`if not b`	right after an `if`: b if the `if` didn't run its command
`for(x in words) a`	a for each word, `$x` it (`for(x) a`: for each of `$*`)
`while(cmd) a`	a while cmd succeeds (`while()` for ever)
`switch(word){ case pattern ... }`	the commands after the first `case` whose pattern matches
`~ subject pattern ...`	success if the subject matches one of the patterns (glob patterns, matched as words, not names)

```
% for(f in $x) echo fruit: $f
fruit: apple
fruit: banana
fruit: cherry
% if(~ $x(1) apple) echo first is apple
first is apple
% switch($x(3)){
          case b*
          echo starts with b
          case c*
          echo starts with c
          }
starts with c
```

Functions

`fn name { commands }` makes one; its arguments are `$*` and `$1` ...; `fn name` alone removes it. Functions are in the environment too (`fn#name`), so a child rc has them. `whatis name` says what a name is: a function's text, a variable's value, or a program's path.

```
% fn greet { echo hello, $1 }
% greet world
hello, world
% whatis greet
fn greet { echo hello, $1 }
% whatis path prompt
path=(. /bin)
prompt=('% ' '       ')
```

Scripts

A file of rc commands runs with `rc file args`, or by its own name if its first line is `#!/bin/rc`: rc runs a file that isn't a program with the interpreter its `#!` line names (Plan 9's way). `/rom/bin/scom` is one:

```
#!/bin/rc
# scom [-l | n]: the old riff ... a song: once; its loop n more times; or -l, till it's stopped
if(~ $1 -l) play -l /rom/songs/scom.zsm
if not play /rom/songs/scom.zsm $*
```

`. file` runs a file's commands in this rc (its variables and functions stay); `exit [status]` ends a script, or rc.

Ctrl-C reaches a script's rc as well as what it's running: rc waits for that to end (a program may take the note itself, finish up and say so), then ends with its status, rather than going on to the next command.

Built-ins

These change rc's own task, so they can't be programs:

Built-in	What it does
`cd [dir]`	The current directory (none: `$home`, else `/`)
`bind [-a | -b] [-c] new old`	new at old in rc's namespace: in place of what's there, or `-a` after it, `-b` before it in a union; `-c`: creates go there
`mount [-a | -b] [-c] dev old [spec]`	A device's tree at old (`mount '#f' /mnt/c 1`: card 1's file system)
`unmount [new] old`	old's member new gone, or all of old
`newns`	The default namespace made again

Built-in	What it does
`exit [status]`	rc ends
`exec [command]`	The command's program runs in rc's place (`EXEC`): rc is gone, and the program's status is what rc's parent waits for. Only if it can't be run does rc go on
`wait [task]`	Wait for background tasks
`eval words`, `. file`, `builtin cmd`, `whatis name`, `shift [n]`	As Plan 9's

Namespaces from the shell

Each shell has its own namespace (namespaces), and rc's built-ins change it. Since there are no search paths, putting programs where `/bin` sees them is a `bind`:

```
% ls /sd/0/sample | head -5
as/
c/
hi
nsum
tick
% hi Ann
rc: hi: not found
% bind -a /sd/0/sample /bin
% hi Ann
Hello, Ann!
I'm task 6, in /.
```

`bind -a` put the card's samples **after** what `/bin` had; `bind -b` would put them **before**, so they'd run in place of the ROM's programs of the same names. The change is this shell's alone: other windows don't see it, and the programs this shell starts share it.

More uses:

- `bind -b /ram/test /bin`: try a new version of a program without touching anything else.
- `mount '#f' /mnt/c 1`: card 1's files at `/mnt/c`.
- `bind '#c2' /dev`: this shell's `/dev/cons` becomes window 2's console.
- `ns` shows the namespace as the `bind` and `mount` lines that would make it; `newns` starts again from the default.

```
% ns | tail -4
bind -c '#fr/2/lib' /lib
bind -a '#fs/lib' /lib
bind -a '#fx/lib' /lib
bind '#P' /pc
```

Windows from the shell

The console's windows (windows) are files and commands:

- `new-window` makes a window with a shell, in this group; `new-window -g` in a group of its own (a new session); `new-window 'ls -l; sleep 5'` runs a command in it. It isn't waited for: the window goes when its program ends.
- `echo layout rows >/dev/wctl` tiles the group's windows; `echo layout tabs >/dev/wctl` shows them one at a time.
- `echo compiling >/dev/label` titles the window; `echo status 3 of 9 >/dev/wctl` sets its footer's status.
- `cat /dev/text` is the window's screen and scrollback as text; `cat /dev/snarf` the cut buffer.
- `$window` is the shell's window.

How a shell starts: the profile

A login shell builds its namespace from `/lib/namespace` (`newns`), then runs its profile: rc's is `/lib/profile`, HyForth's `/lib/forth/profile.fs`, hylang's `/lib/hylang/profile.hl`. Each is found through `/lib`'s union, so one on a card or the RAM disk takes the place of the ROM's. rc's:

```
# /rom/lib/profile: what each shell runs as it starts (rc -l: after newns), before its first prompt.  $window is
# its window (init's: the kernel starts init for each window): its console goes at /dev, and its notes (Ctrl-C) to
# the shell's note group.

prompt=('% ' '       ')
if(~ $#window 0) window=0
if(! ~ $window 0){
          unmount '#c' /dev
          bind -a '#c'^$window /dev
}
echo -n group >/dev/consctl
```

So each window's shell has its own window's console at `/dev`, and takes its window's Ctrl-C. HyForth's profile does the same in Forth. Your own settings go in a profile of your own on a card: copy the ROM's and add to it.

At HyForth's prompt

The login shell is HyForth (`forth -l`), prompt `/>`. A line whose first word isn't a Forth word or a number is an rc line, run whole by an rc of its own (`rc -c`) and waited for, so pipes, redirection, globbing and quoting are rc's there. But:

- **a simple command runs in HyForth's own task**: a line that's a program and its arguments, with none of rc's special characters in it, runs in HyForth's task if its program is a bank program (`RUN`), no rc started, and stays loaded for the next time. Any other line is rc's.
- **each line is a new rc**: a variable or function set on one line is gone on the next (`$status` is kept for the next line). For an rc session, type `rc`.
- **the line's last command runs in rc's place**: `rc -c` runs the last command of its text with `EXEC` when it's a simple command whose first word is a program, so `ps` at the prompt takes one task, as if HyForth had started it, and its status is HyForth's to see. A pipeline, a sequence's earlier commands, a built-in or a function run as rc's always do.
- `%` **before a line** makes it rc's whatever its first word (`% free` when a Forth word `free` is loaded).
- `cd`, `bind`, `mount`, `unmount` **and** `newns` **are Forth words** there, taking rc's arguments, so they change the shell's own directory and namespace, as rc's built-ins do.

HyForth has the rest.

---

rc for a Unix user

In sh or bash	In rc
`x="a b"`	`x='a b'` (one word) or `x=(a b)` (two)
`"$x"`	`$x` (no splitting, ever) or `$"x` (the words joined)
`$(cmd)`	`` `{cmd} ``
`if [ ... ]; then ...; fi`	`if(test) cmd` or `if(~ $x pattern) cmd`
`else`	`if not`
`for x in a b; do ...; done`	`for(x in a b) cmd`
`case $x in a*) ...;; esac`	`switch($x){ case a* ... }`
`f() { ...; }`	`fn f { ... }`
`2>&1`	`>[2=1]`
`$?`	`$status` (a message or a number; empty or `0` is success)
`$#`	`$#*`
`$PATH` and search paths	`bind` into `/bin`

In sh or bash	In rc
`export x`	(every variable is in the environment)
`source file`	`. file`

6. The languages

The Hydra has three languages of its own, each a module of the paged ROM, each a shell too, and each with the whole system a word away:

Language	Command	What it is	Best for
HyForth	`forth`	Forth 2012, subroutine threaded, its word sets as libraries; the login shell	Speed, poking at hardware, small tools, interactive exploration
hylang	`hylang`	danlang, a lisp with integers of any size, exact fractions, hashes and closures; a bytecode machine and native code	Programs that work with lists, text and structure; scripting the system
HyBASIC	`basic`	A structured BASIC in QuickBASIC's way: labels, `SUB`s and `FUNCTION`s, records; compiled to a stack machine	Programs that read like the classic 8-bit ones; graphics, sound, and learning

All three share:

- **The numbers**: one number system, exact, of any size, in any base (below).
- **The system**: every system call is a word or a function in each (`sys-open`, `(sys-open ...)`, `SYS "OPEN"`), and the devices are files each can open.
- **The shell rule**: as a login shell (`forth -l`, `hylang -l`, `basic -l`), a line that's the language's is run as the language, and any other line goes to rc (the shell).
- **The devices' libraries**: the same names for GPIO, I2C, SPI, sound, video, the windows and the tasks in HyForth and hylang, and C's `hydra.h`, `snd.h` and `vera.h` follow them.

C and assembly are the programming guide's.

Contents

1. HyForth
2. hylang
3. HyBASIC

The numbers they share

The Hydra has **one number system**, and every language has all of it:

- **Integers of any size**, to 255 bytes (614 digits): `2 ^ 100` is all 31 of its digits.
- **Fixed decimals**, exact: `0.1 + 0.2` is `0.3`, exactly.
- **Rationals**: `10 / 4` is `5/2`, in lowest terms.
- **Complex numbers**: `1+2i`; the square root of -4 is `2i`.
- **The math functions** (square root, exponential, logarithm, sine, cosine, tangent, arctangent, powers): exact when their answer is (`sqrt 9/4` is `3/2`), else correctly rounded to a precision, 12 significant digits at the start, up to 100.
- **Any base**, for reading and writing: `#xFF`, `#b0.1` (a half), `#c+-0` (balanced ternary), `#16r1F`; a base set once is followed everywhere numbers are read and written.

Language	Its numbers
hylang	Every number is one: `(/ 10 4)` is `5/2`, `(sqrt 2)`, `(base "x")`
HyForth	`lib numbers`: a number stack of its own (`n+`, `n.`, `nsqrt`, `nformat`); a number in the text that isn't a cell goes on it
HyBASIC	Every number is one, of one type: `PRINT 1 / 3` is `1/3`, `BASE "x"`, `DIGITS 30`
C	`num.h`: a function for each operation, and `printf`'s `%N`
Assembly	`numbers.inc`'s `NUMCALL` and `MATHCALL`
rc	`calc`: `calc 2/3 + 0.5` is `7/6`, `calc -b x 255` is `FF`

They're all **the same code**: two library modules of the paged ROM, `numbers` and `math`, working on **the same stored format**, so a number one language writes to a file is the same number when another reads it. Libraries describes the format and how to call them. A language keeps its quickest numbers itself (hylang's 15-bit fixnums, HyBASIC's 32-bit integers, HyForth's cells) and hands them to the libraries past that.

How fast

Twenty benchmarks are written in each language (`/sd/0/bench` on HydraOS's card: `bench.fs`, `bench.hl`, `bench.bas`), the same algorithms, sizes and results, each in the language's own way. `node sim/bench.js` runs them in the emulator. At 3.58 MHz, in October 2026, the time for all twenty, and the geometric means of the ratios:

	HyForth	hylang	HyBASIC
All twenty	9.6 s	17.7 s	55.1 s
Against HyForth	1x	1.9x	7.4x
Against hylang		1x	3.2x

By kind, HyBASIC's time against HyForth's: calls 14.9x, loops 10.1x, arithmetic 4.5x, arrays 6.2x, lists 5.8x, text 4.4x. HyBASIC comes nearest where a statement does much (a number's text, a matrix), and is farthest where a step is small (an empty `FOR` loop, a `FUNCTION`'s call). For comparison, the Hydra's first BASIC (Microsoft's, by way of EhyBASIC) took some 44 times HyForth's time.

HyForth

HyForth (`forth`) is the Hydra-16's Forth: a **Forth 2012** system whose core is in the paged ROM, with the rest of the standard's word sets as pre-compiled libraries it loads as they're wanted. It passes the Forth 2012 test suite's tests of Core, Core Extension, Exception, Facility, File Access, Programming-Tools, Search-Order, String, Double-Number, Locals, Memory-Allocation and Block. It's also **the login shell**: at its prompt, a line is Forth if its first word is a Forth word or a number, and an rc command line otherwise.

Its sources are `reborn/modules/forth` (the core) and `reborn/forthlib` (the libraries).

Contents

1. Starting it
2. The basics
3. Libraries
4. HyForth as a shell
5. Files
6. The Hydra's words
7. The terminal and keys
8. Sound
9. Devices
10. Tools: words, see, disasm
11. Memory-Allocation, Locals and Blocks
12. Numbers
13. Errors and Ctrl-C
14. Memory
15. How it's built

Starting it

Typed at the shell's prompt	What runs
`forth`	Forth at the console: each line typed is run, then `ok`
`forth -l`	Forth as a login shell: its namespace made, `startup.fs` and `profile.fs` run, then its prompt (`/> `)
`forth file.fs a b`	A script: the file run, then forth ends (with code 1 after an error). A file whose first line is `#!/bin/forth` runs by its name too

`bye` ends a plain forth, and `exit` the shell (`exits ( n -- )` with a code). A script's arguments are `argc ( -- n )` and `arg ( n -- c-addr u )`: `forth file.fs a b` has 3, the file's name `0 arg`.

The basics

Words run left to right; numbers go on the data stack, and words take their arguments from it. Names are in lower case and found in either case (`DUP` is `dup`).

```
/> 2 3 + .
5
/> 1 2 3 .s
<3> 1 2 3
/> . . .
3 2 1
```

Numbers are 16 bits, read in `base` (decimal at the start). A prefix gives another: `$ff` hex, `#10` decimal, `%101` binary, `'a'` a character's code. `hex` and `decimal` set `base`; `.` prints signed, `u.` unsigned.

```
/> $ff . #10 . %101 . 'a' .
255 10 5 97
/> -1 u.
65535
```

Definitions are Forth 2012's: `: name ... ;`, `constant`, `variable`, `value` (and `to`), `create` and `allot`, `defer` and `is`, `marker`. The control structures (`if` `else` `then`, `begin` `until`, `begin` `while` `repeat`, `do` `loop`, `?do`, `case` `of` `endof` `endcase`) work in definitions.

```
/> : sq ( n -- n*n ) dup * ;
/> 7 sq .
49
/> : count-to ( n -- ) 0 ?do i . loop ;
/> 3 count-to
0 1 2
/> see sq
: sq dup * ;
```

Compile-only words (the return stack's `>r`, `r>`, `i`; the control structures; `;`, `literal`, `postpone`, `."`) give an error outside a definition, and forth goes on. `.( text)` is `."`'s form for the prompt.

```
/> 1 >r
>r: compile only
```

Libraries

The core is Forth 2012's Core word set. As it starts, forth loads `/lib/forth/startup.fs`, which loads Core Extension, Exception, File Access and Programming-Tools; `forth -l` adds the shell's. The rest are loaded when they're wanted: `lib name` at the prompt, or `require name.fl` in a program.

```
/> libs
forth coreext exception file tools shell
/> lib hydra
/> lib random
/> -lib random
/> libs
forth coreext exception file tools shell hydra (random)
```

`libs` lists the core, then each library; (`random`) is one `-lib` took out of the search (its code stays, so what was compiled with it still runs), and `lib random` puts it back. `lib name` loads `name.fl`, else `name.fs`, from the current directory or `/lib/forth` (the current directory's trouble, a directory of that name or an `i/o error` on `/pc` say, is said only if `/lib/forth` has no such file either). A file that can't be opened or loaded isn't noted as included, so `lib` and `require` can try it again. A `marker` made before a library takes it out with the rest.

Library	Words
`coreext.fl`	Core Extension: `?do`, `case`, `value`, `defer`, `:noname`, `pick`, `roll`, `.(`, `c"`, `s\"` ... (at the start)
`exception.fl`	`catch`, `throw` (at the start)
`file.fl`	File Access: `open-file`, `read-file`, `read-line`, `write-file`, `include`, `require` ... (at the start)
`tools.fl`	Programming-Tools: `.s`, `?`, `dump`, `see`, `words`, `[if]`, `synonym` ...; `libs`, `lib`, `-lib` (at the start)
`shell.fl`	The shell (`forth -l`'s)
`facility.fl`	Facility: `key?`, `ms`, `time&date`, `page`, `at-xy`, structures; the terminal's words and keys
`string.fl`	String: `compare`, `search`, `/string`, `-trailing`, `sliteral`, `substitute`, `replaces` ...
`search.fl`	Search-Order: `wordlist`, `set-order`, `also`, `only`, `previous`, `definitions` ...; `library name` ... `end-library`
`double.fl`	Double-Number: `d+`, `d-`, `d.`, `d.r`, `d<`, `d=`, `dmax`, `m*/`, `2constant`, `2value`, `2rot` ...

Library	Words
memory.fl	Memory-Allocation: allocate, free, resize
locals.fl	Locals: {: ... :}, (local), locals|
block.fl	Block: block, buffer, update, flush, load, list, thru ...
hydra.fl	The Hydra's words: the sys- words, sh, run, banks, directories, notes, argc, arg, sys, ctl
hydra.fs	The system's constants and error codes (o_rdwr, e_noent ...), in a word list of their own, hydra
disasm.fl	disasm; with it, see of an assembly word shows its instructions
bits.fl	tbit, sbit, cbit
random.fl	random (u -- u') (0 to u-1), rand, rand32, rseed
sound.fl	Sound
numbers.fl	Numbers: exact numbers of any size on a number stack
gpio.fs i2c.fs spi.fs cons.fs proc.fs clock.fs disk.fs pc.fs video.fs	Devices, in Forth source

Your own libraries are .fs files: put them where lib looks, or in a card's /lib/forth (a card's /lib is bound before the ROM's, so a card's startup.fs or profile.fs takes the place of the ROM's too).

HyForth as a shell

Under forth -l, a line whose first word is a Forth word or a number is Forth; any other line goes to rc, whole (rc -c), so pipes, redirections, quotes and $name are rc's. % at a line's start sends it to rc whatever its first word is.

```
/> ls /rom/lib/forth
bits.fl
clock.fs
...
/> echo hello | wc -c
      6
/> foo
rc: foo: not found
```

A line that's a plain command, a program and its arguments with none of rc's special characters in it, runs in forth's own task when its program is a bank program (the kernel's RUN: most of the tools), no rc started, and the program stays loaded for the next time. Any other rc line is an rc of its own, so a variable it sets ends with it, and rc runs the line's last command in its own place (EXEC), so a plain command takes one task (at HyForth's prompt).

The shell's own words. An rc line runs in a task of its own, which can't change forth's directory or namespace, so these are Forth words that read their arguments as rc does: cd [dir], bind [-a|-b] [-c] new old, mount, unmount, newns.

The prompt is a format: s" %v%d> " prompt (the start's). %v the card (0:), %d the directory on it, %p the whole path, %t the task, %w the window, %% a %. While a definition goes on over more than one line, the prompt is prompt2's.

```
/> s" %t:%p%% " prompt
4:/% cd /rom/lib
4:/rom/lib% s" %v%d> " prompt
/rom/lib>
```

Statuses. status (-- n) is the last rc line's code, and $status rc's, the program's message. A line ending in & isn't waited for: its task is $apid, and wait (task --) waits for it.

```
/> nosuchprogram
rc: nosuchprogram: not found
/> status .
1
/> echo $status
not found
```

The environment: `getenv ( c-addr1 u1 -- c-addr2 u2 )`, `setenv ( c-addr1 u1 c-addr2 u2 -- )` (the name first), `unsetenv ( c-addr u -- )`. Forth's environment is the shell's, so what it sets the rc lines see:

```
/> s" greeting" s" hello" setenv
/> echo $greeting
hello
```

Programs as values:

Word	Stack	Does
`sh`	`( c-addr u -- n )`	An rc command line run, waited for: its code
`run`	`( c-addr u -- n )`	A program run (its name and arguments, no rc), waited for
`sh-out`	`( c-addr u -- c-addr2 u2 )`	An rc line's output, as a string (`status` its code)
`output-of`	`( xt -- c-addr u )`	What a word writes, as a string
`|`	`( xt "line" -- )`	A word's output into the rest of the line: `' words | wc -l`
`piped`	`( xt c-addr u -- )`	The same, the command line a string
`spawn`	`( c-addr u -- task )`	A program not waited for
`send`	`"n line"`	The line typed in window n, as if at its keyboard

```
/> s" echo hi there" sh-out type
hi there
/> ' words | wc -l
    115
```

The strings `sh-out` and `output-of` give are in the dictionary's free space: use them before anything's added to it (`move` one somewhere to keep it).

Files

File Access's words work on the system's files: a fileid is an fd, and an ior is 0 or -512 less the system's error code, whose text `ior>text` gives (in `hydra.fl`).

```
/> s" /none" r/o open-file . .
-544 0
/> s" /none" r/o open-file nip ior>text type
not found
/> create buf 64 allot
/> s" /rom/README" r/o open-file throw value fd
/> buf 40 fd read-file throw buf swap type
The Hydra-16's ROM disk
================
/> fd close-file throw
```

`include file` and `require file` load source; a name with no `/` that isn't in the current directory is `/lib/forth`'s. In `hydra.fl`, Gforth's words for directories: `open-dir ( c-addr u -- dirid ior )`, `read-dir ( c-addr u1 dirid -- u2 flag ior )`, `close-dir`, `get-dir ( c-addr u1 -- c-addr u2 )`, `set-dir ( c-addr u -- ior )`, `=mkdir ( c-addr u mode -- ior )`. `ctl ( c-addr1 u1 c-addr2 u2 -- )` writes the text c-addr2 u2 to the file c-addr1 u1 in one write, as a device's `ctl` wants (`s" /dev/sndctl" s" volume 100" ctl`).

The Hydra's words

`lib hydra` (`hydra.fl`) has a word for **each system call**, `sys-` and its name in lower case (`sys-getpid`, `sys-sleep`, `sys-banks-alloc` ...; the call reference gives each one's stack), and `require hydra.fs` the system's constants (`o_rdwr`, `e_noent` ..., in their own word list).

Words	Does
`note ( task n -- )`, `note-group ( group n -- )`	A note sent to a task, or a note group
`on-note ( xt -- )`	The notes that come (but Ctrl-C and kill) given to `xt ( n -- flag )`: true, forth goes on; false, as Ctrl-C
`pause`	The other tasks' turn
`bank! ( bank -- )`, `bank@`, `bank-window ( -- addr )`, `seg-bank!`	A RAM bank of the task's at `$8000` (memory)
`code-banks ( flag -- )`	False: colon definitions from now on compiled into the dictionary; true (as forth starts): into the task's banks
`sys ( addr a x y -- a x y p )`	Machine code called with its registers, its flags after
`sh`, `run`, `ctl`, `argc`, `arg`, `ior>text`, the directories'	As above

```
/> : h ( n -- flag ) ." note " . true ;
/> ' h on-note sys-getpid 16 note 7 .
note 16 7
```

The terminal and keys

The console is an ANSI terminal. `lib facility` has, beside `page` and `at-xy ( x y -- )`: `clear-line`, `clear-below`, `cursor-up ( n -- )` (and `-down`, `-right`, `-left`), `cursor-save`, `cursor-restore`, `cursor-off`, `cursor-on`, `form ( -- rows cols )`; colours as C's conio numbers them, `color ( n -- )` and `bgcolor ( n -- )`, with `black` ... `white` and `bright`; `plain`, `bold`, `dim`, `underline`, `blink`, `reverse`; `beep`. Keys: `key` and `key?`, and `ekey`, `ekey?`, `ekey>char`, `ekey>fkey` with `k-up`, `k-down` ... `k-f12` for the special keys.

```
: hello  page  red color bold ." Hello"  plain  0 2 at-xy ;
```

Sound

`lib sound` drives the sound driver (sound), with C's and hylang's names, the channel first:

Word	Does
`snd-reset`, `snd-volume ( v -- )`	Everything cleared; the master volume (0-200)
`snd-claim ( mask -- )`, `snd-release`, `snd-claim-psg ( mask -- )`, `snd-release-psg`	Claim the YM2151's channels (bit n, channel n), or the PSG's (bit n, channel 8 + n)
`snd-patch ( ch n -- )`, `snd-note ( ch note -- )`, `snd-off ( ch -- )`	A patch; a note on (MIDI's: 60 middle C); off
`snd-level ( ch v -- )`, `snd-pan ( ch pan -- )`, `snd-bend ( ch n -- )`	Volume (0-127), speakers (1 left, 2 right, 3 both), pitch bend
`snd-drum ( ch n -- )`, `snd-freq ( ch hz -- )`, `snd-glide ( ch n -- )`	A General MIDI drum; a frequency; a slide to a note
`snd-wave ( ch w width -- )`	A PSG voice's waveform (0 pulse, 1 saw, 2 triangle, 3 noise) and width
`snd-lfo ( rate pmd amd wave -- )`, `snd-sens ( ch pms ams -- )`, `snd-noise ( n -- )`	The YM2151's LFO, sensitivity, noise
`snd-reg ( reg value -- )`, `snd-regs ( c-addr -- )`	A register written; all 256 as written
`note-of ( c-addr u -- n )`, `tune ( c-addr u ch tempo -- )`	A note's name as its number; a string of notes and beats played
`snd-play ( c-addr u times -- status )`, `snd-mml ( ch c-addr u -- status )`, `snd-chord ( ch c-addr u -- status )`	`play` run: a song, a line of MML, a chord

```
/> s" C#4" note-of .
61
/> s" C4 1 E4 1 - 1 G4 2" 0 240 tune
```

Devices

The device libraries are Forth source over the devices' files, with hylang's names; each uses only the words forth starts with, so each is also an example of driving its device by hand. A failure THROWs the system's error.

Library	Words
gpio	`gpio ( pin -- level )`, `gpio! ( pin level -- )`, `gpio-in ( pin -- )`, `gpio-out`, `gpio-port ( -- byte )`, `gpio-port!`, `gpio-ddr! ( byte -- )`, `gpio-ca1! ( rise? -- )`, `gpio-ca2! ( n -- )`, `gpio-wait ( -- count )` (CA1's next edge), `gpio-state ( -- c-addr u )`
i2c	`i2c-read`, `i2c-write ( addr reg c-addr u -- )`, `i2c-speed ( khz -- )`, `i2c-reg-size ( n -- )`, `i2c-devices`, `i2c? ( addr -- flag )`
spi	`spi ( dev c-addr u -- )` (a transaction: what came back in the bytes' place), `spi-read ( dev c-addr u -- )`, `spi-mode ( dev mode -- )`
cons	`window ( -- n )`, `windows ( -- c-addr u )`, `new-window ( c-addr u -- )`, `new-group`, `show-window ( n -- )`, `window-label`, `window-status`, `window-ctl ( c-addr u -- )`
proc	`task-args`, `task-cwd`, `task-env`, `task-ns`, `task-regs ( task -- c-addr u )`, `task-mem ( task addr c-addr u -- )`, `task-ram ( task bank offset c-addr u -- )`
clock	`set-date ( c-addr u -- )`, `rtc ( -- c-addr u )` (`running`, `stopped`, `none`)
disk	`disk-ctl ( disk -- c-addr u )` (a disk by its letter), `disk-start`, `disk-stop ( disk -- )`, `cards ( -- mask )`
video	The Vera X: `bitmap`, `pen`, `plot`, `line`, `box`, `bar`, `circle`, `disc`, `text`, `clear`; the turtle (`cs`, `fd`, `rt` ...); `vpoke`, `vpeek`, `vram!`, `vram@`, `palette!`, `sprite!`, `sprite-at`, `sprite-off`, `vsync`, `border`, `mouse`, `mouse-wait` (video)
pc	`pc? ( -- flag )`: the PC tool answers

```
/> lib gpio
/> 2 gpio .
1
/> 4 1 gpio!
/> lib i2c
/> i2c-devices
50
/> 1 i2c-reg-size  $50 0 s" hello" i2c-write  $50 0 pad 5 i2c-read  pad 5 type
hello
/> lib proc
/> sys-getpid task-cwd type
/rom/lib
```

(Pin 2 high; a memory chip at I2C address `$50`, its registers a byte's address.)

Tools: words, see, disasm

`words` lists the first word list in the search order, newest first: each word's code address in hex, three flags, and its name. `1` a literal (a constant), `i` immediate, `a` or `f` assembly or Forth.

```
/> 5 constant five
/> words
 41EC l-f five            41D5 --f h               409F --a sys-rtc
 ...
 A271 --a swap            A268 --a drop            A256 --a dup
```

`see name` shows a definition; with `lib disasm`, an assembly word's instructions, and `disasm ( addr n -- )` any code's, each as `as` writes it. `dump ( addr u -- )` shows memory, `.s` the stack, `? ( addr -- )` a cell.

```
/> lib disasm
/> see 2drop
code 2drop
 A2E3  E8        inx
 A2E4  E8        inx
 A2E5  60        rts
end-code
```

Memory-Allocation, Locals and Blocks

`lib memory`: `allocate ( u -- a-addr ior )`, `free ( a-addr -- ior )`, `resize ( a-addr u -- a-addr2 ior )`. The heap is the top of the dictionary's space. At the shell's prompt, Forth's `free` then shadows the `free` program: `% free` runs the program.

`lib locals`: a definition's named values, Forth 2012's way. Between `{:` and `:}`: arguments taken from the stack (the last is its top), then after `|` values that start undefined, then after `--` a comment. A local's name pushes its value, `to name` sets it.

```
/> lib locals
/> : lt7 {: a b :} b a ; 7 8 lt7 . .
7 8
/> : lt12 {: a | b c :} 20 to b a 21 to a 22 to c a c b ; 19 lt12 .s
<4> 19 21 22 20
```

`lib block`: Forth's blocks, each 1024 bytes of a file, `blocks.fb` in the current directory (made when first wanted) or the one `s" name" open-blocks` names. `n block` gives a block's buffer, `update` marks it changed, `flush` writes the changed ones; `n load` interprets a block, `a b thru` blocks a to b, `n list` shows one. Blocks are kept in the task's RAM banks too (128 of them).

```
/ram> lib block
/ram> 1 block 1024 bl fill  s" 2 3 + . blk @ . \ the rest 7 ." 1 block swap move  update flush
/ram> 1 load
5 1
```

Numbers

`lib numbers`: the Hydra's numbers in Forth, the same as hylang's and HyBASIC's (the numbers): integers of any size, fixed decimals, rationals and complex numbers, all exact, and the math functions. They have a stack of their own, the **number stack** (64 numbers, in a RAM bank), and the words have hylang's names with an `n` before them where Forth has the name. A number in your text that Forth doesn't read as a cell or a double goes on the number stack: `1.25`, `2/3`, `#xFF`, `2i`, `1+2i`, `10000000000000000000`, and a cell past 16 bits (`70000`).

```
/> lib numbers
/> 1/3 1/6 n+ n.
1/2
/> 0.1 0.2 n+ 0.3 n= .
-1
/> 2 s>n 100 s>n npow n.
1267650600228229401496703205376
/> 2 s>n nsqrt n.  npi n.
1.41421356237 3.14159265359
/> : harm ( n -- ) ( N: -- h ) 0 s>n 1+ 1 do 1 s>n i s>n n/ n+ loop ;  20 harm n.
55835135/15519504
/> 255 s>n 255 s>n 255 s>n s" {} is {x} and {#b}" nformat
255 is FF and #b11111111
```

What	Words (the number stack's effect after `N:`)
The number stack	`ndepth ( -- u )`, `ndrop`, `ndup`, `nover`, `nswap`, `nrot`, `n.s`
Arithmetic	`n+`, `n-`, `n*`, `n/` (exact: `10/4` is `5/2`), `n/mod`, `nnegate`, `nabs`, `ngcd`, `npow ( N: x y -- x^y )`
Comparisons and tests	`n=`, `n<`, `n> ( -- flag ) ( N: x y -- )`, `n0=`, `n0<`, `ncompare`; `int?`, `fixed?`, `rational?`, `complex?`
Cells and doubles	`s>n ( n -- )`, `d>n ( d -- )`; `n>s ( -- n )`, `n>d ( -- d )`
Conversions	`truncate`, `nfloor`, `nround`, `to-fixed ( u -- )`, `to-rational`, `rational.n`, `rational.d`, `complex`, `nrandom`, `nseed`, `nfib`
Bits (any size)	`nand`, `nor`, `nxor`, `ninvert`, `nlshift`, `nrshift ( u -- )`, `nbit? ( u -- flag )`
The math functions	`nsqrt`, `nexp`, `nlog`, `nsin`, `ncos`, `ntan`, `natan`, `npi`; `digits ( -- a-addr )`, their precision (1-100 significant digits: 12)
Text	`n.`, `n.base ( c-addr u -- )`, `n>str ( -- c-addr u )`, `>n ( c-addr u -- flag )`, `nformat ( c-addr u -- )`
Bytes	`nbytes ( -- c-addr u )`, `nfrom-bytes ( c-addr u -- )`

What	Words (the number stack's effect after `N:`)
Variables	`nvariable name`, `n@`, `n!`, `nconstant name`, `nvalue name`, `nliteral`

The base: `base` is the numbers' base as it is the cells': after `hex`, `n.` shows `FF`. `s" b" set-base` selects any of hylang's bases by its name: `s" #x"` hexadecimal with its prefix shown, `s" c"` balanced ternary, `s" [01]"` digits of your own ...; `get-base` gives it.

Division by zero is THROW -10, a number past the libraries' limits -11, the number stack empty -45 or full -44. Each word is a call of the library: about 1.7 ms for `1 s>n n+`, more as the numbers grow.

Errors and Ctrl-C

An error says what it was (with the file and line, in a file being included) and empties both stacks; the prompt comes back. A word that isn't there is `name ?` in a plain forth (under `forth -l` the line goes to rc, which says it isn't a program); the standard's errors have their text (`stack underflow`, `division by zero`), and the system's are its own (`not found`). `catch` and `throw` are Forth 2012's.

Ctrl-C stops the word running (THROW -28, `interrupt`), whatever it's doing: a loop, a wait for a key, a program `sh` or `run` started (which it ends too).

Memory

- **The dictionary** is forth's RAM after its own variables, up to `$7F00`: about 18K free at `forth -l`'s prompt (`unused`).
- **A colon definition's code goes to a RAM bank**, taken as they're wanted: the dictionary keeps only its header and 6 bytes, so 100 definitions of a couple of hundred bytes take about 1.2K of the dictionary, not 22K. A call to a word in another bank costs about 90 cycles more.
- **Libraries** load into the dictionary. `-lib` doesn't free anything; a `marker` does (`marker -work` ... `-work`).
- **The task's banks** (16 of 8K for each memory card) are at `$8000` through `bank!`: take them with `sys-banks-alloc` so your program and the libraries don't use the same ones. Forth keeps the index of its words in the task's last bank.
- A definition whose code is in a bank can't select another with `bank!` (its code would vanish from under it): it THROWs -21. Compile such a word with code banks off:

```
false code-banks
: fill-bank ( bank -- ) bank@ swap bank!  bank-window 8192 0 fill  bank! ;
true code-banks
```

How it's built

- **The core** (`modules/forth`) is a module of the paged ROM, run in place: the inner interpreter, Core's words in assembly, the text interpreter, the compiler, and the words that load files. Its code is shared by every task running forth.
- **Subroutine threaded**: a word's execution token is its code's address, and a colon definition compiles to a run of `jsr` instructions (literals and `if`'s test inline, a few short words copied in whole). The **data stack** is in the program's zero page, its low and high bytes apart, indexed by `,x`; the return stack is the 65C02's.
- **The libraries** (`forthlib/*.s`) are assembled by the build into `.fl` files: pre-compiled images loaded into the dictionary at `here` and relocated, with their headers. A library calls the core by its labels, never another library; a library is for the core it was built with.
- **The index**: a hash of every word's name, kept in a RAM bank, so a word is found without reading the whole dictionary.
- **Benchmarks**: HyForth is the Hydra's fastest language: over twenty benchmarks, hylang takes 1.9 times its time and HyBASIC 7.4 times (how fast).

hylang

hylang is **danlang on the Hydra-16**: a lisp with integers of any size, exact fractions, hashes and closures, and the Hydra's files, tasks, memory and devices a function away. The language is danlang's exactly: its specification is danlang's `reference.md` (https://github.com/SNSTRUTHERS/danlang), and danlang's own regression suite (1,530 checks) runs on the Hydra unchanged. It's written in 65C02 assembly (`reborn/modules/hylang/`), eight banks of the paged ROM, with a bytecode machine and native code.

Contents

1. Starting it
2. The basics
3. Numbers
4. Lists, strings and hashes
5. Control
6. Files and programs
7. The Hydra's built-ins
8. The device libraries
9. Every system call
10. Errors and Ctrl-C
11. hylang as a shell
12. Where hylang differs from danlang
13. How it's built

Starting it

Typed at the shell's prompt	What runs
`hylang`	The REPL: each expression typed is evaluated, its value shown after `=>` ; `exit` ends it
`hylang file.hl a b`	A script: the file run, then hylang ends (`args` is `{"file.hl" "a" "b"}`); a file whose first line is `#!/bin/hylang` runs by its name too
`hylang -l`	hylang as a login shell, over rc (below)

```
/> hylang
hylang (danlang on the Hydra-16)
Type 'exit' to Exit

hylang> (+ 1 2)
=> 3
```

At the REPL the console edits the line (Up and Down for the last ones), and an expression that isn't closed asks for another line. `(exit n)` ends a script with a status.

The basics

An **S-expression** `(f a b)` calls `f` with the values of `a` and `b`; a **Q-expression** `{a b}` is a list, as it's written (data, or code to run later); `[a b]` is a list of the values. Names are case-insensitive.

```
hylang> (def {x} 5)
=> NIL
hylang> (* x x)
=> 25
hylang> (fun {sq n} {* n n})
=> NIL
hylang> (sq 12)
=> 144
hylang> (map (fn {n} {* n n}) {1 2 3})
=> {1 4 9}
```

- `def` binds globally, `set` in the current scope, `set!` changes the nearest binding.
- `fn` makes a function (a closure); `fun` defines one by name.
- A function given fewer arguments than its formals is **partially applied**: with `(fun {add a b} {+ a b})`, `(map (add 10) {1 2 3})` is `{11 12 13}`. Extra arguments past the formals are `&1`, `&2` ...
- **Seven prefixes** are shorthand for a call: `?(c a b)` is `(if c a b)`, and `=` `set`, `:` `def`, `#` `hash-create`, `@` `fn`, `.` `unpack`, `~` `format`.

Numbers

Integers have no fixed size (on the Hydra, up to 255 bytes: some 614 digits), **division is exact**, and a decimal written with a point is a fixed decimal, exact too:

```
hylang> (pow 2 100)
=> 1267650600228229401496703205376
hylang> (/ 1 3)
=> 1/3
hylang> (+ 1/3 1/6)
=> 1/2
hylang> (+ 0.1 0.2)
=> 0.3
```

Bases are read with `#` (`#xFF`, `#b101`, `#16rFF`) and written with `(to-str n "x")` (`FF`; `"#x"` writes `#xFF`) or `format`'s `{x}`. A fraction in another base has a radix point where it ends (`(to-str 1/2 "#b")` is `#b0.1`), and complex numbers are written as they're read, `1+2i`. `(base "x")` makes a base the one everywhere: numbers are printed in it and read in it, a program's too (there a bare number starts with a digit: `0FF`), until `(base "d")`.

```
hylang> (base "x")
=> "d"
hylang> (+ 0FF 1)
=> 100
hylang> (base "d")
=> "x"
```

`hex`, `bin`, the bit functions (`bit-and`, `shl` ...), `lo`, `hi` and `word` are for bytes and words. Complex numbers, `random`, `truncate`, `to-fixed`, `div` and `mod` are there too, and `(number-bytes n)` and `(bytes-number b)` give a number's bytes in the stored format every language of the Hydra shares.

The math functions, `sqrt`, `exp`, `log`, `sin`, `cos`, `tan`, `atan`, `(pi)` and `pow` of any real power, are exact when their answer is, and otherwise a fixed decimal of `(digits)` significant digits (12 at the start; `(digits n)` sets it, 1 to 100), correctly rounded:

```
hylang> (list (sqrt 9/4) (sqrt 2) (pow 8 1/3) (pow 2 0.5))
=> {3/2 1.41421356237 2 1.41421356237}
hylang> (list (sin 1) (log -1) (exp 100))
=> {0.841470984808 3.141592653591 2688117141820000000000000000000000000000000}
hylang> (digits 30)
=> 12
hylang> (pi)
=> 3.14159265358979323846264338328
```

Lists, strings and hashes

```
hylang> (filter even? (range 1 10))
=> {2 4 6 8}
hylang> (len "hydra")
=> 5
hylang> (def {h} #({{:name "hydra"} {:tasks 16}}))
=> NIL
hylang> (h :name)
=> "hydra"
hylang> (format "{} has {} tasks" (h :name) (h :tasks))
=> "hydra has 16 tasks"
```

- **Lists**: `head`, `tail`, `join`, `nth`, `take`, `drop`, `reverse`, `sort`, `map`, `filter`, `foldl`, `find`, `zip` ... Lists, strings and numbers are immutable and shared, never copied.
- **Strings** are bytes: `substring`, `index-of`, `str-split`, `str-join`, `str-replace`, `str-upper` ...; `+` of a string joins what follows as `print` shows it (`(+ "n=" 5)`).
- **Hashes** are called with a key to look it up, and a function in one is a method (`(obj :add 3)`, `&0` the hash).
- **Buffers** are bytes changed in place: `(buffer 16)`, `buffer-get`, `buffer-put`, `buffer-fill`, `buffer-copy`.

Control

`if`, `cond`, `case`, `and`, `or`, `while`, `each`, `dotimes`, `do`, `let`:

```
hylang> (if (> 5 3) "bigger" "smaller")
=> "bigger"
hylang> (dotimes {i 3} (print "line" i))
line 0
line 1
line 2
=> NIL
hylang> (fun {fact n} {if (zero? n) 1 (* n (fact (- n 1)))})
=> NIL
hylang> (fact 30)
=> 265252859812191058636308480000000
```

A branch of `if` written as a Q-expression is that data, not run: write `(if c (a) (b))`, not `{a}`. A call in tail position takes its caller's place, so a tail-recursive loop runs in constant space; other calls nest 2,500 deep at most.

Files and programs

```
hylang> (write-file "/ram/x.txt" "hello file")
=> NIL
hylang> (read-file "/ram/x.txt")
=> "hello file"
hylang> ((stat "/ram/x.txt") :length)
=> 10
hylang> (sh-out "ls /rom/bin | wc -l")
=> "     43\n"
hylang> (run "ls" "/rom")
README
bin/
doc/
lib/
songs/
=> 0
```

Area	Functions
Files	`read-file`, `read-lines`, `write-file`, `append-file`, `ls`, `dir`, `stat` (a hash: `:name`, `:length`, `:dir`, `:mtime`, `:mode`, `:qid`, `:dev`), `exists?`, `dir?`, `file?`, `mkdir`, `remove`, `rename`, `copy-file`, `glob`, `cd`, `cwd`
Streams	`open`, `read-line`, `read-byte`, `read-bytes`, `write-bytes`, `seek`, `close`; `stdin`, `stdout`, `stderr` are fds 0-2
Programs	`(run prog args...)` (its exit code), `(sh line)` (rc's), `(sh-out line)` (its output), `(spawn prog args...)`, `(wait task)`, `(kill task)`
The environment	`$name`, `(env name)`, `(env)` (all of it), `setenv`, `unsetenv`: rc's variables
The clock	`(time)` (seconds since 2000-01-01), `(date)`, `(ticks)`, `(tick-rate)` (200), `(sleep 1/10)`, `(clock)`
Keys	`(key)` (the next key, raw: a character, or `:up`, `:f1` ...), `(key?)`
Libraries	`(load "file")` runs a file; `(use "name")` loads a library once, from `/lib/hylang`

The Hydra's built-ins

What isn't a file is built in:

Area	Functions
Tasks	`(ps)` (a hash each: `:task`, `:name`, `:state`, `:parent`, `:cpu`, `:group`, `:args`), `(task-info t)`, `(pid)`, `(yield)`, `(sleep-until tick)`, `(hold body...)` (no task switch meanwhile)
Notes	`(note task :interrupt)` (or `:kill`, `:hangup`, `:alarm`, `:brk`, or 16-31), `(note-group g n)`, `(on-note f)`: each note (but a kill) given to `f`, which returns T to go on
Namespaces	`(bind new old [:before | :after] [:create])`, `(mount "#f" old [spec])`, `(unmount old [new])`, `(ns)` (a list of hashes), `(newns)`
Memory	`(peek a)`, `(poke a b)`, `(peek-word a)`, `(poke-word a n)`; `(banks)`, `(bank-alloc n)`, `(bank-free bank n)`, `(bank-read bank ofs n)`, `(bank-write bank ofs bytes)`; shared segments: `(seg-create banks)`, `(seg-attach seg)`, `(seg-detach seg)`, `(seg-read seg bank ofs n)`, `(seg-write seg bank ofs bytes)`; `(free)`
The system	`(sysinfo)`, `(mods)`, `(errstr :noent)`, `(platform)` (`:hydra`), `(hydra?)`

The device libraries

Each device has a library in hylang over its files (`/lib/hylang/NAME.hl`), loaded by `use`; each is also a working example of driving the device by hand:

`(use ...)`	For
`"cons"`	The windows: `(window)`, `(windows)`, `(new-window [cmd])`, `(new-group [cmd])`, `(show-window n)`, `(window-size)`, `(window-label s)`, `(window-status s)`, `(window-ctl s)`, `(snarf)`, `(snarf! s)`, `(raw-on)`, `(raw-off)`, `(beep)`
`"gpio"`	`(gpio pin)`, `(gpio! pin level)`, `(gpio-in pin)`, `(gpio-out pin)`, `(gpio-port)`, `(gpio-port! byte)`, `(gpio-ddr! byte)`, `(gpio-ca1! :rise)`, `(gpio-wait)`, `(gpio-state)`

(use ...)	For
"i2c"	(i2c-devices), (i2c-read addr n [reg]), (i2c-write addr bytes [reg]), (i2c-speed khz), (i2c-reg-size n)
"spi"	(spi dev bytes), (spi-read dev n), (spi-mode dev 3)
"snd"	(snd-claim ch...), (snd-patch ch n), (snd-note ch 60), (snd-off ch), (snd-level ch v), (snd-wave 8 :saw), (tune {{60 1} {64 1} {67 2}}), (note-of "C4"), (snd-mml 0 "t180 o4 l8 c d e"), (snd-chord 0 "o4 c e g"), (play path) ... (sound)
"disk"	(disks), (df), (cards), (disk-start d), (disk-stop d)
"proc"	(task-args t), (task-cwd t), (task-env t), (task-ns t), (task-regs t), (task-mem t addr n), (task-ram t bank ofs n)
"clock"	(set-date "2026-10-04 12:00:00"), (rtc)
"pc"	(pc?): whether the PC tool answers
"screen"	The terminal: cls, at, color, bold, plain, clear-line, cursor-off, cursor-on
"video"	The Vera X: (bitmap 320 8), (pen c), (plot x y), (line ...), (box ...), (bar ...), (circle x y r), (disc ...), (text x y s), (clear); the turtle; (vpeek a), (vpoke a v), (palette! i rgb), (sprite-at n x y), (vsync), (mouse) (video)

```
hylang> (use "cons")
=> NIL
hylang> (window)
=> 0
```

Every system call

Each call a program makes is a function, `sys-` and its name in lower case: `(sys-getpid)`, `(sys-open "/rom/README" 0)`. Its arguments are the registers the call takes, and its value what it gives (one, or a list; NIL for none); a failure is the system's error. The call reference gives each call's hylang form (its arguments by kind: `s` a string, `n` a number, `buf(N)` a buffer the call fills). `(sys :name args...)` makes any call by name.

Errors and Ctrl-C

An error is a value: it stops what it reaches, and the REPL shows it (`=> Error: ...`). `(try x handler)` catches one, `&err` its message and `&code` its code; a system failure's code is an atom (`:noent`, `:exist` ...: the names of the error codes, lower case, without `E_`).

```
hylang> (try (/ 1 0) {+ "caught: " &err})
=> "caught: Division by zero."
hylang> (error-code (read-file "/nosuch"))
=> :noent
```

Ctrl-C is the error `interrupted` (`:intr`) at the next call or loop step, which `try` can catch; `(on-note f)` gives it to a function instead.

hylang as a shell

`hylang -l` is a login shell over rc, by the same rule as HyForth's: **a line that starts with `(`, `{` or `[` (or a character right against `(` or `{`: `?(`, `=(`, `#()` is hylang's; any other is an rc command line**, run whole by an rc of its own (`rc -c`). So `ls -l | wc` works as in rc, and `(map print (ls "/bin"))` as lisp.

- `cd`, `bind`, `mount`, `unmount` and `newns`, as a line's one command, are done by hylang itself (with rc's quoting), so they change the shell's own directory and namespace; `exit` ends it.
- A line ending in `&` isn't waited for: its task is `$apid`.
- The prompt is the directory and `> ` (`/rom/lib> `); a function of yours bound to `shell-prompt` replaces it.
- `hylang -l` runs `/lib/hylang/login.hl` first (its namespace made), then `/lib/hylang/profile.hl` (a card's in the ROM's place).

Name `/bin/hylang -l` in a card's `/lib/shell` to have it in every window.

Where hylang differs from danlang

Area	hylang	danlang
Integers	Up to 255 bytes (about 614 digits): past that, the error `Too big`	No limit
Call depth	2,500 frames nested	10,000
Reading	255 brackets open at once; a name or number of 255 bytes; an expression at the REPL of 4,096 bytes	
Strings made	8,184 bytes at most for one string made by `+`, `format`, `repr` ...	
Buffers	8,184 bytes at most	
`load` and `use`	A bare name is `/lib/hylang/name` or `name.hl`, through the namespace	Its own folder
Streams	Over the system's fds	The PC's
`random`	A 16-bit xorshift, seeded by the tick count	
`clock`	To the tick (5 ms)	To the millisecond
A file read	Run an item at a time as it's read	Read whole first
Errors	No trace of where an error was made	The file, the line and the calls
Files	`.hl`	`.dl`

How it's built

- **Values** are 16-bit words: a fixnum (15 bits), or a reference to a cell in a **heap of 128K** in the task's RAM banks. Strings, big numbers and names are blobs in banks of their own. A **mark-and-sweep collector** frees what isn't reachable, sharing its cost across the cells made.
- **The evaluator** is a machine with its own stack of frames (6K in RAM, spilled to banks), never the 65C02's stack, so calls in tail position push nothing and Ctrl-C can stop anything.
- **The bytecode machine**: a function `fun` defines is compiled as it's defined (any other at its second call) to the code of a small machine, kept in an arena of up to eight RAM banks. **Native code** then makes the hot paths 65C02 instructions. Together they make hylang 1.9 times HyForth's time over twenty benchmarks (how fast).
- **The snapshot**: the library of functions written in hylang (`globals.hl`) is loaded at the build and kept in the ROM as a snapshot of the heap (the module `hysnap`), so hylang starts in 0.08 s instead of 2.3.
- **Eight banks** of the paged ROM: the evaluator, the reader and printer, the built-ins, the collector, the compiler, the bytecode machine; the number system is the shared `numbers` and `math` libraries'.
- The design in full is `docs/design/hylang.md`.

HyBASIC

HyBASIC (`basic`) is the Hydra-16's BASIC: a structured BASIC in QuickBASIC's way, with line numbers optional, labels to go to, `SUB`s and `FUNCTION`s with their own variables, records, files, sound, graphics and inline assembly, on the Hydra's numbers: **every number exact** (`PRINT 1 / 3` shows `1/3`, `2 ^ 100` all its digits), in any base. A program is compiled whole into a stack machine's code, then run. It keeps a program at its prompt, as BASICs always have, and it's a shell too (`basic -l`). It's written in 65C02 assembly (`reborn/modules/basic/`), five banks of the paged ROM.

This chapter is its reference: every statement and function.

Contents

1. Starting it
2. A program
3. Numbers
4. Strings
5. Names, arrays and records
6. Expressions
7. Control
8. Procedures
9. Data
10. The console
11. Files
12. Errors
13. Sound
14. The system
15. Inline assembly
16. Graphics
17. At the prompt
18. The shell
19. Functions, all of them
20. From other BASICs
21. How it's built

Starting it

Typed at the shell's prompt	What runs
`basic`	BASIC at the console: its prompt, `>`
`basic prog.bas a b`	A program run, then BASIC ends (its status: `END n`'s, or 1 after an error). A file whose first line is `#!/bin/basic` runs by its name too (`./prog.bas`); `ARG$(1)` is `a`
`basic <prog.bas`	The file's lines as if typed at the prompt, quietly; its end ends BASIC
`basic -l`	BASIC as a login shell: its namespace made, `/lib/basic/profile.bas` run, then its prompt (`/> `)

`SYSTEM` (or `BYE`) ends BASIC, at the prompt or in a program (`SYSTEM n`: its status n); at the shell, `exit` too.

A program

```
' primes.bas: the primes below a number
INPUT "Below"; limit
FOR n = 2 TO limit - 1
    IF isPrime(n) THEN PRINT n;
NEXT
PRINT

FUNCTION isPrime (n)
    FOR d = 2 TO SQR(n)
        IF n MOD d = 0 THEN RETURN 0
    NEXT
    RETURN -1
END FUNCTION
```

- A program is a text file (`.bas`) of lines; a line is statements separated by `:`. `'` and `REM` start a comment, to the line's end. A line ending in `_` goes on on the next.
- Keywords and names are the same in either case (`print` is `PRINT`), and a keyword is a whole word (`SCORE` is a name).
- **A label** is a name and a colon at a line's start (`again:`), alone or before the line's statements. **A line number** at a line's start is a label too (`100 PRINT "hi"`, and `GOTO 100`), so old programs run; the numbers need not be in order, nor on every line. `GOTO`, `GOSUB`, `RESTORE`, `ON ... GOTO`, `ON ERROR GOTO` and `RESUME` take a label or a line number.
- The program is read whole before it runs: a `SUB` or `FUNCTION` is known wherever it's written (`DECLARE` is allowed, and not needed), and a mistake in its form is said before it starts, with its file and line (`primes.bas:4: NEXT without FOR`). The lines outside procedures are the main program; it ends at its last line, or `END`.
- `INCLUDE "lib.bas"` reads another file's lines there (QuickBASIC's `'$INCLUDE: 'lib.bas'` too), from the program's directory or `/lib/basic`.

Numbers

Every number is exact, of one type: the Hydra's numbers (the numbers), the same as hylang's, HyForth's and C's.

Kind	Written	
An integer, of any size	`42`, `-7`, `2 ^ 100` (31 digits)	Exact, to 255 bytes (614 digits)
A fixed decimal	`0.5`, `3.14159`, `1.250`	Exact: `0.1 + 0.2 = 0.3` is true, and `FOR x = 0 TO 1 STEP 0.1` runs 11 times
A fraction	`10 / 4` is `5/2`; `1 / 3`	A division's result when it isn't whole: exact, in lowest terms
A complex number	`2i`, `1 + 2i`, `SQR(-4)` is `2i`	Its parts any of the above

- **Written exactly**: `PRINT 1 / 3` shows `1/3`, never `.3333333`; `PRINT 2 ^ 100` shows all of it. A number has one form, so a fixed decimal shows no 0s at its end (`1.50` is `1.5`, `100.0` is `100`). `PRINT USING` rounds to a picture when asked (below).
- **Exponents**: `1E6`, `2.5E-3`, `1D2` (QuickBASIC's), read exactly, in the program and by `VAL`, `READ` and `INPUT` (in decimal: in another base, `E` is a digit); `.5` is `0.5`.
- **In any base**: QuickBASIC's `&HFF`, `&O17`, `&B101`, and the Hydra's own, a `#` and the base: `#xFF`, `#b101`, `#o17`, `#16r1F`, `#c+-0` (balanced ternary), `#b0.1` (a half). `BASE "x"` makes hexadecimal the base everything is shown and read in: `PRINT`, `STR$`, `VAL`, `INPUT`, `READ`, and the program's own numbers after it (in base x a number starts with a digit: `0FF`, as `FF` is a name). `BASE "d"` is decimal again; `BASE$` is the base's string. The bases are hylang's: `d`, `x` (`FF`), `#x` (`#xFF`, its prefix shown), `b`, `o`, `c`, `16r`, `<x` (least digit first), `[01]` (digits of its own) ... (hylang's numbers). `STR$(x, "b")` writes one number in a base.
- **The math functions** (`SQR`, `EXP`, `LOG`, `SIN`, `COS`, `TAN`, `ATN`, `PI`, `^` of a power that isn't whole) are exact when the answer is (`SQR(9/4)` is `3/2`, `4 ^ 0.5` is `2`), else a fixed decimal of `DIGITS` significant digits, correctly rounded: 12 at the start, `DIGITS 30` for 30 (1 to 100); `DIGITS()` is the precision.
- QuickBASIC's number types are all this one: the suffixes `%`, `&`, `!` and `#` are taken and dropped (`x%`, `x&`, `x!`, `x#` and `x` are one variable), `AS INTEGER`, `AS LONG`, `AS SINGLE`, `AS DOUBLE` and `AS NUMBER` are the same, and `DEFINT`, `DEFLNG`, `DEFSNG`, `DEFDBL` change nothing. `CINT` and `CLNG` round to an integer (a half to the even one); `CSNG` and `CDBL` leave a number as it is.
- True is -1, false 0; any number not 0 is true.

Strings

`"text"`, and names ending in `$` (`name$`), of any length to 64K. `+` joins them; `=`, `<`, `>` ... compare them by their bytes. The functions: `LEN`, `LEFT$`, `RIGHT$`, `MID$`, `INSTR`, `UCASE$`, `LCASE$`, `LTRIM$`, `RTRIM$`, `SPACE$`, `STRING$`, `CHR$`, `ASC`, `STR$`, `VAL`, `HEX$`, `OCT$` (below). `MID$(a$, 2, 3) = "xyz"` changes part of one. A fixed-length string, `DIM s AS STRING * 10`, is always 10 characters: what's put in it is padded with spaces or cut.

Names, arrays and records

- **Names**: letters, digits, `_` and `.`, a letter first, any length, all of it significant; no keyword. A name ending in `$` (or declared `AS STRING`, or starting with a letter `DEFSTR` named) is a string's, any other a number's. A variable needs no declaring: it starts as 0, or `""`.
- **Arrays**: `DIM a(10)` (0 to 10), `DIM grid(1 TO 8, 1 TO 8)`, `DIM names$(100)`; any number of dimensions, 64K of elements' room each (some 13,000 numbers). An array used before a `DIM` is 0 to 10 in each dimension it's used with. `REDIM a(n)` makes it anew (`REDIM PRESERVE` keeps what's in it), `ERASE a` drops it; `LBOUND(a)`, `UBOUND(a, 2)` give its bounds. `OPTION BASE 1` makes 1 the lowest index when only the highest is given. An array and a variable may have the same name (`a` and `a(3)`).
- **Constants**: `CONST pi2 = 2 * PI, title$ = "Hydra"`: a name for a value, which can't change.
- **Records**: QuickBASIC's `TYPE`.

```
TYPE point
    x AS INTEGER
    y AS INTEGER
    label AS STRING * 8
END TYPE
DIM p AS point, path(1 TO 100) AS point
p.x = 3: path(1) = p: PRINT path(1).x
```

A field is a number, a string, a fixed-length string or a record of another `TYPE`; `p = q` copies one, and so do `s.at = p` and `q = path(2).at` (a field that's a record).

- `SHARED` and `STATIC`: Procedures. `DIM SHARED x` at the main level makes `x` every procedure's.

Expressions

From the first done to the last:

Operators	
`^`	A power (`2 ^ 10`; `2 ^ -1` is 1/2; `-2 ^ 2` is -4)
`-` (one operand)	Negation
`*`, `/`	Multiplication, division (exact)
`\`	Integer division: each operand rounded to an integer, the quotient cut toward 0 (`7 \ 2` is 3)
`MOD`	The remainder of \ (its sign the first's: `-7 MOD 2` is -1)
`+`, `-`	Addition, subtraction; `+` joins strings
`=`, `<>`, `<`, `>`, `<=`, `>=`	Comparisons: -1 or 0 (complex numbers by their real parts, then their imaginary)
`NOT`	Bitwise not (`NOT 0` is -1)
`AND`	Bitwise and
`OR`	Bitwise or
`XOR`, `EQV`, `IMP`	Bitwise exclusive or, equivalence, implication

The bitwise operators work on integers of any size, in two's complement; a number that isn't whole is rounded first. Parentheses group.

Control

Statement	
`IF c THEN s [ELSE s]`	On one line; `s` is statements (`:` between them), or a label or line number to go to

Statement	
`IF c THEN` … `ELSEIF c THEN` … `ELSE` … `END IF`	A block, nested to any depth
`SELECT CASE x` … `CASE 1, 3, 5` … `CASE 10 TO 20` … `CASE IS > 100` … `CASE ELSE` … `END SELECT`	The first case that matches; numbers or strings
`FOR i = a TO b [STEP s]` … `NEXT [i]`	`a` to `b`, exactly (`STEP` may be a fraction or below 0); `NEXT i, j` ends two
`DO [WHILE c | UNTIL c]` … `LOOP [WHILE c | UNTIL c]`	The test at the start, at the end, or neither
`WHILE c` … `WEND`	
`EXIT FOR`, `EXIT DO`, `EXIT SUB`, `EXIT FUNCTION`	Out of the innermost one
`GOTO label`	
`GOSUB label` … `RETURN`	`RETURN label` goes back to a label instead
`ON n GOTO a, b, c`, `ON n GOSUB a, b, c`	The nth label (none if n is out of range)
`END [n]`, `STOP`	The end (its status n); `STOP` stops it at the prompt, for `CONT`
`SLEEP s`	Waits s seconds (`SLEEP 0.25`), or till a key with none

Procedures

```
SUB swap2 (a, b)
    t = a: a = b: b = t
END SUB

FUNCTION area (r)
    area = PI * r ^ 2
END FUNCTION

x = 1: y = 2: swap2 x, y: PRINT x; y; area(1)
```

- `SUB name (params)` … `END SUB`, called as a statement, `name args` or `CALL name (args)`. `FUNCTION name (params)` … `END FUNCTION` gives a value: assigned to its name (`area = ...`), or by `RETURN value`, which leaves it too; a name ending in `$` gives a string. Both can call themselves.
- **Parameters**: `a`, `b$`, `c()` (an array), `d AS INTEGER`, `p AS point`. **By reference**: a variable, an array's element, a record's field or a whole array given is the procedure's to change (`swap2 x, y` above); an expression, a constant or a variable in parentheses (`(x)`) is a copy. A variable given must be of the parameter's kind.
- **Its variables are its own**: a name in a procedure is new each call, but its parameters, `SHARED x, y()` (the main program's), the main program's `DIM SHARED` names and `CONST`s. `STATIC x` keeps one between calls (`SUB name (...) STATIC`: all of them).
- `EXIT SUB`, `EXIT FUNCTION` leave it.

Data

`DATA 1, 2.5, "three", four` lists values in the program; `READ a, b, c$, d$` takes them in turn (a number read as `VAL` reads it); `RESTORE` goes back to the first, `RESTORE label` to the first after a label.

The console

`PRINT [items]`, `?`	Items separated by `;` (none between) or `,` (the next zone of 14 columns); a `;` or `,` at the end: no new line. A number has a space before it (or its `-`) and one after. `TAB(n)` to column n, `SPC(n)` n spaces
`PRINT USING fmt$; items`	QuickBASIC's pictures: `#` a digit, `.` the point, `,` thousands, `+` `-` a sign, `$$` `**`, `!` `\ \` `&` strings, `_` the next character as it is; and the Hydra's placeholders: `{}` a value in the base, `{x}` `{#b}` `{c}` in a base named (`PRINT USING "{x} is {}"; 255, 255`)
`WRITE items`	Comma-separated, strings quoted
`INPUT [;] ["prompt";] a, b$`	A line typed, its values comma-separated (`?` after the prompt; `,` in its place: none); not enough of them, or a number that isn't: `Redo from start`. `INPUT ;`: the line goes on after the answer

`LINE INPUT [;] ["prompt";] a$`	A whole line, as it was typed
`INKEY$`	The key typed, or `""` if none (it doesn't wait); the arrow keys and the like as two characters, `CHR$(0)` and a code
`INPUT$(n)`	n keys, waited for
`CLS`, `LOCATE row, col`, `CSRLIN`, `POS(0)`	The screen cleared; the cursor moved (from 1, 1); where it is
`COLOR fg [, bg]`	The text's colours (0-15, the terminal's); in a graphics `SCREEN`, the pen's
`WIDTH n`, `BEEP`	The line's width for `PRINT`'s zones and wrapping (80); the bell

Files

```
OPEN "scores.txt" FOR OUTPUT AS #1
FOR i = 1 TO 3: PRINT #1, i; i * i: NEXT
CLOSE #1
OPEN "scores.txt" FOR INPUT AS #1
DO UNTIL EOF(1)
    INPUT #1, a, b: PRINT a + b
LOOP
CLOSE
```

`OPEN path FOR INPUT | OUTPUT | APPEND | BINARY AS #n`	A file (made or emptied for `OUTPUT`), or a device (`/dev/cons`, `/pc/x`); `#n` 1 to 255, `FREEFILE` the next free. A path goes through the namespace, as rc's do
`CLOSE [#n, ...]`	Those files (one not open: nothing), or all
`PRINT #n, ...`, `PRINT #n, USING ...`, `WRITE #n, ...`	As the console's
`INPUT #n, a, b$`, `LINE INPUT #n, a$`, `INPUT$(k, #n)`	As the console's; past the end, `Input past end of file`
`GET #n, [pos], v`, `PUT #n, [pos], v`	In `BINARY`: a string's bytes (its length's worth read), or a number's (its bytes in the stored format, as `MKN$` makes them), at the byte pos (from 1) or where it is
`SEEK #n, pos`, `SEEK(n)`, `LOC(n)`, `LOF(n)`, `EOF(n)`	Its position (from 1); its length; at its end?
`KILL path`, `NAME old AS new`, `MKDIR`, `RMDIR`, `CHDIR`, `FILES [path]`	Files and directories: removed, renamed, made; the current directory; a directory's names
`DIR$(path)`, `DIR$`	A directory's first name, then the next (`""` after the last)

In a file's statements a `#` is the file's: `PRINT #x1, ...` writes to the file whose number is in `x1`, though `#x1` is a number elsewhere (`PRINT (#xFF)` prints 255).

Errors

- Without a handler, an error stops the program with its message, file and line, on a line of its own: `primes.bas:12: division by zero` (a typed program's, by the line's number: `line 20: ...`; an `INCLUDE`d file's, its own name and line; a file BASIC can't open: `none.bas: not found`), and BASIC's status is 1 in a script.
- `ON ERROR GOTO label` sends an error to a handler (`ON ERROR GOTO 0`: none). In it, `ERR` is its code, `ERL` its line (in a program with line numbers its line's number, or the nearest one before it, as QuickBASIC's; else its line in its file), `ERR$` its message; `RESUME` runs the statement again, `RESUME NEXT` the one after it, `RESUME label` goes there. `ERROR n` makes one.
- The codes are QuickBASIC's (5 illegal function call, 6 overflow, 9 subscript out of range, 11 division by zero, 13 type mismatch, 53 file not found, 62 input past end of file ...), and the system's errors are one list with them: a few as QuickBASIC's (53 not found, 58 already exists, 61 disk full, 70 permission denied, 76 path not found), the rest 256 and the system's code (`ERR$` its text: `not a directory`).
- Ctrl-C stops a program (`Break in primes.bas:12`); `CONT` goes on.

Sound

`SOUND ch, note [, patch [, vol]]` plays a MIDI note (60 is middle C) on a channel (0-7 the YM2151's; 8-23 a Vera X's PSG voices), its instrument and volume (0-127) if they're given; `SOUND ch` lets it go. `SOUND "text"` sends the sound driver its commands (`"pan 0 left"`, `"wave 8 saw"`: sound). `PLAY "t180 o4 l8 c d e"` plays a line of the score language on channel 0, `PLAY ch, "..."` on another, `PLAY "song.zsm"` a song or a score; each waits till it's played. `BEEP` rings the bell.

The system

`SHELL "rc line"`	rc runs it, and BASIC waits; `STATUS` is its status after
`SHELL$("line")`	Its output, as a string (its new lines at the end dropped)
`ENV$("name")`, `ENVIRON$("name")`, `ENVIRON "name=value"`	The environment's variables (rc's)
`ARG$(n)`, `COMMAND$`	A script's arguments (`ARG$(0)` its name); all of them as one line
`TIMER`, `DATE$`, `TIME$`	Seconds since midnight, by the ticks (1/200): one `TIMER` less another is exact; `"2026-10-08"`, `"14:05:09"`
`SYS "NAME" [, a [, x [, y]]]`, `RREG a, x, y, p`	A system call by its name (the call reference): `r0`-`r15` from bytes 2-33 (`POKE` them first); the registers after (`p` bit 0: it failed, `a` the error)
`SYS addr`, `CALL ABSOLUTE (addr)`	Machine code (at `$8000`-`$9FFF`: in `BANK`'s bank)
`CALL ASM label [, a [, x [, y]]]`	An `ASM` block's label: Inline assembly
`PEEK(addr)`, `POKE addr, b`, `BANK n`, `BANK()`	A byte of the task's memory; the RAM bank at `$8000` (BASIC's own data lives in banks too: leave theirs alone)
`FRE()`	The memory free, in bytes

Inline assembly

`ASM` and `END ASM`, each alone on its line, hold assembly in `as`'s language (ca65's: the assembler), anywhere in a program (in a `SUB` too) but not at the prompt. The blocks are assembled as the program is compiled, after its code, as one source in their order (a label in one is known in the others), into a RAM bank of their own, seen at `$8000`-`$9FFF` while their code runs: 8K for their code, their data and their `.bss`, each block starting in `.code`. `CALL ASM label [, a [, x [, y]]]` calls one of their labels as `SYS` calls machine code (`.A`, `.X` and `.Y` from the numbers, 0 if they're left out; `RREG` reads them after). The label is written as the block writes it (the assembler tells capitals from lower case), a keyword's spelling too (`double`). At the prompt, `CALL ASM` calls a label the program's own `CALL ASM`s name.

The program's names are symbols in the blocks: a global number variable (not a procedure's own) is its value's address, 5 bytes: its kind (0 for an integer of 32 bits), then an integer's 4 bytes, lowest first (code that writes one writes its kind too); an integer `CONST` is its value. Each is there in capitals and in lower case (`COUNT` and `count`), but `A`, `X` and `Y`, the registers' names; a block's own label of the same name is the block's. `.include "hydra.inc"` (from `/lib/as`) gives the system calls by name and `r0`-`r15`. The code may use `r0`-`r15`, keeps BASIC's zero page (`$22`-`$7F`: no `.zeropage`), and ends with `rts`.

```
CONST K = 3
total = 0
FOR i = 1 TO 4: CALL ASM addk: NEXT
CALL ASM double, 21: RREG a
PRINT total; a                    ' 12  42

ASM
addk:   lda total + 1           ; total: its kind (0, an integer), then its 4 bytes
        clc
        adc #K
        sta total + 1
        rts
double: asl a
        rts
END ASM
```

An error in a block is the assembler's, at its line (`line 50: ASM: undefined: nowhere`); a label no block has, at the `CALL ASM` that first names it; an `ASM` without its `END ASM`, at the `ASM`. The blocks add the assembler's time to the program's compiling: a quarter of a second for a few, some 3 seconds with `hydra.inc`. The assembler is the asm library's, the one `as` runs. A longer example is `/sd/0/bench/benchasm.bas`: the twenty benchmarks of `bench.bas`, each in assembly (how fast).

Graphics

On a Vera X, through `/dev/vid/draw` (video): the console's text stays over the drawing.

`SCREEN n`	0: text alone; 1: 320 by 240, 4 colours; 2: 640 by 480, 2 colours; 7: 320 by 240, 16 colours; 12: 640 by 480, 4 colours; 13: 320 by 240, 256 colours
`PSET (x, y) [, c]`, `PRESET (x, y) [, c]`	A point, in c or the pen's colour (`PRESET`: the background's)
`LINE [(x1, y1)]-(x2, y2) [, c [, B | BF]]`	A line from where the last one ended; `B` a box, `BF` a filled box. `STEP (dx, dy)` is relative
`CIRCLE (x, y), r [, c]`, `CIRCLE ... , , , , F`	A circle; filled with `F`
`PAINT (x, y) [, c [, border]]`	Fills the area around a point, to the border's colour
`DRAW "u10 r10 d10 l10"`	QuickBASIC's turtle: `U D L R E F G H n` (a move), `M x,y`, `B` (don't draw), `N` (come back), `C n`, `A n`, `TA deg`, `S n`
`GPRINT (x, y), text$`	Text on the drawing
`PALETTE n, rgb`	A colour's 12 bits (`&HF00` red)
`POINT(x, y)`	A pixel's colour
`WINDOW (x1, y1)-(x2, y2)`, `VIEW (x1, y1)-(x2, y2)`	Coordinates of one's own; drawing kept to a part of the screen
`SPRITE n, x, y [, image$]`	The Vera's sprite n (1-127) at x, y, its image a string of 8-bit pixels (`SPRITE n OFF`)

At the prompt

BASIC keeps a program at its prompt, as QuickBASIC's window did.

`LOAD "f.bas"`, `SAVE ["f.bas"]`, `NEW`	A file read into it; written (with no name, to the one it came from); emptied
`RUN`, `RUN "f.bas"`	It run (read and checked first, its numbers in decimal); a file loaded and run
`LIST [from-to]`	Its lines, or those from a line or label to another
`EDIT [label]`	The editing mode: the program in the screen editor (`edit`), at the label, or the last error's line; ^X back to the prompt, the program read and checked again
`CONT`	On after `STOP`, Ctrl-C or an error's stop
`DELETE from-to`	Numbered lines taken out
`CLEAR`	The variables emptied

A line typed with a number first goes into the program at its number (a number alone takes that line out), so a program can be typed line by line as BASIC's always were. Any other line runs at once: `PRINT 2 ^ 64`, `x = 5`, `plot 1, 2` (the program's `SUB plot`). After a run its variables and procedures are still there to use. These commands are the prompt's, at a line's start: in a program, or after a `:`, they're `only at the prompt`.

The shell

`basic -l` is a shell: a line is BASIC's if it's a program line (a number first), `?`, a statement's keyword first (`print`, `run`, `list`), an assignment (`x = 5`, `a$(1) = "hi"`), or a call of a `SUB` of its program; any other is an rc command line, run by rc and waited for.

```
/> print 2 ^ 70
 1180591620717411303424
/> ls /rom/lib/basic
profile.bas
/> cd /ram
```

A name in both is BASIC's (`sleep`, `if`, `for`): `%` before a line makes it rc's whatever it is. `cd`, `bind`, `mount`, `unmount` and `newns` are the shell's own, as an rc line can't change BASIC's directory or namespace; `exit` ends it; a line ending in `&` runs in the background. The prompt is the current directory. `basic -l` runs `/lib/basic/profile.bas` before its first prompt (a card's or the RAM disk's in the ROM's place).

Functions, all of them

Numbers	
`ABS(x)`, `SGN(x)`	A real number's size; -1, 0 or 1
`INT(x)`, `FIX(x)`, `CINT(x)`, `CLNG(x)`	Down to an integer; toward 0; to the nearest (a half to the even)

Numbers	
`CSNG(x)`, `CDBL(x)`	x as it is
`SQR`, `EXP`, `LOG`, `SIN`, `COS`, `TAN`, `ATN`, `PI`	The math functions, in radians, to `DIGITS` digits (exact when they can be)
`RND`, `RND(x)`	A number from 0 up to 1, of `DIGITS` digits (`RND(0)`: the last again); `RANDOMIZE [n]` seeds it (none: from the clock)
`FIX(x * 100) / 100`, `FIXED(x, n)`	A number cut to n places, as a fixed decimal (`FIXED(1/3, 4)` is `0.3333`)
`RATIONAL(x)`, `NUMERATOR(x)`, `DENOMINATOR(x)`	As a fraction; its parts, in lowest terms
`COMPLEX(re, im)`, `REAL(x)`, `IMAG(x)`	A complex number; its parts
`GCD(a, b)`, `FIB(n)`	The greatest common divisor; the nth Fibonacci number
`SHL(x, n)`, `SHR(x, n)`, `BIT(x, n)`	Shifts; bit n (0 or 1)
`VAL(s$)`, `STR$(x [, base$])`	A string's number (the longest start that is one; 0 if none); a number's text
`HEX$(x)`, `OCT$(x)`	In base 16 and 8
`MKN$(x)`, `CVN(s$)`	A number as the bytes of the stored format, and back

Strings	
`LEN(s$)`	Its length
`LEFT$(s$, n)`, `RIGHT$(s$, n)`, `MID$(s$, i [, n])`	Its first n, last n, n from the ith (from 1)
`INSTR([i,] s$, t$)`	Where t$ is in s$ (from i), or 0
`UCASE$`, `LCASE$`, `LTRIM$`, `RTRIM$`	In capitals, in small letters; its spaces at the start, at the end, taken off
`SPACE$(n)`, `STRING$(n, c)`	n spaces; n of a character (c a code or a string's first)
`CHR$(n)`, `ASC(s$)`	A character from its code; the first one's code

The rest	
`ERR`, `ERL`, `ERR$`	The last error's code, line, message
`EOF(n)`, `LOF(n)`, `LOC(n)`, `SEEK(n)`, `FREEFILE`	Files
`LBOUND(a [, d])`, `UBOUND(a [, d])`	An array's bounds
`TIMER`, `DATE$`, `TIME$`, `INKEY$`, `INPUT$(n [, #f])`, `CSRLIN`, `POS(0)`	The clock, the console
`ENV$`, `ENVIRON$`, `ARG$`, `COMMAND$`, `SHELL$`, `STATUS`, `DIR$`	The system
`PEEK`, `BANK()`, `FRE()`, `POINT(x, y)`	Memory, the drawing
`BASE$`, `DIGITS()`	The base's string, the precision

From other BASICs

- **From QuickBASIC**: the same language, but for one number type, exact, and what the Hydra adds. Not here: `DEF FN` (a `FUNCTION` instead), `FIELD`, `LSET` and `RSET`, `RANDOM` files, the `ON` events, `PCOPY`, `VARPTR` and `SADD`, `$DYNAMIC` and `$STATIC`, `PRINT USING`'s `^^^^`, `RUN` from a line, and `RUN` and `CLEAR` in a program (they're the prompt's).
- **From the Hydra's first BASIC** (Microsoft's, 2A): a program with line numbers mostly runs as it is, with spaces between its keywords and names where it ran them together (`FORI=1TO9` is `FOR I = 1 TO 9`), and these changed: `DEF FN` and `USR` are gone; `OPEN n, "name", "W"` is `OPEN "name" FOR OUTPUT AS #n`; `GET` is `INKEY$`; `HIMEM` and `WAIT` are gone; a name has all its letters (`SCORE` and `SC` are two names); and `PRINT 1/3` shows `1/3`.

How it's built

- **Compiled, then run.** A program is read twice, once for its procedures and types and again for its code, and compiled into the code of a **stack machine**: an op a byte, its operands after it. The interpreter runs that code. A line typed at the prompt is compiled the same way, into a region of its own, and run at once.
- **Five banks** of the paged ROM: the interpreter; the compiler; the rest of the compiler, with input and output; the numbers, with the shell, sound, `SYS` and graphics; the heap and the strings, with the errors, the system's functions and the program's text. A call between banks goes through trampolines in its RAM.

- **Values** are 5 bytes: a tag and 4 more, so a number up to 32 bits is held whole, and anything bigger (a big integer, a fraction, a string, an array, a record) is in the **heap**, in the task's RAM banks, with a collector.
- **Memory**: the interpreter's state, the globals and the value stack are in the task's RAM; the program's code, its text, its symbols, its line table, its `DATA` and the heap are in the task's RAM banks, through a table of logical banks taken as they're wanted.
- **Numbers** are the shared `numbers` and `math` libraries', called with `XCALL`: the same as hylang's, HyForth's and C's.
- **Inline assembly** is the asm library's assembler, the one `as` runs, given the blocks a line at a time.
- **How fast**: over twenty benchmarks, 7.4 times HyForth's time and 3.2 times hylang's (how fast). The first BASIC the Hydra had (Microsoft's, by way of EhyBASIC) was some 44 times HyForth's.
- The design in full is `docs/design/basic.md`, and the plan it was built to `docs/design/plans/BASIC.md`.

7. The tools

The programs in `/bin`: the core tools in Plan 9's way, the editors, the task tools, the debugger, the assembler and disassembler, and the system's. They're on the ROM disk (`/rom/bin`), most near 2K, which keeps them in 1K clusters, where a module of the paged ROM took a bank. Most are **bank programs** (the kernel): a command at rc's or HyForth's prompt runs one in the shell's own task, from a RAM bank of the shell's where it stays loaded, so it starts at once the next time and takes no task; in a pipeline or the background it gets a task of its own, as any program does. `as`, `newwin` and `tail` are bank programs that always get a task of their own (`as`'s library and `tail` use the task's RAM, `newwin` changes the environment); `ed`, `edit`, `dis`, `db`, `calc`, `grep` and `sort` are RAM programs, read into their task's RAM as they start (about 50 ms for one of 2.4K). The paged ROM keeps what must run in place (the drivers, init, rc, the languages, `play`; `/dev/mod/NAME`, bound into `/bin` by `#m/bin`). `/bin` is a union, so a program of your own in `/ram/bin`, `/sram/bin` or a card's `/bin` comes before the ROM's of the same name.

How they behave

They behave as Plan 9's do:

- Flags come first (`ls -l`, `-abc` together), then names. A tool that reads files reads fd 0 when it's given none, so each works in a pipeline (`cat f | wc`).
- A name that fails is said on fd 2 as `tool: name: why` (`ls: /none: not found`), the rest go on, and the tool ends with code 1. A bad flag or too few names is `usage: ...`, and the status `usage`.
- `$status` after a tool is its exit message, or its code; empty or `0` is success.

Every tool runs the same way at rc's prompt and at HyForth's. At HyForth's prompt, a tool whose name is also a Forth word needs a `%` before it (`% free` when `memory.fl`'s `free` is loaded).

Contents

1. Files and text
2. The editors: edit and ed
3. Tasks
4. db: the debugger
5. as: the assembler
6. dis: the disassembler
7. The system
8. Others

The disks' tools (`df`, `mkfs`, `fsck`, `label`) are in storage, and `play` in sound.

Files and text

Tool	What it does
`ls [-ld] [name ...]`	A directory's entries, a line each (a `/` after a directory's); a file's own name. `-l`: the mode, the device letter and instance, the length, the time, the name: `--rw-rw-rw- fr 13 2000-01-01 00:00 a`. `-d`: a directory itself, not its entries. None: the current directory
`cat [file ...]`	Each file to fd 1 (none: fd 0)
`cp [-r] from to`, `cp [-r] from ... dir`	Each file copied to a file made (one there already is emptied first), or into a directory by its own name. `-r`: a directory's whole tree. A file isn't copied onto itself
`mv from to`, `mv from ... dir`	Moved, or renamed: in its own directory a rename (a file there by the new name goes first); to another, a file is copied then removed, and a directory can't be moved
`rm [-rf] name ...`	Each file removed, or empty directory. `-r`: a directory's tree too, the deepest first; `-f`: quiet about what can't be
`mkdir [-p] dir ...`	Each directory made. `-p`: the directories it's in too, as needed, and no complaint about one there
`rmdir dir ...`	Each empty directory removed
`touch name ...`	Each file made empty if it isn't there, or its time changed if it is
`du [-a] [name ...]`	The kilobytes each tree takes (its files' lengths rounded up to whole KB), a line a directory, the deepest first; `-a`: a line a file too
`pwd`	The current directory
`cmp file1 file2`	Compared byte by byte: the first that differs said (`file1 file2 differ: byte 5`), or nothing, and the status

```
/> mkdir /ram/notes
/> echo first >/ram/notes/a; echo second >>/ram/notes/a
/> cp /ram/notes/a /ram/notes/b; ls -l /ram/notes
--rw-rw-rw- fr       13 2000-01-01 00:00 a
--rw-rw-rw- fr       13 2000-01-01 00:00 b
/> rm -r /ram/notes
```

Text

Tool	What it does
`echo [-n] [words]`	The words, a space between each, and a new line (`-n`: none)
`wc [-lwc] [file ...]`	Lines, words and bytes (`-l`, `-w`, `-c`: only those), a line a file, and a total for several
`head [-N] [file ...]`	Each file's first N lines (10)
`tail [-N] [file]`	The last N lines (10); the file's last 16K are what it looks at
`grep [-chilnsv] [-e] pattern [file ...]`	The lines that match a regular expression, Plan 9's (`.`, `[a-z]`, `[^a]`, `^`, `$`, `*`, `+`, `?`, `|`, `( )`): `-c` the count, `-l` the files' names, `-n` with line numbers, `-i` case ignored, `-v` the lines that don't, `-s` no messages, `-h` no file names; status `no matches` when none did
`sort [-bfnru] [file ...]`	The lines in order: by bytes; `-n` by the number each starts with; `-f` case folded; `-b` leading blanks ignored; `-r` reversed; `-u` one of each run of equal lines. Stable; some 20K of lines at most
`uniq [-c] [file]`	Each run of the same line once; `-c` with its count
`tee [-a] [file ...]`	Fd 0 to fd 1 and to each file (`-a`: added to its end)
`xd [file ...]`	Bytes in hex, 16 a line after their offset, then as text: `0000010 68 65 6c 6c 6f 0a hello.`
`more [file ...]`	A screen at a time (22 lines, then `--more--`): Enter for the next, `q` to stop. Its keys come from the console even when fd 0 is the file
`ed [file]`	The line editor, ed's way: `p`, `a`, `i`, `c`, `d`, `w [name]`, `q` (twice if the text's changed), `Q`, `h`, with line numbers (`1,5p`, `$`). Lines are typed after `a`, `i` or `c` until a line of just `.`; Ctrl-C comes back to its `*` prompt

```
/> ls /rom/bin | sort -r | head -3
xmodem
xd
wc
/> grep -n Hydra /rom/README
1:The Hydra-16's ROM disk
5:machine.  It's a HydraFS volume, as a card's is, in the paged ROM's banks after
```

The editors: edit and ed

`edit [+N] [file ...]` is the screen editor, nano's way (`+N`: the next file at its line N; BASIC's `EDIT` uses it): what you type goes in at the cursor, and the Ctrl keys and the Meta keys (Esc then a key, or Alt with it: `M-`) are commands. The two lines at the bottom name the commonest, and `^G` shows them all. Up to 6 files are open at once, each in a buffer of its own; their text is in your task's RAM banks, so a file can be as big as they are (a few hundred K: `kdev.s`'s 140K reads in some 6 seconds). A file whose lines end CR LF is written with CR LF again.

Keys	What they do
`^O`, `^S`	Save (`^O` asks for the name: Enter keeps it)
`^X`	Close the file (if it's changed, saved or not: `y`, `n`); with the last closed, `edit` ends
`^R`; `M-,` `M-.`	Open another file; go to the file before, or after
`^W`; `M-W`, `M-Q`	Find (letters either case; round from the other end if need be); find again, find backwards
`M-R`	Replace: each match shown, `y` yes, `n` no, `a` all the rest
`M-G`	Go to a line (`$`: the last)
`^K`, `M-6`, `^U`	Cut the line (`^K` again: the next joins it in the cut buffer), copy it, paste (into any file); with the mark set (`M-A`), the block from the mark to the cursor
`M-U`, `M-E`	Undo, redo (a line typed, a cut, a paste: one step each)
the arrows, Home, End, PgUp, PgDn	Move (and `^B` `^F` `^P` `^N`, `^A` `^E`, `^Y` `^V`); `M-\` and `M-/` the text's start and end

Keys	What they do
Backspace; Del, `^D`	Rub out; delete
`M-I`, `M-X`, `^L`	Auto-indent on and off; the help lines off and on; the screen drawn again

Ctrl-C does nothing in `edit` (it's the console's interrupt), and `^\` and `^]` never reach a program, so nano's keys there are Meta keys here. The screen is the window's size (`consctl`'s: the smaller of the terminals it's shown on; with no console `$COLUMNS` and `$LINES`, else 80 by 24), and `edit` draws itself again as it changes. `ed` is the line editor (text), for scripts and a terminal without a screen.

Tasks

Tool	What it does
`ps [-a]`	The tasks in use: number, state, parent, CPU time (seconds, to a tenth), note group, name; `-a`: their arguments too. A stopped task's state is `stopped`
`top`	The same each second, with the CPU each took in it, the screen drawn again; Ctrl-C ends it
`kill [-i] task ...`	Each task (its number) ended with the kill note, which nothing catches; `-i`: interrupted instead (Ctrl-C's note), which a program may catch
`slay [-i] name ...`	Each task running a program of that name, as `kill`
`sleep seconds`	Nothing for that long; Ctrl-C ends it
`ns [task]`	A task's namespace (none: this one's) as the binds and mounts that make it
`new-window [-g] [command ...]`	A window made and shown, in this one's group (`-g`: a group of its own, another shell session), running the command (rc's: `new-window 'ls -l; sleep 5'`), or with none the shell (`/lib/shell`'s, as Ctrl-] c starts); `$window` is its number. It isn't waited for, and the window goes when its program ends

The tasks' own files are under `/proc/N`: `status`, `args`, `cwd`, `env`, `ns`, `fd` (its open files: `0 rw #c 291 #c/cons`), `regs`, `mem` and `ram` (its memory), `note` (write `interrupt`, `kill`, `hangup` or a number to send it one), and `ctl` (`kill`, `interrupt`, `note N`, `stop` and `start`: a stopped task doesn't run till it's started again; `step`, `next`, `break` and `nobreak`: the debugger's: db). Another task's memory and registers are anyone's but the kernel task's and a driver's.

```
/> sleep 100 &
/> ps
task  state   parent     cpu group  name
...
   5  sleep    4         0.0     4  sleep
...
/> echo stop >/proc/5/ctl; cat /proc/5/status
sleep stopped 4 0 4
/> echo kill >/proc/5/ctl
```

---

db: the debugger

`db program [arg ...]` starts a program stopped at its first instruction, in a task and note group of its own (so Ctrl-C reaches `db`, which stops it); `db -p task` stops a task that's running, where it is. Then it takes commands, a line each, at its `db>` prompt. An address is hex (`0830`, `$0830`) or a symbol, with `+` or `-` a hex offset (`main+14`); a count is decimal.

Command	What it does
`r`	Its registers, and the instruction at its PC
`s [n]`	n instructions (1), a step at a time, into a subroutine at a `JSR`
`n [n]`	The same, but a `JSR`'s subroutine runs whole (a `JSR` into the kernel's jump table always does)
`c`	On, till a breakpoint, a `BRK`, its end, or Ctrl-C
`u addr`	Steps (as `n`) till its PC is addr
`b [addr]`	A breakpoint at addr (in RAM: a `BRK` written there while it runs), or the list
`x [addr]`	The breakpoint at addr gone, or all of them

Command	What it does
`d [addr] [n]`	n instructions (12) from addr (its PC; then on from the last), disassembled as `as` writes them (the asm library's, as `dis`'s): a symbol for the address an operand names, where each is after a `;`
`m [addr] [n]`	n bytes (64) from addr, in hex and as text
`w addr byte ...`	Bytes (hex) written at addr
`l file`	Symbols from an ld65 label file (`ld65 -Ln`: the build makes one for each program, `obj/.../NAME.lbl`; `as -l`'s), or `/lib/as/hydra.inc`'s: the system calls' names and r0-r15
`q`	Quit: a program `db` started is killed; a task it stopped runs on

```
/> db /sd/0/sample/hi Ann Bob
task 4
PC=0830 A=30 X=FF Y=48 S=FD P=nv--dizc W=0 U=0 RAM=00 ROM=00
0830  A5 02     lda $02
db> l /pc/hi.lbl
8 symbols
db> l /lib/as/hydra.inc
105 symbols
db> b main+14
db> c
breakpoint 1
PC=0844 A=41 X=FF Y=48 S=FD P=nv--dizc W=0 U=0 RAM=00 ROM=00
0844  A9 52     lda #$52                ; main+14
db> n 4
0846  85 02     sta r0                  ; main+16
0848  A9 08     lda #$08                ; main+18
084A  85 03     sta r0+1                ; main+1A
PC=084C A=08 X=FF Y=48 S=FD P=nv--dizc W=0 U=0 RAM=00 ROM=00
084C  20 53 F9  jsr PUTS                ; main+1C
db> x
db> c
Hello, Ann!
Hello, Bob!
I'm task 4, in /, in window 0.
task 4 ended: code 0
```

A step can't be taken in the kernel: a task stopped in a call (`(in a call: its state 6)`, as Ctrl-C finds a program in `SLEEP`) is run on with `c`. Breakpoints are in its memory only while it runs, so `d` and `m` show its own bytes; one in a ROM can't be set (`u` reaches an address there, a step at a time). A `BRK` of the program's own stops it too; `c` from there gives it its note (`sys: brk`), as it would have had. A subroutine that reads the bytes after its `JSR` (its arguments) can't be stepped over with `n`; step into it with `s`. `db` works through `/proc/N/ctl` and `mem`: `echo step >/proc/N/ctl` does the same by hand, on a task that's stopped.

---

as: the assembler

`as [-bl] file.s|- [out|-]` assembles a program for the 65C02 on the Hydra itself, from the same source ca65 takes on a PC: a RAM program, `out` (`file` without its `.s`, if there's no `out`), that runs as any program does. `-l` writes its labels too, `out.lbl` (ld65's `-Ln` form, which `db`'s `l` reads); `-b` writes the bytes alone, no header, from `.org`'s address (`$0800` if there's none): a ROM's image, say. A source `-` is standard input (a pipe, or `< file`), and then the output is `-` too unless it's named; an output `-` is standard output: `dis prog | as - copy` makes a program from its own disassembly in one line. `-l` needs a named output.

The SDK's files are in `/lib/as`: `hydra.inc`, `hyx2.inc`, `macros.inc`, `toollib.inc` and `toollib.s`, `srvlib.inc` and `srvlib.s`, `nslib.s`, and three of its samples, `hi.s`, `tick.s` and `upper.s`. `.include` and `.incbin` find a file as it's named, then beside the file that names it, then in `/lib/as`. A program made from the same source by ca65 and ld65 (`sdk/asm/hyx2.cfg`) is the same, byte for byte.

What it takes is ca65's, as the SDK's sources use it:

Kind	What as takes
Instructions	The W65C02S's, each mode written as ca65 writes it; `a:` or `z:` before an address makes it absolute or zero page. An address is on the zero page if it's known by then and under `$100` (a label further on is absolute)
Labels	`name:`; cheap locals, `@name:`, between one normal label and the next; unnamed ones, `:`, reached as `:+` `:++` ... and `:-` `:--` ...; constants, `name = expr` and `name := expr`
Expressions	32 bits, with ca65's operators and their order (`* / .mod & ^ << >>`, `+ - |`, the comparisons, `&& || .and .or .xor .not`, unary `- ~ < > ^ !`), `*` (here), `$hex`, `%binary`, `'c'`, and `.lobyte .hibyte .bankbyte .loword .hiword .strlen .defined .blank .match`
Data	`.byte` (and strings), `.word`, `.addr`, `.dword`, `.res`, `.asciiz`, `.incbin "file" [, start [, count]]`
Files	`.include`
Conditions	`.if`, `.ifdef`, `.ifndef`, `.ifblank`, `.ifnblank`, `.elseif`, `.else`, `.endif`
Macros	`.macro name params` ... `.endmacro`, `.exitmacro`; an argument in `{ }` may have commas in it
Segments	`.segment "NAME"`, `.zeropage`, `.code`, `.rodata`, `.data`, `.bss`, `.pushseg`, `.popseg`
Checks	`.assert expr, error|warning, "text"`, `.error`, `.warning`
Taken and left	`.import`, `.export`, `.global` (and their `zp` kinds), `.setcpu`, `.feature`, `.macpack`, `.debuginfo`, `.list`, `.case` ...

Not there: `.proc` and `.scope`, `.repeat`, `.struct`, `.sprintf` and `.ident` (but in a branch that's not taken), and objects to link: one source file and what it includes make one program. Its segments are laid out as `hyx2.cfg` lays them out: `ZEROPAGE` from `$22` (the program's `$5E` bytes), then from `$0800` `HEADER`, `CODE`, `RODATA`, `DATA` and `BSS`, to `$8000` at most, with ld65's names for them (`__DATA_LOAD__`, `__BSS_RUN__`, `__BSS_SIZE__`, `__RAM_LAST__` ...) and `HYX2_RAM` defined, as `hyx2.inc` needs.

The assembler is the **asm library**'s (`modules/asm`, `spec/asm.def`: dis); `as` is its command line. BASIC's `ASM` blocks are assembled by the same code, a line at a time (HyBASIC).

An error is said as `as: file:line: what`, and `as` ends with status 1; a warning is said, and the program made. It reads its source three times (the segments' sizes, then each symbol's value, then the bytes), each file read from the disk once and kept in the task's RAM banks; a pass that finds errors is the last, so another pass's errors show once they're mended. `hi.s`, with `hydra.inc` (some 47K of source), takes about 4 seconds; `upper.s`, with `toollib.s` too (83K), about 9.

```
/> as -l /sd/0/sample/as/hi.s /ram/hi
/> /ram/hi Ann
Hello, Ann!
I'm task 4, in /, in window 0.
/> db /ram/hi Ann
...
db> l /ram/hi.lbl
8 symbols
```

dis: the disassembler

`dis [-cnw] [-l labels] [-o addr] file` is `as`'s inverse: a program written out as a source in `as`'s language, on stdout, which `as` assembles to the same bytes (`dis prog | as - copy`, in one line). `file` is a RAM program (HYX2, as `as` and the build make them), or with `-o addr` raw bytes from addr (as `as -b` makes them).

- **Its code** is found by following it from where it starts (a program's `main`, or addr): each instruction to the next, and to where it jumps, branches or calls, till one that goes no further (`rts`, `jmp` ...). What isn't reached is data: `.byte`, its text in quotes. `-c`: every byte from the start is an instruction, in turn.
- **Its labels**: each place an instruction names in it, its symbol's name from `-l`'s label file (`as -l`'s, the build's `obj/.../NAME.lbl`) when one names just that place, else `Lnnnn` (`main` at a program's entry); a place just past a symbol is the symbol and an offset (`buf+1`). Symbols outside it are defined first (`arg = $22`), its BSS's are labels in `.bss`, and the system calls and `r0-r15` are `hydra.inc`'s (`/lib/as/hydra.inc`; `-n`: not).
- **A program's header** is `HYX2_PROGRAM "name", main` when that makes the same bytes (`as`'s programs'), else its 48 bytes; its data goes in `.data`, its BSS in `.bss`.
- **Its lines** are compact, a tab then the instruction (a 20K program's source is some 100K, which `as` reads into RAM banks whole); `-w`, the SDK's sources' columns.

```
/> dis /sd/0/sample/tick
; /sd/0/sample/tick, as dis read it: as assembles this to its bytes again
.include "hydra.inc"

.include "hyx2.inc"

        HYX2_PROGRAM "tick", main
.code
main:
        stz B0895
        ...
        jsr NOTIFY
L0841:
        lda #$C8
        ldx #$00
        jsr SLEEP
...
/> dis -l /pc/db.lbl /rom/bin/db >/ram/db.s; as /ram/db.s /ram/db2; cmp /rom/bin/db /ram/db2
```

The instructions are the **asm library**'s (`modules/asm`, `spec/asm.def`), which `db`'s `d` and HyForth's `disasm` use too, so the three write them alike, and its assembler makes its encoding from the same table (`w65c02.inc`): lower case, an address in hex or its name, `a:` before an absolute address under `$100` where the instruction has a zero page form (`as` would take that one), an opcode the W65C02S doesn't define as the NOP it runs (`.byte` and its bytes). C has it as `dis_insn` (`asm.h`, with the symbol files' `lbl_load` ...), assembly as `ASMCALL ASM_DIS` (`asmlib.inc`).

The library is one bank of the paged ROM: the assembler (`FILE`, `as`'s whole run; and a source given a line at a time, BASIC's way: `BEGIN`, then for each pass `PASS`, the caller's symbols with `DEFINE`, the lines with `LINE`, and `END`; then `SYMBOL`, `IMAGE` and `DONE`), and the disassembler (`DIS`). It has no RAM of its own: the assembler's state is in RAM its caller lends it while it runs (`ASM_RAM`, `$5000-$7DFF`), and its zero page (`$22` on) is the caller's, put back as each call ends.

The system

Tool	What it does
`date [-n]`	The clock (`2026-10-03 15:04:05`), or `-n` seconds since 2000-01-01. Set it with `echo 2026-10-03 15:04:05 >/dev/time`, which sets the DS1747 too if there's one
`free`	RAM: each task's own banks (16 for each RAM module, 8K each), and the shared RAM: what segments have, what's free
`mods`	The paged ROM's modules: bank(s) (`4+33`: a small module sharing bank 4, its header at `$A000 + $3300`), type (program, driver, library; `boot` a driver started at boot), name
`hwtest`	The system starts again into the hardware test (as a T typed during POST does); the reset button ends it

The kernel's messages (the boot's, POST's, a driver's) are `/dev/kmesg`, its last 4K.

Others

Tool	What it does
`xmodem -r file`, `xmodem -s [-k] file`	A file received or sent with XMODEM over the serial line, with any terminal program on the PC: `-r` receives, `-s` sends (`-k`: 1K blocks)
`calc [-b base] [-d digits] [expression ...]`	An expression worked out exactly, with the Hydra's numbers (the number libraries', hylang's), its value in decimal or in the base `-b` names (hylang's base strings: `x`, `#x`, `b`, `c`, `16r` ...): `calc 2/3 + 0.5` is `7/6`, `calc sqrt 2` `1.41421356237`, `calc -b x 255` `FF`. Numbers are read in decimal (`0.5`, `2i`), or in a base by their own prefix (`#xFF`); `+ - * /` (exactly), `%` (an integer's remainder), `^`, parentheses; `sqrt exp log sin cos tan atan` (`-d` digits: 12), `abs floor round truncate fib numerator denominator re im rational random`, `gcd(a, b)`, `pow(a, b)`, `complex(a, b)`, `fixed(a, places)`; `pi`, `e`, `i`. rc's `^ * ( )` # need quotes: `calc '2^100'`. With no expression, each line of its input
`forth`, `hylang`, `basic`, `rc`	The languages and the shell: HyForth, hylang, HyBASIC, rc

`play` is sound's.

8. The emulator and the PC tools

You can do everything in this book without a board: the **emulator** runs the Hydra-16 cycle by cycle on a PC, with the same ROM images the board runs. It's what the tests run on, and what the system was developed in. This chapter is the emulator (in a terminal and in a browser), the build, the tests, and the PC's tools for the real board: the PC tool (a terminal and `/pc`), card images, and writing a card.

Everything here needs **Node.js 18 or later**; building the ROMs needs **cc65** too.

Contents

1. Running it
2. The keys
3. The options
4. The monitor
5. Debugging a program
6. The screen and the sound
7. In a browser
8. What's modelled
9. Building the ROMs
10. The tests
11. The benchmarks
12. The PC tools
13. The emulator in danlang

Running it

From `reborn/` (HydraOS) or `base/` (the base alone):

```
$ node sim/run.js -i
```

boots the system's images (`bin/bios_rockwell.bin`, or `bios_wdc.bin` with `--acia wdc`, and `bin/prom*.bin`) in **real time**, with your terminal as the Hydra's serial console. Without `-i`, it runs for a while (30 million cycles: 8.4 s of the Hydra's time, as fast as the PC can) and **reports**: the console's output, each task's state, the longest stretches with interrupts off, the stacks' depths, and the last instructions run.

```
$ node sim/run.js --cycles 20000000 --input 'ls /rom\r'
```

runs for 20 million cycles, typing `ls /rom` and Enter once the system's up, then reports. `--input`'s text takes `\r` (Enter) and `\w` (a pause of 2 million cycles), so a whole session can be scripted; the examples in this book were captured this way.

`reborn/sim/run.js` is the base's (`base/sim/run.js`) run with HydraOS's folder, plus `/pc`.

The keys

With `-i`, **Ctrl-A** and a key are the emulator's:

Keys	What they do
Ctrl-A x	Quit
Ctrl-A r	The reset button
Ctrl-A s	Show the state (the tasks, the registers)
Ctrl-A b	Stop, and the monitor
Ctrl-A v	Show the Vera X's screen as text
Ctrl-A p	Save the Vera X's screen as a PNG
Ctrl-A h	Help

Every other key goes to the Hydra. The terminal's size is told to the Hydra as the PC tool tells it, and again as the window changes.

The options

Option	What it does
`-i`, `--interactive`	The terminal is the serial console, in real time
`--speed N`	With `-i`: N times real time (0: as fast as it goes)
`--cycles N`	Stop at cycle N (default 30,000,000; with `-i`, never)
`--input TEXT`	Keys to type (`\r` Enter, `\w` a wait), one every 20,000 cycles from cycle 200,000; `--paste` as fast as the line goes
`--clock MHz`	The CPU's clock (3.579545; `7.15909` for a J8 board with faster parts than the stated ones, with a `build.js --clock 2` build: the stated parts run at 3.58 MHz or lower)
`--acia wdc`	A WDC W65C51N in the serial port, and its BIOS ROM (`bin/bios_wdc.bin`)
`--modules N`	Memory cards installed (default 2)
`--seed N`	The power-up's random RAM and registers, repeatable
`--sd FILE`	A card image in SD device 0 (again for 1, 2 ...), read and written in the file itself. With none, the system's own card (HydraOS's `bin/sdcard.img`), its writes kept in memory; `--no-sd`: no card
`--pc-dir DIR`	(HydraOS) `/pc` is the folder DIR: the emulator plays the PC tool's part. `--pc-read-only`, `--pc-log`
`--vera [V]`	A Vera X in slot 0 (its gateware version: 47.0.2 by default); `--vera-sd FILE` a card in its SD slot; `--vera-config MS` its start-up time
`--smc`	The Vera X's input controller, with a PS/2 keyboard and mouse; `--kbd TEXT` keys typed on it
`--screen`	After the report, the Vera X's text screen as text; `--frame-png FILE` the screen as a PNG
`--view [PORT]`	With `-i`: the Vera X's screen live in a browser (http://localhost:8016), its keys and mouse the controller's
`--sound [PORT]`	With `-i`: the sound in a browser (its Sound button); `--wav FILE` into a WAV file
`--bios FILE`, `--prom FILE`	Other images
`--trace N`	The last N instructions in the report (25)
`--trace-calls [L]`	Log each system call a program makes, by name, with its registers and what it gives back (L: only these calls or tasks)
`--break SPEC`	Stop before an instruction: a kernel label (`K_OPEN`), a module's (`rc:main`), `page:ADDR`, or `ADDR`
`--watch ADDR[:T]`, `--watch-read ADDR[:T]`, `--watch-pc ADDR`	Log each write (or read) of an address, by any task or task T's; each time the PC reaches an address
`--log FILE`	The log's lines into a file

The top of `base/sim/run.js` describes every option.

The monitor

Ctrl-A b (or a `--break` reached) stops the Hydra and gives a command line of the emulator's own:

Command	What it does
`c`	Continue
`n [N]`	The next N instructions, each shown
`r`	The registers
`t`	The tasks
`m ADDR [N] [T]`	N bytes (64) at ADDR, in task T's view (the running task's)
`b [SPEC]`	A break (none: the list)
`d N`	The Nth break gone
`w ADDR[:T]`	A write watch
`x`	Quit

Debugging a program

The emulator sees everything, so it's the easiest place to find a bug:

- `--trace-calls` logs every system call by name, its arguments (a string's text) and what it gave back: a program that fails with `not found` shows which `OPEN` failed and on what name. `--trace-calls OPEN,t4` traces only `OPEN`, only task 4's.
- `--break rc:main` stops at a module's label (the build's label files name every routine), and the monitor steps from there.
- `--watch` catches whatever writes an address: a variable trampled, a bank register changed.
- On the Hydra itself, `db` does the same for a program's own code (db).

The screen and the sound

- `--vera` puts a Vera X in slot 0, and `--smc` its keyboard and mouse controller.
- `-i --view` shows the screen live at http://localhost:8016, its keys and mouse the controller's, while the terminal stays the serial console.
- `-i --sound` plays the sound in a browser (the same page as `--view`'s; press its Sound button), some 0.15 s behind: the YM2151 (a sample-exact port of ymfm), and with `--vera` the PSG and the PCM. `--wav FILE` records it.
- `--screen` prints the text screen after a run, and `--frame-png FILE` saves it.

```
$ node sim/run.js -i --vera --smc --view --sound
```

is the whole machine: open http://localhost:8016 for the screen and sound, and type in either window.

In a browser

```
$ node sim/web.js
```

builds `bin/hydra-16.html`: one file holding the page, the emulator (in a Web Worker) and the ROM images, which runs HydraOS in Chrome, Edge, Firefox or Safari with nothing installed and nothing sent anywhere, opened straight from the disk. `build.js` makes it too, after the images, and it's in Git with them (the same images make the same page: it names them by their checksum, not the commit), so it's there ready made, with no Node.js. `--serve` serves it at http://localhost:8017.

- The serial console is a terminal in the page, with the Vera X's screen beside it.
- **Setup** has the Vera X card, its keyboard and mouse, the clock chip, the memory cards, and SD cards: HydraOS's card as it starts, a new blank one, or an image loaded from the PC. Cards are kept in the browser, and saved as image files.
- The page is one file: copy it anywhere, or put it on a web server.

What's modelled

The emulator (`base/sim/lib/machine.js` and its parts) is the board, cycle by cycle:

- the W65C02S, every instruction and its cycles;
- `T`, `U`, `V` and `W`, random at power-up as the latches are; each task's `$00`, `$01` and RAM;
- the RAM window's task banks (only the memory cards installed: `--modules N`; the rest float) and the shared banks;
- the paged ROM with its halves swapped and the board's crossed bank bits; the BIOS ROM's pages;
- the I/O ports and the system port; the interrupt lines, their priority and the vector RAM (`n ^ 7`);
- the VIA: its timers, port A's pins and CA1, I2C devices on PA0/PA1, SD cards and other devices on its SPI port;
- the ACIA (Rockwell's, or `--acia wdc`), the YM2151 (its timers, its busy time, and with `--sound` its sound);
- a DS1747 in U7; the reset button (Ctrl-A r);
- the Vera X: its registers, VRAM, the layers, sprites and collisions, the scan's timing and interrupts, FX, the PCM FIFO, the PSG, its SD card, and the input controller.

Not modelled: the analog side (the audio path's filters, the serial line's levels), wait states (the board has none), and DMA.

Building the ROMs

```
$ cd reborn
$ node build.js
```

builds the base first, then HydraOS: the BIOS ROM, one for each serial chip (`bin/bios_rockwell.bin` and `bin/bios_wdc.bin`: the kernel is built twice), the paged ROM's chips (`bin/prom0.bin` ...: the same for both), the ROM disk, the SD card's image (`bin/sdcard.img`), the SDKs (`bin/sdk`), and a report of each part's size and room left. It needs cc65's `ca65` and `ld65` (and `cc65` and `ar65` for the C programs), found in `$CC65_BIN`, `$CC65_HOME/bin`, or on the PATH.

Command	What it builds
`node build.js`	Everything, for a 3.58 MHz board (a BIOS ROM for each ACIA, Rockwell's and WDC's)
`node build.js --clock 2`	For a 7.16 MHz board (jumper J8): one with faster parts than the stated ones, which run at 3.58 MHz or lower
`node build.js --acia wdc`	The same images, with `obj/` (the labels, the objects the tests use) the WDC build's, not the Rockwell's: `sim/test.js` then runs the tests on a W65C51N
`node build.js prog DIR`	A program of your own: `DIR/*.s` (or `*.c`) into `DIR/NAME.hyx` (programming)
`node ../base/build.js`	The base alone (from `reborn/`), or `node build.js` in `base/`

The images in `bin/` are in Git, so the emulator and the chips need no toolchain. A change to what's in them is committed with them rebuilt.

The tests

```
$ node sim/test.js
```

runs the base's tests and HydraOS's: each boots its own ROM image (the system's modules and a test module as init), runs, and is judged on its output (`ok` and `not ok` lines, and `PASS`), its time budgets (the cycles between its marks), the longest stretch with interrupts off, and its own checks. They run as many at a time as the PC has cores.

Command	What it does
`node sim/test.js`	All of them
`node sim/test.js irq -v`	One test, with its output
`node sim/test.js --build`	Build first
`node sim/test.js -j 1`	One at a time
`node sim/test.js --dl`	In the danlang emulator (below)

The tests are in `base/tests/tests.js` and `reborn/tests/tests.js`; HyForth's (the Forth 2012 suite), hylang's (danlang's suite) and HyBASIC's suites run among them.

The benchmarks

```
$ node sim/bench.js
```

runs the twenty benchmarks in HyForth, hylang and HyBASIC (and HyBASIC's inline assembly) in the emulator, checks every result is the same, and prints a table of their times by kind (how fast). `--quick` uses the small sizes; `--only NAME,...` and `--kind KIND,...` pick some. On the board: `forth /sd/0/bench/bench.fs`, `hylang /sd/0/bench/bench.hl`, `basic /sd/0/bench/bench.bas`.

The PC tools

In `reborn/sim/tools/`:

Tool	What it does
`hydrapc.js`	**The PC tool**: a terminal on the Hydra's serial port that also serves a folder of the PC as `/pc` (/pc)
`hydrafs.js`	**Card images**: make one, list it, put files in and get them out

Tool	What it does
`sdwrite.js`	**Write an image onto a card** in the PC's reader
`hysong.js`	The score language into a ZSM song (and a VGM, to hear on the PC)
`pcfs.js`	`/pc`'s file server (the PC tool's, and the emulator's)

hydrapc.js: the terminal and /pc

```
$ cd reborn/sim; npm install; cd ..                  # once: the serialport package
$ node sim/tools/hydrapc.js --list                   # the PC's serial ports
$ node sim/tools/hydrapc.js COM3 work                # a terminal on COM3, and the folder work as /pc
```

`--baud N` sets the rate (9600 as the Hydra starts), `--read-only` keeps the folder from being changed, `--log` lists `/pc`'s requests (and the frames from the Hydra that came damaged or stopped part-way, and the replies it asked for again: a bad line shows there; Ctrl-A l turns it on and off). Ctrl-A s says the counts so far, and quitting says them too. Ctrl-A x quits. It tells the Hydra its window's size, so the Hydra's windows fit it. In Windows Terminal, `--win32-input` lets Ctrl-Tab reach the Hydra.

hydrafs.js: card images

```
$ node sim/tools/hydrafs.js mkfs card.img 16 MYCARD   # a new 16 MB HydraFS image, labelled
$ node sim/tools/hydrafs.js put card.img hello.hyx bin # a PC file into the image's /bin
$ node sim/tools/hydrafs.js import card.img myfolder   # a whole folder tree
$ node sim/tools/hydrafs.js ls card.img -l bin
$ node sim/tools/hydrafs.js get card.img notes.txt notes.txt
$ node sim/tools/hydrafs.js check card.img
```

`mkfs ... -p FATMB` puts the HydraFS in a partition after a FAT one, so the card works in a PC too. Paths start at the card's root (in Git Bash, leave the first `/` off: it turns into a Windows path). An image is what `--sd` takes in the emulator, and what `sdwrite.js` writes to a card.

sdwrite.js: an image onto a card

```
$ node sim/tools/sdwrite.js --list                   # the PC's disks; the removable ones marked
$ node sim/tools/sdwrite.js bin/sdcard.img 3         # the image onto disk 3 (Windows' number)
```

It writes the image to the card's first blocks, raw, then reads them back and compares. It asks for the disk's number again before writing; it **refuses** the disk the system is on, one smaller than the image, and one that isn't removable (without `--force`). It needs an administrator's prompt on Windows, root on Linux (`/dev/sdb`, the card's partitions unmounted), an administrator on macOS (`/dev/disk4`). `--dry-run` shows what it would do.

The emulator in danlang

`reborn/sim/dl/` is the whole emulator written again in **danlang** (the language hylang is), run by danlang's C# interpreter: the CPU, the memory map, the ACIA, the VIA, the YM2151, the SD cards, the I2C bus, the DS1747, the Vera X and its input controller. It runs every test (`node sim/test.js --dl`) and the serial console (`danlang sim/dl/run.dl -i`), about seven times slower than the JavaScript one. It's a test of danlang, and of the emulator, by writing it twice.

9. The programming guide

How to write programs of your own for the Hydra-16: in assembly or C, as programs in files or modules in the ROM, as servers that make devices of their own, and with the system's libraries. It builds on chapter 3 (the kernel and its calls); the call reference is every call in full.

Contents

1. Assembly: the assembly SDK, a program walked through, building and running it, the memory a program has, modules of the paged ROM, modules of several banks, the tools' library.
2. C: the C SDK (cc65), the standard library on the Hydra, the Hydra's own calls (`hydra.h`), sound, the screen, numbers, and the multitasking demos.
3. Servers and drivers: serving files with srvlib, requests and waiting, interrupts, putting a driver in the ROM.
4. Libraries: library modules and `XCALL`, the number libraries and their stored format, the asm library.
5. An operating system of your own: what boots, init, and a system of your own on the base.
6. The system call reference: every call, generated from the specification.

What a program is

Every executable on the Hydra is a **HYX2** file: a 48-byte header, then its code and data. The header says what it is (a program, a driver or a library), where it loads, its entry points, and its name. There are two ways to run one:

	A program in a file	**A module of the paged ROM**
Where it lives	A card, a RAM disk, `/pc`, the ROM disk	The paged ROM: banks of its own, or a page of a bank it shares (a small one)
How it runs	`SPAWN` gives the new task the file, and it **reads itself into RAM at** `$0800` as it starts	**Runs in place**: the task's `$01` selects its bank, so its code is at `$A000-$DFFF`; only its data is copied into RAM
Its size	Up to `$7FFF`, less its data	16K a bank, up to 8 banks
Built with	`-D HYX2_RAM`, linked by `hyx2.cfg`	Linked by `module.cfg` (`module2.cfg` ... `module8.cfg`), listed in `rom.txt`
For	Your programs	The system's programs and drivers, or yours in a ROM of your own

You'll start with programs in files: build on the PC, copy to a card or `/pc`, run by name.

The workflow

1. **Write** the source on the PC (or on the Hydra with `edit`).
2. **Build** it: `node build.js prog DIR` in `reborn/` builds the folder DIR's `.s` or `.c` files into `DIR/NAME.hyx`. Or use the SDK alone with cc65's tools: `bin/sdk/asm` and `bin/sdk/c` are the SDKs, whole, to take anywhere. Or assemble on the Hydra itself: `as file.s` makes the same program, byte for byte.
3. **Run** it in the emulator: `node sim/run.js -i --pc-dir DIR`, then `/pc/NAME` at the prompt. Or put it on a card image (`hydrafs.js put`) and run it with `--sd`.
4. **Debug** it: `--trace-calls` in the emulator shows every call it makes; `db` on the Hydra steps it (db); `--break` and the emulator's monitor stop anywhere.
5. **Run it on the board**: copy it to a card, or over `/pc` with the PC tool, and type its path. `bind -a /sd/0/bin /bin` (or a card's `/bin`, which is bound already) runs it by name.

The rules a program keeps

- **Call the system with** `jsr` to the jump table; arguments in `.A`, `.X`, `.Y` and `r0-r15`; C = 1 on failure with the error in `.A` (the system calls).
- **Your zero page is** `$22-$7F`: no call touches it. `$80-$FF` and `$0200-$03FF` are the system's (but `TASK_EVENT`, a server's).
- **Your RAM is** `$0400` **up** (a RAM program's from `$0800`), to `$7FFF`; your banks are at `$8000` when you select them.
- **Never write** `T`, `V` **or** `W`, and leave the I/O area to the drivers: use the devices' files. (`U` and `$00` are yours, for your shared segments and banks.)

- **Never hold interrupts off** for more than 200 cycles.
- **A long job is fine**: the scheduler preempts you. But a busy loop waiting for something wastes the CPU: wait in a call (`READ`, `SLEEP`, `SEM_ACQUIRE`, `PAUSE`) instead.
- **Take your notes**: Ctrl-C ends a program unless it has a handler (`NOTIFY`), and a handler should end it soon.

A first program

`sdk/asm/samples/hi/hi.s` greets its arguments and says where it runs: its arguments come at `r0`, it prints with `PUTS` and `PUTC`, asks the kernel its task (`GETPID`) and its directory (`GETCWD`), and reads its environment (`ENV_GET`); returning from `main` ends it with code 0. The system calls walks through it, and it's on HydraOS's card:

```
/> /sd/0/sample/hi Ann Bob
Hello, Ann!
Hello, Bob!
I'm task 4, in /, in window 0.
```

The samples, in `reborn/sdk/asm/samples` and `reborn/sdk/c/samples` (built onto the card at `/sd/0/sample` and `/sd/0/sample/c`):

Sample	Shows
`hi` (asm)	Arguments, the task, the directory, the environment
`upper` (asm)	A filter on the tools' library
`tick` (asm)	A note handler
`counter` (asm)	A server: a driver, a module run in place, on srvlib
`nsum` (asm)	The number libraries
`hello`, `upper`, `code` (C)	Arguments, a filter, exit statuses
`keys` (C)	conio: the screen and raw keys
`tones`, `jukebox` (C)	Sound
`sketch`, `shapes` (C)	The Vera X: drawing and the mouse; cc65's TGI
`race`, `chorus`, `philo`, `prodcons`, `round` (C)	Tasks working together: shared segments and semaphores
`ntest`, `ctest` (C)	The number library's test; the C library's test

Assembly

Programs for the Hydra in 65C02 assembly, with cc65's assembler and linker (`ca65`, `ld65`) on a PC, or with `as` on the Hydra itself: the same source makes the same program, byte for byte. This page is the assembly SDK, a program from source to running, the memory a program has, and the larger forms: modules of the paged ROM and modules of several banks.

Contents

1. The SDK
2. A program
3. Building it
4. Running it
5. A program's memory
6. Notes and Ctrl-C
7. The tools' library
8. Modules of the paged ROM
9. Modules of several banks
10. Assembling on the Hydra
11. The source style

The SDK

The assembly SDK is `base/sdk/asm` (the core every system shares) and `reborn/sdk/asm` (HydraOS's additions). The build copies both, with the samples, to `reborn/bin/sdk/asm`, to take anywhere.

File	What it is
`hydra.inc`	The system calls (their addresses), the error codes and the constants. Made from `base/spec/api.def` by the build and kept in Git: never edit it
`hyx2.inc`	The header: `HYX2_PROGRAM "name", main`, `HYX2_DRIVER`, `HYX2_LIBRARY`
`hyx2.cfg`	The link for a program in a file: header, code and data from `$0800`, the BSS after them
`macros.inc`	`CALL name`, `CHECK label`, `LDR reg, value`, `MOVR to, from`, `PRINT label` or `PRINT "text"`; `CR`, `LF`, `TAB`
`toollib.inc`, `toollib.s`	What the system's tools share (below)
`srvlib.inc`, `srvlib.s`	The server library (servers and drivers)
`nslib.s`	A task's default namespace from the namespace file (Plan 9's `newns`)
`numbers.inc`, `numlib.s`	The number libraries' calls (libraries)
`asmlib.inc`	The asm library's calls: the disassembler and the assembler (libraries)
`samples/`	`hi`, `upper`, `tick`, `counter`, `nsum`

A program

```
.include "hydra.inc"
.include "hyx2.inc"
.include "macros.inc"

            HYX2_PROGRAM "hello", main

.code
main:                                   ; r0: the arguments
            PRINT       "Hello"
            lda         #LF
            CALL        PUTC
            lda         #0              ; (Returning: EXITS with code 0)
            rts
```

- `main` gets `r0` pointing at its **arguments**: zero-terminated strings one after another, an empty one after the last (`hello a b`: `"a", 0, "b", 0, 0`). The program's name is the header's (11 characters at most), not one of them.
- **Returning** ends the task with code 0. `EXITS` ends it anywhere, with a code (`.A`) and a message (`r0`, or 0): rc's `$status` is the message if there is one, else the code.
- **A call** is a `jsr` to its name (`CALL name` says the same). C = 0 is success; C = 1 is failure, with the error in `.A` (`ERRSTR` gives its text); `CHECK label` goes on to label on a failure.
- **Its fds** 0, 1 and 2 are its window's console, or what rc gave it (`<`, `>`, `|`). `PUTC`, `PUTS` and `GETC` write and read them a byte or a string at a time; `OPEN`, `READ`, `WRITE` and `CLOSE` do the rest. Names go through its namespace, as rc's do.
- **Its environment** (`ENV_GET`, `ENV_PUT`) is a copy of rc's: rc's variables, `$window`, `$path`.

Building it

In the repository, a program in a folder of its own (its `.s` files):

```
$ cd reborn
$ node build.js prog path/to/hello          # path/to/hello/hello.hyx
```

With cc65's tools and the SDK alone:

```
$ ca65 --cpu 65C02 -D HYX2_RAM -I sdk/asm -o hello.o hello.s
$ ld65 -C sdk/asm/hyx2.cfg -o hello.hyx hello.o
```

`-D HYX2_RAM` makes a program in a file. Without it, `hyx2.inc` makes a module of the paged ROM (below).

A bank program is built from the same sources: a file named `bank` in its folder, with `run` in it (or `task`, for one that must have a task of its own), and `node build.js prog` assembles it with `-D HYX2_BANK`, links it with `sdk/asm/bank.cfg` (an o65 module at `$8000`, its relocations kept) and puts a copy of its HYX2 header, `HF_BANK` set, before it in its file. The shells run a bank program in their own task, kept in a RAM bank for the next time (the kernel); in its own task it runs from a bank of that task's. Its code, data and BSS fit one bank, 7.5K less its data's size again (a copy, for each run), and it mustn't map a bank at `$8000` itself. A program that changes what's its task's (the environment, the namespace, the directory) or needs much RAM (`BREAK`) should be `task`: in place those are its shell's.

Running it

Copy it to a card, a RAM disk or `/pc`, and type its path (`/sd/0/hello`), or its name if it's in `.` or a `bin` (`$path` is `(. /bin)`; a card's `/bin` is part of `/bin`). In the emulator, from `reborn/`:

```
$ node sim/run.js -i --pc-dir path/to/hello         # then: /pc/hello
```

or on a card image:

```
$ node sim/tools/hydrafs.js mkfs card.img 8 MINE
$ node sim/tools/hydrafs.js put card.img path/to/hello/hello.hyx hello
$ node sim/run.js -i --sd card.img                # then: /sd/0/hello
```

The samples are on HydraOS's card: `/sd/0/sample/hi Ann Bob`, `echo hi | /sd/0/sample/upper`, `/sd/0/sample/nsum 1/3 0.5 2`, `/sd/0/sample/tick` (Ctrl-C stops it).

A program's memory

Where	What
`$00`, `$01`	Its bank registers: the RAM bank at `$8000`, the paged ROM bank at `$A000`
`$02-$21`	`r0-r15`, the call registers
`$22-$7F`	**Its own zero page** (`.zeropage`): no call touches it
`$80-$FF`	The system's (one byte is the program's: `TASK_EVENT`, a server's event count)
`$0100-$01FF`	Its stack
`$0200-$03FF`	The system's: its fds, name, arguments (`TASK_ARGS`, `$0350`), current directory ...
`$0800-`	**Its program**: its header, code, data, then BSS; then its break
`-$7FFF`	Pages given out from the top down (`PAGES_ALLOC`)
`$8000-$9FFF`	The RAM bank `$00` selects: one of its own, or a shared one
`$A000-$DFFF`	The paged ROM bank `$01` selects
`$E000-$FFFF`	The BIOS ROM: the jump table at `$F800`; the I/O at `$FF00` (the drivers')

More memory:

- `BREAK` moves the end of its data (as Unix's `brk`); `PAGES_ALLOC` gives runs of 256-byte pages from the top.
- **Banks**: 16 of 8K for each memory card. `BANKS_ALLOC` gives a run of them; write one's number to `$00`, and it's at `$8000`:

```
        lda     #4
        jsr     BANKS_ALLOC                     ; Four banks: .A = the first
        bcs     @nomem
        sta     $00                             ; The first at $8000
        lda     #$5A
        sta     $8000
```

- **Shared segments** are banks several tasks see: `SEG_CREATE`, `SEG_ATTACH`, then `SEG_MAP` for the values of `U` and `$00` (memory).
- **Semaphores** keep tasks in step: `SEM_NEW`, `SEM_ACQUIRE`, `SEM_RELEASE` (semaphores).

Another task's memory is reachable only through `/proc/N/mem`.

Notes and Ctrl-C

A note (Ctrl-C at its window: `NOTE_INTERRUPT`) ends a program, unless it has a handler. `NOTIFY` sets one: it's called with `.A` the note, and returns C = 0 to go on where the program was, C = 1 for the default. `sdk/asm/samples/tick/tick.s`:

```
main:
            LDR         r0, handler                         ; Its note handler
            jsr         NOTIFY
@second:
            lda         #<TICK_HZ                           ; A second (TICK_HZ ticks): a note ends it sooner
            ldx         #>TICK_HZ
            jsr         SLEEP
            lda         noted
            bne         @end
            ...

; The note handler: .A = the note.  C = 0: the task goes on where it was (here, SLEEP ends early); C = 1 would be the
; default
handler:
            sta         noted
            clc
            rts
```

A call that was waiting when the note came returns `E_INTR`. A program that runs code in more than one bank keeps its handler in RAM: the kernel calls it with whatever bank is at `$A000`.

The tools' library

`toollib` is what the system's tools share, so a tool of your own behaves as theirs do. Include `toollib.inc` at the top (its zero page) and `toollib.s` at the end; define `tl_name` (the tool's name), `tl_flagset` (the flag letters it takes) and `tl_usage`.

Routine	What it does
`tl_start`	Parses `main`'s arguments: the flags into `tl_flags` (a bit each), `tl_arg` the first argument after them; a bad flag says the usage and ends
`tl_next`, `tl_count`	The next argument; how many are left
`tl_putc`, `tl_puts`, `tl_space`, `tl_nl`, `tl_dec`	Output, buffered 256 bytes at a time
`tl_flush`, `tl_end`	The output written; the end, with `tl_code`
`tl_err`, `tl_warn`	`NAME: name: why` and `NAME: message` on fd 2, the status 1
`tl_eachin`	Each input named (none: fd 0) handed to a routine, with `tl_getc` reading it
`tl_readdir`, `tl_walk`	A directory read whole; a tree walked, depth first
`tl_pcat`, `tl_pcut`, `tl_base`	Paths
`tl_atoi`, `tl_div`, `tl_date`, `tl_field`	Numbers, dates, columns

`samples/upper` is a filter on it: each input's text in capitals.

Modules of the paged ROM

Without `-D HYX2_RAM`, `hyx2.inc` makes a **module**: linked by `modules/module.cfg` to run in place in the paged ROM, its data copied into its task's RAM (from `$0400`) and its BSS cleared as it starts. The system's drivers, shells and languages are modules: they cost no load (a `SPAWN` of one takes about 14 ms) and leave the task nearly all its RAM. A module of one bank is linked first at `$A000`; then the build finds it a place (`romimg.js`'s plan: bank 0 after the directory, the free top of another module's bank, or a bank of its own) and links it again at that page, `$A000 + 256 * page`, so small modules share banks. Its header's `HX_LOAD` says where (`HYX2_LOAD`, the link's `__ROM_START__`). A module's code mustn't assume `$A000`: its own labels are where it is.

- **To put one in a ROM**: a folder in `modules/` (`modules/NAME/*.s`), listed in `modules/rom.txt`, and `node build.js` builds it with the rest. `#m/NAME` is then its file, and `#m/bin` (bound at `/bin`) runs it by name.
- **A program** module: `HYX2_PROGRAM "name", main`.
- **A driver**: `HYX2_DRIVER "name", init, serve, irq, stop, flags`, with `HF_BOOT` to start it at boot (servers and drivers).
- **A library**: `HYX2_LIBRARY` (libraries).

The module directory (bank 0) lists each module's bank, type, flags, name (10 characters at most) and page, 127 modules at most (the boot).

Modules of several banks

A module bigger than a bank (16K) spans two to eight, one after another:

- `HYX2_PROGRAM "name", main, 2` (or `HYX2_DRIVER ..., 2`), linked by `modules/module2.cfg` (up to `module8.cfg`).
- Its second bank's code and read-only data are in the segments `CODE2` and `RODATA2`, its third's in `CODE3` and `RODATA3`, and so on, all at the same addresses as the first's (`$A000-$DFFF`).
- No bank calls another directly: `FAR2 routine` calls a routine in the second bank from the first, `FAR1 routine` one in the first from the second, and `FARN bank, routine` one in any bank from any, the caller's bank set again after. `.A`, `.X`, `.Y` and C pass both ways. The trampolines are in the module's RAM, and switch its own bank register (`$01`); `HYX2_BANKS_INIT`, first thing in `init` or `main`, notes the module's banks.
- **Its note handler, and its interrupt entry** (and what that calls) must be in its RAM (its `DATA`): a note or an interrupt may come while any of its banks is at `$A000`.
- Code that every bank calls often can be in `DATA` too, if it calls nothing of any bank's.
- **The build checks it** (`base/tools/bankcheck.js`, from the link's debug information): an instruction in one bank that uses a label in another directly (`jsr`, `jmp`, `lda label`) stops the build, as it would run what its own bank has at that address. A directive (`.word label`), an immediate (`#<label`, `LDR`) and the far calls are fine. A comment `bankcheck: far NAME` names a macro of the module's own that only takes an address or calls far, and `bankcheck: copied` marks a file whose code is copied to RAM and run with another bank in place.

`play` and the storage driver are two banks each, the console driver three, HyBASIC five and hylang eight (rc was two, its lexer and parser the second, till they fit in one).

Assembling on the Hydra

`as [-bl] file.s [out]` assembles the same source on the Hydra itself (as):

```
/> as -l /sd/0/sample/as/hi.s /ram/hi
/> /ram/hi Ann
Hello, Ann!
I'm task 4, in /, in window 0.
```

`.include` finds the SDK's files in `/lib/as`. `-l` writes the labels too, which `db` reads. `dis` turns a program back into source `as` assembles to the same bytes (dis).

The source style

The system's sources keep one style, which yours may follow:

- ca65's syntax, 65C02. A label at column 1; the mnemonic at column 13; the operand at column 25; a comment at column 61 (or on its own lines above). A blank line after a jump or return that ends a block.
- Each file starts with a `; ****` line and a paragraph saying what it is and how it works; each routine with a comment saying what it does, its `IN:`, `OUT:` and what it changes.
- `; ----` marks the steps of a long routine.
- A macro defines no labels but unnamed ones (`:`), so the cheap locals (`@name`) of the routine using it keep their scope.
- Names: `UPPER_SNAKE` for constants and calls; a call `NAME` is implemented by the kernel's `K_NAME`.

C

Programs for the Hydra in C, with **cc65** (`cc65`, `ca65`, `ld65`). A C program is a HYX2 file, as an assembly one is: a 48-byte header, then its code and data, which `SPAWN` reads into a task's RAM at `$0800` and starts. cc65's own target `none` does the compiling; the Hydra's part is the start-up, the link, and a library under the standard one, so standard C (stdio, files, `malloc`, `printf`, `system`, `signal`, conio) works on the Hydra's files and devices.

Contents

1. The SDK
2. A program
3. Numbers
4. Building it
5. Running it
6. The multitasking demos
7. How C reaches the system

The SDK

The C SDK is `reborn/sdk/c`; the build makes its library and copies it, with the generated headers and the samples, to `reborn/bin/sdk/c`, to take anywhere.

File	What it is
`include/hydra.h`	The Hydra's own calls (tasks and exit statuses, the namespace, the tick, RAM banks, semaphores, shared segments, the windows, any call by `hy_call`), and what cc65's headers leave to a target: `setenv`, `fstat`, `isatty`, conio's colours and keys
`include/hydracalls.h`	Every system call's address, error code and constant, each with `HY_` before its name. Made from `base/spec/` by the build, kept in Git: never edit it
`include/snd.h`	Sound: the YM2151 and the Vera X's PSG through the sound driver: claims, patches, notes, volumes, bends, drums, waveforms, MML, songs
`include/vera.h`	The Vera X: the bitmap and drawing on it, VRAM, the palette, sprites, frames, the mouse; the chip claimed and its registers; `hydra_tgi`, cc65's TGI's driver
`include/num.h`, `numdefs.h`	The Hydra's numbers (below)
`include/asm.h`, `asmdefs.h`	The asm library: `dis_insn` (an instruction disassembled), symbol files (`lbl_load` ...)
`hydra.cfg`	The link: the header, code and data from `$0800`, the BSS, the heap, and the C stack (2K) down from `$7F00`
`lib/`	The library's sources, built with cc65's `none.lib` into `hydra.lib`
`samples/`	`hello`, `upper`, `code`, `keys`, `tones`, `jukebox`, `sketch`, `shapes`, `ctest`, `ntest`, and the multitasking demos

---

A program

```
#include <stdio.h>

int main (int argc, char* argv[])
{
    printf ("hello, %s\n", argc > 1 ? argv[1] : "world");
    return 0;
}
```

- `argv[0]` is its name: its file's (`/sd/0/sample/c/hello`'s is `hello`); its arguments are rc's words.
- `main`'s value, or `exit`'s, is its exit code (0 is success); `hy_exits ("why")` ends it with a message and code 1, as Plan 9's `exits` does. rc's `$status` is the message if there is one, else the code.

- Its fds 0, 1 and 2 are stdin, stdout and stderr: its window's console, or what rc gave it (`<`, `>`, `|`). stdio is buffered: a file's bytes are read and written 255 at a time, a console's output a line at a time (and before any read), stderr's at once; it all goes out at `fflush`, `fclose` and the program's end.
- Files: `fopen`, `open`, `read`, `write`, `lseek`, `stat`, `fstat`, `remove`, `rename`, `mkdir`, `rmdir`, `chdir`, `getcwd`, and `opendir`/`readdir` (`hy_dirstat`: an entry's whole stat record). Names go through its namespace, as rc's do; `hy_bind`, `hy_mount` and `hy_unmount` change it.
- A failed call returns -1 (or NULL) and sets `errno` (cc65's: `ENOENT` ...) and `_oserror` (the kernel's own code, `HY_E_NOENT` ...; `_stroserror` and `hy_errstr` give its text, the kernel's).
- Its environment is a copy of rc's variables: `getenv`, `setenv`, `putenv`, `unsetenv`.
- `system ("cmd")` runs a command line with rc (`rc -c`): its value is the command's exit code. `hy_spawn` starts a program in a task of its own, `hy_wait` waits for it and takes its code and message; `hy_exec` runs a program in this one's place, in its task (Plan 9's `exec`: back only if it fails); `hy_parent` is the task that started this one.
- Semaphores, for tasks that share something: `hy_sem_new` (a count, or `HY_SEM_MUTEX`), `hy_sem_acquire` (it waits), `hy_sem_try`, `hy_sem_release`, `hy_sem_free`. Every task's, by number; the program's end frees those it made and gives back the mutexes it holds.
- Shared segments, memory several tasks see: `hy_seg_create` (banks of 8K: its number, this task attached), `hy_seg_attach` (another task, given the number), `hy_seg_map` (a bank of it at `HY_BANK_WINDOW`, `$8000`), `hy_seg_detach`. The last task attached to go frees it.
- `signal (SIGINT, f)`: Ctrl-C at its window (the interrupt note) runs `f`; `SIG_IGN` ignores it; the default ends the program.
- conio (`conio.h`) works the console as an ANSI terminal: `clrscr`, `gotoxy`, `textcolor`, `revers`, `cursor`; `cgetc` reads keys raw (no echo, each as it's typed, the cursor and function keys as one code each: `CH_*` in `hydra.h`) until the program ends. `screensize` is the window's size (its `consctl`'s `size` line); `cgetc` gives `CH_RESIZE` when it changes, and `screensize` then has the new one. With `keys mods` written to `consctl`, a key the terminal sent modified comes as `CH_MOD`, its modifiers (`HY_MOD_SHIFT`, `HY_MOD_ALT`, `HY_MOD_CTRL`), then the key.
- The window's chrome (`hydra.h`): `hy_wlabel` its title, `hy_wstatus` its status line (the footer's `%s`), `hy_wctl` any line of its `wctl` (`chrome screen off` ...); `hy_wnew (flags)` a window made, not shown (`HY_WGROUP`: in a group of its own), its number: its files `#cN` (`#cN/cons` ...), the window gone with its last cons closed.
- Sound (`snd.h`): claim the channels it uses (`snd_claim`), then patches, notes, volumes, bends and drums on them; they're given back as it ends. `snd_play` plays a song in the background (`play`, in a task of its own).
- `time` is the system's clock (`/dev/time`; `hy_time`, in seconds since 2000), `clock` the ticks since the program's start; `sleep` and `hy_sleep_ticks` let the other tasks run, and `hy_sleep_until` sleeps till a tick count (for steady timing: a time already passed returns at once).
- Its RAM: `$0800` to `$7F00`, for the program, its BSS, the heap (`malloc`) and the C stack (2K); its own RAM banks at `$8000`-`$9FFF` (`hy_banks_alloc`, `hy_bank`). The 6502's stack is 256 bytes: deep recursion runs out of it. cc65's runtime has the zero page from `$22` (26 bytes); the program's own goes after it, to `$7F`.

Numbers

`num.h` gives C the numbers hylang, HyForth and BASIC have: integers of any size (to 255 bytes), fixed decimals (`1.25`), rationals (`2/3`) and complex numbers (`1+2i`), all exact, worked by the number libraries in the paged ROM. A number is an array of bytes (`num_t`) in their stored format: `NUM_MAX` (1,040) at most, most of them a few.

```
#include <stdio.h>
#include <num.h>

int main (void)
{
    num_t a[32], b[32], c[64];

    num_parse (a, sizeof a, "2/3", NULL, NULL);
    num_parse (b, sizeof b, "0.5", NULL, NULL);
    num_add (c, sizeof c, a, b);
    printf ("%N, in binary %{b}N\n", c, c);        /* 7/6, in binary 111/110 */
    return 0;
}
```

- A function that makes a number takes its place and its room first, then its operands, and gives back the result's length; or -1, and `num_error` says why (`NE_DIV0` ...; `num_strerror` its text). A result may go over one of its operands. `num_size` is a number's length, from its bytes.

- The first call readies the libraries (`num_init`): it takes two of the program's RAM banks.
- Numbers are read and written in the base, decimal at the start; `num_set_base` changes it, with hylang's base strings (`"x"` writes `FF`, `"#x"` `#xFF`; `"b"`, `"o"`, `"c"` balanced ternary, `"16r"`, `"[01]"` digits of its own ...). `num_parse` and `num_display` take a base of their own too (`NULL`: the base); `num_format` is hylang's `format` (`"{} is {x}"`).
- `num_digits` sets the math functions' precision, 12 significant digits at the start: `num_sqrt` of 2 is `1.41421356237`, of `9/4` exactly `3/2`.
- `printf` (`fprintf`, `sprintf`, `snprintf` and their `v` forms) writes a number with `%N` (a `num_t*`), in the base; and with a base in braces right after the `%`, for `%N` and C's integers alike: `%{x}N`, `%{#b}d` (`#b101`), `%{c}ld`, `%{16r}u`; `%{}N` the base, and `%{*}N` the base from the arguments, a string (before a `*` width's). The width, `-` and `0` pad the whole text, a `+` or a space goes before a number not below 0, and a precision is passed over: a number is written whole, however long (2^1000's 302 digits; its text 8K at most). Without braces, `%d` and `%x` are C's own.
- `scanf` (`fscanf`, `sscanf`) reads them the same way: `%N` a number into a place and its room, two arguments (`scanf ("%N", n, sizeof n)`), and `%{x}d` an `int` in base x. Such a conversion reads a word: up to white space, its width, or the character the format has next (`"%N,%N"` reads `1/2,3`), all of which must be a number.
- `calc` (`programs/calc`) is a program on it; the sample `ntest` is its test. Assembly has the same calls: `sdk/asm`'s `numbers.inc` and `numlib.s` (libraries).

Building it

In this repository, a program in a folder of its own (its `.c` files, and `.s` files if it has any):

```
node build.js prog path/to/hello          # path/to/hello/hello.hyx
```

With cc65 and `bin/sdk/c` alone:

```
cc65 -t none --cpu 65C02 -O -I sdk/c/include hello.c
ca65 --cpu 65C02 hello.s
ld65 -C sdk/c/hydra.cfg -o hello.hyx hello.o sdk/c/lib/hydra.lib
```

Running it

As an assembly program is run (assembly): copy it to a card or the RAM disk, and type its path, or its name if it's in `.` or a `bin`. The samples are on the SD card (`bin/sdcard.img`): `/sd/0/sample/c/hello you`, `echo hi | /sd/0/sample/c/upper`, `/sd/0/sample/c/keys`, `/sd/0/sample/c/tones`; `cd /ram; /sd/0/sample/c/ctest a 'b c'` runs the library's test.

The multitasking demos

Five samples show tasks working together. Each is one program that starts copies of itself as its workers (each in a task of its own, given what it needs as arguments: a shared segment's number, semaphores' numbers), draws what they do as they do it, and says how it went. They find themselves by `$prog`, the path rc started them from (`getenv ("prog")`: so a copy runs from anywhere, `/ram/round`), else as `/bin/NAME`, `./NAME` or `/sd/0/sample/c/NAME` (a file, not a directory of that name), so after `bind -a /sd/0/sample/c /bin` (at rc, or HyForth's `bind`) they run by name. Each needs four or five tasks free besides its own; Ctrl-C ends them all, and the program says how far it got.

Sample	What it shows
`race [TASKS [ADDS]]`	Why shared memory needs a lock: four tasks add 1 to a counter in a shared segment, 500 times each (read it, work a little, write it back). With nothing to keep them apart, a task stopped between its read and its write undoes the others' adds, and many are lost; with a mutex around each read and write, none are
`chorus`	The console as a shared resource: four tasks print a line each a letter at a time. With nothing between them the letters tangle; with a mutex held for a whole line each line is whole; with a baton (a semaphore each, passed round: wait for yours, print, give the next one's) the lines come in turn
`philo [-d] [-n MEALS] [N]`	The dining philosophers: five tasks, a fork (a mutex) between each two, a live table of who's thinking, hungry or eating and who holds which fork. Taking the lower-numbered fork first, they never deadlock; with `-d` each takes their left fork first, and they soon do: the program sees it and ends them
`prodcons [-n ITEMS]`	Producers and consumers: two tasks make items and two use them, through a ring of 8 slots in a shared segment, kept in step by two counting semaphores (free slots, filled ones) and a mutex. The ring fills while the producers are quicker and empties while the consumers are, each side waiting (using no CPU) when it must

Sample	What it shows
`round [TIMES [TICKS]]`	"Row, Row, Row Your Boat" as a round: four tasks a voice each, on YM2151 channels 0-3, two bars apart. They meet at a barrier (semaphores), then each keeps its own time by the tick (`hy_sleep_until`), and the words light up as they're sung. In the emulator, `sim/run.js -i --sound` plays it in a browser

Each runs as a test too (`tests/tests.js`: race, chorus, philo, prodcons, round): no adds lost with the mutex, the lines whole and in turn, no fork ever in two hands and the deadlock seen, every item used once, every note in time.

How C reaches the system

- **The start-up** (`lib/crt0.s`) is the program's HYX2 header and its `main`: it sets up cc65's C stack and the heap, turns the kernel's argument list into `argc` and `argv`, calls `main`, and ends with `EXITS` and `main`'s value.
- **The library** is cc65's `none.lib` with the Hydra's modules **in place of** cc65's: each module of `lib/` is named as the cc65 module it replaces (`open`, `read`, `write`, `fclose`, `getenv`, `conio` ...), so cc65's own code (stdio's formatting, `malloc`, the string functions) runs unchanged on top.
- **Any call**: `hy_call (HY_NAME, &regs)`, with `struct hy_regs` holding `.A`, `.X`, `.Y` and `r0`-`r3`:

```
#include <hydra.h>

struct hy_regs r;
r.a = 0;                                /* .A: a task */
if (hy_call (HY_TASKINFO, &r) != 0) ...     /* 0, or the error code (and errno, _oserror) */
```

- **Errors**: a failed call returns -1 (or NULL) and sets `errno` (the POSIX code `base/spec/errors.def` gives each of the kernel's codes) and `_oserror` (the kernel's own, `HY_E_NOENT` ...).
- **The zero page**: cc65's runtime has `$22` up (26 bytes), and the program's own goes after it, to `$7F`. A library routine that cc65's own assembly calls (`_cputc`, `_fgetc`) keeps the runtime's scratch registers it relies on.
- **Notes**: `signal (SIGINT, f)` is a note handler for Ctrl-C's note; the default ends the program.

Servers and drivers

A **server** is a task that serves a device letter: its clients' `OPEN`, `READ`, `WRITE`, `STAT` … of `#x/...` come to it as requests. A **driver** is a server that's a module of the paged ROM, started in a task of its own (the highest free: task F down), at boot if it's flagged so. Every one of the system's drivers is built this way, and so can yours be: a device of your own, a file system, a service other programs talk to through files.

Files, servers and namespaces explained the model; this page is how to write one.

Contents

1. srvlib
2. A server, walked through
3. The tree
4. The kinds of entry
5. Hooks
6. Requests and waiting
7. Interrupts
8. Several devices, and raw devices
9. Putting it in the ROM, and testing it

---

srvlib

Every server is built on **srvlib** (`base/sdk/asm/srvlib.inc` at the top of its source, `srvlib.s` at the end), so a new one is mostly tables: its files, and a routine for each that makes or takes its contents. srvlib does the rest: it takes each request, keeps the fids, walks the tree, makes stat records and directory listings, slices text files at a read's offset, and parses `ctl` commands. It answers what it doesn't know with `E_NOSYS`.

A server, walked through

`sdk/asm/samples/counter/counter.s` serves `#k`: `count`, a text file (the count), and `ctl` (`add N`, `reset`).

```
.include "hydra.inc"
.include "hyx2.inc"
.include "macros.inc"
.include "srvlib.inc"                                   ; (srvlib.s at the end)

            HYX2_DRIVER "counter", init, srv_serve, 0, 0, HF_BOOT
```

`HYX2_DRIVER`'s entries: `init` (run once as it starts), the serve entry (srvlib's `srv_serve`, which takes each request and does the rest), the irq entry and the stop entry (none here), and its flags (`HF_BOOT`: started at boot, which waits for its init). `init` registers the letter:

```
init:
            stz         count
            stz         count + 1
            lda         #'k'
            jmp         SRV_REGISTER                    ; (C = 1, .A = an error: it ends)
```

The tree is a table: each entry its name, its parent (an entry's number), its kind, its handler, its mode and an aux byte:

```
srv_tree:
            SRV_ENTRY   s_root,  $FF, SK_DIR,  0,         SM_READ,            0     ; 0
            SRV_ENTRY   s_count, 0,   SK_TEXT, gen_count, SM_READ,            0     ; 1
            SRV_ENTRY   s_ctl,   0,   SK_CTL,  ctl_cmds,  SM_READ | SM_WRITE, 1     ; 2 (reads as count)
            .word       0
ctl_cmds:                                                 ; Its commands: the word, the handler
            .word       s_add, c_add
            .word       s_reset, c_reset
            .word       0
```

A text file's handler makes its text each time it's read (srvlib slices it at the read's offset, so the server keeps nothing between reads); a ctl command's handler gets the words after it, parsed (`srv_argn` of them, `srv_argp` their places, `srv_arg` their numbers: decimal, or `$hex`):

```
gen_count:
            lda         count
            ldx         count + 1
            jsr         srv_tputdec
            lda         #LF
            jsr         srv_tputc
            clc
            rts

c_add:
            lda         z:srv_argn                        ; (No number: E_INVAL)
            beq         @inval
            clc
            lda         count
            adc         srv_arg
            sta         count
            ...
```

```
% cat '#k/count'
0
% echo add 5 >'#k/ctl'; cat '#k/count'
5
```

The tree

`srv_tree` is the server's files, an entry each (`SRV_ENTRY`, `SE_SIZE` bytes), entry 0 the root, ending with a 0 word:

Field	What it is
`SE_NAME`	The name's address
`SE_PARENT`	The parent's entry number (`$FF` for the root)
`SE_KIND`	`SK_DIR`, `SK_TEXT`, `SK_CTL`, `SK_DATA`, `SK_DYN`, `SK_RAW`
`SE_HANDLER`	The kind's routine or table
`SE_MODE`	Its permissions: `SM_READ`, `SM_WRITE`
`SE_AUX`	A byte of the kind's own

The kinds of entry

Kind	What it is
`SK_DIR`	A directory: the entries whose parent it is, read as stat records
`SK_TEXT`	A text file: its handler makes the text (`srv_tputs`, `srv_tputc`, `srv_tputdec`, `srv_tputhex`), 255 bytes at most

Kind	What it is
SK_CTL	A ctl file: a table of commands (.word name, handler, ending with 0); a write is split into words and the command's handler called; a bad command is E_INVAL and changes nothing. Its aux: the entry of a text file it reads as (its state)
SK_DATA	Anything else: the handler gets each request (.A = R_OPEN, R_READ, R_WRITE, R_CLUNK, R_DUP; .X the fid), moves the data itself (CLIENT_READ, CLIENT_WRITE), sets the count done (RQ_DONE), and answers C = 0, or C = 1 and an error
SK_DYN	A directory whose children a handler makes as it's read (the tasks in /proc, the modules in #m): the handler names its children (DYN_NAME), finds one by name (DYN_FIND), and names one by its id (DYN_IDNAME); its aux is the entry each child is, so a child can be a directory of its own
SK_RAW	A device whose every request goes to one handler, its fids its own (a file system: #f)

Control is text: a device is set by writing a command to its ctl, its words after it, and read back as text. So a device needs no special program: rc's echo and cat, and every language's file calls, drive it.

Hooks

A server may define these before including srvlib.s, for what the tables don't cover:

Hook	Called
SRV_OPENED	For each fid made (an open, a dup): it may keep a byte for it (srv_fid_aux), from the open's spec, or refuse
SRV_CLUNKED	For each fid forgotten (a close)
SRV_FLUSH	When a client's wait is ended by a note (R_FLUSH): forget what was kept for it
SRV_PRE, SRV_POST	Before every request, and after it
SRV_STAT	For each stat record made: fill in more (a data file's length)
SRV_TREES	Several devices (below)

What's a client's: a task's claim on something (sound channels, the VERA) is that task's, given back as its last file of the device closes: count the fids in SRV_OPENED and SRV_CLUNKED.

Requests and waiting

A request is a block the client fills in its own memory (RQ_*: the request, its fid, mode, offset, count, buffer, its task and note group); SRV_TAKE copies it into the server's TASK_INBOX, and SRV_REPLY sends back the count done and a new fid. srvlib does both. Data moves by kcopy: CLIENT_READ and CLIENT_WRITE between the server's memory and the client's buffer, as each task sees its own.

A server never waits. When it can't answer yet (a pipe that's empty, a console with no line), it answers E_AGAIN, and the kernel has the client wait until the server's **event count** changes from what it was as the server took the request, then sends the request again. So a server adds 1 to its event count (inc TASK_EVENT) whenever something its clients may be waiting for has happened: a byte in, room to send.

- A note ends a client's wait: the server gets R_FLUSH (SRV_FLUSH, if it keeps anything for a client), the client E_INTR.
- A client whose fd is O_NONBLOCK gets E_AGAIN instead of waiting.
- **A server that must give up on something that doesn't come times it itself**: the kernel has no timed wait. It makes its event count change now and then (from an interrupt it owns), and the client, asking again, looks at the time.
- A server serves one request at a time; a second client waits until it's free.

A driver with work of its own between its requests (a device to look at now and then, something to do whenever a key comes) needs no second task for it. Its init doesn't return: it loops, waiting in SRV_NAP between its turns, for a number of ticks (.A/.X) or, with 0 ticks, till its own event count isn't the one it last saw (.Y). While it waits there the driver serves its requests as any other; when it runs again it's busy, and a request waits, till its next SRV_NAP: so its loop and its requests never meet in its data. The boot counts it as started at its first SRV_NAP. In the loop it may make requests of other drivers (not of itself: call its own routines). If its init does return later, it's an ordinary driver from then on.

```
init:       ...                                     ; (Its letters, its lines, its hardware)
@turn:      lda         #<(TICK_HZ / 10)            ; Ten times a second
            ldx         #>(TICK_HZ / 10)
            jsr         SRV_NAP
            jsr         look                        ; (Its own work: the device read, what it has passed on)
            bra         @turn
```

Interrupts

A driver owns an interrupt line with `IRQ_OWN` (`.A` the line) in its `init`; its **irq entry** (the third of `HYX2_DRIVER`'s) runs in its own task with the line in `.A`, interrupts off, and returns `.A` = 0, or `IRQ_RESCHED` for a task switch. The rules:

- **About 85 cycles**: no stretch of code may hold interrupts off for more than 200, and the dispatch takes 115.
- **Never wait**, and never call what waits.
- **Wake clients by adding 1 to the event count** (`inc TASK_EVENT`), never with `WAKE`.
- **Send a note** to a note group with `NOTE_QUEUE` (the console's Ctrl-C does).
- **Quiet the device**: the lines are level-triggered and can't be masked, so the handler must clear what interrupted, or it comes straight back.
- **Own the line before the device can interrupt**: a line nobody owns is counted and ignored, and comes straight back.
- A module of more than one bank owns a line only with its irq entry (and what that calls) in its RAM.

VIA timer 2 (line 16) and CA1 (line 17) are lines of their own; owning one is owning that part of the VIA. The lines' owners are in the kernel.

Several devices, and raw devices

- `SRV_TREES` gives a server several letters, a tree each: `.byte` the letter, `.word` its tree, ending with 0. A request's `RQ_DEV` chooses the tree. `kdev` serves nine devices this way.
- **A spec**: what follows a device's letter in a `#` name (`#c2`, `#fx`) reaches the server in the open request, so one tree can serve several instances (the console's windows). `SRV_OPENED` reads it and keeps it in the fid's aux byte.
- `SK_RAW`: a tree of one entry, whose handler gets every request and keeps its own fids: for a server whose files aren't a fixed tree, such as a file system (HydraFS's `#f`).

Putting it in the ROM, and testing it

A driver is a folder in `modules/` (`modules/NAME/*.s`), listed in `modules/rom.txt`; `node build.js` builds it with the rest. `HF_BOOT` starts it at boot, in the next free task down from F; without it, a program starts it with `SPAWN`.

A **test** of it is a test module (`tests/mod`) that runs as init, with the driver in the ROM (`tests/tests.js`: a test's `modules`), and `node sim/test.js NAME -v` runs it (the tests). `/proc` refuses a driver's memory and registers, and `NOTE` its notes: a driver's task is the system's.

Libraries

A **library module** is code in the paged ROM that other programs call, in their own tasks, with their own RAM: it has no data, BSS or entries of its own. The system has three: `numbers` and `math` (the number system every language shares) and `asm` (the W65C02S's instructions: the assembler and the disassembler). This page is how a library is called, and each of the system's.

Contents

1. XCALL
2. Writing a library
3. The number libraries: the stored format, calling them, the entries, errors, bases and text
4. The asm library

XCALL

`XCALL` calls a routine in any bank of the paged ROM, as the Commander X16's `jsrfar` does: `r15` the routine's address, `r14` (its low byte) the bank it's in. The caller's bank register (`$01`) is set to the routine's for the call and back after it; `.A`, `.X`, `.Y`, the flags and `r0`-`r13` pass through both ways. The routine runs in the caller's task, with its RAM and zero page.

A program finds a library's bank in the module directory: `MODINFO` gives each entry's name and first bank (`ME_NAME`, `ME_BANK`), so a program looks for `"numbers"` as it starts and keeps the bank.

```
            lda         lib_bank                        ; (MODINFO's ME_BANK, found as it started)
            sta         r14
            LDR         r15, LIB_ADD                    ; The library's routine ($A030: its jump table)
            lda         #3
            ldx         #4
            jsr         XCALL                           ; .A = 7
```

A note may come while the library runs: a program that makes far calls keeps its **note handler in its RAM**, since the kernel calls it with whatever bank is at `$A000`.

Writing a library

- Its header is `HYX2_LIBRARY "name", "segment", "memory"`; it's a module of its own, listed in `rom.txt`.
- **A jump table at its start**, after its header (at `$A030`), so its routines' addresses don't change as it does. `tests/mod/t_lib/t_lib.s` is one, and `t_xcall` calls it.
- It has no RAM of its own: it works in what its caller gives it (addresses in registers, a RAM bank named in a register, the caller's zero page saved and put back).
- The system's libraries are made from a specification (`reborn/spec/numbers.def`, `asm.def`): `reborn/tools/apigen.js` makes the library's jump table and each language's binding (assembly's `numbers.inc` and `asmlib.inc`, C's `numdefs.h` and `asmdefs.h`) from it.
- A program may also have libraries of its own, assembled with it, each a bank with a header (`HYX2_LIBRARY`), called with `FARN` (modules of several banks).

The number libraries

The Hydra has one number system (the numbers), and every language works it through the same two library modules:

- `numbers`: the arithmetic, conversions, text, the bits of integers of any size, random numbers, Fibonacci.
- `math`: the functions (`SQRT`, `EXP`, `LOG`, `TRIG`, `ATAN`, `RPOW`, `PI`) and their precision (`DIGITS`).

The stored format

A number is a tag byte, and what the tag says follows (integers' bytes least significant first):

Tag	A number	Then
`$00-$7F`	An integer -64 to 63: the tag itself, 7-bit two's complement	Nothing
`$80-$8F`, `$90-$9F`	A positive or negative integer of n bytes (the tag's low 4 bits: n - 1)	Its magnitude
`$A0`, `$A1`	A positive or negative integer of 17 to 255 bytes	Its length, then its magnitude
`$B0-$BF`	A fixed decimal of 0 to 15 places (the tag's low 4 bits)	Its digits: an integer
`$C0`	A fixed decimal of 16 to 65,535 places	Its places (2 bytes), then its digits
`$C1`	A rational, in lowest terms, its denominator above 1	Its numerator, then its denominator
`$C2`	A complex number, its imaginary part not 0	Its real part, then its imaginary part
`$FF`	Never a number: a program's own (HyBASIC's values use it)	

Each number has **one form** (an integer in its shortest, a fixed decimal without 0s at its end), so two numbers of one kind are equal exactly when their bytes are. Most numbers are 1 to 5 bytes: `100` is `80 64`, `0.5` is `B1 05`, `1/3` is `C1 01 03`, `1+2i` is `C2 01 02`. `NUM_MAX` (1,040) is the longest there can be. This is what hylang's heap, HyForth's number stack, HyBASIC's values and a C program's `num_t` arrays hold.

Calling them

A library has no RAM of its own, so **the libraries work in a RAM bank the program gives them**: one bank (8K) of its own (`BANKS_ALLOC`), the same for its life, named in `r13` at every call. `INIT` fills it once (the base decimal, the precision 12 digits, the random generator seeded from the clock). A call selects that bank at `$8000` and gives the caller's back as it ends; it uses the zero page's `$70-$7F` too, saved and put back.

Registers	
`r15`, `r14`, `r13`	The entry's address (`NUM_ADD` ...), the library's bank, the libraries' RAM bank
`r0`, `r1`	The operands: numbers' addresses, anywhere in the caller's memory but its paged ROM
`r2`, `r3`	The result's place and the room it has (a result may be written over an operand)
`r4`	A base string for the text entries (0: the base the library holds)
`.Y`	A few entries' options (`PARSE`'s flags, `BITS`'s operation, `TRIG`'s function)
Back	C = 0: `.A`/`.X` the result's length; C = 1: `.A` an error (`NE_`)

In assembly, `numbers.inc` has each entry's address and the macros `NUMCALL entry` and `MATHCALL entry`, which load `r13-r15`; `numlib.s`**'s** `num_open` finds the libraries, takes the bank and calls `INIT` as the program starts. The sample `nsum` sums the numbers it's given:

```
            jsr         num_open                        ; The libraries (C = 1: .A the error)
            ...
            stz         r4                              ; Read in the base (r4 = 0), the whole word a number
            stz         r4 + 1
            LDR         r2, num
            LDR         r3, NUM_MAX
            ldy         #NPARSE_WHOLE
            NUMCALL     NUM_PARSE                       ; (r0, r1: the text and its length)
            ...
            LDR         r0, sum                         ; sum = sum + num
            LDR         r1, num
            LDR         r2, sum
            NUMCALL     NUM_ADD
```

A call costs a few hundred cycles before its work (`XCALL` some 120, the zero page's saving some 200), so a language keeps its quickest numbers itself where it can, and hands them to the library past that. In C, `num.h` wraps every entry (C).

The entries

	`numbers`**'s**
Its bank	`INIT`; `SET_BASE`, `GET_BASE` (the base, a string)
Text	`PARSE` (the longest start of a text that's a number, or with `NPARSE_WHOLE` the whole text), `DISPLAY` (a number written exactly), `FORMAT` (a format string's {} placeholders filled)
Arithmetic	`ADD`, `SUB`, `MUL`, `DIV` (exact: `10/4` is `5/2`), `NEG`, `ABS` (a real number's: `math`'s takes complex ones too), `CMP` (by value, whatever the kinds), `KIND` (an integer, a fixed decimal, a rational, a complex number; its sign), `IDIV` (an integer quotient toward 0, and the remainder), `GCD`, `POW` (a whole power)

	numbers's
Conversions	TRUNCATE, FLOOR, ROUND (a half to the even one), TO_FIXED, TO_RATIONAL, NUMERATOR, DENOMINATOR, COMPLEX, PART, FROM_INT and TO_INT (16 or 32 bits), BYTES (bytes checked as a number)
Bits	BITS: and, or, xor, not, shifts, a bit's test, on integers of any size in two's complement
Others	RANDOM (an integer below n, or a fixed decimal from 0 to 1) and SEED; FIB

	math's
The functions	SQRT, EXP, LOG, TRIG (sine, cosine, tangent), ATAN, RPOW (any power), PI, ABS (a complex number's modulus too)
The precision	DIGITS: 1 to 100 significant digits, 12 at the start

The functions are exact when their answer is (SQRT of 9/4 is 3/2), else a fixed decimal of the precision's significant digits, **correctly rounded**, so the Hydra, danlang and the PC's reference agree to the last digit. A negative number's square root or logarithm is complex. A complex argument (or a negative number to a power that isn't whole) gives the principal value, as mathematics defines it, each part correctly rounded: sqrt(3+4i) is 2+1i, ABS of 3-4i is 5, i^i is real. These are worked in the library's third bank (mtcplx.inc), from the real functions' cores. At 12 digits a sine takes some 220,000 cycles.

Errors

Error	
NE_BIG	Too big: an integer past 255 bytes, a fixed decimal past 65,535 places
NE_DIV0	Division by zero
NE_NOTNUM	Not a number: text that isn't one, bytes that aren't one in the stored format
NE_ROOM	No room for the result in the place given
NE_DOMAIN	Outside the function's domain (LOG of 0)
NE_BASE	Not a base
NE_INT, NE_REAL	An integer is needed; a real number, not a complex one
NE_INIT	The bank in r13 isn't one INIT filled
NE_FORMAT	A format's placeholders and its arguments don't match

Bases and text

A base is a string, hylang's: d decimal, x hexadecimal written bare (FF), #x with its prefix (#xFF), b, o, c (balanced ternary), 16r (a radix), <x (least digit first), [01] (digits of its own), and the rest of hylang's named bases.

- **Reading** (PARSE): integers, fractions (1/3), a radix point (0.8 in hexadecimal is a half), complex numbers (1+2i), and a number with a base of its own (#xFF, #b0.1). NPARSE_PROGRAM is a program's rule: a bare number starts with a digit (0FF), so FF is a name.
- **Writing** (DISPLAY): exactly, in the base: a fraction or a fixed decimal with a radix point when it ends in that base, else as a fraction (0.1 in hexadecimal is 1/A), so what's written reads back as the same number.
- **Formats** (FORMAT): {} a value in the base, {x}, {#b} ... in one named: hylang's format, HyForth's nformat, HyBASIC's PRINT USING "{}", C's printf with %N and %{x}N.

The asm library

asm (reborn/modules/asm, reborn/spec/asm.def) is the W65C02S's instructions as the assembler as writes them, one table for every tool that reads or writes them: db's d, HyForth's disasm, dis, as, and HyBASIC's ASM blocks. It's one bank: the assembler and the disassembler.

Entry	What it does
DIS	One instruction disassembled, as as writes it (with symbols, if given)
FILE	A source file assembled whole, as as does it
BEGIN, PASS, DEFINE, LINE, END	A source given a line at a time, for each pass, with the caller's own symbols: HyBASIC's way
SYMBOL, IMAGE, DONE	The assembled symbols and image, and the end
ERROR	The last error's text

It has no RAM of its own: the assembler's state is in RAM its caller lends it while it runs (ASM_RAM, $5000-$7DFF), and its zero page ($22 on) is the caller's, put back as each call ends. In assembly, asmlib.inc has the entries and the macro ASMCALL; in C, asm.h has dis_insn and the symbol files' lbl_load.

An operating system of your own

The base gives an operating system on the Hydra-16 its hardest parts: the kernel, its calls, the interrupt path, a console driver and the kernel's devices. So a new system is mostly modules: what runs at boot, and what the user sees. HydraOS is one such system (`reborn/`: its drivers, shells, languages and tools are all modules on this kernel); this page is how to make another.

Contents

1. What the kernel gives
2. What boots
3. A module
4. Building it
5. What else the base has

What the kernel gives

The kernel is the BIOS ROM (`base/bin/bios_rockwell.bin`, or `bios_wdc.bin` for a WDC serial chip), the same for every system, with its 89 calls (the system calls): tasks and the scheduler, memory and banks, shared segments, semaphores, notes, the clock, files and the server protocol, namespaces, the loader. Your system uses them as HydraOS does; it never needs to change the kernel.

What boots

The kernel boots, runs POST, then starts what the paged ROM's **module directory** says (the boot):

1. **The boot drivers**: each module of type driver with the flag `HF_BOOT`, in the directory's order, the first in task F, the next in E, and down. Each runs its `init` (it registers its device letters, `SRV_REGISTER`, and owns its interrupt lines, `IRQ_OWN`); the boot waits for them all.
2. **Init**: the directory's init entry, a program, in task 1, with an empty namespace of its own and no fds: it builds them. **Init is the system.**

The base's `rom.txt` is `ser` (task F), `kdev` (task E) and `wozmon` (init). Yours keeps `ser` and `kdev` (or your own console driver in their place), and names your init:

```
init        myinit
module      ser
module      kdev
module      myinit
module      ...         ; your programs and drivers
```

A module

A module is a program, a driver or a library in the paged ROM, run in place in its banks (one to eight; a small one shares its bank with others, at a page of it), with a 48-byte header first (`base/sdk/asm/hyx2.inc`):

```
.include "hydra.inc"
.include "hyx2.inc"
.include "macros.inc"

            HYX2_PROGRAM "myinit", main           ; a program: main gets r0 = its arguments; rts ends it

.code
main:
            LDR         r0, s_cons                ; fds 0-2: the console (ser's #c)
            lda         #O_RDWR
            jsr         OPEN
            ...
```

- **A program** (`HYX2_PROGRAM name, main`) runs in a task of its own, its RAM from `$0400` (its data and BSS) up to its break, its zero page `$22-$7F`. `SPAWN "#m/NAME"` starts one (kdev's `#m` serves the directory's modules as files; bound at `/bin`, `SPAWN "/bin/NAME"`).
- **A driver** (`HYX2_DRIVER name, init, serve, irq, stop, flags`) serves a device: its files, by the kernel's server protocol, through srvlib (servers and drivers). Its `irq` entry gets the lines it owns.
- **A library** (`HYX2_LIBRARY`) is code another task calls in its own task with `XCALL` (libraries).

Init's first jobs are the console (open `#c/cons` three times, for fds 0, 1 and 2, and write `group` to `#c/consctl` so Ctrl-C reaches it), and the namespace: bind what your programs will look for (`/dev`, `/bin`, `/env`, `/proc`). The SDK's `nslib.s` builds a namespace from a file of `bind` and `mount` lines, as HydraOS's init does.

Building it

The quickest way is the base's own build: put a module in a folder, `base/modules/NAME/NAME.s`, and `node build.js` in `base/` assembles it with the base's includes (`sdk/asm`, `include`, `obj/gen`, `lib`) and links it with `modules/module.cfg` (`moduleN.cfg` for N banks); then name it in `rom.txt`.

For a system of your own beside the base, do what HydraOS's `reborn/build.js` does:

- `require('../base/build.js')`: its `build()` for the kernel and the base's modules, its `buildModule()` for yours;
- its `tools/romimg.js` for your paged ROM (the module directory, the hardware test in bank 1, your modules);
- `sim/run.js`'s `main(argv, { root })` and `sim/test.js`'s `setup({ root, tests, image })` run your tree's images and tests, as HydraOS's `sim/run.js` and `sim/test.js` do.

What else the base has

- **The kernel's devices** (`kdev`): the modules, `null` and the kernel's messages, the tasks, pipes, the environment, shared segments' names, the clock, raw RAM (the kernel's devices).
- **The console** (`ser`): `#c/cons` (a line at a time, cooked; or raw with `#c/consctl`'s `rawon`), `#c/ser` and `#c/serctl` (the serial port raw, its rate). HydraOS's console (`cons`) serves the same files and more, on the same serial layer (`base/lib/serial.inc`): write your own on it, or keep `ser` (the serial console).
- **The monitor** (`wozmon`): to poke at the machine while you bring your system up (the monitor).
- **The emulator**: the board, cycle by cycle, with a monitor of its own, the system calls traced by name, breaks on a module's labels (the emulator).

The system call reference

Every system call the kernel has, with its registers, its errors, and its forms in HyForth and hylang. The system calls explains how calling works; this page is the reference to keep open while you write.

The rules for every call:

- Call with `jsr NAME` (assembly: `hydra.inc`), `hy_call (HY_NAME, &regs)` or a wrapper (C: `hydra.h`), `sys-name` (HyForth and hylang), or `SYS "NAME"` (HyBASIC).
- Arguments and results in `.A`, `.X`, `.Y` and the call registers `r0`-`r15` (`$02`-`$21`); a 16-bit value is `.A` (low) and `.X` (high), or a call register. `r0/r1` is a 32-bit value, `r0` the low half.
- **C = 0 is success; C = 1 is failure, with the error code in `.A`**, for every call.
- A call may change `.A`, `.X`, `.Y`, `r0`-`r15` and the flags. It never touches `$22`-`$7F`.
- A call that **waits** can be ended by a note: it returns `E_INTR`, and the note is taken on the way out.

In the HyForth column, the stack effect lists the registers in order: `( r0 r1 .A -- .A ior )` takes `r0`, `r1` and `.A` and gives `.A` and an ior (0, or -512 less the error code). **In the hylang column**, each argument is `REGISTER=KIND` (`s` a string, `n` a number, `buf(N)` a buffer the call fills, `?` may be left out) and `->` what the function gives.

<!-- calls:begin -->

Made from `base/spec/api.def` and `base/spec/errors.def` by `node docs/build/callref.js`: 96 calls in 9 groups.

The groups: system, task, memory, file, name, cons, time, server, dbg; the error codes; the constants.

---

system (`$F800`)

System information and services: 16 slots from `$F800`, 5 used.

Call	Address	What it does
`SYSINFO`	`$F800`	What the system has.
`ERRSTR`	`$F803`	An error code as text, for messages.
`KMESG`	`$F806`	The kernel's messages.
`REBOOT`	`$F809`	Start the system again, as the reset button does (the VIA's and ACIA's interrupts quieted first: only the button resets the chips); with REBOOT_HWTEST, into the hardware test, as a T typed during POST does.
`XCALL`	`$F80C`	Call a routine in another bank of the paged ROM (a library module's: HT_LIBRARY, its bank from MODINFO), as the X16's jsrfar.

SYSINFO

`$F800`.

In	-
Out	.A = the ABI version (1); .X = how many RAM modules are installed; r0 = the free tasks (bit = task)
Errors	-
Waits	no
HyForth	`sys-sysinfo ( -- .A .X r0 )`
hylang	`sys-sysinfo -> .A .X r0`

What the system has.

ERRSTR

`$F803`.

In	.A = an error code; r0 = a buffer (32 bytes)
Out	the error's text in the buffer, zero-terminated ("error $xx" for an unknown code)
Errors	-
Waits	no

HyForth	`sys-errstr ( .A r0 -- )`
hylang	`sys-errstr .A=n r0=buf(32) -> buf=z`

An error code as text, for messages.

KMESG

`$F806`, a far call (its routine on another BIOS page).

In	r0 = a buffer; r1 = its size, the most it copies; r2 = an offset in the messages held, 0 the oldest
Out	.A/.X = the bytes held, KMESG_SIZE at most; the buffer has those from r2, r1 at most
Errors	-
Waits	no
HyForth	`sys-kmesg ( r0 r1 r2 -- .A/.X )`
hylang	`sys-kmesg r0=buf r1=count r2=n? -> buf .A/.X`

The kernel's messages: what it printed on the bring-up console (the banner, POST, the boot's tasks, a driver's own lines), its last KMESG_SIZE bytes. /dev/kmesg (kdev's #n/kmesg) is them as a file.

REBOOT

`$F809`.

In	.A = 0, or REBOOT_HWTEST
Out	(doesn't return)
Errors	-
Waits	no
HyForth	`sys-reboot ( .A -- )`
hylang	`sys-reboot .A=n? ->`

Start the system again, as the reset button does (the VIA's and ACIA's interrupts quieted first: only the button resets the chips); with REBOOT_HWTEST, into the hardware test, as a T typed during POST does. Every task ends where it is: files written are on their disks already (the disks write through).

XCALL

`$F80C`.

In	r15 = a routine's address ($A000-$DFFF); r14 (its low byte) = the paged ROM bank it's in; .A, .X, .Y, the flags and r0-r13: the routine's
Out	.A, .X, .Y, the flags and r0-r15 as the routine leaves them
Errors	-
Waits	no

Call a routine in another bank of the paged ROM (a library module's: HT_LIBRARY, its bank from MODINFO), as the X16's jsrfar: the caller's bank register ($01) is set to the routine's for the call and back after it, and everything else passes through both ways. The routine runs in the caller's task, with its RAM and zero page. A note may come while it runs: a program that makes far calls keeps its note handler in RAM, as a module of several banks does.

---

task (`$F830`)

Tasks: starting and ending them, waiting, scheduling, ticks: 32 slots from `$F830`, 28 used.

Call	Address	What it does
`EXITS`	`$F830`	End the calling task, with an exit status (Plan 9's exits).
`WAIT`	`$F833`	Wait for a child to end, and take its exit status.
`SPAWN`	`$F836`	Start a program in a task of its own.
`GETPID`	`$F839`	The calling task's number (0-15).
`YIELD`	`$F83C`	Let the other tasks run; come back when it's this one's turn again (at once, if nothing else can run).

Call	Address	What it does
SLEEP	$F83F	Sleep for a number of ticks.
SLEEP_UNTIL	$F842	Sleep until the tick count reaches a value.
TICKS	$F845	The scheduler's tick count.
PREEMPT_OFF	$F848	Hold the CPU.
PREEMPT_ON	$F84B	Undo PREEMPT_OFF; a task switch that came due meanwhile happens now.
TASKINFO	$F84E	What the kernel knows of a task, for ps (and /proc's source).
NOTIFY	$F851	Set the task's note handler (Plan 9's notify).
NOTE	$F854	Send a note (Plan 9's postnote).
MODINFO	$F857	A module in the paged ROM (SPAWN's #m/NAME), as the module directory has it.
ENV_GET	$F85A	A variable of a task's environment (each task has its own: ENV_MAX bytes; SPAWN copies its parent's).
ENV_PUT	$F85D	Set a variable, or make it (offset 0).
ENV_DEL	$F860	Remove a variable.
ENV_NAME	$F863	The variables of an environment, one at a time, in the order they were made.
TASKREAD	$F866	Another task's arguments, current directory, environment, registers or open files, as they are now.
GETPPID	$F869	The task that started this one (Plan 9's getppid).
SEM_NEW	$F86C	Make a semaphore, for tasks that share something (a shared segment, a device) or wait for each other.
SEM_ACQUIRE	$F86F	Take one of a semaphore's count; if there's none, wait (using no CPU) till SEM_RELEASE gives one back.
SEM_TRY	$F872	SEM_ACQUIRE without the wait.
SEM_RELEASE	$F875	Give one back to a semaphore's count, and wake the tasks waiting for it.
SEM_FREE	$F878	Free a semaphore (any task may).
EXEC	$F87B	Run a program in this task, in place of the one that calls it (Plan 9's exec).
SEM_WAIT	$F87E	SEM_ACQUIRE with a time limit.
RUN	$F881	Run a bank program in this task, as a subroutine, and wait for it.

EXITS

$F830.

In	.A = the exit code (0: success); r0 = a message (zero-terminated, 31 characters at most), or 0 for none
Out	(doesn't return)
Errors	-
Waits	no
HyForth	`sys-exits ( .A r0 -- )`
hylang	`sys-exits .A=n? r0=s? ->`

End the calling task, with an exit status (Plan 9's exits). Returning from a program's entry point is EXITS with code 0 and no message. Everything the task had is released; its parent can WAIT for the status.

WAIT

$F833.

In	.A = a task this one started, or $FF for any; r0 = a buffer for the message (32 bytes), or 0
Out	.A = the task that ended; .X = its exit code; its message in r0's buffer ("" for none)
Errors	E_CHILD (no such child, or none at all), E_INTR (a note came: taken on the way out)
Waits	yes
HyForth	`sys-wait ( .A r0 -- .A .X ior )`
hylang	`sys-wait .A=n r0=buf(32) -> .A .X buf=z`

Wait for a child to end, and take its exit status. A child that has ended already isn't waited for.

SPAWN

$F836.

In	r0 = the program's path (through the namespace: /bin/NAME; or #m/NAME, a module in the paged ROM); r1 = its arguments (zero-terminated strings, an empty one after the last: 176 bytes at most), or 0; .A = flags (SPAWN_NEWGROUP, SPAWN_NEWNS, SPAWN_FDMAP, SPAWN_NOENV, SPAWN_STOPPED, SPAWN_RUN, RUN_LEND); r2 = an fd map (with SPAWN_FDMAP)
Out	.A = the new task (SPAWN_RUN: this one, if the program ran in it)
Errors	E_NOENT (no such file), E_NOEXEC (not a program this kernel runs), E_NOTASK (no free task), E_TOOBIG (the arguments), E_INVAL (the map), E_MFILE, E_INTR (and what else OPEN and READ give)
Waits	yes
HyForth	`sys-spawn ( r0 r1 .A r2 -- .A ior )`
hylang	`sys-spawn r0=s r1=args? .A=n? r2=map? -> .A`

Start a program in a task of its own. SPAWN opens the file and reads its HYX2 header (HX_*): a module of the paged ROM (HF_INPLACE: #m's files) runs in place, in its bank (the module directory's of its name); any other program is read into the new task's RAM at its load address ($0800), by the task itself as it starts, and its BSS cleared (one that can't be read ends at once, with the error as its exit code). It starts at its entry point with r0 pointing to its arguments (at TASK_ARGS: the list), its name the header's. Its fds: the caller's 0, 1 and 2; or with SPAWN_FDMAP, the map's: a count (SPAWN_FDS at most), then for each of the child's fds from 0 the caller's fd that becomes it ($FF, or a closed one: closed). Each shares the caller's channel and offset; the rest are closed. Its parent is the caller, which can WAIT for it. It's in the caller's note group unless SPAWN_NEWGROUP starts one (its number: the child's), and it shares the caller's namespace unless SPAWN_NEWNS gives it an empty one. Its environment is a copy of the caller's (SPAWN_NOENV: empty). SPAWN_STOPPED: it's stopped at its entry point, for the debugger (TASKSTEP; /proc/N/ctl's step, or start). A bank program (HF_BANK) is loaded into a RAM bank of its task's own and runs there. SPAWN_RUN, for a caller that waits for the program at once: a bank program that may run in place (not HF_TASK) runs in this task (RUN, with the map and RUN_LEND as given) before SPAWN returns, and the task given back is this one, whose WAIT gives the program's exit status; any other program, or with SPAWN_NEWGROUP, SPAWN_NEWNS, SPAWN_NOENV or SPAWN_STOPPED, starts in a task of its own as ever. rc runs a simple command this way.

GETPID

`$F839`.

In	-
Out	.A = this task's number
Errors	-
Waits	no
HyForth	`sys-getpid ( -- .A )`
hylang	`sys-getpid -> .A`

The calling task's number (0-15).

YIELD

`$F83C`.

In	-
Out	-
Errors	-
Waits	no
HyForth	`sys-yield ( -- )`
hylang	`sys-yield ->`

Let the other tasks run; come back when it's this one's turn again (at once, if nothing else can run).

SLEEP

`$F83F`.

In	.A/.X = ticks (TICK_HZ a second; 0-32767)
Out	-
Errors	E_INTR (a note woke it early: taken on the way out)
Waits	yes

HyForth	`sys-sleep ( .A/.X -- ior )`
hylang	`sys-sleep .A/.X=n ->`

Sleep for a number of ticks. The other tasks run meanwhile, or the system idles.

SLEEP_UNTIL

`$F842`.

In	.A/.X = a tick count (at most 32767 ticks ahead)
Out	-
Errors	E_INTR (a note woke it early: taken on the way out)
Waits	yes
HyForth	`sys-sleep-until ( .A/.X -- ior )`
hylang	`sys-sleep-until .A/.X=n ->`

Sleep until the tick count reaches a value: for steady timing (a time that's passed returns at once).

TICKS

`$F845`.

In	-
Out	.A/.X = the tick count (TICK_HZ a second; it wraps after about 5.5 minutes)
Errors	-
Waits	no
HyForth	`sys-ticks ( -- .A/.X )`
hylang	`sys-ticks -> .A/.X`

The scheduler's tick count.

PREEMPT_OFF

`$F848`.

In	-
Out	-
Errors	-
Waits	no
HyForth	`sys-preempt-off ( -- )`
hylang	`sys-preempt-off ->`

Hold the CPU: no task switch until the matching PREEMPT_ON (they nest). Interrupts keep running.

PREEMPT_ON

`$F84B`.

In	-
Out	-
Errors	-
Waits	no
HyForth	`sys-preempt-on ( -- )`
hylang	`sys-preempt-on ->`

Undo PREEMPT_OFF; a task switch that came due meanwhile happens now.

TASKINFO

`$F84E`, a far call (its routine on another BIOS page).

In	.A = a task (0-15); r0 = a buffer (TI_SIZE bytes)
Out	the buffer: TI_STATE, TI_FLAGS, TI_PARENT, TI_CPU (3 bytes: ticks), TI_BANK, TI_TYPE, TI_NAME (16)
Errors	E_SRCH (not a task)
Waits	no
HyForth	`sys-taskinfo ( .A r0 -- ior )`
hylang	`sys-taskinfo .A=n r0=buf(TI_SIZE) -> buf`

What the kernel knows of a task, for ps (and /proc's source). A task's CPU time is the ticks that interrupted it. TI_STATE is 0 for a free task.

NOTIFY

`$F851`, a far call (its routine on another BIOS page).

In	r0 = the note handler, or 0 for none
Out	-
Errors	-
Waits	no

Set the task's note handler (Plan 9's notify). A note runs it in the task, in its own code (never in the middle of a system call), with .A = the note: it returns C = 0 to go on where the task was, or C = 1 for the default; it may call EXITS. A note that comes meanwhile waits for it to return.

NOTE

`$F854`.

In	.A = a task, or NOTE_GROUP | a note group; .X = the note (1-31: NOTE_*)
Out	-
Errors	E_INVAL (no such note), E_SRCH (no task, or nobody in the group), E_PERM (the kernel task, a driver)
Waits	no
HyForth	`sys-note ( .A .X -- ior )`
hylang	`sys-note .A=n .X=n ->`

Send a note (Plan 9's postnote). It wakes the task if it waits: WAIT, PAUSE, SLEEP and GETC end with E_INTR. With no handler, and for NOTE_KILL always, the task ends with 128 + the note's Unix number (130 for NOTE_INTERRUPT, 137 for NOTE_KILL) and its name as the message.

MODINFO

`$F857`, a far call (its routine on another BIOS page).

In	.A = an entry of the module directory (0 on); r0 = a buffer (ME_SIZE bytes)
Out	the buffer: ME_BANK, ME_BANKS, ME_TYPE (HT_*), ME_FLAGS (HF_*), ME_NAME (11, zero-padded), ME_PAGE
Errors	E_NOENT (past the last)
Waits	no
HyForth	`sys-modinfo ( .A r0 -- ior )`
hylang	`sys-modinfo .A=n r0=buf(ME_SIZE) -> buf`

A module in the paged ROM (SPAWN's #m/NAME), as the module directory has it. A module of a bank may share it with others: its header is at $A000 + 256 * ME_PAGE there (0 for a module of banks).

ENV_GET

`$F85A`, a far call (its routine on another BIOS page).

In	.A = a task ($FF: this one); r0 = a variable's name; r1 = a buffer; r2 = its size; r3 = an offset in the value
Out	.A/.X = the value's length from the offset (0 at its end or past it); as much of it as fits, in the buffer
Errors	E_NOENT (no such variable), E_SRCH (no such task), E_INVAL (an empty name, or one with a /), E_NAMETOOLONG
Waits	no

HyForth	`sys-env-get ( .A r0 r1 r2 r3 -- .A/.X ior )`
hylang	`sys-env-get .A=n r0=s r1=buf r2=size r3=n? -> .A/.X buf=.A/.X`

A variable of a task's environment (each task has its own: ENV_MAX bytes; SPAWN copies its parent's). A value is any bytes (rc's lists: each word and a 0). #e serves the caller's as files, /env/NAME.

ENV_PUT

`$F85D`, a far call (its routine on another BIOS page).

In	.A = a task ($FF: this one); r0 = a variable's name; r1 = bytes; r2 = their count; r3 = where in its value they go (the value cut there first: 0 for a new value; its length at most)
Out	-
Errors	E_NOMEM (no room in the environment: ENV_MAX bytes; or no RAM module for one) E_INVAL (past the value's end; a bad name), E_SRCH E_NOENT (an offset given, and no such variable), E_NAMETOOLONG
Waits	no
HyForth	`sys-env-put ( .A r0 r1 r2 r3 -- ior )`
hylang	`sys-env-put .A=n r0=s r1=bytes(r2) r3=n? ->`

Set a variable, or make it (offset 0). An offset is for writing a long value in parts.

ENV_DEL

`$F860`, a far call (its routine on another BIOS page).

In	.A = a task ($FF: this one); r0 = a variable's name
Out	-
Errors	E_NOENT, E_SRCH, E_INVAL, E_NAMETOOLONG
Waits	no
HyForth	`sys-env-del ( .A r0 -- ior )`
hylang	`sys-env-del .A=n r0=s ->`

Remove a variable.

ENV_NAME

`$F863`, a far call (its routine on another BIOS page).

In	.A = a task ($FF: this one); .X = which variable (0 on); r0 = a buffer (ENV_NAME_MAX + 1 bytes)
Out	its name in the buffer, zero-terminated; .A/.X = its value's length
Errors	E_NOENT (past the last), E_SRCH
Waits	no
HyForth	`sys-env-name ( .A .X r0 -- .A/.X ior )`
hylang	`sys-env-name .A=n .X=n r0=buf(ENV_NAME_MAX+1) -> buf=z .A/.X`

The variables of an environment, one at a time, in the order they were made.

TASKREAD

`$F866`, a far call (its routine on another BIOS page).

In	.A = a task ($FF: this one); .X = what: TR_ARGS (its arguments: ARGS_MAX bytes), TR_CWD (its current directory: PATH_MAX + 1 bytes), TR_ENV (its environment's first variables, whole, and a 0: ENV_SIZE bytes at most), TR_FRAME (its state and the frame it left as it was switched out: TF_SIZE bytes, TF_*) or TR_ENVAT (part of its environment), or TR_FD (its fd r2: FI_SIZE bytes, FI_*); r0 = a buffer that big; r1 = TR_ENVAT's count (the most it copies); r2 = TR_ENVAT's offset, or TR_FD's fd
Out	the bytes in the buffer; .A/.X = TR_ENVAT's: the environment's bytes in use (its 0 too: it copied as many as there are from r2, r1 at most)
Errors	E_SRCH, E_INVAL (another what), E_BADF (TR_FD: not an open fd)
Waits	no
HyForth	`sys-taskread ( .A .X r0 r1 r2 -- .A/.X ior )`
hylang	`sys-taskread .A=n .X=n r0=buf r1=count r2=n? -> buf .A/.X`

Another task's arguments, current directory, environment, registers or open files, as they are now: /proc's args, cwd, env, regs and fd, ps. The kernel task copies them a byte at a time, with IRQs off for each. An environment may be ENV_MAX bytes: TR_ENVAT reads one whole, a part at a time.

GETPPID

`$F869`.

In	-
Out	.A = this task's parent ($FF: none)
Errors	-
Waits	no
HyForth	`sys-getppid ( -- .A )`
hylang	`sys-getppid -> .A`

The task that started this one (Plan 9's getppid): init once that one has ended; none for init, the drivers and the kernel task.

SEM_NEW

`$F86C`, a far call (its routine on another BIOS page).

In	.A = its count: how many can take it before a taker waits (0-255); .X = 0, or SEM_MUTEX: a mutex (its count 1, and it has a holder: only the task that took it gives it back)
Out	.A = the semaphore (0 to SEM_MAX - 1)
Errors	E_NOMEM (none free: SEM_MAX in all)
Waits	no
HyForth	`sys-sem-new ( .A .X -- .A ior )`
hylang	`sys-sem-new .A=n .X=n? -> .A`

Make a semaphore, for tasks that share something (a shared segment, a device) or wait for each other. Every task sees the same ones, by number. It lasts till SEM_FREE, or the end of the task that made it.

SEM_ACQUIRE

`$F86F`, a far call (its routine on another BIOS page).

In	.A = a semaphore
Out	-
Errors	E_INVAL (not a semaphore, or freed while this task waited), E_BUSY (a mutex this task holds already), E_INTR (a note came: taken on the way out)
Waits	yes
HyForth	`sys-sem-acquire ( .A -- ior )`
hylang	`sys-sem-acquire .A=n ->`

Take one of a semaphore's count; if there's none, wait (using no CPU) till SEM_RELEASE gives one back. A mutex's holder is then this task.

SEM_TRY

`$F872`, a far call (its routine on another BIOS page).

In	.A = a semaphore
Out	-
Errors	E_INVAL (not a semaphore), E_BUSY (a mutex this task holds already), E_AGAIN (none to take)
Waits	no
HyForth	`sys-sem-try ( .A -- ior )`
hylang	`sys-sem-try .A=n ->`

SEM_ACQUIRE without the wait.

SEM_RELEASE

`$F875`, a far call (its routine on another BIOS page).

In	.A = a semaphore
Out	-
Errors	E_INVAL (not a semaphore), E_PERM (a mutex this task doesn't hold), E_RANGE (its count is 255)
Waits	no
HyForth	`sys-sem-release ( .A -- ior )`
hylang	`sys-sem-release .A=n ->`

Give one back to a semaphore's count, and wake the tasks waiting for it: the first to run takes it.

SEM_FREE

`$F878`, a far call (its routine on another BIOS page).

In	.A = a semaphore
Out	-
Errors	E_INVAL (not a semaphore)
Waits	no
HyForth	`sys-sem-free ( .A -- ior )`
hylang	`sys-sem-free .A=n ->`

Free a semaphore (any task may): its number can be made again, and the tasks waiting for it wake with E_INVAL. The end of the task that made it frees it too, and a task's end gives back the mutexes it holds.

EXEC

`$F87B`.

In	r0 = the program's path (as SPAWN's); r1 = its arguments (as SPAWN's: 176 bytes at most), or 0
Out	(doesn't return, once the program is found: it runs in this task's place)
Errors	E_NOENT (no such file), E_NOEXEC (not a program this kernel runs), E_TOOBIG (the arguments), E_PERM (a driver), E_INTR (and what else OPEN and READ give)
Waits	yes
HyForth	`sys-exec ( r0 r1 -- ior )`
hylang	`sys-exec r0=s r1=args? ->`

Run a program in this task, in place of the one that calls it (Plan 9's exec): the program is found as SPAWN finds it (a module of the paged ROM runs in place; any other is read into this task's RAM at its load address), and starts at its entry point with r0 pointing to its arguments, its name the header's. The task keeps what isn't the old program's: its number, its parent (whose WAIT gets the new program's exit status), its note group, every fd (a RAM program's file takes fd 15 as it loads), its current directory, its namespace and its environment, and its CPU time. What was the old program's goes as at its end: its IRQ lines, shared segments, semaphores (and the mutexes it holds), and the device letters it served; its note handler; its pages and banks. A note that comes meanwhile waits for the new program. If the program can't be found or isn't one, EXEC returns the error and the caller goes on; if it's found but can't be read (a RAM program's file cut short), the task ends with the error as its code. rc runs the last command of rc -c this way, and its exec built-in.

SEM_WAIT

`$F87E`, a far call (its routine on another BIOS page).

In	.A = a semaphore; .X = the most ticks to wait (TICK_HZ a second; 0: none, as SEM_TRY)
Out	-
Errors	E_AGAIN (the time came and none to take), E_INVAL (not a semaphore, or freed while this task waited), E_BUSY (a mutex this task holds already), E_INTR (a note came: taken on the way out)
Waits	yes
HyForth	`sys-sem-wait ( .A .X -- ior )`
hylang	`sys-sem-wait .A=n .X=n ->`

SEM_ACQUIRE with a time limit: take one of a semaphore's count; if there's none, wait (using no CPU) till SEM_RELEASE gives one back or the ticks have passed, whichever comes first. A doorbell with a timeout: a task with work of its own to do every so often (a clock to keep) sleeps on its semaphore, and is woken at once when it's rung.

RUN

`$F881`.

In	r0 = a bank program's path (as SPAWN's); r1 = its arguments (as SPAWN's), or 0; .A = flags (SPAWN_FDMAP, RUN_LEND); r2 = an fd map (with SPAWN_FDMAP, as SPAWN's). None of them in the window ($8000-$9FFF)
Out	.A = its exit code; r0 = its exit message (31 characters at most: TA_SCRATCH, "" for none)
Errors	E_NOEXEC (not a bank program that runs in place: SPAWN it), E_NOENT, E_BUSY (running already), E_NOMEM (no bank for it), E_TOOBIG, E_FAULT (a pointer in the window), E_INTR (and what STAT and READ give)
Waits	yes
HyForth	`sys-run ( r0 r1 .A r2 -- .A r0 ior )`
hylang	`sys-run r0=s r1=args? .A=n? r2=map? -> .A`

Run a bank program in this task, as a subroutine, and wait for it. A bank program (HF_BANK, not HF_TASK) is kept in this task's code cache: RAM banks of its own (eight at most, BANKS_ALLOC's), several programs to a bank, one above the other, each loaded the first time it's run (its file's o65 module, relocated where it's put) and kept for the next; when there's no room the one run least lately is given up. The program is known by its file (STAT's), so a changed file is loaded again. It runs with its bank at $8000 and its zero page in the program's ($22 up), on this task's stack: its arguments at r0, its fds this task's 0, 1 and 2 (SPAWN_FDMAP: the map's), no note handler (the task keeps its name). It ends as a program does (returning from main, EXITS, a note's default: Ctrl-C), and then the task is the caller's again: its fds (the program's closed), note handler, zero page, banks at $8000 and $A000 and break; a note that ended the program (but a kill) is posted again, for the caller's own handler. RUN_LEND lets the program move the break past the caller's (BREAK: memory the caller doesn't use; it's put back after); without it, BREAK fails. The banks and pages the program takes (and not gives back) are the task's till it ends. A program running in place can RUN another.

---

memory (`$F890`)

Memory: the break, pages, banks, shared segments (phase 1.8): 16 slots from `$F890`, 15 used.

Call	Address	What it does
`BREAK`	`$F890`	Move the end of the program's data area.
`PAGES_ALLOC`	`$F893`	A run of pages of the task's RAM, above the break, taken from the top down.
`PAGES_FREE`	`$F896`	Give pages back.
`BANKS`	`$F899`	The task's own RAM banks.
`BANKS_ALLOC`	`$F89C`	A run of the task's banks, from the lowest.
`BANKS_FREE`	`$F89F`	Give banks back.
`SEG_CREATE`	`$F8A2`	A shared segment.
`SEG_ATTACH`	`$F8A5`	Attach the calling task to a segment (attaching twice is once).
`SEG_DETACH`	`$F8A8`	Detach the calling task from a segment; the last task to go frees it.
`SEG_MAP`	`$F8AB`	How to reach a bank of a segment.
`SEGINFO`	`$F8AE`	The shared RAM, as free shows it.
`BANKS_ALLOC_IN`	`$F8B1`	BANKS_ALLOC, from a range of the task's banks (the RAM disk's start FROM-TO: the modules its banks are on).
`SEG_CREATE_IN`	`$F8B4`	SEG_CREATE, from a range of the shared RAM (the shared RAM disk's start FROM-TO).
`BANK_LOAD`	`$F8B7`	A child (a bank program of a caller's, bankprog.h's: its functions called by the caller itself) in this task's code cache, as RUN keeps programs there (several to a bank, the one used least lately given up when there's no room), loaded if it isn't.
`BANK_FLUSH`	`$F8BA`	Give up every program and child in this task's code cache (but a program running), and free the banks left empty.

BREAK

`$F890`, a far call (its routine on another BIOS page).

In	r0 = the new end of the program's data (the first byte past it), or 0 to ask
Out	r0 = the end

Errors	E_INVAL (below the program's own data), E_NOMEM (over pages PAGES_ALLOC gave, or past the top of RAM)
Waits	no
HyForth	`sys-break ( r0 -- r0 ior )`
hylang	`sys-break r0=n? -> r0`

Move the end of the program's data area: Unix's brk (BRK is the 65C02's instruction, so the call is BREAK). It starts at the top of the module's data and BSS. The top of a task's RAM is $7FFF ($7EFF in task F).

PAGES_ALLOC

`$F893`, a far call (its routine on another BIOS page).

In	.A = pages (256 bytes each)
Out	r0 = the first one's address; .A = its page
Errors	E_INVAL (0), E_NOMEM
Waits	no
HyForth	`sys-pages-alloc ( .A -- r0 .A ior )`
hylang	`sys-pages-alloc .A=n -> r0 .A`

A run of pages of the task's RAM, above the break, taken from the top down: for a language runtime's heap, or a buffer.

PAGES_FREE

`$F896`, a far call (its routine on another BIOS page).

In	.A = pages; r0 = the first one's address
Out	-
Errors	E_INVAL (not pages PAGES_ALLOC gave this task)
Waits	no
HyForth	`sys-pages-free ( .A r0 -- ior )`
hylang	`sys-pages-free .A=n r0=n ->`

Give pages back.

BANKS

`$F899`, a far call (its routine on another BIOS page).

In	-
Out	.A = the banks the task has (16 for each good RAM module); r0 = the good modules (bit = module)
Errors	-
Waits	no
HyForth	`sys-banks ( -- .A r0 )`
hylang	`sys-banks -> .A r0`

The task's own RAM banks: module m's are $m0-$mF, selected by writing $00 (the X16's way). Modules POST found bad aren't counted.

BANKS_ALLOC

`$F89C`, a far call (its routine on another BIOS page).

In	.A = banks
Out	.A = the first (for $00)
Errors	E_INVAL (0), E_NOMEM
Waits	no
HyForth	`sys-banks-alloc ( .A -- .A ior )`
hylang	`sys-banks-alloc .A=n -> .A`

A run of the task's banks, from the lowest. Only bookkeeping, so that the libraries in one program don't use the same banks: a program may use its banks without it.

BANKS_FREE

`$F89F`, a far call (its routine on another BIOS page).

In	.A = the first bank; .X = how many
Out	-
Errors	E_INVAL (not banks BANKS_ALLOC gave this task)
Waits	no
HyForth	`sys-banks-free ( .A .X -- ior )`
hylang	`sys-banks-free .A=n .X=n ->`

Give banks back.

SEG_CREATE

`$F8A2`, a far call (its routine on another BIOS page).

In	.A = banks (1-128, 8K each)
Out	.A = the segment (0-15)
Errors	E_INVAL (0), E_NOMEM
Waits	no
HyForth	`sys-seg-create ( .A -- .A ior )`
hylang	`sys-seg-create .A=n -> .A`

A shared segment: shared RAM banks any task can attach to, with the calling task attached. It stays while a task is attached (SEG_DETACH, or the task's end).

SEG_ATTACH

`$F8A5`, a far call (its routine on another BIOS page).

In	.A = a segment
Out	-
Errors	E_INVAL (no such segment)
Waits	no
HyForth	`sys-seg-attach ( .A -- ior )`
hylang	`sys-seg-attach .A=n ->`

Attach the calling task to a segment (attaching twice is once).

SEG_DETACH

`$F8A8`, a far call (its routine on another BIOS page).

In	.A = a segment
Out	-
Errors	E_INVAL (not attached)
Waits	no
HyForth	`sys-seg-detach ( .A -- ior )`
hylang	`sys-seg-detach .A=n ->`

Detach the calling task from a segment; the last task to go frees it.

SEG_MAP

`$F8AB`, a far call (its routine on another BIOS page).

In	.A = a segment; .X = one of its banks (0 on)
Out	.A = the value for U; .X = the value for $00
Errors	E_INVAL (not attached), E_RANGE (no such bank in it)
Waits	no
HyForth	`sys-seg-map ( .A .X -- .A .X ior )`
hylang	`sys-seg-map .A=n .X=n -> .A .X`

How to reach a bank of a segment: write U, then $00, and it's at $8000-$9FFF.

SEGINFO

`$F8AE`, a far call (its routine on another BIOS page).

In	-
Out	.A = the shared banks segments can have (8K each: the good chips'); .X = those segments have now; r0 = the segments in use (bit = segment)
Errors	-
Waits	no
HyForth	`sys-seginfo ( -- .A .X r0 )`
hylang	`sys-seginfo -> .A .X r0`

The shared RAM, as free shows it.

BANKS_ALLOC_IN

`$F8B1`, a far call (its routine on another BIOS page).

In	.A = how many; .X = the lowest bank it may take; .Y = the highest (module m's are $m0-$mF)
Out	.A = the first
Errors	E_INVAL (0), E_NOMEM (no run that long there)
Waits	no
HyForth	`sys-banks-alloc-in ( .A .X .Y -- .A ior )`
hylang	`sys-banks-alloc-in .A=n .X=n .Y=n -> .A`

BANKS_ALLOC, from a range of the task's banks (the RAM disk's start FROM-TO: the modules its banks are on).

SEG_CREATE_IN

`$F8B4`, a far call (its routine on another BIOS page).

In	.A = how many banks (1-128); .X = the lowest shared bank ID it may take; .Y = the highest (U << 4 | the bank's low nibble; segments have $80-$FF)
Out	.A = the segment
Errors	E_INVAL (0), E_NOMEM (no run that long there)
Waits	no
HyForth	`sys-seg-create-in ( .A .X .Y -- .A ior )`
hylang	`sys-seg-create-in .A=n .X=n .Y=n -> .A`

SEG_CREATE, from a range of the shared RAM (the shared RAM disk's start FROM-TO).

BANK_LOAD

`$F8B7`, a far call (its routine on another BIOS page).

In	r0 = a child's path (an o65 module, linked at $8000: sdk/c/child.cfg); r1 = its caller's exports (NAME.exp), or 0; .A = where its zero page goes; .X = the bytes there it may have. None of them in the window
Out	.A = its bank; .X = its directory entry (RB_DIR + 16 x .X in its bank: RE_GEN its load, RE_TABLE its table)
Errors	E_NOENT (its file; or an import the exports haven't), E_NOEXEC (not a child), E_RANGE (its zero page), E_IO (the exports), E_NOMEM, E_TOOBIG (no room), E_FAULT (a pointer in the window)
Waits	yes

HyForth	`sys-bank-load ( r0 r1 .A .X -- .A .X ior )`
hylang	`sys-bank-load r0=s r1=s? .A=n .X=n -> .A .X`

A child (a bank program of a caller's, bankprog.h's: its functions called by the caller itself) in this task's code cache, as RUN keeps programs there (several to a bank, the one used least lately given up when there's no room), loaded if it isn't: its imports found by name in the caller's exports (its NAME.exp, the build's: "BX", 1, then each export its name's length, its name and its address, a 0 after the last), its zero page where the caller says. Its exports' table (RE_TABLE: each its name, a 0, its address; a 0 after the last) gives its entries. A call: its bank in the window ($00 alone), the entry's jsr, the bank put back; RE_GEN, as it was when it was found, says it's still there (it may have been given up for another meanwhile: then BANK_LOAD again), and the caller sets RE_STAMP from TK_RCLOCK (one more) as it calls. On a failure .X says why (BL_*), and a missing import's name is in TA_SCRATCH.

BANK_FLUSH

`$F8BA`, a far call (its routine on another BIOS page).

In	.A = the banks the code cache may have from now on (RC_BANKS to start with), or 0 (as it was)
Out	-
Errors	-
Waits	yes
HyForth	`sys-bank-flush ( .A -- )`
hylang	`sys-bank-flush .A=n ->`

Give up every program and child in this task's code cache (but a program running), and free the banks left empty.

---

file (`$F8C0`)

Files (phase 2): 32 slots from `$F8C0`, 17 used.

Call	Address	What it does
`OPEN`	`$F8C0`	Open a file.
`CREATE`	`$F8C3`	Make a file or a directory, and open it.
`CLOSE`	`$F8C6`	Close an fd.
`READ`	`$F8C9`	Read from the fd's offset, and move it on.
`WRITE`	`$F8CC`	Write at the fd's offset, and move it on.
`SEEK`	`$F8CF`	Move an fd's offset (shared by the fds that DUP or SPAWN made of it).
`STAT`	`$F8D2`	A file's stat record.
`FSTAT`	`$F8D5`	An open file's stat record.
`WSTAT`	`$F8D8`	Change a file from a stat record.
`FWSTAT`	`$F8DB`	Change an open file from a stat record.
`REMOVE`	`$F8DE`	Remove a file, or an empty directory.
`DUP`	`$F8E1`	Another fd for an open file.
`DUP2`	`$F8E4`	Make fd .X name the same channel as fd .A, closing what it named first.
`CHDIR`	`$F8E7`	Change the current directory.
`GETCWD`	`$F8EA`	The current directory.
`PIPE`	`$F8ED`	A pipe (#|).
`FD2PATH`	`$F8F0`	The name an fd's file was opened by (Plan 9's fd2path), whole and clean, as OPEN or CREATE made it (an fd SPAWN or DUP gave shares its channel, and its name).

OPEN

`$F8C0`, a far call (its routine on another BIOS page).

In	r0 = a path; .A = the mode (O_READ, O_WRITE, O_RDWR; | O_TRUNC, O_NONBLOCK)
Out	.A = an fd
Errors	E_NOENT, E_NODEV (no such device), E_NAMETOOLONG, E_MFILE (no fd free), E_NFILE (no channel free)

Waits	yes
HyForth	`sys-open ( r0 .A -- .A ior )`
hylang	`sys-open r0=s .A=n? -> .A`

Open a file. Its server may answer other errors. (Until namespaces, a path names a device: "#x", then "/" and the path in it.)

CREATE

`$F8C3`, a far call (its routine on another BIOS page).

In	r0 = a path; .A = the mode; .X = the new file's mode, high byte (DM_DIR: a directory)
Out	.A = an fd
Errors	E_NOENT, E_NODEV, E_NAMETOOLONG, E_MFILE, E_NFILE, E_EXIST
Waits	yes
HyForth	`sys-create ( r0 .A .X -- .A ior )`
hylang	`sys-create r0=s .A=n? .X=n? -> .A`

Make a file or a directory, and open it.

CLOSE

`$F8C6`, a far call (its routine on another BIOS page).

In	.A = an fd
Out	-
Errors	E_BADF (not open)
Waits	yes
HyForth	`sys-close ( .A -- ior )`
hylang	`sys-close .A=n ->`

Close an fd. The last fd naming a channel (DUP, SPAWN share them) tells its server.

READ

`$F8C9`, a far call (its routine on another BIOS page).

In	.A = an fd; r0 = a buffer; r1 = a count
Out	.A/.X = the count read (0: the end of the file)
Errors	E_BADF (not open for reading), E_AGAIN (non-blocking: nothing yet), E_INTR (a note: taken on the way out)
Waits	yes
HyForth	`sys-read ( .A r0 r1 -- .A/.X ior )`
hylang	`sys-read .A=n r0=buf r1=count -> buf=.A/.X`

Read from the fd's offset, and move it on. A count short of the one asked for is all there is for now (a console line, say). A directory reads as stat records (SR_SIZE bytes each).

WRITE

`$F8CC`, a far call (its routine on another BIOS page).

In	.A = an fd; r0 = the data; r1 = its count
Out	.A/.X = the count written
Errors	E_BADF (not open for writing), E_AGAIN, E_INTR
Waits	yes
HyForth	`sys-write ( .A r0 r1 -- .A/.X ior )`
hylang	`sys-write .A=n r0=bytes(r1) -> .A/.X`

Write at the fd's offset, and move it on.

SEEK

`$F8CF`, a far call (its routine on another BIOS page).

In	.A = an fd; r0, r1 = an offset (low word, high word: signed); .X = from: 0 the start, 1 the offset, 2 the end
Out	r0, r1 = the new offset
Errors	E_BADF, E_INVAL (before the start, or no such from)
Waits	yes
HyForth	`sys-seek ( .A r0/r1 .X -- r0/r1 ior )`
hylang	`sys-seek .A=n r0/r1=i .X=n? -> r0/r1`

Move an fd's offset (shared by the fds that DUP or SPAWN made of it).

STAT

`$F8D2`, a far call (its routine on another BIOS page).

In	r0 = a path; r1 = a buffer (SR_SIZE bytes)
Out	the file's stat record in the buffer (SR_*)
Errors	E_NOENT, E_NODEV, E_NAMETOOLONG, E_MFILE, E_NFILE
Waits	yes
HyForth	`sys-stat ( r0 r1 -- ior )`
hylang	`sys-stat r0=s r1=buf(SR_SIZE) -> buf=stat`

A file's stat record: its name, qid, mode, length, time, device letter.

FSTAT

`$F8D5`, a far call (its routine on another BIOS page).

In	.A = an fd; r0 = a buffer (SR_SIZE bytes)
Out	the stat record in the buffer
Errors	E_BADF
Waits	yes
HyForth	`sys-fstat ( .A r0 -- ior )`
hylang	`sys-fstat .A=n r0=buf(SR_SIZE) -> buf=stat`

An open file's stat record.

WSTAT

`$F8D8`, a far call (its routine on another BIOS page).

In	r0 = a path; r1 = a stat record
Out	-
Errors	E_NOENT, E_NODEV, E_NAMETOOLONG, E_MFILE, E_NFILE, E_NOSYS (its server changes nothing)
Waits	yes
HyForth	`sys-wstat ( r0 r1 -- ior )`
hylang	`sys-wstat r0=s r1=stat ->`

Change a file from a stat record: its name, its mode, its length (what its server allows: HydraFS a file's length, longer with zeros or shorter). What's all $FF in the record (an empty name) is left as it is.

FWSTAT

`$F8DB`, a far call (its routine on another BIOS page).

In	.A = an fd; r0 = a stat record
Out	-
Errors	E_BADF, E_NOSYS

Waits	yes
HyForth	`sys-fwstat ( .A r0 -- ior )`
hylang	`sys-fwstat .A=n r0=stat ->`

Change an open file from a stat record.

REMOVE

`$F8DE`, a far call (its routine on another BIOS page).

In	r0 = a path
Out	-
Errors	E_NOENT, E_NODEV, E_NAMETOOLONG, E_NOTEMPTY, E_NOSYS
Waits	yes
HyForth	`sys-remove ( r0 -- ior )`
hylang	`sys-remove r0=s ->`

Remove a file, or an empty directory.

DUP

`$F8E1`, a far call (its routine on another BIOS page).

In	.A = an fd
Out	.A = a new fd (the lowest free), on the same channel (the same offset)
Errors	E_BADF, E_MFILE
Waits	no
HyForth	`sys-dup ( .A -- .A ior )`
hylang	`sys-dup .A=n -> .A`

Another fd for an open file.

DUP2

`$F8E4`, a far call (its routine on another BIOS page).

In	.A = an fd; .X = another (0-15)
Out	-
Errors	E_BADF
Waits	yes
HyForth	`sys-dup2 ( .A .X -- ior )`
hylang	`sys-dup2 .A=n .X=n ->`

Make fd .X name the same channel as fd .A, closing what it named first.

CHDIR

`$F8E7`, a far call (its routine on another BIOS page).

In	r0 = a directory's path
Out	-
Errors	E_NOTDIR, E_NOENT, E_NODEV, E_NAMETOOLONG
Waits	yes
HyForth	`sys-chdir ( r0 -- ior )`
hylang	`sys-chdir r0=s ->`

Change the current directory: the name a relative path starts from. It's kept whole and clean (GETCWD).

GETCWD

`$F8EA`, a far call (its routine on another BIOS page).

In	r0 = a buffer (PATH_MAX + 1 bytes)
Out	.A = the current directory's length; the directory in the buffer, zero-terminated
Errors	-
Waits	no
HyForth	`sys-getcwd ( r0 -- .A )`
hylang	`sys-getcwd r0=buf(PATH_MAX+1) -> buf=z`

The current directory.

PIPE

`$F8ED`, a far call (its routine on another BIOS page).

In	-
Out	.A = an fd to read from; .X = an fd to write to
Errors	E_NODEV (no pipe server), E_MFILE, E_NFILE, E_NOMEM (no pipe free)
Waits	yes
HyForth	`sys-pipe ( -- .A .X ior )`
hylang	`sys-pipe -> .A .X`

A pipe (#|): what's written to one end is read from the other, 512 bytes held between them. A read waits for something to read, and is the end (0) once no writer is left; a write waits for room, and is E_PIPE once no reader is left.

FD2PATH

`$F8F0`, a far call (its routine on another BIOS page).

In	.A = an fd; r0 = a buffer (PATH_MAX + 1 bytes)
Out	the name in the buffer, zero-terminated
Errors	E_BADF (not an open fd)
Waits	no
HyForth	`sys-fd2path ( .A r0 -- ior )`
hylang	`sys-fd2path .A=n r0=buf(PATH_MAX+1) -> buf=z`

The name an fd's file was opened by (Plan 9's fd2path), whole and clean, as OPEN or CREATE made it (an fd SPAWN or DUP gave shares its channel, and its name): the namespace's, or a # name; a pipe's #|/pipe.

---

name (`$F920`)

Namespaces (phase 2): 16 slots from `$F920`, 4 used.

Call	Address	What it does
`BIND`	`$F920`	Bind new at old.
`MOUNT`	`$F923`	Mount a device's server, with a spec (which of its file trees: '#f' with x is the ROM disk), at old, as BIND does.
`UNMOUNT`	`$F926`	Undo a bind or a mount at old.
`NSINFO`	`$F929`	A namespace's mount entries, one at a time, in the order of the kernel's table.

BIND

`$F920`, a far call (its routine on another BIOS page).

In	r0 = new (a path); r1 = old (a path: absolute); .A = MREPL, MBEFORE or MAFTER, and MCREATE
Out	-
Errors	E_NOENT, E_INVAL, E_NSFULL, E_NODEV, E_NAMETOOLONG

Waits	yes
HyForth	`sys-bind ( r0 r1 .A -- ior )`
hylang	`sys-bind r0=s r1=s .A=n? ->`

Bind new at old: in old's place (MREPL), or a union with what's there, new first (MBEFORE) or last (MAFTER); with MCREATE, a CREATE in the union comes to new. New is what it is now: a mount point itself is all of its union's members, anything else the first of its candidates that's there. A union's first bind at a directory that's there (and isn't a mount point itself) keeps it as a member.

MOUNT

`$F923`, a far call (its routine on another BIOS page).

In	.X = a device letter; r0 = its spec (a string, 8 characters at most), or 0; r1 = old; .A = BIND's flags
Out	-
Errors	E_NODEV, E_NOENT, E_INVAL, E_NSFULL, E_NAMETOOLONG
Waits	yes
HyForth	`sys-mount ( .X r0 r1 .A -- ior )`
hylang	`sys-mount .X=n r0=s? r1=s .A=n? ->`

Mount a device's server, with a spec (which of its file trees: '#f' with x is the ROM disk), at old, as BIND does.

UNMOUNT

`$F926`, a far call (its routine on another BIOS page).

In	r0 = new, or 0; r1 = old, or 0
Out	-
Errors	E_NOENT, E_INVAL, E_NSFULL, E_NAMETOOLONG
Waits	yes
HyForth	`sys-unmount ( r0 r1 -- ior )`
hylang	`sys-unmount r0=s? r1=s? ->`

Undo a bind or a mount at old: all of what's there (r0 = 0), or new's member (as BIND finds new). With no old either (r1 = 0), everything: the caller has an empty namespace of its own (the tasks it shared one with keep theirs): what Plan 9's newns starts from, and nslib's ns_default does first.

NSINFO

`$F929`, a far call (its routine on another BIOS page).

In	.A = a task ($FF: this one); .X = which entry of its namespace (0 on); r0 = a buffer (NI_SIZE bytes)
Out	the entry in the buffer (NI_*): its mount point, its device letter and spec, the path in that device, its flags (MCREATE) and its place among the union's members
Errors	E_RANGE (past the last), E_SRCH
Waits	no
HyForth	`sys-nsinfo ( .A .X r0 -- ior )`
hylang	`sys-nsinfo .A=n .X=n r0=buf(NI_SIZE) -> buf`

A namespace's mount entries, one at a time, in the order of the kernel's table: each a member of the union at its mount point, the members in NI_SEQ's order (lowest first). /proc's ns, and ns.

---

cons (`$F950`)

Console helpers: a byte or a string to stdout, a byte from stdin: 16 slots from `$F950`, 4 used.

Call	Address	What it does
PUTC	`$F950`	Write a byte to stdout (the X16's CHROUT).
PUTS	`$F953`	Write a string to stdout.

Call	Address	What it does
GETC	$F956	Read a byte from stdin, waiting for one (the X16's CHRIN, unechoed).
PUTHEX	$F959	Write a byte to stdout as two hex digits (for debugging).

PUTC

$F950.

In	.A = a byte
Out	-
Errors	-
Waits	yes
HyForth	`sys-putc ( .A -- )`
hylang	`sys-putc .A=n ->`

Write a byte to stdout (the X16's CHROUT).

PUTS

$F953.

In	r0 = a string, zero-terminated
Out	-
Errors	E_INTR (a note came while it waited: taken on the way out)
Waits	yes
HyForth	`sys-puts ( r0 -- ior )`
hylang	`sys-puts r0=s ->`

Write a string to stdout.

GETC

$F956.

In	-
Out	.A = a byte
Errors	E_EOF (the end of stdin), E_INTR (a note came: taken on the way out)
Waits	yes
HyForth	`sys-getc ( -- .A ior )`
hylang	`sys-getc -> .A`

Read a byte from stdin, waiting for one (the X16's CHRIN, unechoed).

PUTHEX

$F959.

In	.A = a byte
Out	-
Errors	-
Waits	yes
HyForth	`sys-puthex ( .A -- )`
hylang	`sys-puthex .A=n ->`

Write a byte to stdout as two hex digits (for debugging).

---

time (`$F980`)

The clock (phase 5): 16 slots from `$F980`, 3 used.

Call	Address	What it does
`TIME`	`$F980`	The clock.
`TIME_SET`	`$F983`	Set the clock (from now: the boot's time becomes the clock less the ticks since).
`RTC`	`$F986`	The DS1747 in U7, its registers a byte at a time (task F's $7FF8-$7FFF).

TIME

`$F980`, a far call (its routine on another BIOS page).

In	-
Out	r0/r1 = the clock: seconds since 2000-01-01 00:00:00
Errors	-
Waits	no
HyForth	`sys-time ( -- r0/r1 )`
hylang	`sys-time -> r0/r1`

The clock: the boot's time and the ticks since. It starts at 0 (2000-01-01) unless kdev finds a DS1747 with a date and time as it starts; a write of /dev/time sets it (and the chip).

TIME_SET

`$F983`, a far call (its routine on another BIOS page).

In	r0/r1 = seconds since 2000-01-01 00:00:00
Out	-
Errors	-
Waits	no
HyForth	`sys-time-set ( r0/r1 -- )`
hylang	`sys-time-set r0/r1=n ->`

Set the clock (from now: the boot's time becomes the clock less the ticks since). The DS1747 is the caller's to set (RTC).

RTC

`$F986`, a far call (its routine on another BIOS page).

In	.A = 0, read, or 1, write; r0 = 8 bytes: the DS1747's registers (RTC_REGS, hw.inc), BCD
Out	-
Errors	E_INVAL (.A neither)
Waits	no
HyForth	`sys-rtc ( .A r0 -- ior )`
hylang	`sys-rtc .A=n r0=io(8) -> buf`

The DS1747 in U7, its registers a byte at a time (task F's $7FF8-$7FFF): read with its updates halted (R) for the copy; written with W set, the control (its century) last with W clear, which starts it from them. No chip: the RAM there, as it is. A read must check the fields hold a date and time.

---

server (`$F9B0`)

Servers and drivers: 32 slots from `$F9B0`, 17 used.

Call	Address	What it does
`IRQ_OWN`	`$F9B0`	Own an IRQ line.
`IRQ_RELEASE`	`$F9B3`	Give an IRQ line back.

Call	Address	What it does
WAKE	$F9B6	Make a waiting task (PAUSE) runnable.
PAUSE	$F9B9	Wait until another task or an irq entry WAKEs this one, or a note comes (taken on the way out).
NOTE_POST	$F9BC	NOTE without taking the caller's own notes.
SRV_REGISTER	$F9BF	Serve a device.
SRV_TAKE	$F9C2	In the serve entry.
SRV_REPLY	$F9C5	At the end of the serve entry.
CLIENT_READ	$F9C8	Bytes from the client of the request being served (kcopy).
CLIENT_WRITE	$F9CB	Bytes to the client of the request being served (kcopy).
NOTE_QUEUE	$F9CE	A note to a note group, from an irq entry (a console's Ctrl-C).
ROMREAD	$F9D1	Copy from any bank of the paged ROM, for a module run in place (the storage driver's ROM disk).
TASKMEM	$F9D4	Bytes from a task's memory into the buffer, or from the buffer into it.
TASKSTOP	$F9D7	Stop a task where it is (the scheduler passes it by: its waits go on, and end, but it doesn't run), or start it again.
TASKSTEP	$F9DA	The debugger's step (/proc's ctl: kdev).
SRV_NAP	$F9DD	For a driver with work of its own between its calls (a device to look at now and then).
INIT_RUN	$F9E0	Start init (the module the directory names) again, in a task of its own, with the number in decimal as its argument.

IRQ_OWN

$F9B0.

In	.A = an IRQ line (0-14), or LINE_VIA_T2 (VIA timer 2), or LINE_VIA_CA1 (VIA CA1)
Out	-
Errors	E_BUSY (another task owns it), E_RANGE (not a line), E_INVAL (the task has no IRQ entry)
Waits	no

Own an IRQ line: its interrupts run the calling task's irq entry, in the calling task. The entry gets the line in .A, and returns .A = 0, or IRQ_RESCHED to ask for a task switch (it woke a task). It runs with interrupts off and must not make calls that wait, and with the dispatch's own cycles it must stay under the 200-cycle budget: about 70 cycles of its own (TASK_EVENT, not WAKE, to wake clients; NOTE_QUEUE for a group). A task's lines are released when it ends. LINE_VIA_T2 is VIA timer 2: owning it sets the timer one-shot with its interrupt on; the owner starts it (T2CL, then T2CH), and its entry clears the interrupt (reading T2CL, or starting it again). LINE_VIA_CA1 is CA1: owning it turns CA1's interrupt on (off as it's given back), and the entry clears CA1's flag (IFR).

IRQ_RELEASE

$F9B3.

In	.A = an IRQ line
Out	-
Errors	E_PERM (not the caller's), E_RANGE
Waits	no

Give an IRQ line back.

WAKE

$F9B6.

In	.A = a task
Out	-
Errors	-
Waits	no

Make a waiting task (PAUSE) runnable. From a task or an irq entry. Waking a task that isn't waiting does nothing.

PAUSE

`$F9B9`.

In	-
Out	-
Errors	-
Waits	yes

Wait until another task or an irq entry WAKEs this one, or a note comes (taken on the way out). (Wake-ups can come early: look again after.)

NOTE_POST

`$F9BC`.

In	.A = a task, or NOTE_GROUP | a note group; .X = the note
Out	-
Errors	E_INVAL, E_SRCH, E_PERM
Waits	no

NOTE without taking the caller's own notes: the note posted, the tasks woken. From a task, or an irq entry for a task (a group is too many tasks to look at with interrupts off: NOTE_QUEUE).

SRV_REGISTER

`$F9BF`, a far call (its routine on another BIOS page).

In	.A = a device letter (printable; not #)
Out	-
Errors	E_INVAL (not a letter), E_EXIST (another task's), E_NOMEM (no room)
Waits	no

Serve a device: its requests ("#x...") come to the calling task's serve entry. A task may serve several letters (the request block names it: RQ_DEV). Freed when the task ends.

SRV_TAKE

`$F9C2`, a far call (its routine on another BIOS page).

In	-
Out	.A = the request (R_*); .Y = the client; the request block in TASK_INBOX, and (R_OPEN, R_CREATE, R_REMOVE) the name in TASK_PATH
Errors	-
Waits	no

In the serve entry: take the request being served from the client (srvlib does it). It notes the server's event count (TASK_EVENT) in RQ_EVENT: a client answered E_AGAIN waits for it to change.

SRV_REPLY

`$F9C5`, a far call (its routine on another BIOS page).

In	TASK_INBOX's RQ_FID, RQ_DONE, RQ_PERM, RQ_EVENT: what the request returns; .A, C: the answer
Out	.A and C kept
Errors	-
Waits	no

At the end of the serve entry: what the request returns, back to the client (srvlib does it).

CLIENT_READ

`$F9C8`, a far call (its routine on another BIOS page).

In	r0 = here; r1 = there, in the client's view; r2 = a count
Out	-
Errors	-
Waits	no

Bytes from the client of the request being served (kcopy).

CLIENT_WRITE

`$F9CB`, a far call (its routine on another BIOS page).

In	r0 = here (the data); r1 = there, in the client's view; r2 = a count
Out	-
Errors	-
Waits	no

Bytes to the client of the request being served (kcopy).

NOTE_QUEUE

`$F9CE`.

In	IRQs off; .X = a note group (0-15); .A = the note's bit: 1 << (note - 1), for a note 1-8
Out	.X kept
Errors	-
Waits	no

A note to a note group, from an irq entry (a console's Ctrl-C): queued, and posted by the kernel task as soon as it runs. Short enough for an irq entry: nothing is checked.

ROMREAD

`$F9D1`.

In	.A = a paged ROM bank; r0 = where in it ($A000-$DFFF); r1 = a buffer (not at $A000-$DFFF); r2 = a count
Out	the bytes in the buffer
Errors	E_PERM (the task owns an IRQ line: its irq entry would find the wrong bank at $A000)
Waits	no

Copy from any bank of the paged ROM, for a module run in place (the storage driver's ROM disk): the bank is the task's own meanwhile, and its module's is put back.

TASKMEM

`$F9D4`, a far call (its routine on another BIOS page).

In	.A = a task; .X = TM_READ or TM_WRITE, | TM_BANK; r0 = a buffer; r1 = an address in the task's view, $0000-$DFFF (with TM_BANK: $8000-$9FFF, in its RAM bank r3); r2 = a count (the range ends at $E000, or with TM_BANK at $A000, at most)
Out	-
Errors	E_PERM (the caller isn't a driver), E_SRCH (not a task), E_INVAL (past the range)
Waits	no

Bytes from a task's memory into the buffer, or from the buffer into it: /proc's mem and ram (kdev, which decides who may: a program reaches another task's memory only through those files). Its task RAM, and its RAM bank and paged ROM bank as it has them selected; or with TM_BANK any of its RAM banks (its bank register points at it meanwhile: not for a driver, whose irq entry might run). A byte at a time, IRQs off for each.

TASKSTOP

`$F9D7`, a far call (its routine on another BIOS page).

In	.A = a task; .X = 1 to stop it, 0 to start it again
Out	-
Errors	E_PERM (the caller isn't a driver; the kernel task, a driver), E_SRCH (not a task, or not started)
Waits	no

Stop a task where it is (the scheduler passes it by: its waits go on, and end, but it doesn't run), or start it again: /proc's ctl (kdev, which decides who may), for the debugger. TASKINFO's TI_FLAGS has TF_STOPPED while it's stopped. A kill note starts it, to end it.

TASKSTEP

`$F9DA`, a far call (its routine on another BIOS page).

In	.A = a task; .X = TS_STEP, TS_NEXT, TS_BREAKS or TS_NOBREAKS
Out	-
Errors	E_PERM (the caller isn't a driver; the kernel task, a driver), E_SRCH (not a task, or not started), E_INVAL (another .X; to step: a BRK or STP), E_BUSY (to step: not stopped, or a step still out) (or in the kernel: in a call, its PC at $E000 or above, or W not 0)
Waits	no

The debugger's step (/proc's ctl: kdev). TS_STEP: a stopped task runs its next instruction and stops again (TI_FLAGS: TF_STOPPED once it has). JMP, JSR, RTS and RTI are done at once, on its frame; others run out of line, in its zero page, a BRK after them. TS_NEXT: the same, but a JSR runs its subroutine whole, as a JSR to the kernel's jump table always does (a subroutine that reads bytes after its JSR can't be stepped over). TS_BREAKS: from now any BRK stops the task, its PC back on the BRK (the debugger's breakpoints, BRKs it writes into the program); TS_NOBREAKS: a BRK is the program's own again (NOTE_BRK). SPAWN_STOPPED starts a program stopped at its entry point.

SRV_NAP

`$F9DD`.

In	.A/.X = ticks (1-32767); or 0, and .Y = the driver's event count (TASK_EVENT) as it last saw it
Out	-
Errors	-
Waits	yes

For a driver with work of its own between its calls (a device to look at now and then): its init doesn't return, but waits here between its turns. While it waits it serves its calls, as a driver that has started does (the boot goes on); when the time has come it runs again, busy (a call to it waits) till its next SRV_NAP, or till its init returns (it's an ordinary driver from then). With 0 ticks it waits till its own event count isn't .Y any more (its irq entry adds to it, or one of its calls): for work that follows what happens, not the clock. ps shows it asleep, or waiting for an event.

INIT_RUN

`$F9E0`.

In	.A = a number, 0-15
Out	-
Errors	-
Waits	no

Start init (the module the directory names) again, in a task of its own, with the number in decimal as its argument: as the boot starts it (no parent, no fds, an empty namespace and environment, a note group of its own), but the boot's has no argument. The kernel task starts it as it next runs; with no task free it isn't started (but 0's, which waits for one). The kernel itself asks for 0's when the task init was last started in for 0 (or by the boot) has EXECed and then ends: so an init that EXECs the login shell is started again when that shell ends (an init that ends itself is left ended). HydraOS's console asks for n's when the user makes window n. From a task or an irq entry.

---

dbg (`$FA10`)

Debugging and tests: unstable, may change in any version: 16 slots from `$FA10`, 3 used.

Call	Address	What it does
DBG_SCALL	$FA10	Call a driver's serve entry directly, in the driver's task (the file layer will make these calls in phase 2).
DBG_KCOPY	$FA13	Copy bytes between this task's memory and another's (the kernel's kcopy).
DBG_PS	$FA16	Print a line for each task in use.

DBG_SCALL

$FA10.

In	.Y = a driver task; .A, .X = what its serve entry gets
Out	.A, .X and C from its serve entry
Errors	E_SRCH (not a task), E_NODEV (it's free or ended, or has no serve entry: a program)
Waits	yes

Call a driver's serve entry directly, in the driver's task (the file layer will make these calls in phase 2). A driver serves one call at a time: a second caller waits.

DBG_KCOPY

$FA13.

In	.A = another task; r0 = an address in this task; r1 = an address in that task; r2 = a byte count; C = 0 to copy from this task to that one, C = 1 the other way
Out	-
Errors	E_SRCH (not a task)
Waits	no

Copy bytes between this task's memory and another's (the kernel's kcopy).

DBG_PS

$FA16, a far call (its routine on another BIOS page).

In	-
Out	-
Errors	-
Waits	yes

Print a line for each task in use: its number, state, flags, parent, CPU time and name.

The error codes

A call that fails returns C = 1 and one of these in .A. ERRSTR gives the text; the C library sets errno to the code's POSIX equivalent (and _oserror to the code itself).

Code	Name	Text
$01	E_PERM	not allowed
$02	E_INVAL	invalid argument
$03	E_NOSYS	no such call
$04	E_AGAIN	not yet: try again
$05	E_INTR	interrupted
$06	E_NOMEM	out of memory
$07	E_BUSY	busy
$08	E_RANGE	out of range
$09	E_FAULT	bad address
$0A	E_NAMETOOLONG	name too long
$0B	E_TOOBIG	too big

Code	Name	Text
$20	E_NOENT	not found
$21	E_EXIST	already exists
$22	E_NOTDIR	not a directory
$23	E_ISDIR	is a directory
$24	E_NOTEMPTY	directory not empty
$25	E_BADF	bad file descriptor
$26	E_MFILE	no free file descriptor
$27	E_NFILE	too many open files
$28	E_NOSPC	disk full
$29	E_ROFS	read-only
$2A	E_IO	i/o error
$2B	E_NODEV	no such device
$2C	E_PIPE	broken pipe
$2D	E_NOEXEC	not a program
$2E	E_EOF	end of file
$2F	E_NOTFS	no file system
$30	E_MEDIA	medium error
$40	E_NOTASK	no free task
$41	E_SRCH	no such task
$42	E_CHILD	no such child
$50	E_NSFULL	namespace full
$51	E_NSLOOP	too many binds

The constants

What `hydra.inc` defines besides the calls and the error codes (`hydracalls.h` has them for C, with `HY_` before each name).

Name	Value	What it is
ABI_VERSION	1	The ABI version this include describes
TICK_HZ	200	Ticks a second
INIT_TASK	1	init's task (and its note group: the console's foreground at first)
PROG_ZP	$22	A program's own zero page: $22-$7F
PROG_ZP_END	$80	
RB_DIR	$9F00	A code cache bank's directory (BANK_LOAD): its entries at RB_DIR + 16 x n, n 1-15
RE_START	0	A cache directory entry: where its module is (2)
RE_END	2	A cache directory entry: the first byte past it (2)
RE_TABLE	4	A cache directory entry: a child's exports' table (2)
RE_GEN	6	A cache directory entry: its load's number (0: free)
RE_FLAGS	7	A cache directory entry: RF_*
RE_STAMP	8	A cache directory entry: when it was last used (TK_RCLOCK) (2)
TK_RCLOCK	$EC	A task's code cache's uses counted (2): its zero page's
RC_BANKS	8	A task's code cache's banks at most, to start with (BANK_FLUSH)
BL_FILE	2	BANK_LOAD's why (.X): its file not there, or not read
BL_FORMAT	3	BANK_LOAD's why (.X): not a child (o65, linked at $8000, in a bank)
BL_ZP	4	BANK_LOAD's why (.X): its zero page more than the caller gives
BL_EXPORTS	5	BANK_LOAD's why (.X): the caller's exports' file not there, or not of its form
BL_IMPORT	6	BANK_LOAD's why (.X): an import the caller hasn't (its name in TA_SCRATCH)
BL_BANK	7	BANK_LOAD's why (.X): no bank, or no room in one

Name	Value	What it is
TA_SCRATCH	$0280	A task's call scratch (32 bytes): an exit message, BANK_LOAD's missing import
TASK_ARGS	$0350	A task's arguments: zero-terminated strings, an empty one after the last (176 bytes at most)
ARGS_MAX	176	SPAWN's arguments: their bytes at most (each string's 0, and the empty one that ends them)
TASK_NAME	$0230	A task's name, zero-terminated
IRQ_RESCHED	$80	An irq entry's answer: a task switch, please
LINE_VIA_T2	16	IRQ_OWN's line for VIA timer 2 (the hardware's lines: hw.inc)
LINE_VIA_CA1	17	IRQ_OWN's line for VIA CA1 (J27's pin 11): owning it turns CA1's interrupt on; the owner clears its flag
TASK_EVENT	$BD	A task's event count: a server adds 1 (inc TASK_EVENT) when what its clients wait for has happened
HT_PROGRAM	1	HYX2 module types
HT_DRIVER	2	
HT_LIBRARY	3	
HF_INPLACE	$01	HYX2 flags: runs in place in the paged ROM
HF_BOOT	$02	HYX2 flags: a driver the kernel starts at boot
HF_BANK	$04	HYX2 flags: a bank program: an o65 module after the header, run in a RAM bank (RUN)
HF_TASK	$08	HYX2 flags: a bank program that runs only in a task of its own (not in place: RUN)
HX_MAGIC	0	The HYX2 header (HX_SIZE bytes, a program's first): HYX2
HX_HSIZE	4	HYX2: the header's size (48)
HX_TYPE	5	HYX2: the type (HT_*)
HX_FLAGS	6	HYX2: the flags (HF_*)
HX_ABI	7	HYX2: the ABI version it was built for
HX_LOAD	8	HYX2: its load address (2: $A000 in place, $0800 a RAM program)
HX_LENGTH	10	HYX2: its image's length (2: in its last bank, in place)
HX_DATA_LOAD	12	HYX2: its data, where it is in the image (2) ...
HX_DATA_RUN	14	HYX2: ... where it runs (2) ...
HX_DATA_LEN	16	HYX2: ... and its length (2)
HX_BSS	18	HYX2: its BSS (2) ...
HX_BSS_LEN	20	HYX2: ... and its length (2: cleared as it starts)
HX_TOP	22	HYX2: the top of the RAM it uses (2: its break starts there)
HX_MAIN	24	HYX2: entry: main (a program), init (a driver)
HX_SERVE	26	HYX2: entry: serve (a driver)
HX_IRQ	28	HYX2: entry: irq (a driver)
HX_STOP	30	HYX2: entry: stop (a driver)
HX_DEVICE	32	HYX2: its device letter (a driver)
HX_BANKS	33	HYX2: the banks it spans (in place)
HX_VERSION	34	HYX2: its version (2)
HX_NAME	36	HYX2: its name (12: zero-terminated)
HX_SIZE	48	
ME_BANK	0	MODINFO's answer (ME_SIZE bytes): the module's first paged ROM bank
ME_BANKS	1	MODINFO: the banks it spans
ME_TYPE	2	MODINFO: its type (HT_*)
ME_FLAGS	3	MODINFO: its flags (HF_*)
ME_NAME	4	MODINFO: its name (11 bytes, zero-padded: 10 characters at most)
ME_PAGE	15	MODINFO: its header's page in its first bank, 0-63 ($A000 + 256 * it: a module of a bank, packed with others)
ME_SIZE	16	
TI_STATE	0	TASKINFO's answer: the task's state (0: free, 1 ready, 2 wait, 3 call, 4 idle, 5 new, 6 sleep, 7 blocked, 8 event)
TI_FLAGS	1	TASKINFO: its flags (TF_DRIVER, TF_STOPPED)
TF_DRIVER	$01	TI_FLAGS: a driver
TF_STOPPED	$80	TI_FLAGS: stopped (/proc's ctl: TASKSTOP)

Name	Value	What it is
TI_PARENT	2	TASKINFO: its parent ($FF: none)
TI_CPU	3	TASKINFO: its CPU time, in ticks (3 bytes)
TI_BANK	6	TASKINFO: its module's paged ROM bank ($FF: none)
TI_TYPE	7	TASKINFO: its module's type (HT_*; 0: none)
TI_NAME	8	TASKINFO: its name, zero-terminated (16 bytes)
TI_GROUP	24	TASKINFO: its note group
TI_SIZE	25	TASKINFO: the answer's size
NOTE_INTERRUPT	1	Notes: Ctrl-C (default: the end, 130)
NOTE_KILL	2	Notes: kill, never caught (137)
NOTE_HANGUP	3	Notes: hangup (129)
NOTE_ALARM	4	Notes: alarm (142)
NOTE_BRK	5	Notes: sys: brk, a BRK instruction (133)
NOTE_USER	16	Notes: 16-31 are the programs' own (128 + the note)
NOTE_GROUP	$80	NOTE's .A: a note group, not a task
SEM_MUTEX	$01	SEM_NEW's .X: a mutex (its count 1, given back only by its holder)
REBOOT_HWTEST	$01	REBOOT's .A: into the hardware test
SEM_MAX	16	Semaphores: as many as there can be at once
SPAWN_NEWGROUP	$01	SPAWN's flags: the child starts a note group of its own
SPAWN_NEWNS	$02	SPAWN's flags: the child starts with an empty namespace of its own (else it shares its parent's)
SPAWN_FDMAP	$04	SPAWN's flags: the child's fds are r2's map (else the caller's 0, 1 and 2)
SPAWN_NOENV	$08	SPAWN's flags: the child starts with an empty environment (else a copy of its parent's)
SPAWN_STOPPED	$10	SPAWN's flags: the child stops at its entry point, for the debugger (/proc/N/ctl's step)
SPAWN_RUN	$20	SPAWN's flags: a bank program that runs in place does, in this task (RUN), and SPAWN gives this task back
RUN_LEND	$40	RUN's flags (and SPAWN_RUN's): the program may move the break past the caller's
ENV_SIZE	1024	TASKREAD's TR_ENV: its buffer, an environment's first variables, whole, and a 0
ENV_MAX	8192	An environment's bytes at most: each variable its name's length, its name, its value's length (2), its value; a 0
ENV_NAME_MAX	31	A variable's name's length at most
SPAWN_FDS	15	SPAWN's fd map: the child's fds it can give, at most (its fd 15 is its loader's as it starts)
MREPL	0	BIND's and MOUNT's flags: in place of what's at old
MBEFORE	1	BIND's and MOUNT's flags: a union, new first
MAFTER	2	BIND's and MOUNT's flags: a union, new last
MCREATE	4	BIND's and MOUNT's flags: a CREATE in the union comes to new
EXIT_MSG_MAX	31	An exit message's length at most (WAIT's buffer: 32 bytes)
KMESG_SIZE	4096	KMESG: the kernel's messages held at most (then the oldest go)
TR_ARGS	0	TASKREAD: a task's arguments (ARGS_MAX bytes)
TR_CWD	1	TASKREAD: a task's current directory (PATH_MAX + 1 bytes)
TR_ENV	2	TASKREAD: a task's environment's first variables, whole, and a 0 (ENV_SIZE bytes at most)
TR_FRAME	3	TASKREAD: a task's state and frame (TF_SIZE bytes)
TR_ENVAT	4	TASKREAD: part of a task's environment (r1 bytes at most, from offset r2; .A/.X its bytes in use)
TR_FD	5	TASKREAD: a task's fd r2 (FI_SIZE bytes)
FI_NAME	0	TR_FD's: the name its file was opened by (PATH_MAX + 1 bytes, zero-terminated) ...
FI_MODE	64	TR_FD's: its open mode (O_*) ...
FI_DEV	65	TR_FD's: its server's device letter ...
FI_QTYPE	66	TR_FD's: its qid type (QT_DIR ...) ...
FI_OFFSET	67	TR_FD's: and its offset (4 bytes, low first)
FI_SIZE	71	
TF_STATE	0	TR_FRAME's: the task's state (0 free, 1 ready, 2 waiting, 3 calling, 4 idle, 5 new, 6 sleeping, 7 blocked, 8 waiting for an event)
TF_S	1	TR_FRAME's: its stack pointer, as it was before the frame

Name	Value	What it is
TF_U	2	TR_FRAME's: the frame: U
TF_Y	3	
TF_W	4	
TF_X	5	
TF_A	6	
TF_P	7	
TF_PC	8	TR_FRAME's: PC (2)
TF_RAM	10	TR_FRAME's: its RAM bank register ($00)
TF_ROM	11	TR_FRAME's: its paged ROM bank register ($01)
TF_SIZE	12	
TM_READ	0	TASKMEM: from the task's memory
TM_WRITE	$01	TASKMEM: to the task's memory
TM_BANK	$80	TASKMEM's flag: its RAM bank r3, at $8000-$9FFF
TS_STEP	0	TASKSTEP: one instruction
TS_NEXT	1	TASKSTEP: one instruction, a JSR's subroutine whole
TS_BREAKS	2	TASKSTEP: from now, the task's BRKs stop it (the debugger's breakpoints)
TS_NOBREAKS	3	TASKSTEP: its BRKs are its own again (NOTE_BRK)
NI_FROM	0	NSINFO's answer (NI_SIZE bytes): the mount point (PATH_MAX + 1 bytes, zero-terminated)
NI_PATH	64	NSINFO: the path in the device (zero-terminated, no leading /)
NI_DEV	128	NSINFO: the device letter
NI_SPEC	129	NSINFO: the spec (8 bytes, zero-padded, and a 0)
NI_FLAGS	138	NSINFO: the flags (MCREATE)
NI_SEQ	139	NSINFO: its place among the union's members (lowest first)
NI_SIZE	140	
KEY_UP	$80	Keys: a raw console read gives the terminal's cursor and function keys as one code each
KEY_DOWN	$81	
KEY_RIGHT	$82	
KEY_LEFT	$83	
KEY_HOME	$84	
KEY_END	$85	
KEY_INS	$86	
KEY_DEL	$87	
KEY_PGUP	$88	
KEY_PGDN	$89	
KEY_F1	$8A	Keys: F1-F12 are $8A-$95
KEY_F2	$8B	
KEY_F3	$8C	
KEY_F4	$8D	
KEY_F5	$8E	
KEY_F6	$8F	
KEY_F7	$90	
KEY_F8	$91	
KEY_F9	$92	
KEY_F10	$93	
KEY_F11	$94	
KEY_F12	$95	
KEY_FOCUS	$97	Keys: the focus moved to another window of this one's group; its number is the next key (a raw read's, keys hydra's)
KEY_RESIZE	$96	Keys: the window's size changed (a raw read's, keys hydra's: consctl reads with the new one)

Name	Value	What it is
KEY_MOD	$98	Keys: a key the terminal sent modified (keys mods's): the modifiers next (1 Shift, 2 Alt, 4 Ctrl), then the key
O_READ	0	Open modes
O_WRITE	1	
O_RDWR	2	
O_RW_MASK	3	
O_TRUNC	$10	Open modes: empty the file first
O_NONBLOCK	$40	Open modes: E_AGAIN rather than wait
FD_MAX	16	Fds a task has (0 stdin, 1 stdout, 2 stderr)
PATH_MAX	63	A path's length at most
IO_UNIT	512	The most a READ or WRITE asks of a server in one request
SR_NAME	0	The stat record (SR_SIZE bytes): its name (32, zero-terminated)
SR_QTYPE	32	Stat: the qid's type (QT_*)
SR_QVERS	33	Stat: the qid's version
SR_QPATH	34	Stat: the qid's path (4: unique in its server)
SR_MODE	38	Stat: the mode (2: permissions; high byte DM_DIR)
SR_LENGTH	40	Stat: the length (4)
SR_MTIME	44	Stat: the time it changed (4: seconds since 2000)
SR_DEV	48	Stat: the device letter
SR_INST	49	Stat: the device's instance (its spec's first character)
SR_SIZE	64	
QT_DIR	$80	Qid types: a directory
QT_FILE	$00	
DM_DIR	$80	A mode's high byte: a directory
DM_APPEND	$40	A mode's high byte: append-only (writes go to the end)
TASK_INBOX	$0200	A server's copy of the request it's serving (RQ_*)
TASK_PATH	$02C0	A server's copy of the name a request names (64 bytes)
RQ_TYPE	0	The request block (RQ_SIZE bytes): the request (R_*)
RQ_FID	1	Request: the fid (out: the new fid, R_OPEN and R_CREATE)
RQ_MODE	2	Request: the open mode
RQ_CLIENT	3	Request: the client task
RQ_U	4	Request: the client's U
RQ_FLAGS	5	Request: RF_*
RQ_OFFSET	6	Request: the offset (4)
RQ_COUNT	10	Request: the count asked for (2)
RQ_DONE	12	Request: the count done (2, out)
RQ_BUF	14	Request: the buffer, in the client's view (2)
RQ_PERM	16	Request: R_CREATE's new mode (high byte); out (R_OPEN, R_CREATE): the qid type
RQ_NAMELEN	17	Request: the name's length
RQ_GROUP	18	Request: the client's note group
RQ_DEV	19	Request: the device letter
RQ_SPEC	20	Request: the mount's spec (8, zero-padded)
RQ_EVENT	28	Request: the server's event count as it took it (after E_AGAIN, the client waits for a change)
RQ_SIZE	32	
RF_NONBLOCK	$01	Request flags: answer E_AGAIN rather than wait
R_OPEN	1	Requests
R_CREATE	2	
R_READ	3	
R_WRITE	4	

Name	Value	What it is
R_CLUNK	5	
R_STAT	6	
R_WSTAT	7	
R_REMOVE	8	
R_FLUSH	9	
R_DUP	10	
SND_CHANNELS	24	Sound (#a): its channels: 0-7 the YM2151's, 8-23 the Vera X's PSG (its voices 0-15)
SND_PSG	8	Sound: the PSG's first channel (its voice 0)
SND_PATCHES	163	Sound: its patches (0-127: General MIDI's programs; 128-162: drums and percussion)
SND_DRUMS	128	Sound: General MIDI's drums (MIDI channel 10's notes)
SND_R_CH	$02	Sound: a command, at a register the chip doesn't have: the channel the next commands are for (0-23)
SND_R_PATCH	$03	Sound: load patch n into the channel (its speakers stay)
SND_R_NOTE	$04	Sound: key on MIDI note n (60: middle C; 69: A, 440 Hz), with the channel's bend
SND_R_OFF	$05	Sound: key off (the release)
SND_R_VOL	$06	Sound: the channel's volume (0-127, as General MIDI's)
SND_R_PAN	$07	Sound: the channel's speakers (SND_PAN_*)
SND_R_BEND	$09	Sound: the channel's bend (signed: 64ths of a semitone; the note playing too)
SND_R_DRUM	$0A	Sound: a General MIDI drum (its patch and pitch), keyed on
SND_R_FREQ_LO	$0B	Sound: a frequency's low byte (Hz), for SND_R_FREQ (only kept)
SND_R_FREQ	$0C	Sound: key on at a frequency, SND_R_FREQ_LO's byte and this its high one (Hz; 0: key off)
SND_R_GLIDE	$0D	Sound: the channel's pitch to note n without a new attack (legato)
SND_R_WAVE	$0E	Sound: a PSG channel's waveform: SND_WAVE_* << 6 | its pulse width (0-63: 63 a square)
SND_PAN_LEFT	1	Sound: SND_R_PAN's speakers
SND_PAN_RIGHT	2	
SND_PAN_BOTH	3	
SND_WAVE_PULSE	0	Sound: SND_R_WAVE's waveforms (the PSG's)
SND_WAVE_SAW	1	
SND_WAVE_TRIANGLE	2	
SND_WAVE_NOISE	3	
r0	$02	The call registers r0-r15: arguments and results
r1	$04	
r2	$06	
r3	$08	
r4	$0A	
r5	$0C	
r6	$0E	
r7	$10	
r8	$12	
r9	$14	
r10	$16	
r11	$18	
r12	$1A	
r13	$1C	
r14	$1E	
r15	$20	

<!-- calls:end -->